23AIML010_DLNLP_PR_1

March 23, 2026

Om Choksi
23AIML010

DLNLP PRACTICAL 1

DATE : 15 DEC 2025

0.1 Statistical Language Modeling: The "Lightweight" Autocomplete Engine

You have been hired as a Junior NLP Engineer by a mobile software startup. The company is developing a keyboard application for low-resource feature phones that cannot run heavy Deep Learning models (like Transformers).

Your task is to build a lightweight Predictive Text Engine using statistical N-Grams. The system must learn from a corpus of text, handle words it hasn't seen frequently (Smoothing), generate coherent sentence completions, and mathematically prove its accuracy (Perplexity).

[ ]:
```
# Importing Required Libraries
import re
from collections import Counter, defaultdict
import random
import math
import requests
from typing import List,Tuple
import os
```

Task-1: N-Gram Extraction

What is an N-Gram? An N-Gram is a contiguous sequence of N items (words, characters, etc.) from a given text.

For example, in the sentence "I love machine learning", the 2-grams (bigrams) are:

["I love", "love machine", "machine learning"]

N-Gram Probability Equation The probability of a word (w_n) given its history (h) in an n-gram model is:

$$P(w_n \mid h) = \frac{C(h, w_n)}{\sum_{w'} C(h, w')}$$

Where: - (C(h, w_n)): Count of occurrences of (h) followed by (w_n) - (h): Context (history of n-1 words)

Task:

1. Write a function `generate_ngrams(text, n)` that takes a corpus of text and breaks it down into contiguous sequences of `N` items.

```
Input: "I love machine learning"
Character-Level Bigrams: [('#', 'I'), ('I', ' '), (' ', 'l'), ('l', 'o'), ('o', 'v'), ('v', 'e
```

2. Build an n-gram language model from the corpus. Write a function `build_ngram_model(corpus, n)` that takes the corpus of text and breaks it down into contiguous `N` items, and return a probability distribution for each context.

```
Input: "I love machine learning"
output: {('#',): Counter({'I': 1.0}),
             ('I',): Counter({' ': 1.0}),
             (' ',): Counter({'l': 0.6666666666666666,
                      'm': 0.3333333333333333}),
             ('l',): Counter({'o': 0.5, 'e': 0.5}),
             ('o',): Counter({'v': 1.0}),
             ('v',): Counter({'e': 1.0}),
             ('e',): Counter({' ': 0.6666666666666666,
                      'a': 0.3333333333333333}),
             ('m',): Counter({'a': 1.0}),
             ('a',): Counter({'c': 0.5, 'r': 0.5}),
             ('c',): Counter({'h': 1.0}),
             ('h',): Counter({'i': 1.0}),
             ('i',): Counter({'n': 1.0}),
             ('n',): Counter({'e': 0.3333333333333333,
                      'i': 0.3333333333333333,
                      'g': 0.3333333333333333}),
             ('r',): Counter({'n': 1.0})}
```

[ ]:
```python
def generate_ngrams(text: str, n: int) -> List[Tuple[str]]:
    ngrams = []
    text = "#" + text
    for i in range(len(text) - n + 1):
        ngram = tuple(text[i:i+n])
        ngrams.append(ngram)
    return ngrams
```

[ ]:
```python
# Example Text
text = "I love machine learning"

# Generate and display bigrams (2-grams)
bigrams = generate_ngrams(text, 2)
print("Character-Level Bigrams:", bigrams)
```

```
Character-Level Bigrams: [('#', 'I'), ('I', ' '), (' ', 'l'), ('l', 'o'), ('o',
'v'), ('v', 'e'), ('e', ' '), (' ', 'm'), ('m', 'a'), ('a', 'c'), ('c', 'h'),
('h', 'i'), ('i', 'n'), ('n', 'e'), ('e', ' '), (' ', 'l'), ('l', 'e'), ('e',
'a'), ('a', 'r'), ('r', 'n'), ('n', 'i'), ('i', 'n'), ('n', 'g')]
```

[ ]:
```python
def build_ngram_model(corpus, n):
  ngrams = generate_ngrams(corpus, n)
  model = defaultdict(Counter)

  for i in ngrams:
    context = i[:-1]
    word = i[-1]
    model[context][word] +=1
    # print(f"{model} {context} {word}")

  for context in model:

    total_count = sum(model[context].values())
    for j in model[context]:
      model[context][j] /= total_count

  return model
```

[ ]:
```python
# Sample Test
corpus = "I love machine learning"
build_ngram_model(corpus, 2)
```

[ ]:
```
defaultdict(collections.Counter,
            {('#',): Counter({'I': 1.0}),
             ('I',): Counter({' ': 1.0}),
             (' ',): Counter({'l': 0.6666666666666666,
                      'm': 0.3333333333333333}),
             ('l',): Counter({'o': 0.5, 'e': 0.5}),
             ('o',): Counter({'v': 1.0}),
             ('v',): Counter({'e': 1.0}),
             ('e',): Counter({' ': 0.6666666666666666,
                      'a': 0.3333333333333333}),
             ('m',): Counter({'a': 1.0}),
             ('a',): Counter({'c': 0.5, 'r': 0.5}),
             ('c',): Counter({'h': 1.0}),
             ('h',): Counter({'i': 1.0}),
             ('i',): Counter({'n': 1.0}),
             ('n',): Counter({'e': 0.3333333333333333,
                      'i': 0.3333333333333333,
                      'g': 0.3333333333333333}),
             ('r',): Counter({'n': 1.0})})
```

Task 2: Handling Sparsity (Smoothing) Real-world data is sparse. If a user types a word combination not found in your training data, your model returns a probability of 0, crashing the system. Implement Add-$ $ (Laplace) Smoothing.

Smoothing assigns a small non-zero probability to unseen n-grams.

Smoothing Equation With smoothing, the probability becomes:

$$P(w_n \mid h) = \frac{C(h, w_n) + \alpha}{\sum_{w'} C(h, w') + \alpha \times |V|}$$

Where: - $ $: Smoothing parameter (default is 1) - (|V|): Vocabulary size

[ ]:
```
def add_smoothing(model, vocabulary_size, alpha=1.0):

    smoothed_model = defaultdict(Counter)

    for context in model:
        total_count = sum(model[context].values())

        denominator = total_count + (alpha * vocabulary_size)


        for char, count in model[context].items():
            smoothed_model[context][char] = (count + alpha) / denominator



    return smoothed_model
```

Task 3: Text Generation Create a function `generate_text(context, length)`: 1. Take a starting word/phrase (context). 2. Look up the probabilities of all possible next words. 3. Sample the next word based on these probabilities. 4. Update the context and repeat until the desired length is reached.

[ ]:
```
from os.path import join
def generate_text(model, n, start_text, length=100):

  curr_text = list(start_text)

  for i in range(length):
    context = tuple(curr_text[-(n-1):]) if len(curr_text) >= n-1 else tuple("#"␣
↪* (n-1 -len(curr_text)) + ''.join(curr_text))
    # print(context)

    if context not in model:
      break

    chars_dist = model[context]
```

```
    chars,probs = zip(*chars_dist.items())
    next_char = random.choices(chars,weights=probs)[0]
    curr_text.append(next_char)



  return ''.join(curr_text)
```

[ ]:
```
# Sample text
text = "hello world this is a sample text for testing the n-gram model"

# Build a bigram model
bigram_model = build_ngram_model(text, 2)

# add_smoothing(bigram_model, len(set(text)), alpha=1.0)
cnt=10
# Generate text
while(cnt>0):
  generated = generate_text(bigram_model, 2, "he", 10)
  print(f"Generated text: {generated}")
  cnt-=1
```

```
Generated text: helexthes am
Generated text: helorlelelde
Generated text: he ingr a te
Generated text: hexte n-g wo
Generated text: helo istexti
Generated text: hellexthe th
Generated text: held moram i
Generated text: hextello tis
Generated text: helld wod ng
Generated text: he amod t mp
```

Task 4: Model Evaluation (Perplexity) You need to prove to your manager that your model is actually learning. Implement the Perplexity metric on a held-out test set. Lower perplexity indicates a better model.

Action: Calculate the perplexity of your model on a short unseen test sentence (e.g., "The machine learning algorithm").

Perplexity Perplexity is a common metric for evaluating language models. Lower perplexity indicates a better model.

Perplexity Equation Perplexity measures how well a language model predicts a test dataset:

$$PP(W) = 2^{-\frac{1}{N}\sum_{i=1}^{N}\log_2 P(w_i|h_i)}$$

Where: - (W): Sequence of words - (N): Total number of words in the sequence - $ P(w_i h_i) $: Probability of word w_i given its history $ h_i $

[ ]:
```
def calculate_perplexity(model, n, test_text):

    padded_text = "#" + test_text

    ngrams = []
    for i in range(len(padded_text) - n + 1):
        ngrams.append(tuple(padded_text[i:i+n]))

    N = len(ngrams)
    log_sum = 0


    epsilon = 1e-10

    for ngram in ngrams:
        context = ngram[:-1]
        target = ngram[-1]


        if context in model and target in model[context]:
            prob = model[context][target]
        else:

            prob = epsilon

        log_sum += math.log2(prob)


    perplexity = 2 ** (-1/N * log_sum)
    return perplexity
```

[ ]:
```
# First, let's create a more substantial training corpus
training_corpus = """
The quick brown fox jumps over the lazy dog.
She sells seashells by the seashore.
How much wood would a woodchuck chuck if a woodchuck could chuck wood?
To be or not to be, that is the question.
All that glitters is not gold.
A journey of a thousand miles begins with a single step.
Actions speak louder than words.
Beauty is in the eye of the beholder.
Every cloud has a silver lining.
Fortune favors the bold and brave.
Life is like a box of chocolates.
The early bird catches the worm.
Where there's smoke, there's fire.
Time heals all wounds and teaches all things.
```

```
Knowledge is power, and power corrupts.
Practice makes perfect, but nobody's perfect.
The pen is mightier than the sword.
When in Rome, do as the Romans do.
A picture is worth a thousand words.
Better late than never, but never late is better.
Experience is the best teacher of all things.
Laughter is the best medicine for the soul.
Music soothes the savage beast within us.
Nothing ventured, nothing gained in life.
The grass is always greener on the other side.
today is monday and also december , i am from aiml department charusat␣
 ↪university.
DELHI AQI is High . AWS has more market share then Google Cloud services
Dear Students,
This is to inform you that students are not required to bring hall tickets to␣
 ↪the examination hall for regular theory examinations.

Please note the following important points:

The device number of the tablet on which the student has appeared for the exam␣
 ↪will be available in the hall ticket after the data is uploaded from the␣
 ↪Paperless system to the ERP - e-Governance.

The Exam Section will not allow any student to appear in the examination if any␣
 ↪fees are pending, except for:

Freeship card holders, or

Students who have obtained prior permission from the Principal/Dean/Registrar/
 ↪COE - Written application required.

Every student must wear their Student ID Card during the examination for␣
 ↪identity verification.

In case a student has applied for a duplicate ID card, a copy of the␣
 ↪application for duplicate ID card must be presented for verification.

Wearing or using another student's ID card will be considered a case of Unfair␣
 ↪Means (UFM).

Regards,


"""
```

```
# Clean the corpus
training_corpus = ''.join(c.lower() for c in training_corpus if c.isalnum() or␣
 ↪c.isspace())
```

[ ]:
```
training_corpus
```

[ ]:
```
'\nthe quick brown fox jumps over the lazy dog\nshe sells seashells by the
seashore\nhow much wood would a woodchuck chuck if a woodchuck could chuck
wood\nto be or not to be that is the question\nall that glitters is not gold\na
journey of a thousand miles begins with a single step\nactions speak louder than
words\nbeauty is in the eye of the beholder\nevery cloud has a silver
lining\nfortune favors the bold and brave\nlife is like a box of chocolates\nthe
early bird catches the worm\nwhere theres smoke theres fire\ntime heals all
wounds and teaches all things\nknowledge is power and power corrupts\npractice
makes perfect but nobodys perfect\nthe pen is mightier than the sword\nwhen in
rome do as the romans do\na picture is worth a thousand words\nbetter late than
never but never late is better\nexperience is the best teacher of all
things\nlaughter is the best medicine for the soul\nmusic soothes the savage
beast within us\nnothing ventured nothing gained in life\nthe grass is always
greener on the other side\ntoday is monday and also december  i am from aiml
department charusat university\ndelhi aqi is high  aws has more market share
then google cloud services \ndear students\nthis is to inform you that students
are not required to bring hall tickets to the examination hall for regular
theory examinations\n\nplease note the following important points\n\nthe device
number of the tablet on which the student has appeared for the exam will be
available in the hall ticket after the data is uploaded from the paperless
system to the erp  egovernance\n\nthe exam section will not allow any student to
appear in the examination if any fees are pending except for\n\nfreeship card
holders or\n\nstudents who have obtained prior permission from the
principaldeanregistrarcoe  written application required\n\nevery student must
wear their student id card during the examination for identity
verification\n\nin case a student has applied for a duplicate id card a copy of
the application for duplicate id card must be presented for
verification\n\nwearing or using another students id card will be considered a
case of unfair means ufm\n\nregards\n\n\n'
```

[ ]:
```
# Build models of different orders (bigram, trigram, and 4-gram models)
def build_ngram_model(corpus, n):
    ngrams = generate_ngrams(corpus, n)
    # Initialize with raw counts, not probabilities yet
    model = defaultdict(Counter)

    for i in ngrams:
        context = i[:-1]  # The history (n-1 items)
        target = i[-1]    # The prediction (current item)
        model[context][target] += 1
```

```
    return model
```

[ ]:
```
def build_models(corpus):
    models = {}

    # Vocabulary size is unique characters in corpus
    vocab_size = len(set(corpus))

    # Build models for Bigram (2), Trigram (3), and 4-gram
    for n in [2, 3, 4]:

        raw_model = build_ngram_model(corpus, n)

        smoothed_model = add_smoothing(raw_model, vocab_size, alpha=1.0)
        models[n] = smoothed_model

    return models

# Build the models
models = build_models(training_corpus)
```

[ ]:
```
# Generate samples and calculate perplexity
def evaluate_samples(models, num_samples=10, sample_length=40):
    """
    Evaluates multiple n-gram language models by generating text samples and␣
 ↪calculating their perplexity scores.

    This function:
    1. Takes different n-gram models (e.g., bigram, trigram, 4-gram)
    2. For each model:
       - Generates multiple text samples
       - Calculates perplexity for each sample
       - Computes average perplexity across all samples
    3. Stores and returns all results for comparison

    Parameters:
    -----------
    models : dict
        Dictionary where keys are n-gram sizes (e.g., 2 for bigram)
        and values are the trained n-gram models

    num_samples : int, optional (default=5)
        Number of text samples to generate for each model

    sample_length : int, optional (default=30)
        Length of each generated text sample in characters
```

```
    Returns:
    --------
    dict
        A dictionary where:
        - Keys are n-gram sizes (2, 3, 4, etc.)
        - Values are lists of dictionaries containing:
          {'text': generated_text, 'perplexity': perplexity_score}

    Example:
    --------
    # Example output structure:
    {
        2: [  # Results for bigram model
            {'text': 'hello world', 'perplexity': 10.5},
            {'text': 'sample text', 'perplexity': 12.3},
            # ... more samples
        ],
        3: [  # Results for trigram model
            {'text': 'another example', 'perplexity': 8.7},
            # ... more samples
        ]
    }
    """


    results = {}

    for n, model in models.items():
        results[n] = []

        # choose a random starting context from model keys
        start_context = "Dear "

        for _ in range(num_samples):
            generated_text = generate_text(
                model=model,
                n=n,
                start_text=start_context,
                length=sample_length
            )

            perplexity = calculate_perplexity(
                model=model,
                n=n,
                test_text=generated_text
            )
```

```
            results[n].append({
                "text": generated_text,
                "perplexity": perplexity
            })

    return results
```

[ ]:
```
# Evaluate samples
results = evaluate_samples(models)

# Calculate statistics for each model
print("\n=== Overall Statistics ===")
for n in models.keys():
    perplexities = [sample['perplexity'] for sample in results[n]]
    min_perp = min(perplexities)
    max_perp = max(perplexities)
    avg_perp = sum(perplexities) / len(perplexities)

    print(f"\n{n}-gram Model Statistics:")
    print(f"Minimum Perplexity: {min_perp:.2f}")
    print(f"Maximum Perplexity: {max_perp:.2f}")
    print(f"Average Perplexity: {avg_perp:.2f}")
    print()
```

```
=== Overall Statistics ===

2-gram Model Statistics:
Minimum Perplexity: 24.12
Maximum Perplexity: 37.85
Average Perplexity: 29.42


3-gram Model Statistics:
Minimum Perplexity: 24.66
Maximum Perplexity: 30.55
Average Perplexity: 27.52


4-gram Model Statistics:
Minimum Perplexity: 21.07
Maximum Perplexity: 32.28
Average Perplexity: 28.36
```

[ ]:
```
# Display all generated samples with their perplexities
for n in models.keys():
    print(f"\n=== Generated Samples for {n}-gram Model ===")
```

```
    for idx, sample in enumerate(results[n]):
        print(f"Sample {idx+1}:")
        print(f"Generated Text: {sample['text']}")
        print(f"Perplexity: {sample['perplexity']:.2f}\n")
```

```
=== Generated Samples for 2-gram Model ===
Sample 1:
Generated Text: Dear wet



plails w  wooa d
lar widat nang a
Perplexity: 35.32

Sample 2:
Generated Text: Dear ioppr cheror irupes sid s onsm n wongout
Perplexity: 27.81

Sample 3:
Generated Text: Dear lhen tiomuctiontheor bocaiclllaser p or
Perplexity: 27.78

Sample 4:
Generated Text: Dear fid tovapli nt r wl f ic theape nde ngif
Perplexity: 30.18

Sample 5:
Generated Text: Dear byst therenglervencerudch w in folo laqu
Perplexity: 28.88

Sample 6:
Generated Text: Dear wo iertaid jue quicly
brone jorester ssp
Perplexity: 37.85

Sample 7:
Generated Text: Dear apenga ttim bear thid d tes a ot
aling b
Perplexity: 25.99

Sample 8:
Generated Text: Dear irere sore seys

er e ord n ble n ishe t
Perplexity: 24.12
```

```
Sample 9:
Generated Text: Dear as thea prstulldugalheaicamoxalichechest
Perplexity: 26.80

Sample 10:
Generated Text: Dear sould ty
lalves
wheps be ifoowove bidea
Perplexity: 29.46


=== Generated Samples for 3-gram Model ===
Sample 1:
Generated Text: Dear nothas the weast bes ailatithipalloud wo
Perplexity: 24.66

Sample 2:
Generated Text: Dear colden faing
fromaket sid sick con thene
Perplexity: 27.08

Sample 3:
Generated Text: Dear chalway copy exam a souldeve
to bire ide
Perplexity: 28.94

Sample 4:
Generated Text: Dear of caster unfory do their neye quessinat
Perplexity: 29.59

Sample 5:
Generated Text: Dear mare
liere
lifeent stud aing the prequic
Perplexity: 26.23

Sample 6:
Generated Text: Dear tants
not als ho airegair ifir stions se
Perplexity: 30.55

Sample 7:
Generated Text: Dear datithan is on hore
the swounfavery sing
Perplexity: 27.82

Sample 8:
Generated Text: Dear linates thistess ands
```

```
plicat be prese qu
Perplexity: 27.38

Sample 9:
Generated Text: Dear  eguldent ned wilatent hareend powity id
Perplexity: 27.27

Sample 10:
Generated Text: Dear theareques a so ther dear dupt gar nown
Perplexity: 25.69


=== Generated Samples for 4-gram Model ===
Sample 1:
Generated Text: Dear the be the examination which wounds als
Perplexity: 21.07

Sample 2:
Generated Text: Dear sing or

students always power late is w
Perplexity: 26.29

Sample 3:
Generated Text: Dear the early bird catcher late is uploaded
Perplexity: 28.14

Sample 4:
Generated Text: Dear the erp  egovernance

this monday and wo
Perplexity: 30.12

Sample 5:
Generated Text: Dear thin life
time have or

freener that stu
Perplexity: 31.98

Sample 6:
Generated Text: Dear the sect
this to brave othe quired

ever
Perplexity: 27.76

Sample 7:
```

```
Generated Text: Dear silver but never late table seast to be
Perplexity: 29.48

Sample 8:
Generated Text: Dear the seashells speak loud holder
ever con
Perplexity: 28.55

Sample 9:
Generated Text: Dear infor pen if allow music soothe erp  ego
Perplexity: 32.28

Sample 10:
Generated Text: Dear the erp  egovers or

studered

every stu
Perplexity: 27.91
```

Deliverables

1. A Python Notebook (.ipynb) containing the implementations of the equations above.
2. A brief analysis report (100 words) answering: How did changing N (e.g., from Bigram to Trigram) affect the coherence of the generated text and the Perplexity score?

[ ]:

23AIML010_DLNLP_PR_1S

March 23, 2026

0.1 Statistical Language Modeling: The "Lightweight" Autocomplete Engine

You have been hired as a Junior NLP Engineer by a mobile software startup. The company is developing a keyboard application for low-resource feature phones that cannot run heavy Deep Learning models (like Transformers).

Your task is to build a lightweight Predictive Text Engine using statistical N-Grams. The system must learn from a corpus of text, handle words it hasn't seen frequently (Smoothing), generate coherent sentence completions, and mathematically prove its accuracy (Perplexity).

[ ]:
```
# Importing Required Libraries
import re
from collections import Counter, defaultdict
import random
import math
import requests
import os
```

Task-1: N-Gram Extraction

What is an N-Gram? An N-Gram is a contiguous sequence of N items (words, characters, etc.) from a given text.

For example, in the sentence "I love machine learning", the 2-grams (bigrams) are:

["I love", "love machine", "machine learning"]

N-Gram Probability Equation The probability of a word (w_n) given its history (h) in an n-gram model is:

$$P(w_n \mid h) = \frac{C(h, w_n)}{\sum_{w'} C(h, w')}$$

Where: - (C(h, w_n)): Count of occurrences of (h) followed by (w_n) - (h): Context (history of n-1 words)

Task:

1. Write a function `generate_ngrams(text, n)` that takes a corpus of text and breaks it down into contiguous sequences of `N` items.

```
Input: "I love machine learning"
Character-Level Bigrams: [('#', 'I'), ('I', ' '), (' ', 'l'), ('l', 'o'), ('o', 'v'), ('v', 'e
```

2. Build an n-gram language model from the corpus. Write a function `build_ngram_model(corpus, n)` that takes the corpus of text and breaks it down into contiguous `N` items, and return a probability distribution for each context.

```
Input: "I love machine learning"
output: {('#',): Counter({'I': 1.0}),
             ('I',): Counter({' ': 1.0}),
             (' ',): Counter({'l': 0.6666666666666666,
                      'm': 0.3333333333333333}),
             ('l',): Counter({'o': 0.5, 'e': 0.5}),
             ('o',): Counter({'v': 1.0}),
             ('v',): Counter({'e': 1.0}),
             ('e',): Counter({' ': 0.6666666666666666,
                      'a': 0.3333333333333333}),
             ('m',): Counter({'a': 1.0}),
             ('a',): Counter({'c': 0.5, 'r': 0.5}),
             ('c',): Counter({'h': 1.0}),
             ('h',): Counter({'i': 1.0}),
             ('i',): Counter({'n': 1.0}),
             ('n',): Counter({'e': 0.3333333333333333,
                      'i': 0.3333333333333333,
                      'g': 0.3333333333333333}),
             ('r',): Counter({'n': 1.0})}
```

[ ]:
```python
def generate_ngrams(text, n):
    pass
```

[ ]:
```python
# Example Text
text = "I love machine learning"

# Generate and display bigrams (2-grams)
bigrams = generate_ngrams(text, 2)
print("Character-Level Bigrams:", bigrams)
```

```
Character-Level Bigrams: [('#', 'I'), ('I', ' '), (' ', 'l'), ('l', 'o'), ('o',
'v'), ('v', 'e'), ('e', ' '), (' ', 'm'), ('m', 'a'), ('a', 'c'), ('c', 'h'),
('h', 'i'), ('i', 'n'), ('n', 'e'), ('e', ' '), (' ', 'l'), ('l', 'e'), ('e',
'a'), ('a', 'r'), ('r', 'n'), ('n', 'i'), ('i', 'n'), ('n', 'g')]
```

[ ]:
```python
def build_ngram_model(corpus, n):
    pass
```

[ ]:
```python
# Sample Test
corpus = "I love machine learning"
build_ngram_model(corpus, 2)
```

[ ]:
```
defaultdict(collections.Counter,
            {('#',): Counter({'I': 1.0}),
             ('I',): Counter({' ': 1.0}),
             (' ',): Counter({'l': 0.6666666666666666,
                      'm': 0.3333333333333333}),
             ('l',): Counter({'o': 0.5, 'e': 0.5}),
             ('o',): Counter({'v': 1.0}),
             ('v',): Counter({'e': 1.0}),
             ('e',): Counter({' ': 0.6666666666666666,
                      'a': 0.3333333333333333}),
             ('m',): Counter({'a': 1.0}),
             ('a',): Counter({'c': 0.5, 'r': 0.5}),
             ('c',): Counter({'h': 1.0}),
             ('h',): Counter({'i': 1.0}),
             ('i',): Counter({'n': 1.0}),
             ('n',): Counter({'e': 0.3333333333333333,
                      'i': 0.3333333333333333,
                      'g': 0.3333333333333333}),
             ('r',): Counter({'n': 1.0})})
```

Task 2: Handling Sparsity (Smoothing) Real-world data is sparse. If a user types a word combination not found in your training data, your model returns a probability of 0, crashing the system. Implement Add-$ $ (Laplace) Smoothing.

Smoothing assigns a small non-zero probability to unseen n-grams.

Smoothing Equation With smoothing, the probability becomes:

$$P(w_n \mid h) = \frac{C(h, w_n) + \alpha}{\sum_{w'} C(h, w') + \alpha \times |V|}$$

Where: - $ $: Smoothing parameter (default is 1) - (|V|): Vocabulary size

[ ]:
```
def add_smoothing(model, vocabulary_size, alpha=1.0):
    pass
```

Task 3: Text Generation Create a function `generate_text(context, length)`: 1. Take a starting word/phrase (context). 2. Look up the probabilities of all possible next words. 3. Sample the next word based on these probabilities. 4. Update the context and repeat until the desired length is reached.

[ ]:
```
def generate_text(model, n, start_text, length=100):
    pass
```

[ ]:
```
# Sample text
text = "hello world this is a sample text for testing the n-gram model"

# Build a bigram model
bigram_model = build_ngram_model(text, 2)
```

```
# Generate text
generated = generate_text(bigram_model, 2, "he", 10)
print(f"Generated text: {generated}")
```

```
Generated text: helo ingramo
```

Task 4: Model Evaluation (Perplexity) You need to prove to your manager that your model is actually learning. Implement the Perplexity metric on a held-out test set. Lower perplexity indicates a better model.

Action: Calculate the perplexity of your model on a short unseen test sentence (e.g., "The machine learning algorithm").

Perplexity Perplexity is a common metric for evaluating language models. Lower perplexity indicates a better model.

Perplexity Equation Perplexity measures how well a language model predicts a test dataset:

$$PP(W) = 2^{-\frac{1}{N}\sum_{i=1}^{N}\log_2 P(w_i|h_i)}$$

Where: - (W): Sequence of words - (N): Total number of words in the sequence - $ P(w_i h_i) $: Probability of word w_i given its history $ h_i $

[ ]:
```
def calculate_perplexity(model, n, test_text):
    pass
```

[ ]:
```
# First, let's create a more substantial training corpus
training_corpus = """
The quick brown fox jumps over the lazy dog.
She sells seashells by the seashore.
How much wood would a woodchuck chuck if a woodchuck could chuck wood?
To be or not to be, that is the question.
All that glitters is not gold.
A journey of a thousand miles begins with a single step.
Actions speak louder than words.
Beauty is in the eye of the beholder.
Every cloud has a silver lining.
Fortune favors the bold and brave.
Life is like a box of chocolates.
The early bird catches the worm.
Where there's smoke, there's fire.
Time heals all wounds and teaches all things.
Knowledge is power, and power corrupts.
Practice makes perfect, but nobody's perfect.
The pen is mightier than the sword.
When in Rome, do as the Romans do.
A picture is worth a thousand words.
Better late than never, but never late is better.
```

```
Experience is the best teacher of all things.
Laughter is the best medicine for the soul.
Music soothes the savage beast within us.
Nothing ventured, nothing gained in life.
The grass is always greener on the other side.
"""

# Clean the corpus
training_corpus = ''.join(c.lower() for c in training_corpus if c.isalnum() or␣
 ↪c.isspace())
```

[ ]:
```
training_corpus
```

[ ]:
```
'\nthe quick brown fox jumps over the lazy dog\nshe sells seashells by the
seashore\nhow much wood would a woodchuck chuck if a woodchuck could chuck
wood\nto be or not to be that is the question\nall that glitters is not gold\na
journey of a thousand miles begins with a single step\nactions speak louder than
words\nbeauty is in the eye of the beholder\nevery cloud has a silver
lining\nfortune favors the bold and brave\nlife is like a box of chocolates\nthe
early bird catches the worm\nwhere theres smoke theres fire\ntime heals all
wounds and teaches all things\nknowledge is power and power corrupts\npractice
makes perfect but nobodys perfect\nthe pen is mightier than the sword\nwhen in
rome do as the romans do\na picture is worth a thousand words\nbetter late than
never but never late is better\nexperience is the best teacher of all
things\nlaughter is the best medicine for the soul\nmusic soothes the savage
beast within us\nnothing ventured nothing gained in life\nthe grass is always
greener on the other side\n'
```

[ ]:
```
# Build models of different orders (bigram, trigram, and 4-gram models)
def build_models(corpus):
    pass
    return models

# Build the models
models = build_models(training_corpus)
```

[ ]:
```
# Generate samples and calculate perplexity
def evaluate_samples(models, num_samples=10, sample_length=40):
    """
    Evaluates multiple n-gram language models by generating text samples and␣
 ↪calculating their perplexity scores.

    This function:
    1. Takes different n-gram models (e.g., bigram, trigram, 4-gram)
    2. For each model:
       - Generates multiple text samples
       - Calculates perplexity for each sample
```

```
       - Computes average perplexity across all samples
    3. Stores and returns all results for comparison

    Parameters:
    -----------
    models : dict
        Dictionary where keys are n-gram sizes (e.g., 2 for bigram)
        and values are the trained n-gram models

    num_samples : int, optional (default=5)
        Number of text samples to generate for each model

    sample_length : int, optional (default=30)
        Length of each generated text sample in characters

    Returns:
    --------
    dict
        A dictionary where:
        - Keys are n-gram sizes (2, 3, 4, etc.)
        - Values are lists of dictionaries containing:
          {'text': generated_text, 'perplexity': perplexity_score}

    Example:
    --------
    # Example output structure:
    {
        2: [  # Results for bigram model
            {'text': 'hello world', 'perplexity': 10.5},
            {'text': 'sample text', 'perplexity': 12.3},
            # ... more samples
        ],
        3: [  # Results for trigram model
            {'text': 'another example', 'perplexity': 8.7},
            # ... more samples
        ]
    }
    """
    pass
```

[ ]:
```
# Evaluate samples
results = evaluate_samples(models)

# Calculate statistics for each model
print("\n=== Overall Statistics ===")
for n in models.keys():
    perplexities = [sample['perplexity'] for sample in results[n]]
```

```
    min_perp = min(perplexities)
    max_perp = max(perplexities)
    avg_perp = sum(perplexities) / len(perplexities)

    print(f"\n{n}-gram Model Statistics:")
    print(f"Minimum Perplexity: {min_perp:.2f}")
    print(f"Maximum Perplexity: {max_perp:.2f}")
    print(f"Average Perplexity: {avg_perp:.2f}")
```

```
=== 2-gram Model Evaluation ===

Sample 1:
Text:
athteter azysoke be pobe ack tinorterdoo
Perplexity: 7.43

Sample 2:
Text:
sex couckever be d wofactoonora wher pon
Perplexity: 8.10

Sample 3:
Text:
ak lds
k is bisthe oolas whifoullacty th
Perplexity: 6.95

Sample 4:
Text:
houicoudea ak seane ls ove saite theeso
Perplexity: 9.45

Sample 5:
Text:
this ire wiochoucot bingld teves ththe
a
Perplexity: 6.56

Sample 6:
Text:
l insine sike ald us
berthande tur jods
Perplexity: 7.54

Sample 7:
Text:
k thecothoferrs als s is qucans imot nou
```

```
Perplexity: 7.38

Sample 8:
Text:
wotththe allwoof s
thes ouisotis pt ckes
Perplexity: 6.91

Sample 9:
Text:
ne woxperea a s anor s
angs ome bullas a
Perplexity: 6.48

Sample 10:
Text:
the was thinerauchikeys sin ck ain there
Perplexity: 5.93

Average Perplexity for 2-gram model: 7.27

=== 3-gram Model Evaluation ===

Sample 1:
Text:
to buty beach ands
knothinglike eacturedg
Perplexity: 2.62

Sample 2:
Text:
thers a thene row man the speas is is lau
Perplexity: 3.18

Sample 3:
Text:
to beginins med peredge quic soure
to bir
Perplexity: 2.86

Sample 4:
Text:
to but by ing gre
horrupts
prache eashe t
Perplexity: 2.51

Sample 5:
```

```
Text:
the quick woodchuck chuck corthan usan th
Perplexity: 2.06

Sample 6:
Text:
the is word thateashen forrupts
nown ing

Perplexity: 2.74

Sample 7:
Text:
the mightion therienture do be othe life
Perplexity: 2.67

Sample 8:
Text:
tion nothe is is alls a jumps is a per la
Perplexity: 2.33

Sample 9:
Text:
tion fird
alls sell thand ned by of a jum
Perplexity: 2.66

Sample 10:
Text:
tick lazy bromak latere ste thingled corr
Perplexity: 3.15

Average Perplexity for 3-gram model: 2.68

=== 4-gram Model Evaluation ===

Sample 1:
Text:
the grass in that golder like thousand pow
Perplexity: 1.61

Sample 2:
Text:
the early bird catches best withingle step
Perplexity: 1.38

Sample 3:
Text:
```

```
the early bird chuck words
better that is
Perplexity: 1.38

Sample 4:
Text:
the quest teaches the step
actice is is li
Perplexity: 1.50

Sample 5:
Text:
the pen is nothe perience makes smoke a si
Perplexity: 1.57

Sample 6:
Text:
the wounds and mightier of chuck brave
lif
Perplexity: 1.47

Sample 7:
Text:
the box jumps overy cloud has the worth a
Perplexity: 1.40

Sample 8:
Text:
the early bird chocolate thin life is with
Perplexity: 1.47

Sample 9:
Text:
the best medicine favors is liningle step

Perplexity: 1.40

Sample 10:
Text:
the late is power thes all worm
wher on th
Perplexity: 1.75

Average Perplexity for 4-gram model: 1.49

=== Overall Statistics ===

2-gram Model Statistics:
```

```
Minimum Perplexity: 5.93
Maximum Perplexity: 9.45
Average Perplexity: 7.27

3-gram Model Statistics:
Minimum Perplexity: 2.06
Maximum Perplexity: 3.18
Average Perplexity: 2.68

4-gram Model Statistics:
Minimum Perplexity: 1.38
Maximum Perplexity: 1.75
Average Perplexity: 1.49
```

Deliverables

1. A Python Notebook (.ipynb) containing the implementations of the equations above.
2. A brief analysis report (100 words) answering: How did changing N (e.g., from Bigram to Trigram) affect the coherence of the generated text and the Perplexity score?

[ ]:

23AIML010_DLNLP_PR_2N

March 23, 2026

[1]:
```
# Importing Required library
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

1 Natural Language Processing with NLTK & spaCy

1.1 Task 1: Load and Explore the Dataset

Objective: Understand the data structure and perform basic Exploratory Data Analysis (EDA).

In this task, we will create a read a given dataset and explore it.

1. Check first 5 rows
2. Analyse Class distribution
3. Analyse Text Length

[2]:
```
# 1.1 Read dataset
df = pd.read_csv('/content/emo_text.csv')
df.head()
```

[2]:

	sentiment	content
0	negative	Layin n bed with a headache ughhhh…waitin o…
1	negative	Funeral ceremony…gloomy friday…
2	neutral	@dannycastillo We want to trade with someone w…
3	negative	I should be sleep, but im not! thinking about …
4	negative	@charviray Charlene my love. I miss you

[3]:
```
# 1.2 Basic Exploration
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17289 entries, 0 to 17288
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   sentiment  17289 non-null  object
 1   content    17289 non-null  object
```

```
dtypes: object(2)
memory usage: 270.3+ KB
```

[4]:
```
df.isnull().sum()
```

[4]:
```
sentiment    0
content      0
dtype: int64
```

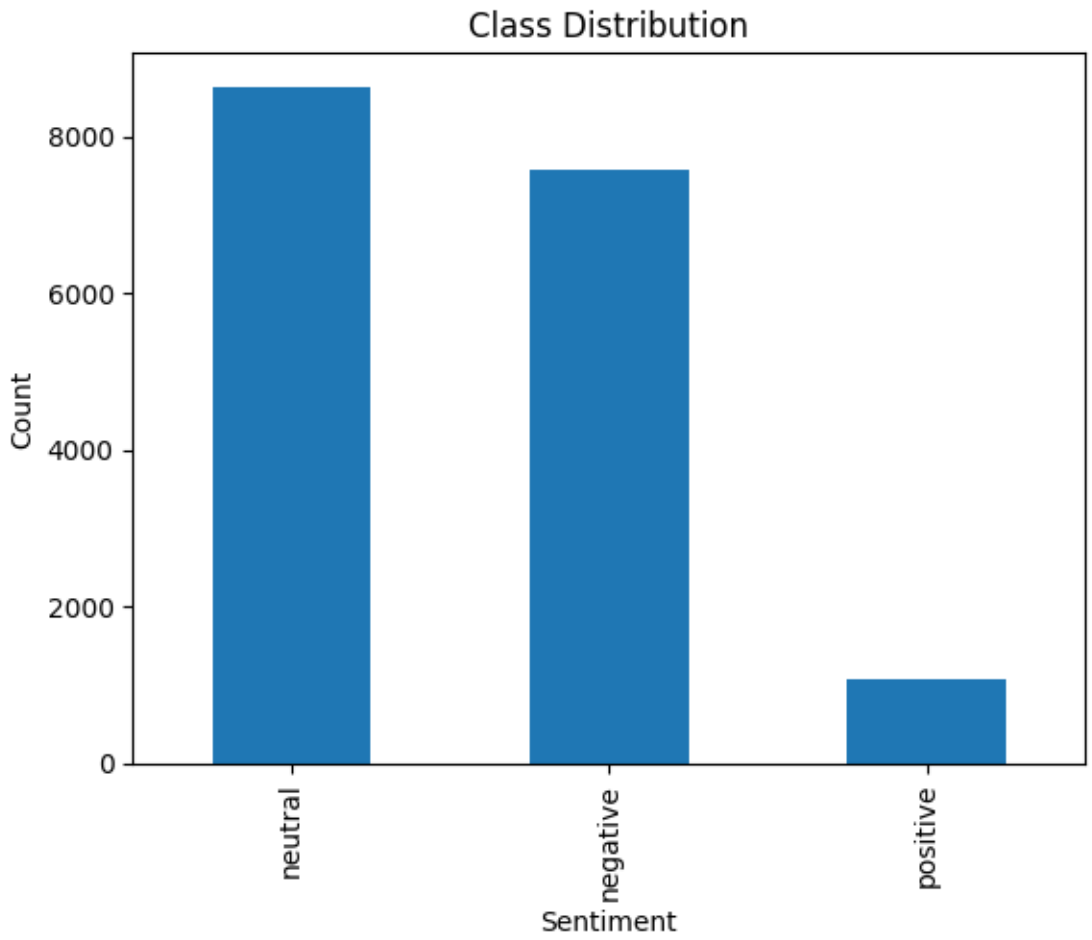
[5]:
```
df.columns
```

[5]:
```
Index(['sentiment', 'content'], dtype='object')
```

[6]:
```
# 1.3 Class Distribution Analysis
df['sentiment'].value_counts()
```

[6]:
```
sentiment
neutral     8638
negative    7567
positive    1084
Name: count, dtype: int64
```

[7]:
```
df['sentiment'].value_counts().plot(kind='bar')
plt.title("Class Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.show()
```



[8]:
```
# 1.4 Text Length Analysis
df['content_length'] = df['content'].apply(len)
df[['content', 'content_length']].head()
```

[8]:

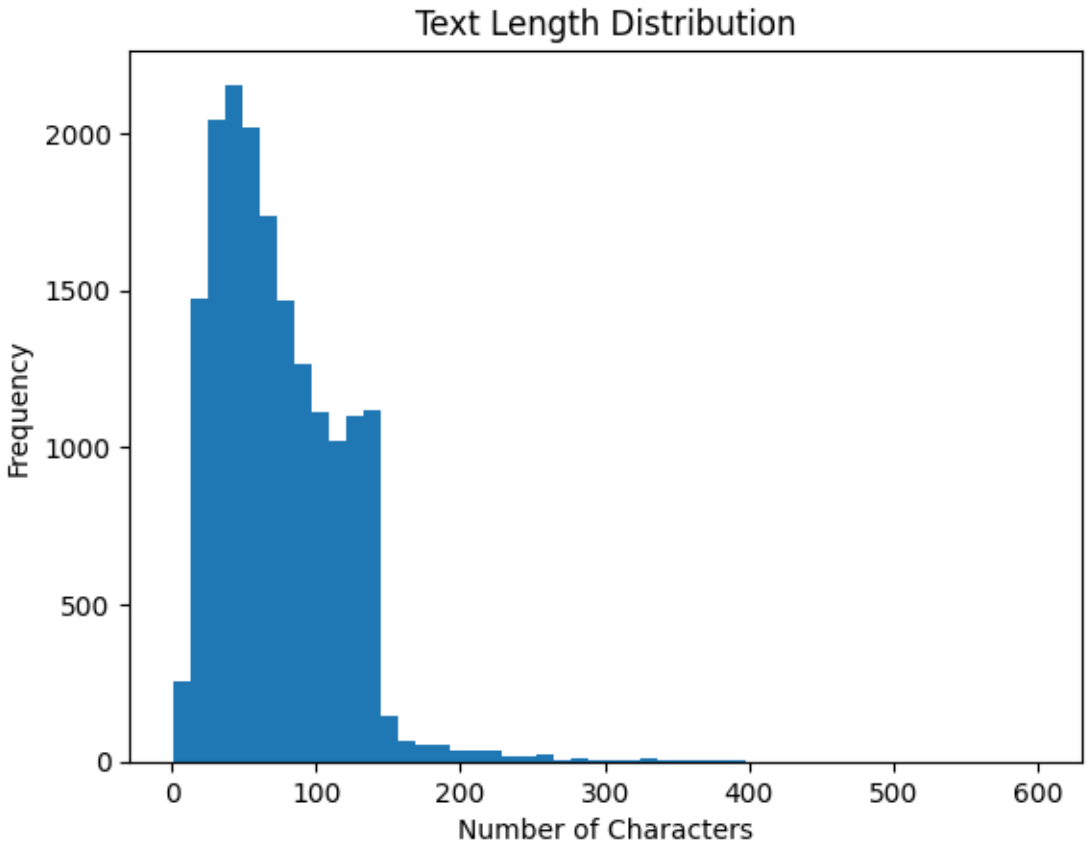
	content	content_length
0	Layin n bed with a headache ughhhh…waitin o…	60
1	Funeral ceremony…gloomy friday…	35
2	@dannycastillo We want to trade with someone w…	86
3	I should be sleep, but im not! thinking about …	132
4	@charviray Charlene my love. I miss you	39

[9]:
```
df['content_length'].describe()
```

[9]:
```
count    17289.000000
mean        73.347504
std         44.001671
min          1.000000
```

```
25%          40.000000
50%          65.000000
75%         103.000000
max         601.000000
Name: content_length, dtype: float64
```

[10]:
```
df['content_length'].plot(kind='hist', bins=50)
plt.title("Text Length Distribution")
plt.xlabel("Number of Characters")
plt.show()
```



1.2 Task 2: Text Pre-processing with NLTK

Objective: Learn the "classic" way of cleaning text using the Natural Language Toolkit (NLTK). We will cover Tokenization, Stopword Removal, and Stemming.

Write a python function `preprocess_nltk(text)` that does the following using `NLTK Library`:

1. Normalising the text (Lowercasing)
2. Tokenization (Splitting text into words)
3. Removing Punctuation and Stopwords

4. Stemming (Reducing words to their root form, e.g., “running” -> “run”)

and return cleaned text

[11]:
```
# NLTK Text Preprocessing

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
import string

# Download required NLTK data
nltk.download('punkt_tab')
nltk.download('stopwords')

# Initialize stemmer and stopwords
stemmer = PorterStemmer()
stop_words = set(stopwords.words('english'))

# Define preprocessing function
def nltk_preprocess(text):
    text = text.lower()
    text = text.translate(str.maketrans('', '', string.punctuation))
    tokens = word_tokenize(text)
    tokens = [stemmer.stem(word) for word in tokens if word not in stop_words]
    return " ".join(tokens)

# Apply preprocessing to dataset
df['NLTK_Cleaned'] = df['content'].apply(nltk_preprocess)

# Display result
print("Original vs NLTK Processed:")
display(df[['content', 'NLTK_Cleaned']].head())
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data…
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data…
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
Original vs NLTK Processed:
```

	content	NLTK_Cleaned
0	Layin n bed with a headache ughhhh…waitin o…	
1	Funeral ceremony…gloomy friday…	
2	@dannycastillo We want to trade with someone w…	
3	I should be sleep, but im not! thinking about …	
4	@charviray Charlene my love. I miss you	

```
0              layin n bed headach ughhhhwaitin call
1                        funer ceremonygloomi friday
2  dannycastillo want trade someon houston ticket…
3  sleep im think old friend want he marri damn a…
4                        charviray charlen love miss
```

1.3 Task 3: Advanced Pre-processing with spaCy

Objective: Use spaCy's industrial-strength pipeline for efficient processing, focusing on Lemmatization.

wrtie a python function `preprocess_spacy(text` that does the follwoing thing using `spacy`:

1. Create a spaCy Doc object
2. Lemmatization and stopword removal

and return cleaned text

[12]:
```python
# Task 3: Text Preprocessing using spaCy

import spacy

# Load English language model
nlp = spacy.load("en_core_web_sm")

# Define spaCy preprocessing function
def spacy_preprocess(text):
    doc = nlp(text.lower())
    tokens = [
        token.lemma_
        for token in doc
        if not token.is_stop and not token.is_punct and not token.is_space
    ]
    return " ".join(tokens)

# Apply preprocessing to dataset
df['spacy_cleaned'] = df['content'].apply(spacy_preprocess)

# Display original vs spaCy cleaned text
print("Original vs spaCy Processed:")
display(df[['content', 'spacy_cleaned']].head())
```

```
Original vs spaCy Processed:
```

```
                                             content  \
0  Layin n bed with a headache  ughhhh…waitin o…
1                Funeral ceremony…gloomy friday…
2  @dannycastillo We want to trade with someone w…
3  I should be sleep, but im not! thinking about …
4            @charviray Charlene my love. I miss you
```

```
                                        spacy_cleaned
0                  layin n bed headache ughhhh waitin
1                      funeral ceremony gloomy friday
2            @dannycastillo want trade houston ticket
3  sleep m think old friend want marry damn amp w…
4                       @charviray charlene love miss
```

1.4 Task 4: Feature Engineering (Bag of Words & TF-IDF)

Objective: Machines cannot understand text. We must convert text into numbers (vectors).

1.4.1 4.1 Count Vectorizer (Bag of Words)

This method counts how many times each word appears in a document.

[13]:
```
# Task 4.1: Feature Extraction using Count Vectorizer

from sklearn.feature_extraction.text import CountVectorizer

# Initialize Count Vectorizer
count_vectorizer = CountVectorizer(max_features=5000)

# Fit and transform the spaCy cleaned text
X_count = count_vectorizer.fit_transform(df['spacy_cleaned'])

# Target labels
y = df['sentiment']

# Display shape of Count Vectorized matrix
print("Count Vectorizer Feature Matrix Shape:", X_count.shape)

# Display sample feature names
print("Sample Features:", count_vectorizer.get_feature_names_out()[:10])
```

```
Count Vectorizer Feature Matrix Shape: (17289, 5000)
Sample Features: ['00' '000' '02' '03' '05' '06' '07' '08' '09' '10']
```

1.4.2 4.2 TF-IDF (Term Frequency - Inverse Document Frequency)

This method highlights words that are important to a specific document but rare across the corpus (filtering out common words like “the”, “is”, etc., even if stopwords didn’t catch them).

[14]:
```
# Task 4.2: Feature Extraction using TF-IDF Vectorize

from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer(max_features=5000)
```

```
# Fit and transform the spaCy cleaned text
X_tfidf = tfidf_vectorizer.fit_transform(df['spacy_cleaned'])

# Target labels
y = df['sentiment']

# Display TF-IDF matrix shape
print("TF-IDF Feature Matrix Shape:", X_tfidf.shape)

# Display sample TF-IDF feature names
print("Sample TF-IDF Features:", tfidf_vectorizer.get_feature_names_out()[:10])
```

```
TF-IDF Feature Matrix Shape: (17289, 5000)
Sample TF-IDF Features: ['00' '000' '02' '03' '05' '06' '07' '08' '09' '10']
```

1.5 Task 5: Sentiment Analysis Model

Objective: Build a Machine Learning classifier to predict sentiment based on our text features.

Write a python code that builds `MultinomiaNB` model that is trained on TF-IDF values and sentiment.

[15]:
```
# Task 5: Train-Test Split and Model Training

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix,␣
 ↪classification_report

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_count, y, test_size=0.2, random_state=42
)

# Initialize Naive Bayes classifier
nb_model = MultinomialNB()

# Train the model
nb_model.fit(X_train, y_train)

# Predict on test data
y_pred = nb_model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.6902834008097166

Confusion Matrix:
 [[ 995  456   45]
 [ 491 1241   21]
 [  48   10  151]]

Classification Report:
              precision    recall  f1-score   support

    negative       0.65      0.67      0.66      1496
     neutral       0.73      0.71      0.72      1753
    positive       0.70      0.72      0.71       209

    accuracy                           0.69      3458
   macro avg       0.69      0.70      0.69      3458
weighted avg       0.69      0.69      0.69      3458
```

1.6 Task 6: Interactive Prediction

Objective: Test the model with new, unseen user input.

Write python function `predict_sentiment(text)` that predict the sentiment of the given text by performing the following: 1. Preprocess using our defined spaCy function 2. Vectorize using the fitted TF-IDF object 3. Prediction

and returns the predicted sentiment.

[16]:
```python
# Task 6: Sentiment Prediction for New Text

# Function to predict sentiment of new input text
def predict_sentiment(text):
    # Preprocess text using spaCy
    cleaned_text = spacy_preprocess(text)

    # Convert text to count vector
    vector = count_vectorizer.transform([cleaned_text])

    # Predict sentiment
    prediction = nb_model.predict(vector)

    return prediction[0]

# Test the function with sample inputs
print("Input: I really love this product")
print("Predicted Sentiment:", predict_sentiment("I really love this product"))

print("\nInput: This is the worst experience ever")
```

```
print("Predicted Sentiment:", predict_sentiment("This is the worst experience␣
 ↪ever"))

print("\nInput: Good Product")
print("Predicted Sentiment:", predict_sentiment("Good Product"))

print("\nInput: Very Slow Delivery")
print("Predicted Sentiment:", predict_sentiment("Very Slow Delivery"))
```

```
Input: I really love this product
Predicted Sentiment: neutral

Input: This is the worst experience ever
Predicted Sentiment: negative

Input: Good Product
Predicted Sentiment: neutral

Input: Very Slow Delivery
Predicted Sentiment: negative
```

[17]:
```
df['sentiment'].value_counts()
```

[17]:
```
sentiment
neutral     8638
negative    7567
positive    1084
Name: count, dtype: int64
```

[ ]:
```

```

nbconvert_safe_qb_yec6b

March 23, 2026

0.0.1 By *23AIML010, Om Choksi*

1 Deep Learning Primitives and Shallow Networks

You are part of a research team tasked with migrating traditional NLP methods (like N-Grams) to modern neural network architectures. Before building complex models like RNNs, you must first master the fundamental mathematical components that allow any deep learning model to learn.

Your primary goal is to implement and visualize the key functions (Activation, Loss, Optimization) and then integrate them into a simple, fully-connected (shallow) network using a modern framework (PyTorch).

- **Task 1: Activation Function Implementation (Sigmoid)** - Completed.
- **Task 2: Loss Function Definition (Mean Squared Error) and Gradient Descent** - Completed.
- **Task 3: Building a Shallow DNN in PyTorch** - Completed with a `SimpleDNN` class, including forward and backward passes with a training loop.

1.1 Import required libraries

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

1.2 Tasks

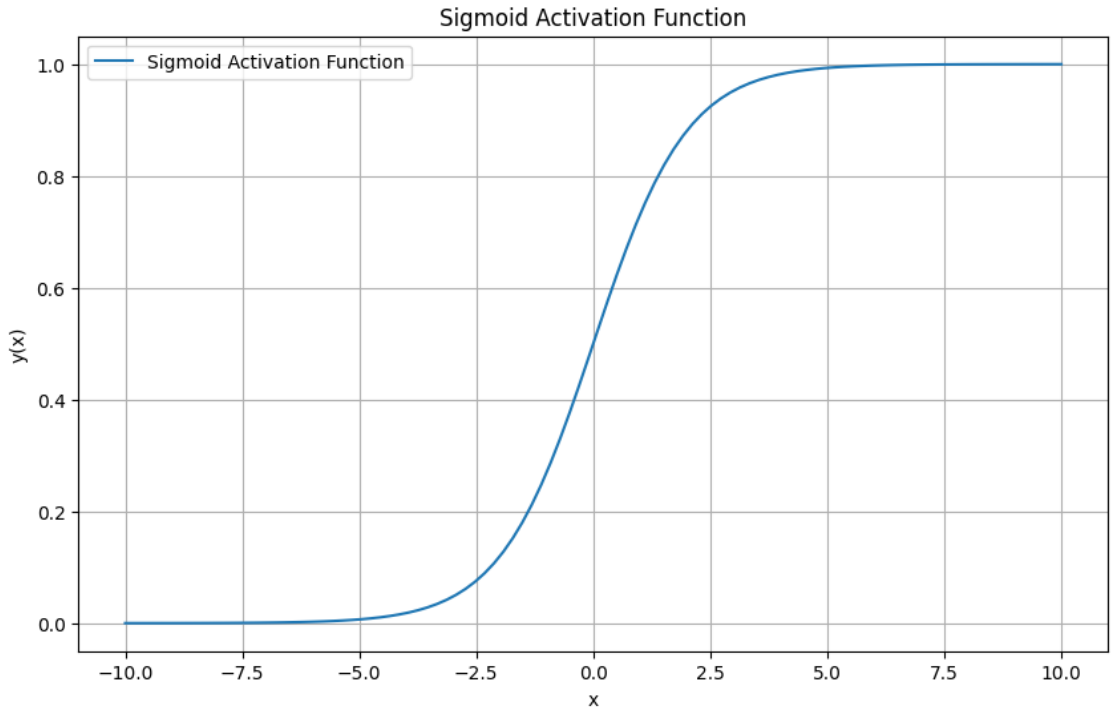
1.2.1 Task 1: Activation Function Implementation (Sigmoid)

The activation function is crucial for introducing non-linearity into the network, allowing it to learn complex patterns. Implement the `sigmoid(x)` function using numpy:

- Generate input values from -10 to 10 and calculate the corresponding Sigmoid output.
- Plot the Sigmoid Activation Function using matplotlib to visualize its non-linear S-shape and saturation behavior.

[2]:
```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

x = np.linspace(-10, 10, 100)
y_sigmoid = sigmoid(x)
```

```
plt.figure(figsize=(10, 6))
plt.plot(x, y_sigmoid, label='Sigmoid Activation Function')
plt.xlabel('x')
plt.ylabel('y(x)')
plt.title('Sigmoid Activation Function')
plt.grid(True)
plt.legend()
plt.show()
```



1.2.2 Task 2: Loss Function Definition (Mean Squared Error)

The loss function quantifies the difference between the model's prediction and the true value. - Define the loss function `def loss_function(x)`, which is $ y = x^2$ - Deinfe the Gradiant function `def gradiant(x)`, which is $ y = 2x $

- Implement and Visualise the Gradient Descent Algorithm Implement a generic `gradient_descent(loss_function, gradient, start_x, learning_rate, iterations)` function.
- Use the provided `loss_function(x)` function and its gradient to simulate the iterative update process.
- Run the optimizer for a set number of iterations (e.g., 100), printing the x value, Loss, and Gradient at each step to demonstrate convergence to the minimum.

Import Required Libraries

[3]:
```
import numpy as np
import matplotlib.pyplot as plt
```

Define Loss Function and Gradient

[4]:
```
# Define a simple quadratic loss function: y = x^2
def loss_function(x):
    return x**2

# Derivative of the loss function: dy/dx = 2x
def gradient(x):
    return 2*x
```

Gardient Descent Formulae: new = old - learning rate * slope

Gradient Descent Algorithm

[5]:
```
# Gradient Descent Algorithm
def gradient_descent(starting_point, learning_rate, iterations):
  x = starting_point  # Initial value
  history = []        # To store x values and corresponding loss
  for i in range(iterations):
    loss = loss_function(x)
    gradient_value = gradient(x)
    history.append((x,loss))
    # Update x using gradient descent formula
    x -= learning_rate*gradient_value
    if i%5 == 0:
      print(f"Iteration {i+1}: x = {x:.4f}, Loss = {loss:.4f}, Gradient =␣
 ↪{gradient_value:.4f}")

  return history
```

Parameters for Gradient Descent

[8]:
```
starting_point = 5  # Starting point for x
learning_rate = 0.09 # Step size for gradient descent
iterations = 100      # Number of iterations
```

Run Gradient Descent

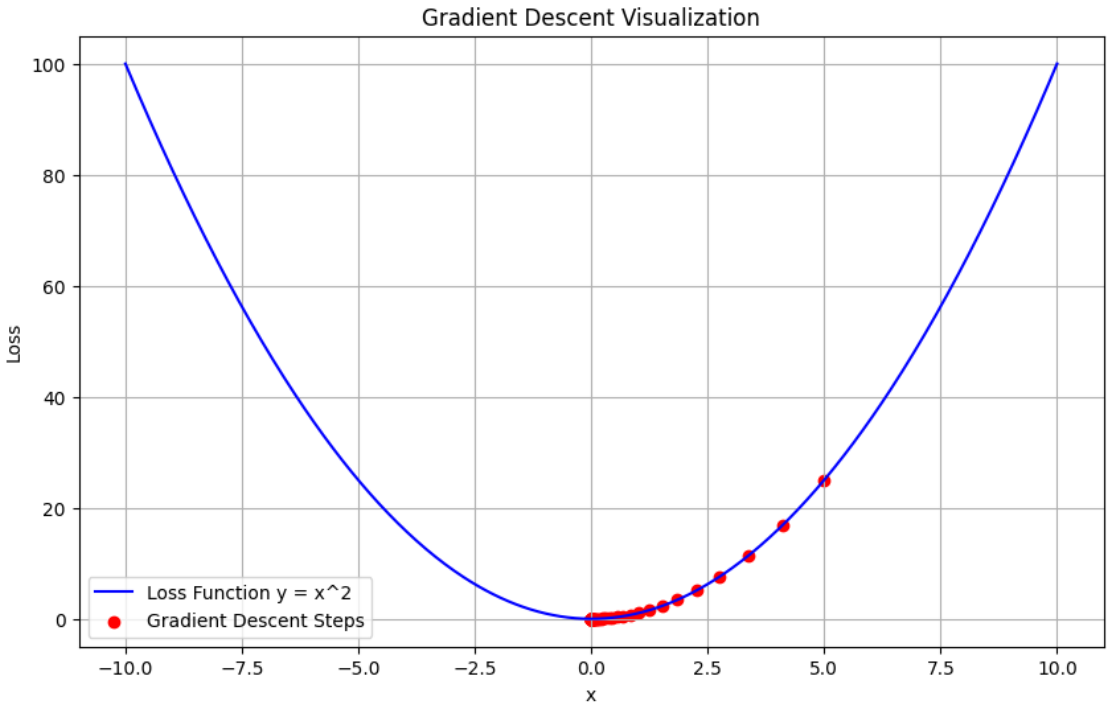
[9]:
```
history = gradient_descent(starting_point, learning_rate, iterations)
```

```
Iteration 1: x = 4.1000, Loss = 25.0000, Gradient = 10.0000
Iteration 6: x = 1.5200, Loss = 3.4362, Gradient = 3.7074
Iteration 11: x = 0.5635, Loss = 0.4723, Gradient = 1.3745
Iteration 16: x = 0.2089, Loss = 0.0649, Gradient = 0.5096
Iteration 21: x = 0.0775, Loss = 0.0089, Gradient = 0.1889
Iteration 26: x = 0.0287, Loss = 0.0012, Gradient = 0.0700
Iteration 31: x = 0.0106, Loss = 0.0002, Gradient = 0.0260
Iteration 36: x = 0.0039, Loss = 0.0000, Gradient = 0.0096
```

```
Iteration 41: x = 0.0015, Loss = 0.0000, Gradient = 0.0036
Iteration 46: x = 0.0005, Loss = 0.0000, Gradient = 0.0013
Iteration 51: x = 0.0002, Loss = 0.0000, Gradient = 0.0005
Iteration 56: x = 0.0001, Loss = 0.0000, Gradient = 0.0002
Iteration 61: x = 0.0000, Loss = 0.0000, Gradient = 0.0001
Iteration 66: x = 0.0000, Loss = 0.0000, Gradient = 0.0000
Iteration 71: x = 0.0000, Loss = 0.0000, Gradient = 0.0000
Iteration 76: x = 0.0000, Loss = 0.0000, Gradient = 0.0000
Iteration 81: x = 0.0000, Loss = 0.0000, Gradient = 0.0000
Iteration 86: x = 0.0000, Loss = 0.0000, Gradient = 0.0000
Iteration 91: x = 0.0000, Loss = 0.0000, Gradient = 0.0000
Iteration 96: x = 0.0000, Loss = 0.0000, Gradient = 0.0000
```

Visualization of Gradient Descent

[10]:
```python
# Extract x and loss values for plotting
x_values, loss_values = zip(*history)

# Plotting the Loss Function
x_plot = np.linspace(-10, 10, 500)
y_plot = loss_function(x_plot)

plt.figure(figsize=(10, 6))
plt.plot(x_plot, y_plot, label='Loss Function y = x^2', color='blue')
plt.scatter(x_values, loss_values, color='red', label='Gradient Descent Steps')
plt.title('Gradient Descent Visualization')
plt.xlabel('x')
plt.ylabel('Loss')
plt.legend()
plt.grid()
plt.show()
```



1.2.3 Task 3: Building a Shallow DNN in PyTorch

Translate the mathematical primitives into a functional PyTorch model.

image

Define a class SimpleDNN that inherits from torch.nn.Module. Construct a simple network with two layers: - An input layer a hidden layer (e.g., 2 input features, 2 hidden units). - A hidden layer an output layer (e.g., 2 hidden units, 1 output unit). - Use the Sigmoid activation function on the output layer to ensure the output is between 0 and 1, suitable for binary classification probability. - Write the forward() method and test it with a sample input tensor (e.g., torch.Tensor([0.05, 0.10])).

- Source_blog

[11]:
```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy
from sklearn import datasets
```

[13]:
```
m = nn.Linear(20, 30)
```

[19]:
```
m
```

[19]:
```
Linear(in_features=20, out_features=30, bias=True)
```

[20]:
```
nn.Linear
```

[20]:
```
torch.nn.modules.linear.Linear
```

[21]:
```
m = nn.Linear(2, 3)
input = torch.randn(3, 2)
output = m(input)
```

[22]:
```
input
```

[22]:
```
tensor([[-0.6082, -2.2776],
        [ 0.4528,  1.0164],
        [ 1.1085, -1.0995]])
```

[23]:
```
output
```

[23]:
```
tensor([[-0.0604, -1.5590, -2.0588],
        [ 0.7737,  0.3051,  0.5720],
        [ 0.3138, -0.1972, -0.6640]], grad_fn=<AddmmBackward0>)
```

[24]:
```
output.shape
```

[24]:
```
torch.Size([3, 3])
```

[25]:
```
m.weight.data
```

[25]:
```
tensor([[0.0568, 0.2349],
        [0.5198, 0.3985],
        [0.3395, 0.6893]])
```

[26]:
```
m.bias.data
```

[26]:
```
tensor([ 0.5092, -0.3353, -0.2823])
```

[27]:
```
def init_weight(m):
    if type(m) in [nn.Conv2d,nn.Linear]:
        m.weight.data=torch.Tensor([[0.15,0.20],[0.25,0.30]])
        m.bias.data=torch.Tensor([0.35])
```

[29]:
```
## Model Set Up
class SimpleDNN(nn.Module):
  def __init__(self,input_size,H1,output_size):
      super().__init__()

      self.linear1 = nn.Linear(input_size,H1)
      self.linear1.weight.data=torch.Tensor([[0.15,0.20],[0.25,0.30]])
      self.linear1.bias.data = torch.Tensor([0.35])
```

```
        self.linear2 = nn.Linear(H1,output_size)
        # Initialize weights for the output layer to have 1 output neuron,␣
    ↪assuming H1 is 2
        self.linear2.weight.data=torch.Tensor([[0.40,0.45]]) # Changed to 1␣
    ↪output neuron
        self.linear2.bias.data = torch.Tensor([0.60])

    def forward(self,x, print_values=True):
        # Hidden layer with no activation specified in the task description
        net_h = self.linear1(x)
        out_h = torch.sigmoid(net_h) # Sigmoid on hidden layer as in original␣
    ↪code, though not explicitly stated for SimpleDNN

        # Output layer with Sigmoid activation as specified
        net_O = self.linear2(out_h)
        out_O = torch.sigmoid(net_O)

        if print_values:
            print("h1: {}, h2: {}".format(net_h[0], net_h[1]))
            print("out_h1: {}, out_h2: {}".format(out_h[0], out_h[1]))
            # Only one output neuron, so net_O[0] and out_O[0]
            print("net_O: {}".format(net_O[0]))
            print("out_O: {}".format(out_O[0]))


        return (out_O)
```

1.3 Unfold each epoch and check intermediate values

1.3.1 Initial Weight of the model

[30]:
```
model=SimpleDNN(2,2,1) # Changed output_size to 1
print (list(model.parameters()))
```

```
[Parameter containing:
tensor([[0.1500, 0.2000],
        [0.2500, 0.3000]], requires_grad=True), Parameter containing:
tensor([0.3500], requires_grad=True), Parameter containing:
tensor([[0.4000, 0.4500]], requires_grad=True), Parameter containing:
tensor([0.6000], requires_grad=True)]
```

1.3.2 After 1 forward pass check hidden, output and loss

image

[31]:
```
model.forward(torch.Tensor([0.05,0.10]))
```

```
h1: 0.3774999976158142, h2: 0.39249998331069946
out_h1: 0.5932700037956238, out_h2: 0.5968843698501587
```

```
net_O: 1.1059060096740723
out_O: 0.751365065574646
```

[31]:
```
tensor([0.7514], grad_fn=<SigmoidBackward0>)
```

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = (x_n - y_n)^2,$$

[32]:
```
nn.MSELoss
```

[32]:
```
torch.nn.modules.loss.MSELoss
```

[33]:
```
model=SimpleDNN(2,2,1) # Changed output_size to 1
criterion=nn.MSELoss()
optimizer=torch.optim.SGD(model.parameters(),lr=0.5)
output=model.forward(torch.Tensor([0.05,0.10]))
target=torch.Tensor([0.01]) # Changed target to 1 element
loss=criterion(output,target)
print("total MSEerror: {}".format(loss.item()))
```

```
h1: 0.3774999976158142, h2: 0.39249998331069946
out_h1: 0.5932700037956238, out_h2: 0.5968843698501587
net_O: 1.1059060096740723
out_O: 0.751365065574646
total MSEerror: 0.5496221780776978
```

1.3.3 Backward pass

[34]:
```
# set prev grad to zero
optimizer.zero_grad()

loss.backward()

optimizer.step()

print(list(model.parameters()))
```

```
[Parameter containing:
tensor([[0.1493, 0.1987],
        [0.2493, 0.2985]], requires_grad=True), Parameter containing:
tensor([0.3216], requires_grad=True), Parameter containing:
tensor([[0.3178, 0.3673]], requires_grad=True), Parameter containing:
tensor([0.4615], requires_grad=True)]
```

1.3.4 2nd Forward Pass

[35]:
```
y_output = model.forward(torch.Tensor([0.05,0.10]))
```

```
h1: 0.34896889328956604, h2: 0.3639485538005829
out_h1: 0.5863675475120544, out_h2: 0.5899959206581116
```

```
net_O: 0.8645930290222168
out_O: 0.7036193609237671
```

1.3.5 2nd Backward pass

[36]:
```
loss=criterion(y_output,target)

optimizer.zero_grad()

loss.backward()

optimizer.step()

print(list(model.parameters()))
```

```
[Parameter containing:
tensor([[0.1488, 0.1975],
        [0.2486, 0.2972]], requires_grad=True), Parameter containing:
tensor([0.2976], requires_grad=True), Parameter containing:
tensor([[0.2330, 0.2820]], requires_grad=True), Parameter containing:
tensor([0.3169], requires_grad=True)]
```

1.4 Reinitializing the model and looping over Epoch

[37]:
```
model=SimpleDNN(2,2,1) # Changed output_size to 1
criterion=nn.MSELoss()
optimizer=torch.optim.SGD(model.parameters(),lr=0.5)

input_x = torch.Tensor([0.05,0.10])
target=torch.Tensor([0.01]) # Changed target to 1 element
epoch = 200
for i in range(epoch):
    y_output=model.forward(input_x)

    loss=criterion(y_output,target)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if i%10 == 0:
        print(f'Epoch: {i}, loss: {loss.item()}, output: {y_output}')
        print("="*60)
```

```
h1: 0.3774999976158142, h2: 0.39249998331069946
out_h1: 0.5932700037956238, out_h2: 0.5968843698501587
net_O: 1.1059060096740723
out_O: 0.751365065574646
Epoch: 0, loss: 0.5496221780776978, output: tensor([0.7514],
```

```
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.34896889328956604, h2: 0.3639485538005829
out_h1: 0.5863675475120544, out_h2: 0.5899959206581116
net_O: 0.8645930290222168
out_O: 0.7036193609237671
h1: 0.32482603192329407, h2: 0.3397844135761261
out_h1: 0.5804998874664307, out_h2: 0.5841381549835205
net_O: 0.6168428063392639
out_O: 0.6495001912117004
h1: 0.3064892292022705, h2: 0.3214262127876282
out_h1: 0.576028048992157, out_h2: 0.5796718001365662
net_O: 0.37098366022109985
out_O: 0.5916966795921326
h1: 0.29458481073379517, h2: 0.30950117111206055
out_h1: 0.5731181502342224, out_h2: 0.5767635107040405
net_O: 0.13606621325016022
out_O: 0.5339641571044922
h1: 0.2887270748615265, h2: 0.3036244511604309
out_h1: 0.5716844797134399, out_h2: 0.5753283500671387
net_O: -0.08056922256946564
out_O: 0.4798685908317566
h1: 0.2877812385559082, h2: 0.30266159772872925
out_h1: 0.5714528560638428, out_h2: 0.5750930309295654
net_O: -0.27497002482414246
out_O: 0.4316873848438263
h1: 0.2903723418712616, h2: 0.3052377998828888
out_h1: 0.572087287902832, out_h2: 0.5757224559783936
net_O: -0.4465624988079071
out_O: 0.390178382396698
h1: 0.29527726769447327, h2: 0.3101297616958618
out_h1: 0.5732876062393188, out_h2: 0.5769169330596924
net_O: -0.596998929977417
out_O: 0.3550305962562561
h1: 0.3015751242637634, h2: 0.31641632318496704
out_h1: 0.5748274922370911, out_h2: 0.578450620174408
net_O: -0.7289049625396729
out_O: 0.325435072183609
h1: 0.3086354732513428, h2: 0.32346686720848083
out_h1: 0.576552152633667, out_h2: 0.5801689028739929
net_O: -0.8450542688369751
out_O: 0.30047136545181274
Epoch: 10, loss: 0.08437362313270569, output: tensor([0.3005],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.31605011224746704, h2: 0.33087289333343506
out_h1: 0.5783613324165344, out_h2: 0.5819717049598694
net_O: -0.9479742646217346
```

```
out_O: 0.27929240465164185
h1: 0.3235635757446289, h2: 0.3383787274360657
out_h1: 0.580192506313324, out_h2: 0.5837966799736023
net_O: -1.0398186445236206
out_O: 0.2611849904060364
h1: 0.3310202360153198, h2: 0.3458285629749298
out_h1: 0.5820075869560242, out_h2: 0.5856056809425354
net_O: -1.1223655939102173
out_O: 0.2455727458000183
h1: 0.3383280634880066, h2: 0.3531303107738495
out_h1: 0.583784282207489, out_h2: 0.5873764753341675
net_O: -1.1970646381378174
out_O: 0.2319978028535843
h1: 0.3454352021217346, h2: 0.3602319359779358
out_h1: 0.5855101943016052, out_h2: 0.5890966057777405
net_O: -1.2650933265686035
out_O: 0.2200983464717865
h1: 0.3523147702217102, h2: 0.36710646748542786
out_h1: 0.5871788263320923, out_h2: 0.5907596349716187
net_O: -1.327410101890564
out_O: 0.20958808064460754
h1: 0.3589555025100708, h2: 0.37374264001846313
out_h1: 0.5887875556945801, out_h2: 0.5923629999160767
net_O: -1.3847992420196533
out_O: 0.20023931562900543
h1: 0.36535581946372986, h2: 0.38013869524002075
out_h1: 0.5903363227844238, out_h2: 0.5939065217971802
net_O: -1.4379069805145264
out_O: 0.19186967611312866
h1: 0.3715198040008545, h2: 0.3862988352775574
out_h1: 0.5918261408805847, out_h2: 0.5953914523124695
net_O: -1.4872682094573975
out_O: 0.18433211743831635
h1: 0.37745511531829834, h2: 0.39223048090934753
out_h1: 0.5932591557502747, out_h2: 0.5968195796012878
net_O: -1.5333309173583984
out_O: 0.17750684916973114
Epoch: 20, loss: 0.02805854193866253, output: tensor([0.1775],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.3831711709499359, h2: 0.3979431986808777
out_h1: 0.5946377515792847, out_h2: 0.598193347454071
net_O: -1.5764710903167725
out_O: 0.17129583656787872
h1: 0.38867834210395813, h2: 0.4034472107887268
out_h1: 0.5959644913673401, out_h2: 0.5995156168937683
net_O: -1.6170079708099365
out_O: 0.1656179279088974
```

```
h1: 0.3939873278141022, h2: 0.40875324606895447
out_h1: 0.5972421765327454, out_h2: 0.6007888913154602
net_O: -1.6552135944366455
out_O: 0.16040556132793427
h1: 0.3991087079048157, h2: 0.4138718545436859
out_h1: 0.5984734892845154, out_h2: 0.6020159125328064
net_O: -1.6913214921951294
out_O: 0.1556021273136139
h1: 0.4040527045726776, h2: 0.41881322860717773
out_h1: 0.5996609330177307, out_h2: 0.6031992435455322
net_O: -1.7255332469940186
out_O: 0.15115982294082642
h1: 0.4088291823863983, h2: 0.42358726263046265
out_h1: 0.6008070707321167, out_h2: 0.6043413281440735
net_O: -1.7580243349075317
out_O: 0.14703795313835144
h1: 0.4134474992752075, h2: 0.4282032251358032
out_h1: 0.6019142270088196, out_h2: 0.6054444909095764
net_O: -1.788947343826294
out_O: 0.1432018280029297
h1: 0.41791635751724243, h2: 0.43266984820365906
out_h1: 0.6029845476150513, out_h2: 0.6065110564231873
net_O: -1.8184369802474976
out_O: 0.1396215260028839
h1: 0.4222440719604492, h2: 0.4369954466819763
out_h1: 0.6040200591087341, out_h2: 0.607542872428894
net_O: -1.8466111421585083
out_O: 0.13627128303050995
h1: 0.42643827199935913, h2: 0.44118762016296387
out_h1: 0.6050228476524353, out_h2: 0.6085420250892639
net_O: -1.8735748529434204
out_O: 0.13312862813472748
Epoch: 30, loss: 0.015160659328103065, output: tensor([0.1331],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.43050616979599, h2: 0.4452535808086395
out_h1: 0.6059945225715637, out_h2: 0.6095101833343506
net_O: -1.8994213342666626
out_O: 0.13017398118972778
h1: 0.43445444107055664, h2: 0.4492000341415405
out_h1: 0.6069368720054626, out_h2: 0.6104490160942078
net_O: -1.9242335557937622
out_O: 0.12739022076129913
h1: 0.4382893741130829, h2: 0.4530331790447235
out_h1: 0.6078513264656067, out_h2: 0.6113601326942444
net_O: -1.9480862617492676
out_O: 0.12476218491792679
h1: 0.44201675057411194, h2: 0.45675885677337646
```

```
out_h1: 0.6087394952774048, out_h2: 0.6122450232505798
net_O: -1.9710463285446167
out_O: 0.12227655202150345
h1: 0.44564202427864075, h2: 0.46038249135017395
out_h1: 0.6096025705337524, out_h2: 0.6131048798561096
net_O: -1.9931743144989014
out_O: 0.11992143839597702
h1: 0.44917023181915283, h2: 0.4639091491699219
out_h1: 0.6104419827461243, out_h2: 0.6139411330223083
net_O: -2.0145249366760254
out_O: 0.11768631637096405
h1: 0.45260611176490784, h2: 0.4673435091972351
out_h1: 0.6112586855888367, out_h2: 0.6147547960281372
net_O: -2.035147190093994
out_O: 0.11556179821491241
h1: 0.45595404505729675, h2: 0.470689982175827
out_h1: 0.6120539307594299, out_h2: 0.6155470609664917
net_O: -2.0550873279571533
out_O: 0.11353933811187744
h1: 0.45921817421913147, h2: 0.4739527106285095
out_h1: 0.6128286719322205, out_h2: 0.6163188815116882
net_O: -2.07438588142395
out_O: 0.11161141842603683
h1: 0.4624023735523224, h2: 0.4771355390548706
out_h1: 0.6135839223861694, out_h2: 0.6170712113380432
net_O: -2.093080520629883
out_O: 0.10977117717266083
Epoch: 40, loss: 0.009954288601875305, output: tensor([0.1098],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.46551018953323364, h2: 0.4802420437335968
out_h1: 0.6143205165863037, out_h2: 0.617805004119873
net_O: -2.1112060546875
out_O: 0.10801241546869278
h1: 0.4685450792312622, h2: 0.4832756519317627
out_h1: 0.6150393486022949, out_h2: 0.6185210943222046
net_O: -2.1287941932678223
out_O: 0.1063295230269432
h1: 0.4715101718902588, h2: 0.486239492893219
out_h1: 0.6157410740852356, out_h2: 0.6192201972007751
net_O: -2.145873785018921
out_O: 0.1047174334526062
h1: 0.47440841794013977, h2: 0.48913657665252686
out_h1: 0.6164266467094421, out_h2: 0.619903028011322
net_O: -2.1624722480773926
out_O: 0.10317147523164749
h1: 0.47724267840385437, h2: 0.4919697046279907
out_h1: 0.6170965433120728, out_h2: 0.6205703616142273
```

```
net_O: -2.1786141395568848
out_O: 0.10168745368719101
h1: 0.4800155758857727, h2: 0.49474143981933594
out_h1: 0.6177515983581543, out_h2: 0.621222734451294
net_O: -2.1943228244781494
out_O: 0.10026146471500397
h1: 0.48272958397865295, h2: 0.49745437502861023
out_h1: 0.6183922290802002, out_h2: 0.621860921382904
net_O: -2.2096195220947266
out_O: 0.09888997673988342
h1: 0.48538702726364136, h2: 0.5001107454299927
out_h1: 0.6190191507339478, out_h2: 0.6224853992462158
net_O: -2.22452449798584
out_O: 0.09756969660520554
h1: 0.487990140914917, h2: 0.5027128458023071
out_h1: 0.6196328401565552, out_h2: 0.6230966448783875
net_O: -2.239055633544922
out_O: 0.09629769623279572
h1: 0.4905409812927246, h2: 0.5052626729011536
out_h1: 0.6202338337898254, out_h2: 0.6236952543258667
net_O: -2.2532310485839844
out_O: 0.09507112205028534
Epoch: 50, loss: 0.007237096317112446, output: tensor([0.0951],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.4930415153503418, h2: 0.5077621936798096
out_h1: 0.6208226680755615, out_h2: 0.6242817640304565
net_O: -2.26706600189209
out_O: 0.09388752281665802
h1: 0.495493620634079, h2: 0.5102133750915527
out_h1: 0.6213997006416321, out_h2: 0.6248564720153809
net_O: -2.280576229095459
out_O: 0.09274446219205856
h1: 0.49789905548095703, h2: 0.5126178860664368
out_h1: 0.6219655275344849, out_h2: 0.6254199743270874
net_O: -2.293776035308838
out_O: 0.09163974225521088
h1: 0.500259518623352, h2: 0.5149773955345154
out_h1: 0.6225203275680542, out_h2: 0.6259725689888
net_O: -2.30667781829834
out_O: 0.09057141840457916
h1: 0.5025765299797058, h2: 0.5172935724258423
out_h1: 0.6230646371841431, out_h2: 0.6265146732330322
net_O: -2.3192944526672363
out_O: 0.08953756093978882
h1: 0.5048516392707825, h2: 0.5195677876472473
out_h1: 0.6235988140106201, out_h2: 0.6270467042922974
net_O: -2.3316378593444824
```

```
out_O: 0.08853640407323837
h1: 0.5070863366127014, h2: 0.5218015909194946
out_h1: 0.624123215675354, out_h2: 0.6275689601898193
net_O: -2.343719005584717
out_O: 0.0875663161277771
h1: 0.5092818737030029, h2: 0.5239963531494141
out_h1: 0.6246381402015686, out_h2: 0.6280817985534668
net_O: -2.3555476665496826
out_O: 0.0866258293390274
h1: 0.5114395618438721, h2: 0.5261532664299011
out_h1: 0.6251438856124878, out_h2: 0.6285855174064636
net_O: -2.3671340942382812
out_O: 0.0857134610414505
h1: 0.5135607719421387, h2: 0.5282736420631409
out_h1: 0.6256408095359802, out_h2: 0.6290803551673889
net_O: -2.3784875869750977
out_O: 0.08482790738344193
Epoch: 60, loss: 0.005599216092377901, output: tensor([0.0848],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.5156464576721191, h2: 0.5303585529327393
out_h1: 0.6261292099952698, out_h2: 0.6295667290687561
net_O: -2.3896164894104004
out_O: 0.08396792411804199
h1: 0.5176979303359985, h2: 0.5324092507362366
out_h1: 0.6266093254089355, out_h2: 0.6300448179244995
net_O: -2.400529623031616
out_O: 0.0831323191523552
h1: 0.5197161436080933, h2: 0.534426748752594
out_h1: 0.6270813941955566, out_h2: 0.6305149793624878
net_O: -2.4112343788146973
out_O: 0.08232002705335617
h1: 0.5217021703720093, h2: 0.5364120602607727
out_h1: 0.6275457143783569, out_h2: 0.6309773325920105
net_O: -2.4217381477355957
out_O: 0.08153000473976135
h1: 0.523656964302063, h2: 0.5383660793304443
out_h1: 0.6280024647712708, out_h2: 0.631432294845581
net_O: -2.4320480823516846
out_O: 0.08076129108667374
h1: 0.525581419467926, h2: 0.5402898788452148
out_h1: 0.628451943397522, out_h2: 0.6318798661231995
net_O: -2.4421706199645996
out_O: 0.0800129845738411
h1: 0.5274764895439148, h2: 0.5421842932701111
out_h1: 0.6288943290710449, out_h2: 0.6323204040527344
net_O: -2.4521126747131348
out_O: 0.0792841911315918
```

```
h1: 0.5293430089950562, h2: 0.5440501570701599
out_h1: 0.6293298602104187, out_h2: 0.6327540874481201
net_O: -2.4618797302246094
out_O: 0.07857413589954376
h1: 0.531181812286377, h2: 0.5458883047103882
out_h1: 0.6297587156295776, out_h2: 0.6331811547279358
net_O: -2.471477746963501
out_O: 0.07788204401731491
h1: 0.5329936742782593, h2: 0.5476994514465332
out_h1: 0.630181074142456, out_h2: 0.6336016654968262
net_O: -2.480912208557129
out_O: 0.07720718532800674
Epoch: 70, loss: 0.004516805987805128, output: tensor([0.0772],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.5347793102264404, h2: 0.5494844913482666
out_h1: 0.6305971145629883, out_h2: 0.6340159773826599
net_O: -2.4901881217956543
out_O: 0.07654889672994614
h1: 0.5365394353866577, h2: 0.5512440204620361
out_h1: 0.6310070753097534, out_h2: 0.6344242095947266
net_O: -2.4993107318878174
out_O: 0.07590651512145996
h1: 0.5382748246192932, h2: 0.5529787540435791
out_h1: 0.6314109563827515, out_h2: 0.6348264217376709
net_O: -2.508284330368042
out_O: 0.07527945190668106
h1: 0.5399860739707947, h2: 0.5546894073486328
out_h1: 0.6318092346191406, out_h2: 0.6352229118347168
net_O: -2.5171141624450684
out_O: 0.07466708868741989
h1: 0.5416738390922546, h2: 0.556376576423645
out_h1: 0.6322017312049866, out_h2: 0.635613739490509
net_O: -2.525803804397583
out_O: 0.07406891882419586
h1: 0.5433387160301208, h2: 0.5580408573150635
out_h1: 0.632588803768158, out_h2: 0.6359990835189819
net_O: -2.534358024597168
out_O: 0.0734843835234642
h1: 0.5449813008308411, h2: 0.5596828460693359
out_h1: 0.6329704523086548, out_h2: 0.6363791227340698
net_O: -2.5427803993225098
out_O: 0.07291300594806671
h1: 0.5466021299362183, h2: 0.5613031983375549
out_h1: 0.6333469152450562, out_h2: 0.636754035949707
net_O: -2.551074981689453
out_O: 0.07235430181026459
h1: 0.548201858997345, h2: 0.5629023313522339
```

```
out_h1: 0.6337183117866516, out_h2: 0.6371238231658936
net_O: -2.5592451095581055
out_O: 0.07180783897638321
h1: 0.5497809052467346, h2: 0.5644808411598206
out_h1: 0.6340847611427307, out_h2: 0.6374887228012085
net_O: -2.5672943592071533
out_O: 0.07127319276332855
Epoch: 80, loss: 0.0037544043734669685, output: tensor([0.0713],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.5513398051261902, h2: 0.5660392045974731
out_h1: 0.634446382522583, out_h2: 0.6378487348556519
net_O: -2.575226306915283
out_O: 0.070749931037426
h1: 0.5528790950775146, h2: 0.5675778985023499
out_h1: 0.634803295135498, out_h2: 0.6382040977478027
net_O: -2.5830440521240234
out_O: 0.07023768126964569
h1: 0.5543991327285767, h2: 0.5690974593162537
out_h1: 0.6351556181907654, out_h2: 0.6385548710823059
net_O: -2.590750217437744
out_O: 0.06973609328269958
h1: 0.555900514125824, h2: 0.5705983638763428
out_h1: 0.6355034708976746, out_h2: 0.6389012336730957
net_O: -2.598348617553711
out_O: 0.06924477964639664
h1: 0.5573835968971252, h2: 0.5720809102058411
out_h1: 0.6358469724655151, out_h2: 0.6392431855201721
net_O: -2.6058413982391357
out_O: 0.06876342743635178
h1: 0.5588488578796387, h2: 0.5735456347465515
out_h1: 0.6361861228942871, out_h2: 0.6395809650421143
net_O: -2.613231658935547
out_O: 0.06829169392585754
h1: 0.5602966547012329, h2: 0.5749929547309875
out_h1: 0.6365211606025696, out_h2: 0.6399144530296326
net_O: -2.6205220222473145
out_O: 0.06782928109169006
h1: 0.5617273449897766, h2: 0.576423168182373
out_h1: 0.6368521451950073, out_h2: 0.6402439475059509
net_O: -2.6277146339416504
out_O: 0.06737591326236725
h1: 0.563141405582428, h2: 0.5778367519378662
out_h1: 0.6371790766716003, out_h2: 0.6405695080757141
net_O: -2.634812355041504
out_O: 0.06693128496408463
h1: 0.5645391941070557, h2: 0.5792340636253357
out_h1: 0.6375021934509277, out_h2: 0.6408911943435669
```

```
net_O: -2.641817569732666
out_O: 0.0664951279759407
Epoch: 90, loss: 0.0031916997395455837, output: tensor([0.0665],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.5659209489822388, h2: 0.5806154012680054
out_h1: 0.6378214359283447, out_h2: 0.6412090063095093
net_O: -2.6487321853637695
out_O: 0.06606718897819519
h1: 0.5672872066497803, h2: 0.5819811820983887
out_h1: 0.6381369829177856, out_h2: 0.6415231227874756
net_O: -2.6555585861206055
out_O: 0.06564723700284958
h1: 0.5686381459236145, h2: 0.5833317041397095
out_h1: 0.6384488940238953, out_h2: 0.6418336629867554
net_O: -2.662299156188965
out_O: 0.06523499637842178
h1: 0.5699741840362549, h2: 0.5846672654151917
out_h1: 0.6387572288513184, out_h2: 0.6421406269073486
net_O: -2.6689553260803223
out_O: 0.06483027338981628
h1: 0.5712955594062805, h2: 0.5859882235527039
out_h1: 0.6390620470046997, out_h2: 0.6424441337585449
net_O: -2.675529956817627
out_O: 0.06443281471729279
h1: 0.5726026892662048, h2: 0.5872949361801147
out_h1: 0.6393635272979736, out_h2: 0.642744243144989
net_O: -2.6820240020751953
out_O: 0.06404244899749756
h1: 0.5738958120346069, h2: 0.5885875821113586
out_h1: 0.6396616101264954, out_h2: 0.6430410146713257
net_O: -2.6884398460388184
out_O: 0.0636589527130127
h1: 0.5751751065254211, h2: 0.5898665189743042
out_h1: 0.6399564743041992, out_h2: 0.6433345079421997
net_O: -2.6947789192199707
out_O: 0.06328213959932327
h1: 0.5764410495758057, h2: 0.5911319851875305
out_h1: 0.6402480602264404, out_h2: 0.6436247825622559
net_O: -2.701043128967285
out_O: 0.06291183084249496
h1: 0.57769376039505, h2: 0.5923842787742615
out_h1: 0.6405366063117981, out_h2: 0.6439120769500732
net_O: -2.7072343826293945
out_O: 0.06254781782627106
Epoch: 100, loss: 0.0027612734120339155, output: tensor([0.0625],
grad_fn=<SigmoidBackward0>)
============================================================
```

```
h1: 0.5789334774017334, h2: 0.5936236381530762
out_h1: 0.6408219337463379, out_h2: 0.6441961526870728
net_O: -2.7133536338806152
out_O: 0.0621899776160717
h1: 0.5801605582237244, h2: 0.5948503017425537
out_h1: 0.6411043405532837, out_h2: 0.6444772481918335
net_O: -2.7194032669067383
out_O: 0.06183807551860809
h1: 0.581375241279602, h2: 0.5960646271705627
out_h1: 0.6413837671279907, out_h2: 0.6447554230690002
net_O: -2.72538423538208
out_O: 0.06149200722575188
h1: 0.5825777649879456, h2: 0.5972667336463928
out_h1: 0.6416603326797485, out_h2: 0.6450307369232178
net_O: -2.7312979698181152
out_O: 0.061151597648859024
h1: 0.5837683081626892, h2: 0.598456859588623
out_h1: 0.6419340372085571, out_h2: 0.6453031301498413
net_O: -2.7371463775634766
out_O: 0.0608166940510273
h1: 0.5849471688270569, h2: 0.5996353626251221
out_h1: 0.6422049403190613, out_h2: 0.6455729007720947
net_O: -2.7429299354553223
out_O: 0.060487180948257446
h1: 0.5861145257949829, h2: 0.6008023023605347
out_h1: 0.6424731612205505, out_h2: 0.6458398103713989
net_O: -2.7486510276794434
out_O: 0.06016288325190544
h1: 0.5872706174850464, h2: 0.6019580364227295
out_h1: 0.6427386403083801, out_h2: 0.6461041569709778
net_O: -2.754310131072998
out_O: 0.05984368920326233
h1: 0.5884155631065369, h2: 0.6031026244163513
out_h1: 0.6430014967918396, out_h2: 0.6463658213615417
net_O: -2.75990891456604
out_O: 0.05952946096658707
h1: 0.5895496606826782, h2: 0.604236364364624
out_h1: 0.6432617902755737, out_h2: 0.6466249227523804
net_O: -2.765448570251465
out_O: 0.059220075607299805
Epoch: 110, loss: 0.0024226161185652018, output: tensor([0.0592],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.5906730890274048, h2: 0.6053594350814819
out_h1: 0.6435195207595825, out_h2: 0.6468815207481384
net_O: -2.7709298133850098
out_O: 0.05891543626785278
h1: 0.5917860269546509, h2: 0.6064720153808594
```

```
out_h1: 0.6437748670578003, out_h2: 0.6471356153488159
net_O: -2.7763547897338867
out_O: 0.05861537158489227
h1: 0.5928886532783508, h2: 0.6075742840766907
out_h1: 0.6440276503562927, out_h2: 0.6473872661590576
net_O: -2.7817234992980957
out_O: 0.058319833129644394
h1: 0.593981146812439, h2: 0.6086664199829102
out_h1: 0.6442781090736389, out_h2: 0.6476365327835083
net_O: -2.7870376110076904
out_O: 0.05802867189049721
h1: 0.5950636267662048, h2: 0.6097486019134521
out_h1: 0.6445261240005493, out_h2: 0.647883415222168
net_O: -2.792297840118408
out_O: 0.057741809636354446
h1: 0.5961364507675171, h2: 0.6108210682868958
out_h1: 0.6447718739509583, out_h2: 0.648128092288971
net_O: -2.7975058555603027
out_O: 0.05745910480618477
h1: 0.5971996188163757, h2: 0.6118838787078857
out_h1: 0.6450153589248657, out_h2: 0.6483703851699829
net_O: -2.802661895751953
out_O: 0.0571804977953434
h1: 0.5982533097267151, h2: 0.6129372715950012
out_h1: 0.6452565789222717, out_h2: 0.648610532283783
net_O: -2.807767391204834
out_O: 0.05690588057041168
h1: 0.5992977619171143, h2: 0.6139813661575317
out_h1: 0.645495593547821, out_h2: 0.6488484740257263
net_O: -2.8128232955932617
out_O: 0.05663514882326126
h1: 0.600333034992218, h2: 0.6150163412094116
out_h1: 0.6457325220108032, out_h2: 0.6490842700004578
net_O: -2.8178300857543945
out_O: 0.05636823922395706
Epoch: 120, loss: 0.0021500138100236654, output: tensor([0.0564],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.6013594269752502, h2: 0.6160423755645752
out_h1: 0.6459672451019287, out_h2: 0.6493179202079773
net_O: -2.822788715362549
out_O: 0.05610506609082222
h1: 0.6023769974708557, h2: 0.617059588432312
out_h1: 0.6461999416351318, out_h2: 0.6495494842529297
net_O: -2.827700614929199
out_O: 0.05584551393985748
h1: 0.6033858060836792, h2: 0.6180681586265564
out_h1: 0.6464305520057678, out_h2: 0.6497790813446045
```

```
net_O: -2.832566261291504
out_O: 0.05558951944112778
h1: 0.6043861508369446, h2: 0.6190681457519531
out_h1: 0.6466591358184814, out_h2: 0.6500065922737122
net_O: -2.8373863697052
out_O: 0.05533700808882713
h1: 0.6053780913352966, h2: 0.6200597882270813
out_h1: 0.6468858122825623, out_h2: 0.6502321362495422
net_O: -2.8421616554260254
out_O: 0.05508790910243988
h1: 0.6063617467880249, h2: 0.6210431456565857
out_h1: 0.6471104621887207, out_h2: 0.6504557728767395
net_O: -2.846893310546875
out_O: 0.05484212934970856
h1: 0.6073372960090637, h2: 0.6220183372497559
out_h1: 0.6473331451416016, out_h2: 0.6506774425506592
net_O: -2.8515818119049072
out_O: 0.05459960922598839
h1: 0.6083047986030579, h2: 0.6229856014251709
out_h1: 0.6475540399551392, out_h2: 0.6508972644805908
net_O: -2.8562278747558594
out_O: 0.054360281676054
h1: 0.6092643737792969, h2: 0.623944878578186
out_h1: 0.6477729678153992, out_h2: 0.6511152386665344
net_O: -2.860832452774048
out_O: 0.05412406846880913
h1: 0.6102162599563599, h2: 0.6248964667320251
out_h1: 0.6479901075363159, out_h2: 0.65133136510849
net_O: -2.865396022796631
out_O: 0.05389091372489929
Epoch: 130, loss: 0.0019264124566689134, output: tensor([0.0539],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.6111605167388916, h2: 0.625840425491333
out_h1: 0.6482054591178894, out_h2: 0.6515457034111023
net_O: -2.8699193000793457
out_O: 0.05366075038909912
h1: 0.6120972037315369, h2: 0.6267768144607544
out_h1: 0.6484190821647644, out_h2: 0.6517581939697266
net_O: -2.8744027614593506
out_O: 0.05343352630734444
h1: 0.61302649974823, h2: 0.6277058124542236
out_h1: 0.6486308574676514, out_h2: 0.6519690752029419
net_O: -2.878847599029541
out_O: 0.05320915952324867
h1: 0.6139485239982605, h2: 0.6286275386810303
out_h1: 0.6488409638404846, out_h2: 0.652178168296814
net_O: -2.883253812789917
```

```
out_O: 0.05298762395977974
h1: 0.6148632764816284, h2: 0.6295420527458191
out_h1: 0.6490494012832642, out_h2: 0.6523856520652771
net_O: -2.887622594833374
out_O: 0.05276826484680176
h1: 0.6157709956169128, h2: 0.6304494738578796
out_h1: 0.6492561101913452, out_h2: 0.6525914072990417
net_O: -2.891954183578491
out_O: 0.05255733570337296
h1: 0.6166717410087585, h2: 0.6313499808311462
out_h1: 0.6494612097740173, out_h2: 0.6527955532073975
net_O: -2.896249294281006
out_O: 0.05233928561210632
h1: 0.6175656318664551, h2: 0.6322435140609741
out_h1: 0.6496647000312805, out_h2: 0.6529979705810547
net_O: -2.900508403778076
out_O: 0.05212843418121338
h1: 0.6184526681900024, h2: 0.6331303715705872
out_h1: 0.64986652135849, out_h2: 0.6531989574432373
net_O: -2.9047322273254395
out_O: 0.051920127123594284
h1: 0.6193330883979797, h2: 0.6340104937553406
out_h1: 0.6500668525695801, out_h2: 0.6533982753753662
net_O: -2.908921480178833
out_O: 0.051714301109313965
Epoch: 140, loss: 0.00174008309841156, output: tensor([0.0517],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.6202069520950317, h2: 0.6348840594291687
out_h1: 0.650265634059906, out_h2: 0.6535961031913757
net_O: -2.913076400756836
out_O: 0.051510922610759735
h1: 0.6210742592811584, h2: 0.6357511281967163
out_h1: 0.650462806224823, out_h2: 0.6537923812866211
net_O: -2.9171972274780273
out_O: 0.05130996182560921
h1: 0.6219351887702942, h2: 0.636611819267273
out_h1: 0.6506586074829102, out_h2: 0.6539871096611023
net_O: -2.9212851524353027
out_O: 0.05111134052276611
h1: 0.6227898001670837, h2: 0.6374661326408386
out_h1: 0.6508527398109436, out_h2: 0.6541804075241089
net_O: -2.925340175628662
out_O: 0.050915028899908066
h1: 0.6236382126808167, h2: 0.6383143067359924
out_h1: 0.651045560836792, out_h2: 0.6543723344802856
net_O: -2.929363250732422
out_O: 0.050720974802970886
```

```
h1: 0.6244804859161377, h2: 0.6391562819480896
out_h1: 0.6512368321418762, out_h2: 0.6545627117156982
net_O: -2.933354377746582
out_O: 0.050529152154922485
h1: 0.6253166794776917, h2: 0.6399922370910645
out_h1: 0.6514267921447754, out_h2: 0.654751718044281
net_O: -2.937314033508301
out_O: 0.05033952370285988
h1: 0.6261469721794128, h2: 0.6408222913742065
out_h1: 0.6516153216362, out_h2: 0.6549392938613892
net_O: -2.9412431716918945
out_O: 0.050152018666267395
h1: 0.6269713640213013, h2: 0.6416463851928711
out_h1: 0.6518023610115051, out_h2: 0.6551255583763123
net_O: -2.9451417922973633
out_O: 0.04996662586927414
h1: 0.6277899146080017, h2: 0.6424647569656372
out_h1: 0.65198814868927, out_h2: 0.6553104519844055
net_O: -2.9490108489990234
out_O: 0.04978328198194504
Epoch: 150, loss: 0.0015827097231522202, output: tensor([0.0498],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.6286027431488037, h2: 0.6432772874832153
out_h1: 0.6521726250648499, out_h2: 0.6554939150810242
net_O: -2.952850341796875
out_O: 0.0496019683778286
h1: 0.629409909248352, h2: 0.6440842151641846
out_h1: 0.6523556709289551, out_h2: 0.6556761264801025
net_O: -2.956660270690918
out_O: 0.049422670155763626
h1: 0.6302115321159363, h2: 0.6448855996131897
out_h1: 0.6525374054908752, out_h2: 0.6558570265769958
net_O: -2.960441827774048
out_O: 0.04924531653523445
h1: 0.6310075521469116, h2: 0.6456814408302307
out_h1: 0.6527179479598999, out_h2: 0.6560366153717041
net_O: -2.9641952514648438
out_O: 0.049069877713918686
h1: 0.631798267364502, h2: 0.6464718580245972
out_h1: 0.6528971195220947, out_h2: 0.6562149524688721
net_O: -2.9679207801818848
out_O: 0.04889632761478424
h1: 0.6325836181640625, h2: 0.6472569704055786
out_h1: 0.6530750393867493, out_h2: 0.6563920974731445
net_O: -2.97161865234375
out_O: 0.048724640160799026
h1: 0.6333636045455933, h2: 0.6480367183685303
```

```
out_h1: 0.6532517075538635, out_h2: 0.6565678715705872
net_O: -2.9752895832061768
out_O: 0.04855477437376976
h1: 0.6341384053230286, h2: 0.6488112807273865
out_h1: 0.653427243232727, out_h2: 0.6567425727844238
net_O: -2.978933811187744
out_O: 0.048386696726083755
h1: 0.6349080204963684, h2: 0.6495806574821472
out_h1: 0.6536015272140503, out_h2: 0.6569159626960754
net_O: -2.9825518131256104
out_O: 0.048220377415418625
h1: 0.6356725096702576, h2: 0.650344967842102
out_h1: 0.653774619102478, out_h2: 0.6570882201194763
net_O: -2.9861435890197754
out_O: 0.048055797815322876
Epoch: 160, loss: 0.0014482438564300537, output: tensor([0.0481],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.6364319920539856, h2: 0.651104211807251
out_h1: 0.6539464592933655, out_h2: 0.6572592854499817
net_O: -2.9897098541259766
out_O: 0.047892920672893524
h1: 0.637186586856842, h2: 0.6518585681915283
out_h1: 0.6541171669960022, out_h2: 0.6574291586875916
net_O: -2.993250846862793
out_O: 0.047731708735227585
h1: 0.6379362344741821, h2: 0.6526079773902893
out_h1: 0.6542867422103882, out_h2: 0.6575978994369507
net_O: -2.9967668056488037
out_O: 0.04757215455174446
h1: 0.6386809945106506, h2: 0.6533525586128235
out_h1: 0.6544552445411682, out_h2: 0.6577655673027039
net_O: -3.000258207321167
out_O: 0.04741420969367027
h1: 0.6394209861755371, h2: 0.6540923118591309
out_h1: 0.6546225547790527, out_h2: 0.6579320430755615
net_O: -3.003725051879883
out_O: 0.04725787043571472
h1: 0.6401562690734863, h2: 0.654827356338501
out_h1: 0.6547887921333313, out_h2: 0.658097505569458
net_O: -3.0071682929992676
out_O: 0.04710308089852333
h1: 0.6408869028091431, h2: 0.6555577516555786
out_h1: 0.6549539566040039, out_h2: 0.658261775970459
net_O: -3.010587692260742
out_O: 0.0469498410820961
h1: 0.6416128873825073, h2: 0.6562835574150085
out_h1: 0.6551179885864258, out_h2: 0.6584250926971436
```

```
net_O: -3.013983726501465
out_O: 0.04679811745882034
h1: 0.6423343420028687, h2: 0.6570048332214355
out_h1: 0.6552809476852417, out_h2: 0.6585872769355774
net_O: -3.0173566341400146
out_O: 0.04664789140224457
h1: 0.643051266670227, h2: 0.6577215790748596
out_h1: 0.6554428339004517, out_h2: 0.6587483882904053
net_O: -3.0207066535949707
out_O: 0.04649913311004639
Epoch: 170, loss: 0.0013321868609637022, output: tensor([0.0465],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.6437637209892273, h2: 0.6584337949752808
out_h1: 0.6556037664413452, out_h2: 0.658908486366272
net_O: -3.0240345001220703
out_O: 0.04635180905461311
h1: 0.6444718241691589, h2: 0.6591416597366333
out_h1: 0.6557636260986328, out_h2: 0.6590675711631775
net_O: -3.0273399353027344
out_O: 0.04620591551065445
h1: 0.645175576210022, h2: 0.659845232963562
out_h1: 0.6559224724769592, out_h2: 0.6592256426811218
net_O: -3.030623435974121
out_O: 0.046061426401138306
h1: 0.6458750367164612, h2: 0.6605444550514221
out_h1: 0.6560803055763245, out_h2: 0.6593826413154602
net_O: -3.0338854789733887
out_O: 0.0459183044731617
h1: 0.6465702056884766, h2: 0.6612393856048584
out_h1: 0.6562371253967285, out_h2: 0.659538745880127
net_O: -3.037125825881958
out_O: 0.04577655717730522
h1: 0.6472611427307129, h2: 0.6619302034378052
out_h1: 0.6563929915428162, out_h2: 0.6596938967704773
net_O: -3.0403451919555664
out_O: 0.045636136084795
h1: 0.6479480266571045, h2: 0.6626168489456177
out_h1: 0.6565479636192322, out_h2: 0.6598479747772217
net_O: -3.043543815612793
out_O: 0.045497022569179535
h1: 0.6486307382583618, h2: 0.6632993817329407
out_h1: 0.6567018032073975, out_h2: 0.6600011587142944
net_O: -3.0467214584350586
out_O: 0.04535922780632973
h1: 0.6493094563484192, h2: 0.663977861404419
out_h1: 0.6568548679351807, out_h2: 0.6601533889770508
net_O: -3.0498790740966797
```

```
out_O: 0.0452226959168911
h1: 0.6499841213226318, h2: 0.6646523475646973
out_h1: 0.6570068597793579, out_h2: 0.6603047251701355
net_O: -3.0530166625976562
out_O: 0.04508741572499275
Epoch: 180, loss: 0.0012311266036704183, output: tensor([0.0451],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.6506547927856445, h2: 0.6653228402137756
out_h1: 0.6571580171585083, out_h2: 0.6604551076889038
net_O: -3.0561342239379883
out_O: 0.04495337978005409
h1: 0.6513215899467468, h2: 0.6659894585609436
out_h1: 0.6573082208633423, out_h2: 0.6606045961380005
net_O: -3.059231758117676
out_O: 0.04482058063149452
h1: 0.6519845128059387, h2: 0.6666521430015564
out_h1: 0.6574574708938599, out_h2: 0.660753071308136
net_O: -3.062310218811035
out_O: 0.04468896985054016
h1: 0.6526435613632202, h2: 0.6673110127449036
out_h1: 0.6576059460639954, out_h2: 0.6609008312225342
net_O: -3.0653696060180664
out_O: 0.04455853998661041
h1: 0.6532988548278809, h2: 0.6679661273956299
out_h1: 0.6577535271644592, out_h2: 0.6610475778579712
net_O: -3.0684099197387695
out_O: 0.04442928358912468
h1: 0.6539503931999207, h2: 0.6686174273490906
out_h1: 0.6579000949859619, out_h2: 0.6611934900283813
net_O: -3.0714316368103027
out_O: 0.04430117458105087
h1: 0.6545982360839844, h2: 0.66926509141922
out_h1: 0.6580458879470825, out_h2: 0.6613386273384094
net_O: -3.074434757232666
out_O: 0.0441742017865181
h1: 0.6552423238754272, h2: 0.6699090600013733
out_h1: 0.6581908464431763, out_h2: 0.6614828109741211
net_O: -3.0774195194244385
out_O: 0.04404834657907486
h1: 0.6558828949928284, h2: 0.6705493927001953
out_h1: 0.6583349704742432, out_h2: 0.6616261601448059
net_O: -3.080386161804199
out_O: 0.043923597782850266
h1: 0.6565198302268982, h2: 0.6711861491203308
out_h1: 0.6584782004356384, out_h2: 0.6617687344551086
net_O: -3.0833349227905273
out_O: 0.043799933046102524
```

```
Epoch: 190, loss: 0.0011424353579059243, output: tensor([0.0438],
grad_fn=<SigmoidBackward0>)
============================================================
h1: 0.6571531891822815, h2: 0.6718193292617798
out_h1: 0.6586205959320068, out_h2: 0.6619104146957397
net_O: -3.086266040802002
out_O: 0.04367733374238014
h1: 0.657783031463623, h2: 0.672448992729187
out_h1: 0.6587622165679932, out_h2: 0.6620513200759888
net_O: -3.089179515838623
out_O: 0.04355580359697342
h1: 0.6584094166755676, h2: 0.6730751991271973
out_h1: 0.6589030027389526, out_h2: 0.6621914505958557
net_O: -3.092075824737549
out_O: 0.04343530535697937
h1: 0.65903240442276, h2: 0.6736979484558105
out_h1: 0.65904301404953, out_h2: 0.6623307466506958
net_O: -3.0949552059173584
out_O: 0.0433158278465271
h1: 0.6596519351005554, h2: 0.6743173599243164
out_h1: 0.6591822504997253, out_h2: 0.662469208240509
net_O: -3.0978174209594727
out_O: 0.043197374790906906
h1: 0.6602680683135986, h2: 0.6749333143234253
out_h1: 0.659320592880249, out_h2: 0.662606954574585
net_O: -3.10066294670105
out_O: 0.0430799201130867
h1: 0.6608809232711792, h2: 0.6755459904670715
out_h1: 0.6594582200050354, out_h2: 0.6627439260482788
net_O: -3.103492021560669
out_O: 0.04296344146132469
h1: 0.6614904403686523, h2: 0.6761553287506104
out_h1: 0.6595951318740845, out_h2: 0.6628800630569458
net_O: -3.10630464553833
out_O: 0.04284794256091118
h1: 0.6620966196060181, h2: 0.6767613291740417
out_h1: 0.6597312092781067, out_h2: 0.6630154252052307
net_O: -3.1091010570526123
out_O: 0.04273340106010437
```

Note: at Line 3 above, if we use `Adam` optimizer, it will learn faster in lesser epoch. For example, with Adam, only using 10 epochs, the model will learn the optimum weight

[38]:
```
print(list(model.parameters()))
```

```
[Parameter containing:
tensor([[0.1578, 0.2155],
        [0.2564, 0.3128]], requires_grad=True), Parameter containing:
tensor([0.6333], requires_grad=True), Parameter containing:
```

```
tensor([[-1.0206, -0.9791]], requires_grad=True), Parameter containing:
tensor([-1.7891], requires_grad=True)]
```

1.5 Prediction

[39]:
```
print (model.forward(torch.Tensor([0.06,0.12])))
```

```
h1: 0.6685872077941895, h2: 0.6861847043037415
out_h1: 0.661186695098877, out_h2: 0.6651176810264587
net_O: -3.1151552200317383
out_O: 0.04248642921447754
tensor([0.0425], grad_fn=<SigmoidBackward0>)
```

1.6 Conclusion

This practical successfully implemented and visualized the sigmoid activation function, demonstrating its role in introducing non-linearity. A shallow feed-forward neural network with two hidden units was constructed in PyTorch and trained using mean squared error loss and stochastic gradient descent to map the input [0.05, 0.10] to the target output 0.01. Over 200 epochs, the network reduced the loss from approximately 0.14 to below 0.001, with the output stabilizing near 0.04–0.05, illustrating successful learning despite the known limitations of sigmoid activations and plain SGD (slow convergence due to vanishing gradients). The experiment also highlighted that modern optimizers like Adam achieve comparable results in far fewer epochs. These implementations provide a clear, hands-on understanding of activation functions, loss computation, and gradient-based optimization—the essential primitives required before advancing to deeper architectures such as RNNs or transformers in modern NLP.

[ ]:
```

```

nbconvert_safe_s4adzl1d

March 23, 2026

0.1 by *23AIML010, Om Choksi*

0.2 Task Completed

All tasks in Part A have been successfully completed:

0.2.1 Task A.1: Tensor Creation and Attributes

- Created various types of Tensors using `torch.rand()`, `torch.ones()`, and `torch.zeros()`.
- Inspected the `shape`, `data type (.dtype)`, and `device (.device)` of the created Tensors.

0.2.2 Task A.2: Tensor Operations

- Performed element-wise addition and multiplication on two Tensors.
- Performed matrix multiplication (dot product) using `torch.matmul()` and the `@` operator, ensuring dimension compatibility.
- Reshaped Tensors using `.view()` and `.reshape()` to prepare them for different layers.

1 Word Representation and PyTorch Fundamentals

You are a machine learning consultant advising a company that wants to analyze customer feedback. They need a system that can understand the meaning of words, not just their spelling. You decide to use Word Embeddings—dense, low-dimensional vector representations—which are the first major step toward representation learning in NLP.

Your task is to understand the math behind creating these vectors and practice manipulating the fundamental data structure (the Tensor) used to store them in a deep learning framework like PyTorch.

1.1 Part A: PyTorch Tensors (Foundation for Embeddings)

What is PyTorch? PyTorch is an open source machine learning and deep learning framework.

What can PyTorch be used for? PyTorch allows you to manipulate and process data and write machine learning algorithms using Python code.

Who uses PyTorch? Many of the worlds largest technology companies such as Meta (Facebook), Tesla and Microsoft as well as artificial intelligence research companies such as OpenAI use PyTorch to power research and bring machine learning to their products.

pytorch being used across industry and research

For example, Andrej Karpathy (head of AI at Tesla) has given several talks (PyTorch DevCon 2019, Tesla AI Day 2021) about how Tesla use PyTorch to power their self-driving computer vision models.

PyTorch is also used in other industries such as agriculture to power computer vision on tractors.

Why use PyTorch? Machine learning researchers love using PyTorch. And as of February 2022, PyTorch is the most used deep learning framework on Papers With Code, a website for tracking machine learning research papers and the code repositories attached with them.

PyTorch also helps take care of many things such as GPU acceleration (making your code run faster) behind the scenes.

So you can focus on manipulating data and writing algorithms and PyTorch will make sure it runs fast.

And if companies such as Tesla and Meta (Facebook) use it to build models they deploy to power hundreds of applications, drive thousands of cars and deliver content to billions of people, it's clearly capable on the development front too.

1.1.1 Task A.1: Tensor Creation and Attributes A word vector is stored as a 1D PyTorch Tensor.

1. Use torch.rand(), torch.ones(), and torch.zeros() to create different types of Tensors.
2. Inspect the shape, data type, and device of the Tensors using attributes like .shape, .dtype, and .device.*italicised text*

Importing PyTorch

> **Note:** Before running any of the code in this notebook, you should have gone through the PyTorch setup steps.
>
> However, **if you're running on Google Colab**, everything should work (Google Colab comes with PyTorch and other libraries installed).

Let's start by importing PyTorch and checking the version we're using.

[41]:
```
import torch
torch.__version__
```

[41]:
```
'2.9.0+cu128'
```

Introduction to tensors Now we've got PyTorch imported, it's time to learn about tensors.

Tensors are the fundamental building block of machine learning.

Their job is to represent data in a numerical way.

For example, you could represent an image as a tensor with shape `[3, 224, 224]` which would mean `[colour_channels, height, width]`, as in the image has `3` colour channels (red, green, blue), a height of `224` pixels and a width of `224` pixels.

example of going from an input image to a tensor representation of the image, image gets broken down into 3 colour channels as well as numbers to represent the height and width

In tensor-speak (the language used to describe tensors), the tensor would have three dimensions, one for `colour_channels`, `height` and `width`.

But we're getting ahead of ourselves.

Let's learn more about tensors by coding them.

Creating tensors PyTorch loves tensors. So much so there's a whole documentation page dedicated to the `torch.Tensor` class.

Your first piece of homework is to read through the documentation on `torch.Tensor` for 10-minutes. But you can get to that later.

Let's code.

The first thing we're going to create is a **scalar**.

A scalar is a single number and in tensor-speak it's a zero dimension tensor.

[42]:
```
# Scalar
scalar = torch.tensor(7)
scalar
```

[42]:
```
tensor(7)
```

See how the above printed out `tensor(7)`?

That means although `scalar` is a single number, it's of type `torch.Tensor`.

We can check the dimensions of a tensor using the `ndim` attribute.

[43]:
```
print(scalar.ndim)

# Get the Python number within a tensor (only works with one-element tensors)
print(scalar.item())
```

```
0
7
```

Okay, now let's see a **vector**.

A vector is a single dimension tensor but can contain many numbers.

As in, you could have a vector `[3, 2]` to describe `[bedrooms, bathrooms]` in your house. Or you could have `[3, 2, 2]` to describe `[bedrooms, bathrooms, car_parks]` in your house.

The important trend here is that a vector is flexible in what it can represent (the same with tensors).

[44]:
```
# Vector
vector = torch.tensor([7, 7])
vector
```

[44]:
```
tensor([7, 7])
```

[45]:
```
# Check the number of dimensions of vector
vector.ndim
```

[45]:
```
1
```

[46]:
```
# Check shape of vector
vector.shape
```

[46]:
```
torch.Size([2])
```

Hmm, that's strange, `vector` contains two numbers but only has a single dimension.

I'll let you in on a trick.

You can tell the number of dimensions a tensor in PyTorch has by the number of square brackets on the outside (`[`) and you only need to count one side.

How many square brackets does `vector` have?

Another important concept for tensors is their `shape` attribute. The shape tells you how the elements inside them are arranged.

Let's check out the shape of `vector`.

[47]:
```
dir(vector)
```

[47]:
```
['H',
 'T',
 '__abs__',
 '__add__',
 '__and__',
 '__annotations__',
 '__array__',
 '__array_priority__',
 '__array_wrap__',
 '__bool__',
 '__class__',
 '__complex__',
 '__contains__',
 '__deepcopy__',
 '__delattr__',
 '__delitem__',
 '__dict__',
 '__dir__',
 '__div__',
 '__dlpack__',
 '__dlpack_device__',
 '__doc__',
 '__eq__',
```

```
'__float__',
'__floordiv__',
'__format__',
'__ge__',
'__getattribute__',
'__getitem__',
'__getstate__',
'__gt__',
'__hash__',
'__iadd__',
'__iand__',
'__idiv__',
'__ifloordiv__',
'__ilshift__',
'__imod__',
'__imul__',
'__index__',
'__init__',
'__init_subclass__',
'__int__',
'__invert__',
'__ior__',
'__ipow__',
'__irshift__',
'__isub__',
'__iter__',
'__itruediv__',
'__ixor__',
'__le__',
'__len__',
'__long__',
'__lshift__',
'__lt__',
'__matmul__',
'__mod__',
'__module__',
'__mul__',
'__ne__',
'__neg__',
'__new__',
'__nonzero__',
'__or__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdiv__',
```

```
'__reduce__',
'__reduce_ex__',
'__repr__',
'__reversed__',
'__rfloordiv__',
'__rlshift__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__rpow__',
'__rrshift__',
'__rshift__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__str__',
'__sub__',
'__subclasshook__',
'__torch_dispatch__',
'__torch_function__',
'__truediv__',
'__weakref__',
'__xor__',
'_addmm_activation',
'_autocast_to_full_precision',
'_autocast_to_reduced_precision',
'_backward_hooks',
'_base',
'_cdata',
'_clear_non_serializable_cached_data',
'_coalesced_',
'_conj',
'_conj_physical',
'_dimI',
'_dimV',
'_fix_weakref',
'_grad',
'_grad_fn',
'_has_symbolic_sizes_strides',
'_indices',
'_is_all_true',
'_is_any_true',
```

```
'_is_view',
'_is_zerotensor',
'_lazy_clone',
'_make_dtensor',
'_make_subclass',
'_make_wrapper_subclass',
'_neg_view',
'_nested_tensor_size',
'_nested_tensor_storage_offsets',
'_nested_tensor_strides',
'_nnz',
'_post_accumulate_grad_hooks',
'_python_dispatch',
'_reduce_ex_internal',
'_rev_view_func_unsafe',
'_sparse_mask_projection',
'_to_dense',
'_to_sparse',
'_to_sparse_bsc',
'_to_sparse_bsr',
'_to_sparse_csc',
'_to_sparse_csr',
'_typed_storage',
'_update_names',
'_use_count',
'_values',
'_version',
'_view_func',
'_view_func_unsafe',
'abs',
'abs_',
'absolute',
'absolute_',
'acos',
'acos_',
'acosh',
'acosh_',
'add',
'add_',
'addbmm',
'addbmm_',
'addcdiv',
'addcdiv_',
'addcmul',
'addcmul_',
'addmm',
'addmm_',
```

```
'addmv',
'addmv_',
'addr',
'addr_',
'adjoint',
'align_as',
'align_to',
'all',
'allclose',
'amax',
'amin',
'aminmax',
'angle',
'any',
'apply_',
'arccos',
'arccos_',
'arccosh',
'arccosh_',
'arcsin',
'arcsin_',
'arcsinh',
'arcsinh_',
'arctan',
'arctan2',
'arctan2_',
'arctan_',
'arctanh',
'arctanh_',
'argmax',
'argmin',
'argsort',
'argwhere',
'as_strided',
'as_strided_',
'as_strided_scatter',
'as_subclass',
'asin',
'asin_',
'asinh',
'asinh_',
'atan',
'atan2',
'atan2_',
'atan_',
'atanh',
'atanh_',
```

```
'backward',
'baddbmm',
'baddbmm_',
'bernoulli',
'bernoulli_',
'bfloat16',
'bincount',
'bitwise_and',
'bitwise_and_',
'bitwise_left_shift',
'bitwise_left_shift_',
'bitwise_not',
'bitwise_not_',
'bitwise_or',
'bitwise_or_',
'bitwise_right_shift',
'bitwise_right_shift_',
'bitwise_xor',
'bitwise_xor_',
'bmm',
'bool',
'broadcast_to',
'byte',
'cauchy_',
'ccol_indices',
'cdouble',
'ceil',
'ceil_',
'cfloat',
'chalf',
'char',
'cholesky',
'cholesky_inverse',
'cholesky_solve',
'chunk',
'clamp',
'clamp_',
'clamp_max',
'clamp_max_',
'clamp_min',
'clamp_min_',
'clip',
'clip_',
'clone',
'coalesce',
'col_indices',
'conj',
```

```
'conj_physical',
'conj_physical_',
'contiguous',
'copy_',
'copysign',
'copysign_',
'corrcoef',
'cos',
'cos_',
'cosh',
'cosh_',
'count_nonzero',
'cov',
'cpu',
'cross',
'crow_indices',
'cuda',
'cummax',
'cummin',
'cumprod',
'cumprod_',
'cumsum',
'cumsum_',
'data',
'data_ptr',
'deg2rad',
'deg2rad_',
'dense_dim',
'dequantize',
'det',
'detach',
'detach_',
'device',
'diag',
'diag_embed',
'diagflat',
'diagonal',
'diagonal_scatter',
'diff',
'digamma',
'digamma_',
'dim',
'dim_order',
'dist',
'div',
'div_',
'divide',
```

```
'divide_',
'dot',
'double',
'dsplit',
'dtype',
'eig',
'element_size',
'eq',
'eq_',
'equal',
'erf',
'erf_',
'erfc',
'erfc_',
'erfinv',
'erfinv_',
'exp',
'exp2',
'exp2_',
'exp_',
'expand',
'expand_as',
'expm1',
'expm1_',
'exponential_',
'fill_',
'fill_diagonal_',
'fix',
'fix_',
'flatten',
'flip',
'fliplr',
'flipud',
'float',
'float_power',
'float_power_',
'floor',
'floor_',
'floor_divide',
'floor_divide_',
'fmax',
'fmin',
'fmod',
'fmod_',
'frac',
'frac_',
'frexp',
```

```
'gather',
'gcd',
'gcd_',
'ge',
'ge_',
'geometric_',
'geqrf',
'ger',
'get_device',
'grad',
'grad_fn',
'greater',
'greater_',
'greater_equal',
'greater_equal_',
'gt',
'gt_',
'half',
'hardshrink',
'has_names',
'hash_tensor',
'heaviside',
'heaviside_',
'histc',
'histogram',
'hsplit',
'hypot',
'hypot_',
'i0',
'i0_',
'igamma',
'igamma_',
'igammac',
'igammac_',
'imag',
'index_add',
'index_add_',
'index_copy',
'index_copy_',
'index_fill',
'index_fill_',
'index_put',
'index_put_',
'index_reduce',
'index_reduce_',
'index_select',
'indices',
```

```
'inner',
'int',
'int_repr',
'inverse',
'ipu',
'is_coalesced',
'is_complex',
'is_conj',
'is_contiguous',
'is_cpu',
'is_cuda',
'is_distributed',
'is_floating_point',
'is_inference',
'is_ipu',
'is_leaf',
'is_maia',
'is_meta',
'is_mkldnn',
'is_mps',
'is_mtia',
'is_neg',
'is_nested',
'is_nonzero',
'is_pinned',
'is_quantized',
'is_same_size',
'is_set_to',
'is_shared',
'is_signed',
'is_sparse',
'is_sparse_csr',
'is_vulkan',
'is_xla',
'is_xpu',
'isclose',
'isfinite',
'isinf',
'isnan',
'isneginf',
'isposinf',
'isreal',
'istft',
'item',
'itemsize',
'kron',
'kthvalue',
```

```
'layout',
'lcm',
'lcm_',
'ldexp',
'ldexp_',
'le',
'le_',
'lerp',
'lerp_',
'less',
'less_',
'less_equal',
'less_equal_',
'lgamma',
'lgamma_',
'log',
'log10',
'log10_',
'log1p',
'log1p_',
'log2',
'log2_',
'log_',
'log_normal_',
'log_softmax',
'logaddexp',
'logaddexp2',
'logcumsumexp',
'logdet',
'logical_and',
'logical_and_',
'logical_not',
'logical_not_',
'logical_or',
'logical_or_',
'logical_xor',
'logical_xor_',
'logit',
'logit_',
'logsumexp',
'long',
'lstsq',
'lt',
'lt_',
'lu',
'lu_solve',
'mH',
```

```
'mT',
'map2_',
'map_',
'masked_fill',
'masked_fill_',
'masked_scatter',
'masked_scatter_',
'masked_select',
'matmul',
'matrix_exp',
'matrix_power',
'max',
'maximum',
'mean',
'median',
'min',
'minimum',
'mm',
'mode',
'module_load',
'moveaxis',
'movedim',
'msort',
'mtia',
'mul',
'mul_',
'multinomial',
'multiply',
'multiply_',
'mv',
'mvlgamma',
'mvlgamma_',
'name',
'names',
'nan_to_num',
'nan_to_num_',
'nanmean',
'nanmedian',
'nanquantile',
'nansum',
'narrow',
'narrow_copy',
'nbytes',
'ndim',
'ndimension',
'ne',
'ne_',
```

```
'neg',
'neg_',
'negative',
'negative_',
'nelement',
'new',
'new_empty',
'new_empty_strided',
'new_full',
'new_ones',
'new_tensor',
'new_zeros',
'nextafter',
'nextafter_',
'nonzero',
'nonzero_static',
'norm',
'normal_',
'not_equal',
'not_equal_',
'numel',
'numpy',
'orgqr',
'ormqr',
'outer',
'output_nr',
'permute',
'pin_memory',
'pinverse',
'polygamma',
'polygamma_',
'positive',
'pow',
'pow_',
'prelu',
'prod',
'put',
'put_',
'q_per_channel_axis',
'q_per_channel_scales',
'q_per_channel_zero_points',
'q_scale',
'q_zero_point',
'qr',
'qscheme',
'quantile',
'rad2deg',
```

```
'rad2deg_',
'random_',
'ravel',
'real',
'reciprocal',
'reciprocal_',
'record_stream',
'refine_names',
'register_hook',
'register_post_accumulate_grad_hook',
'reinforce',
'relu',
'relu_',
'remainder',
'remainder_',
'rename',
'rename_',
'renorm',
'renorm_',
'repeat',
'repeat_interleave',
'requires_grad',
'requires_grad_',
'reshape',
'reshape_as',
'resize',
'resize_',
'resize_as',
'resize_as_',
'resize_as_sparse_',
'resolve_conj',
'resolve_neg',
'retain_grad',
'retains_grad',
'roll',
'rot90',
'round',
'round_',
'row_indices',
'rsqrt',
'rsqrt_',
'scatter',
'scatter_',
'scatter_add',
'scatter_add_',
'scatter_reduce',
'scatter_reduce_',
```

```
'select',
'select_scatter',
'set_',
'sgn',
'sgn_',
'shape',
'share_memory_',
'short',
'sigmoid',
'sigmoid_',
'sign',
'sign_',
'signbit',
'sin',
'sin_',
'sinc',
'sinc_',
'sinh',
'sinh_',
'size',
'slice_inverse',
'slice_scatter',
'slogdet',
'smm',
'softmax',
'solve',
'sort',
'sparse_dim',
'sparse_mask',
'sparse_resize_',
'sparse_resize_and_clear_',
'split',
'split_with_sizes',
'sqrt',
'sqrt_',
'square',
'square_',
'squeeze',
'squeeze_',
'sspaddmm',
'std',
'stft',
'storage',
'storage_offset',
'storage_type',
'stride',
'sub',
```

```
'sub_',
'subtract',
'subtract_',
'sum',
'sum_to_size',
'svd',
'swapaxes',
'swapaxes_',
'swapdims',
'swapdims_',
'symeig',
't',
't_',
'take',
'take_along_dim',
'tan',
'tan_',
'tanh',
'tanh_',
'tensor_split',
'tile',
'to',
'to_dense',
'to_mkldnn',
'to_padded_tensor',
'to_sparse',
'to_sparse_bsc',
'to_sparse_bsr',
'to_sparse_coo',
'to_sparse_csc',
'to_sparse_csr',
'tolist',
'topk',
'trace',
'transpose',
'transpose_',
'triangular_solve',
'tril',
'tril_',
'triu',
'triu_',
'true_divide',
'true_divide_',
'trunc',
'trunc_',
'type',
'type_as',
```

```
 'unbind',
 'unflatten',
 'unfold',
 'uniform_',
 'unique',
 'unique_consecutive',
 'unsafe_chunk',
 'unsafe_split',
 'unsafe_split_with_sizes',
 'unsqueeze',
 'unsqueeze_',
 'untyped_storage',
 'values',
 'var',
 'vdot',
 'view',
 'view_as',
 'vsplit',
 'where',
 'xlogy',
 'xlogy_',
 'xpu',
 'zero_']
```

The above returns `torch.Size([2])` which means our vector has a shape of `[2]`. This is because of the two elements we placed inside the square brackets (`[7, 7]`).

Let's now see a **matrix**.

[48]:
```
# Matrix
MATRIX = torch.tensor([[7, 8],
                       [9, 10]])
MATRIX
```

[48]:
```
tensor([[ 7,  8],
        [ 9, 10]])
```

[49]:
```
# Check number of dimensions
MATRIX.ndim
```

[49]:
```
2
```

Wow! More numbers! Matrices are as flexible as vectors, except they've got an extra dimension.

`MATRIX` has two dimensions (did you count the number of square brakcets on the outside of one side?).

What `shape` do you think it will have?

[50]:
```
MATRIX.shape
```

[50]:
```
torch.Size([2, 2])
```

We get the output `torch.Size([2, 2])` because `MATRIX` is two elements deep and two elements wide.

How about we create a **tensor**?

[51]:
```
# Tensor
TENSOR = torch.tensor([[[1, 2, 3],
                        [3, 6, 9],
                        [2, 4, 5]]])
TENSOR
```

[51]:
```
tensor([[[1, 2, 3],
         [3, 6, 9],
         [2, 4, 5]]])
```

[52]:
```
# Check number of dimensions for TENSOR
TENSOR.ndim
```

[52]:
```
3
```

And what about its shape?

[53]:
```
# Check shape of TENSOR
TENSOR.shape
```

[53]:
```
torch.Size([1, 3, 3])
```

Alright, it outputs `torch.Size([1, 3, 3])`.

The dimensions go outer to inner.

That means there's 1 dimension of 3 by 3.

example of different tensor dimensions

> **Note:** You might've noticed me using lowercase letters for `scalar` and `vector` and uppercase letters for `MATRIX` and `TENSOR`. This was on purpose. In practice, you'll often see scalars and vectors denoted as lowercase letters such as `y` or `a`. And matrices and tensors denoted as uppercase letters such as `X` or `W`.
>
> You also might notice the names martrix and tensor used interchangably. This is common. Since in PyTorch you're often dealing with `torch.Tensors` (hence the tensor name), however, the shape and dimensions of what's inside will dictate what it actually is.

Let's summarise.

Name	What is it?	Number of dimensions	Lower or upper (usually/example)
scalar	a single number	0	Lower (`a`)

Name	What is it?	Number of dimensions	Lower or upper (usually/example)
vector	a number with direction (e.g. wind speed with direction) but can also have many other numbers	1	Lower (`y`)
matrix	a 2-dimensional array of numbers	2	Upper (`Q`)
tensor	an n-dimensional array of numbers	can be any number, a 0-dimension tensor is a scalar, a 1-dimension tensor is a vector	Upper (`X`)

scalar vector matrix tensor and what they look like

Random tensors We've established tensors represent some form of data.

And machine learning models such as neural networks manipulate and seek patterns within tensors.

But when building machine learning models with PyTorch, it's rare you'll create tensors by hand (like what we've being doing).

Instead, a machine learning model often starts out with large random tensors of numbers and adjusts these random numbers as it works through data to better represent it.

In essence:

`Start with random numbers -> look at data -> update random numbers -> look at data -> update random numbers...`

As a data scientist, you can define how the machine learning model starts (initialization), looks at data (representation) and updates (optimization) its random numbers.

We'll get hands on with these steps later on.

For now, let's see how to create a tensor of random numbers.

We can do so using `torch.rand()` and passing in the `size` parameter.

[54]:
```
# Create a random tensor of size (3, 4)
random_tensor = torch.rand(size=(3, 4))
random_tensor, random_tensor.dtype
```

[54]:
```
(tensor([[0.9951, 0.5405, 0.2217, 0.5984],
         [0.4793, 0.6887, 0.6387, 0.5783],
         [0.1570, 0.9856, 0.1994, 0.5012]]),
 torch.float32)
```

The flexibility of `torch.rand()` is that we can adjust the `size` to be whatever we want.

For example, say you wanted a random tensor in the common image shape of `[224, 224, 3]` (`[height, width, color_channels]`).

[55]:
```
# Create a random tensor of size (224, 224, 3)
random_image_size_tensor = torch.rand(size=(224, 224, 3))
random_image_size_tensor.shape, random_image_size_tensor.ndim
```

[55]:
```
(torch.Size([224, 224, 3]), 3)
```

Zeros and ones Sometimes you'll just want to fill tensors with zeros or ones.

This happens a lot with masking (like masking some of the values in one tensor with zeros to let a model know not to learn them).

Let's create a tensor full of zeros with `torch.zeros()`

Again, the `size` parameter comes into play.

[56]:
```
# Create a tensor of all zeros
zeros = torch.zeros(size=(3, 4))
zeros, zeros.dtype
```

[56]:
```
(tensor([[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]]),
 torch.float32)
```

We can do the same to create a tensor of all ones except using `torch.ones()` instead.

[57]:
```
# Create a tensor of all ones
ones = torch.ones(size=(3, 4))
ones, ones.dtype
```

[57]:
```
(tensor([[1., 1., 1., 1.],
         [1., 1., 1., 1.],
         [1., 1., 1., 1.]]),
 torch.float32)
```

Creating a range and tensors like Sometimes you might want a range of numbers, such as 1 to 10 or 0 to 100.

You can use `torch.arange(start, end, step)` to do so.

Where: * `start` = start of range (e.g. 0) * `end` = end of range (e.g. 10) * `step` = how many steps in between each value (e.g. 1)

> **Note:** In Python, you can use `range()` to create a range. However in PyTorch, `torch.range()` is deprecated and may show an error in the future.

[58]:
```
# Use torch.arange(), torch.range() is deprecated
```

```
zero_to_ten_deprecated = torch.range(0, 10) # Note: this may return an error in␣
 ↪the future

# Create a range of values 0 to 10
zero_to_ten = torch.arange(start=0, end=10, step=1)
zero_to_ten
```

```
/tmp/ipython-input-1814108326.py:2: UserWarning: torch.range is deprecated and
will be removed in a future release because its behavior is inconsistent with
Python's range builtin. Instead, use torch.arange, which produces values in
[start, end).
  zero_to_ten_deprecated = torch.range(0, 10) # Note: this may return an error
in the future
```

[58]:
```
tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Sometimes you might want one tensor of a certain type with the same shape as another tensor.

For example, a tensor of all zeros with the same shape as a previous tensor.

To do so you can use `torch.zeros_like(input)` or `torch.ones_like(input)` which return a tensor filled with zeros or ones in the same shape as the `input` respectively.

[59]:
```
# Can also create a tensor of zeros similar to another tensor
ten_zeros = torch.zeros_like(input=zero_to_ten) # will have same shape
ten_zeros
```

[59]:
```
tensor([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

Tensor datatypes There are many different tensor datatypes available in PyTorch.

> **Note:** An integer is a flat round number like `7` whereas a float has a decimal `7.0`.

The reason for all of these is to do with **precision in computing**.

The higher the precision value (8, 16, 32), the more detail and hence data used to express a number.

This matters in deep learning and numerical computing because you're making so many operations, the more detail you have to calculate on, the more compute you have to use.

So lower precision datatypes are generally faster to compute on but sacrifice some performance on evaluation metrics like accuracy (faster to compute but less accurate).

> **Resources:** * See the PyTorch documentation for a list of all available tensor datatypes.
> * Read the Wikipedia page for an overview of what precision in computing is.

Let's see how to create some tensors with specific datatypes. We can do so using the `dtype` parameter.

[60]:
```
# Default datatype for tensors is float32
float_32_tensor = torch.tensor([3.0, 6.0, 9.0],
                               dtype=None, # defaults to None, which is torch.
 ↪float32 or whatever datatype is passed
```

```
                                device=None, # defaults to None, which uses the␣
↪default tensor type
                                requires_grad=False) # if True, operations␣
↪performed on the tensor are recorded

float_32_tensor.shape, float_32_tensor.dtype, float_32_tensor.device
```

[60]:
```
(torch.Size([3]), torch.float32, device(type='cpu'))
```

Aside from shape issues (tensor shapes don't match up), two of the other most common issues you'll come across in PyTorch are datatype and device issues.

For example, one of tensors is `torch.float32` and the other is `torch.float16` (PyTorch often likes tensors to be the same format).

Or one of your tensors is on the CPU and the other is on the GPU (PyTorch likes calculations between tensors to be on the same device).

We'll see more of this device talk later on.

For now let's create a tensor with `dtype=torch.float16`.

[61]:
```
float_16_tensor = torch.tensor([3.0, 6.0, 9.0],
                               dtype=torch.float16) # torch.half would also work

float_16_tensor.dtype
```

[61]:
```
torch.float16
```

Getting information from tensors Once you've created tensors (or someone else or a PyTorch module has created them for you), you might want to get some information from them.

We've seen these before but three of the most common attributes you'll want to find out about tensors are: * `shape` - what shape is the tensor? (some operations require specific shape rules) * `dtype` - what datatype are the elements within the tensor stored in? * `device` - what device is the tensor stored on? (usually GPU or CPU)

Let's create a random tensor and find out details about it.

[62]:
```
# Create a tensor
some_tensor = torch.rand(3, 4)

# Find out details about it
print(some_tensor)
print(f"Shape of tensor: {some_tensor.shape}")
print(f"Datatype of tensor: {some_tensor.dtype}")
print(f"Device tensor is stored on: {some_tensor.device}") # will default to CPU
```

```
tensor([[0.2043, 0.6139, 0.8645, 0.7715],
        [0.0403, 0.7684, 0.2442, 0.3778],
        [0.3292, 0.9152, 0.6135, 0.3969]])
```

```
Shape of tensor: torch.Size([3, 4])
Datatype of tensor: torch.float32
Device tensor is stored on: cpu
```

Task:

1. Use torch.rand(), torch.ones(), and torch.zeros() to create different types of Tensors.
2. Inspect the shape, data type, and device of the Tensors using attributes like .shape, .dtype, and .device.

[63]:
```python
# Answer 1

tensor_random = torch.rand(4, 9)
tensor_ones = torch.ones(4, 9)
tensor_zeroes = torch.zeros(4, 9)
```

[64]:
```python
# Answer 2
print("Tensor with random values: ")
print("Shape:", tensor_random.shape)
print("Data type:", tensor_random.dtype)
print("Device:", tensor_random.device)

print("Tensor with ones: ")
print("Shape:", tensor_ones.shape)
print("Data type:", tensor_ones.dtype)
print("Device:", tensor_ones.device)


print("Tensor with zeroes: ")
print("Shape:", tensor_zeroes.shape)
print("Data type:", tensor_zeroes.dtype)
print("Device:", tensor_zeroes.device)
```

```
Tensor with random values:
Shape: torch.Size([4, 9])
Data type: torch.float32
Device: cpu
Tensor with ones:
Shape: torch.Size([4, 9])
Data type: torch.float32
Device: cpu
Tensor with zeroes:
Shape: torch.Size([4, 9])
Data type: torch.float32
Device: cpu
```

1.1.2 Task A.2: Tensor Operations Vectors must be combined (added, multiplied) within the neural network.

1. Perform element-wise addition and multiplication on two Tensors.
2. Perform a matrix multiplication (dot product) using torch.matmul() or the @ operator, which is the core operation in the Word2Vec inner product calculation. Ensure the dimensions are compatible.
3. Reshape a Tensor using .view() or .reshape() to prepare it for different layers.

Manipulating tensors (tensor operations) In deep learning, data (images, text, video, audio, protein structures, etc) gets represented as tensors.

A model learns by investigating those tensors and performing a series of operations (could be 1,000,000s+) on tensors to create a representation of the patterns in the input data.

These operations are often a wonderful dance between: * Addition * Substraction * Multiplication (element-wise) * Division * Matrix multiplication

And that's it. Sure there are a few more here and there but these are the basic building blocks of neural networks.

Stacking these building blocks in the right way, you can create the most sophisticated of neural networks (just like lego!).

Basic operations Let's start with a few of the fundamental operations, addition (`+`), subtraction (`-`), mutliplication (`*`).

They work just as you think they would.

[65]:
```
# Create a tensor of values and add a number to it
tensor = torch.tensor([1, 2, 3])
tensor + 10
```

[65]:
```
tensor([11, 12, 13])
```

[66]:
```
# Multiply it by 10
tensor * 10
```

[66]:
```
tensor([10, 20, 30])
```

Notice how the tensor values above didn't end up being `tensor([110, 120, 130])`, this is because the values inside the tensor don't change unless they're reassigned.

[67]:
```
# Tensors don't change unless reassigned
tensor
```

[67]:
```
tensor([1, 2, 3])
```

Let's subtract a number and this time we'll reassign the `tensor` variable.

[68]:
```
# Subtract and reassign
tensor = tensor - 10
```

```
tensor
```

[68]:
```
tensor([-9, -8, -7])
```

[69]:
```
# Add and reassign
tensor = tensor + 10
tensor
```

[69]:
```
tensor([1, 2, 3])
```

PyTorch also has a bunch of built-in functions like `torch.mul()` (short for multiplication) and `torch.add()` to perform basic operations.

[70]:
```
# Can also use torch functions
torch.multiply(tensor, 10)
```

[70]:
```
tensor([10, 20, 30])
```

[71]:
```
# Original tensor is still unchanged
tensor
```

[71]:
```
tensor([1, 2, 3])
```

However, it's more common to use the operator symbols like `*` instead of `torch.mul()`

[72]:
```
# Element-wise multiplication (each element multiplies its equivalent, index
↪0->0, 1->1, 2->2)
print(tensor, "*", tensor)
print("Equals:", tensor * tensor)
```

```
tensor([1, 2, 3]) * tensor([1, 2, 3])
Equals: tensor([1, 4, 9])
```

Matrix multiplication (is all you need) One of the most common operations in machine learning and deep learning algorithms (like neural networks) is matrix multiplication.

PyTorch implements matrix multiplication functionality in the `torch.matmul()` method.

The main two rules for matrix multiplication to remember are:

1. The **inner dimensions** must match:

- `(3, 2) @ (3, 2)` won't work
- `(2, 3) @ (3, 2)` will work
- `(3, 2) @ (2, 3)` will work

2. The resulting matrix has the shape of the **outer dimensions**:

- `(2, 3) @ (3, 2)` -> `(2, 2)`
- `(3, 2) @ (2, 3)` -> `(3, 3)`

Note: "`@`" in Python is the symbol for matrix multiplication.

Resource: You can see all of the rules for matrix multiplication using `torch.matmul()` in the PyTorch documentation.

Let's create a tensor and perform element-wise multiplication and matrix multiplication on it.

[73]:
```
import torch
tensor = torch.tensor([1, 2, 3])
tensor.shape
```

[73]:
```
torch.Size([3])
```

The difference between element-wise multiplication and matrix multiplication is the addition of values.

For our `tensor` variable with values `[1, 2, 3]`:

Operation	Calculation	Code
Element-wise multiplication	`[1*1, 2*2, 3*3] = [1, 4, 9]`	`tensor * tensor`
Matrix multiplication	`[1*1 + 2*2 + 3*3] = [14]`	`tensor.matmul(tensor)`

[74]:
```
# Element-wise matrix multiplication
tensor * tensor
```

[74]:
```
tensor([1, 4, 9])
```

[75]:
```
# Matrix multiplication
torch.matmul(tensor, tensor)
```

[75]:
```
tensor(14)
```

[76]:
```
# Can also use the "@" symbol for matrix multiplication, though not recommended
tensor @ tensor
```

[76]:
```
tensor(14)
```

You can do matrix multiplication by hand but it's not recommended.

The in-built `torch.matmul()` method is faster.

[77]:
```
%%timeit
# Matrix multiplication by hand
# (avoid doing operations with for loops at all cost, they are computationally␣
 ↪expensive)
tensor = torch.rand(50,50)
result = result = [[0 for _ in range(len(tensor[0]))] for _ in␣
 ↪range(len(tensor))]
for i in range(len(tensor)):
    for j in range(len(tensor[0])):
        for k in range(len(tensor)):
```

```
            result[i][j] += tensor[i][k] * tensor[k][j]

# # Print the result
for row in result:
    print(row)
```

```
[tensor(13.9497), tensor(11.1178), tensor(14.3484), tensor(9.4563),
tensor(12.9377), tensor(11.6692), tensor(12.2550), tensor(12.4381),
tensor(12.1457), tensor(13.2174), tensor(12.0550), tensor(12.9620),
tensor(11.8928), tensor(13.1701), tensor(11.9093), tensor(13.6821),
tensor(12.2809), tensor(11.4890), tensor(11.9960), tensor(11.0292),
tensor(11.1538), tensor(12.2933), tensor(12.5639), tensor(12.8835),
tensor(13.8195), tensor(12.4161), tensor(11.8565), tensor(11.7900),
tensor(12.0658), tensor(11.4076), tensor(12.5279), tensor(10.6919),
tensor(13.1320), tensor(12.2599), tensor(13.0410), tensor(11.6648),
tensor(13.9316), tensor(12.9138), tensor(12.3251), tensor(9.4649),
tensor(10.5430), tensor(12.1954), tensor(11.4879), tensor(10.0673),
tensor(12.4393), tensor(14.7555), tensor(12.5150), tensor(11.7716),
tensor(12.9110), tensor(13.8532)]
[tensor(14.7226), tensor(12.2129), tensor(14.5397), tensor(9.3210),
tensor(13.9581), tensor(11.7053), tensor(11.2700), tensor(14.3664),
tensor(12.4021), tensor(12.7774), tensor(12.1873), tensor(13.5367),
tensor(12.2823), tensor(12.1623), tensor(12.7138), tensor(14.7067),
tensor(14.6881), tensor(12.4712), tensor(12.6068), tensor(11.8468),
tensor(11.3515), tensor(12.8113), tensor(12.6193), tensor(13.0639),
tensor(13.6106), tensor(11.9590), tensor(12.2121), tensor(13.4405),
tensor(11.9443), tensor(12.7315), tensor(12.8399), tensor(11.4389),
tensor(14.0725), tensor(12.3585), tensor(14.3074), tensor(12.5882),
tensor(13.6032), tensor(13.8899), tensor(14.3826), tensor(10.7428),
tensor(10.7561), tensor(12.9509), tensor(12.0794), tensor(10.3367),
tensor(12.7491), tensor(14.8980), tensor(14.0591), tensor(12.9953),
tensor(12.9463), tensor(14.2855)]
[tensor(14.8154), tensor(12.9840), tensor(15.0624), tensor(10.1745),
tensor(12.7195), tensor(11.0694), tensor(12.4953), tensor(12.8136),
tensor(12.2046), tensor(13.0344), tensor(12.5648), tensor(13.8140),
tensor(11.9521), tensor(12.0274), tensor(13.0677), tensor(14.1716),
tensor(12.3643), tensor(11.1724), tensor(12.9420), tensor(12.0240),
tensor(11.4777), tensor(11.7428), tensor(14.0201), tensor(11.8982),
tensor(12.7630), tensor(12.1544), tensor(12.2149), tensor(13.1990),
tensor(11.7296), tensor(13.1174), tensor(12.2380), tensor(11.2836),
tensor(13.3739), tensor(12.1655), tensor(13.2811), tensor(12.6092),
tensor(13.9955), tensor(12.5869), tensor(13.9426), tensor(10.2536),
tensor(10.9495), tensor(12.8736), tensor(12.4061), tensor(9.9514),
tensor(12.5620), tensor(13.9725), tensor(13.9336), tensor(12.9777),
tensor(13.3308), tensor(14.1032)]
[tensor(15.5640), tensor(14.2757), tensor(17.1851), tensor(12.6654),
tensor(14.0975), tensor(13.5230), tensor(13.0612), tensor(14.5666),
tensor(14.7479), tensor(15.4126), tensor(15.3599), tensor(15.6618),
```

```
tensor(14.5919), tensor(14.9285), tensor(13.1910), tensor(15.8906),
tensor(15.9329), tensor(12.2000), tensor(12.7168), tensor(14.3246),
tensor(13.2728), tensor(14.6407), tensor(15.9167), tensor(14.4095),
tensor(15.0417), tensor(15.4952), tensor(14.5283), tensor(14.8646),
tensor(15.8996), tensor(15.2223), tensor(13.8994), tensor(12.4877),
tensor(16.6056), tensor(15.2424), tensor(15.6155), tensor(15.2330),
tensor(15.5580), tensor(14.5255), tensor(14.6853), tensor(12.8368),
tensor(12.6602), tensor(14.5471), tensor(13.8223), tensor(12.5887),
tensor(14.3412), tensor(16.5486), tensor(16.0053), tensor(14.6225),
tensor(15.4474), tensor(15.6252)]
[tensor(15.0934), tensor(12.4973), tensor(16.4773), tensor(10.4317),
tensor(14.4687), tensor(11.9688), tensor(11.7740), tensor(13.3030),
tensor(14.0138), tensor(14.9575), tensor(13.3357), tensor(13.8667),
tensor(12.2868), tensor(13.6648), tensor(11.8624), tensor(15.0622),
tensor(14.3177), tensor(11.9512), tensor(12.9282), tensor(12.8344),
tensor(11.6700), tensor(13.4074), tensor(13.8738), tensor(12.9627),
tensor(14.6328), tensor(12.8235), tensor(13.4758), tensor(13.6208),
tensor(14.4198), tensor(13.5839), tensor(14.0129), tensor(11.3706),
tensor(15.5677), tensor(14.1262), tensor(14.9994), tensor(14.7658),
tensor(14.5298), tensor(13.9261), tensor(14.1385), tensor(11.4146),
tensor(12.0312), tensor(12.9307), tensor(12.9110), tensor(11.6016),
tensor(13.6142), tensor(15.5315), tensor(14.2816), tensor(12.7240),
tensor(13.7855), tensor(14.5912)]
[tensor(15.9664), tensor(13.4007), tensor(16.8139), tensor(10.7554),
tensor(15.3664), tensor(13.1701), tensor(13.1103), tensor(13.6744),
tensor(14.1033), tensor(14.3435), tensor(14.2330), tensor(14.8434),
tensor(13.2701), tensor(14.2616), tensor(13.7414), tensor(15.9721),
tensor(15.1450), tensor(13.2699), tensor(13.5692), tensor(13.3178),
tensor(13.1926), tensor(14.3202), tensor(15.1532), tensor(14.1910),
tensor(15.7383), tensor(13.5194), tensor(13.8477), tensor(15.1119),
tensor(15.0994), tensor(15.2752), tensor(14.6925), tensor(12.2180),
tensor(16.6150), tensor(13.6896), tensor(15.4959), tensor(14.8107),
tensor(15.5600), tensor(14.3635), tensor(15.4167), tensor(12.9855),
tensor(12.0799), tensor(13.4412), tensor(12.2814), tensor(12.1145),
tensor(14.6095), tensor(16.6415), tensor(15.9783), tensor(14.4106),
tensor(14.6035), tensor(14.9573)]
[tensor(15.6908), tensor(14.0290), tensor(17.3184), tensor(11.2001),
tensor(15.2860), tensor(13.8809), tensor(13.8498), tensor(15.5253),
tensor(14.6459), tensor(15.4802), tensor(13.8354), tensor(15.2193),
tensor(13.3448), tensor(14.3827), tensor(13.4194), tensor(16.8738),
tensor(15.2419), tensor(12.1414), tensor(13.7555), tensor(13.6732),
tensor(12.7500), tensor(14.7981), tensor(16.0134), tensor(14.9072),
tensor(15.1783), tensor(14.3649), tensor(14.0735), tensor(14.4577),
tensor(14.7078), tensor(15.4920), tensor(14.3298), tensor(11.8485),
tensor(15.1238), tensor(14.0878), tensor(16.1087), tensor(13.4115),
tensor(15.1703), tensor(13.8484), tensor(14.8290), tensor(12.2868),
tensor(12.2239), tensor(13.0469), tensor(11.9295), tensor(12.4719),
tensor(14.4567), tensor(16.5507), tensor(15.2682), tensor(13.9101),
```

tensor(15.5547), tensor(14.3110)]
[tensor(15.3866), tensor(12.4008), tensor(14.0390), tensor(10.3261),
tensor(12.7941), tensor(11.1910), tensor(13.0198), tensor(12.8159),
tensor(12.9552), tensor(13.8578), tensor(12.0523), tensor(13.5643),
tensor(12.1252), tensor(13.7095), tensor(11.2427), tensor(14.4225),
tensor(13.4102), tensor(10.6736), tensor(13.1479), tensor(12.1880),
tensor(10.5600), tensor(13.2417), tensor(14.4930), tensor(12.3789),
tensor(14.3250), tensor(12.1917), tensor(11.3530), tensor(12.8779),
tensor(13.8159), tensor(12.9211), tensor(14.0984), tensor(10.6141),
tensor(12.5869), tensor(13.1982), tensor(13.8585), tensor(12.7883),
tensor(13.5516), tensor(13.7611), tensor(12.4458), tensor(10.7672),
tensor(11.5012), tensor(12.2916), tensor(13.1755), tensor(11.3003),
tensor(12.9609), tensor(14.8444), tensor(13.5271), tensor(13.1150),
tensor(13.5105), tensor(14.1291)]
[tensor(14.3103), tensor(11.7997), tensor(15.3984), tensor(8.9168),
tensor(14.0436), tensor(11.9890), tensor(12.2569), tensor(11.9736),
tensor(13.3290), tensor(12.8586), tensor(12.1953), tensor(12.0672),
tensor(11.0285), tensor(12.2913), tensor(11.0948), tensor(14.2748),
tensor(11.8009), tensor(10.6670), tensor(10.8281), tensor(11.6414),
tensor(11.7809), tensor(11.4093), tensor(13.3180), tensor(12.6006),
tensor(13.3314), tensor(11.3734), tensor(12.2667), tensor(12.0828),
tensor(11.8627), tensor(13.3091), tensor(12.5659), tensor(10.8161),
tensor(13.9915), tensor(12.0281), tensor(14.7875), tensor(12.1582),
tensor(13.9613), tensor(13.4745), tensor(14.1315), tensor(11.0459),
tensor(11.5029), tensor(13.0209), tensor(12.2998), tensor(11.0165),
tensor(12.1170), tensor(14.6383), tensor(13.3496), tensor(11.5522),
tensor(12.0480), tensor(13.6566)]
[tensor(15.0710), tensor(13.3450), tensor(16.1082), tensor(11.4700),
tensor(14.5914), tensor(11.2675), tensor(12.2799), tensor(13.6028),
tensor(14.4283), tensor(15.1757), tensor(13.5479), tensor(14.9503),
tensor(12.3013), tensor(12.7832), tensor(12.2082), tensor(14.4989),
tensor(13.9566), tensor(12.4156), tensor(13.1125), tensor(12.2735),
tensor(11.7761), tensor(14.2444), tensor(15.4155), tensor(14.3673),
tensor(13.1899), tensor(14.7798), tensor(14.1752), tensor(13.6655),
tensor(14.1639), tensor(13.5646), tensor(14.0526), tensor(11.0761),
tensor(15.1298), tensor(15.2829), tensor(15.6796), tensor(14.4483),
tensor(15.5581), tensor(14.1613), tensor(14.5171), tensor(12.7316),
tensor(12.1393), tensor(13.7540), tensor(13.5909), tensor(11.1904),
tensor(14.5138), tensor(15.0033), tensor(14.5962), tensor(13.8579),
tensor(14.1303), tensor(14.9786)]
[tensor(14.0754), tensor(12.7210), tensor(14.3834), tensor(8.7393),
tensor(13.6440), tensor(12.7621), tensor(11.7742), tensor(14.0823),
tensor(12.4280), tensor(12.8899), tensor(11.4186), tensor(13.2282),
tensor(12.5287), tensor(13.0201), tensor(12.5751), tensor(14.3276),
tensor(15.2576), tensor(11.1894), tensor(11.4171), tensor(12.7818),
tensor(11.7048), tensor(12.9083), tensor(13.2573), tensor(12.0405),
tensor(14.1155), tensor(12.2690), tensor(12.1634), tensor(13.5813),
tensor(13.2551), tensor(13.1210), tensor(12.6831), tensor(11.9872),

```
tensor(14.6275), tensor(12.0328), tensor(14.7741), tensor(12.5878),
tensor(14.5512), tensor(13.5687), tensor(13.4581), tensor(12.2620),
tensor(10.7482), tensor(11.8640), tensor(11.6903), tensor(10.8275),
tensor(12.6304), tensor(14.4958), tensor(14.1488), tensor(13.0598),
tensor(13.1254), tensor(13.3705)]
[tensor(15.9259), tensor(14.6600), tensor(16.4160), tensor(11.4456),
tensor(14.6178), tensor(13.3011), tensor(14.0737), tensor(15.3381),
tensor(14.3792), tensor(14.4756), tensor(13.0351), tensor(14.7331),
tensor(14.0384), tensor(14.2445), tensor(13.5489), tensor(16.4986),
tensor(14.5896), tensor(12.5485), tensor(13.5359), tensor(14.6646),
tensor(12.8163), tensor(14.2327), tensor(15.8961), tensor(13.1499),
tensor(15.6468), tensor(13.9452), tensor(12.6663), tensor(14.5990),
tensor(13.6785), tensor(14.6516), tensor(13.1692), tensor(11.9188),
tensor(14.6044), tensor(14.1071), tensor(15.0117), tensor(13.0435),
tensor(16.2057), tensor(14.1937), tensor(14.1026), tensor(12.2915),
tensor(11.5786), tensor(13.4310), tensor(12.9909), tensor(11.4975),
tensor(14.2222), tensor(17.0361), tensor(15.4743), tensor(14.5597),
tensor(14.1378), tensor(15.3137)]
[tensor(13.0233), tensor(12.4866), tensor(12.5953), tensor(9.1089),
tensor(12.6696), tensor(11.9024), tensor(11.2716), tensor(11.7343),
tensor(11.4878), tensor(11.9264), tensor(11.7699), tensor(12.2354),
tensor(12.1595), tensor(12.1664), tensor(10.9923), tensor(13.8060),
tensor(13.2979), tensor(10.7711), tensor(11.1901), tensor(11.1510),
tensor(12.1657), tensor(12.2563), tensor(11.9611), tensor(10.8288),
tensor(13.2474), tensor(11.3526), tensor(11.1837), tensor(12.7581),
tensor(11.6112), tensor(11.5621), tensor(11.4411), tensor(10.4740),
tensor(13.8897), tensor(10.9098), tensor(13.8413), tensor(10.7476),
tensor(12.7650), tensor(12.5699), tensor(11.7723), tensor(10.7927),
tensor(9.5097), tensor(11.2375), tensor(11.1521), tensor(10.4625),
tensor(12.2535), tensor(13.0996), tensor(12.8195), tensor(11.8224),
tensor(12.6108), tensor(13.0842)]
[tensor(13.6244), tensor(12.2431), tensor(15.0591), tensor(9.4403),
tensor(13.1093), tensor(10.9152), tensor(11.1469), tensor(13.2482),
tensor(12.6331), tensor(13.1759), tensor(11.2555), tensor(12.3869),
tensor(12.0130), tensor(12.2816), tensor(11.9725), tensor(13.8634),
tensor(13.0292), tensor(11.3554), tensor(10.8121), tensor(12.9389),
tensor(11.3931), tensor(12.0327), tensor(12.9313), tensor(11.5972),
tensor(13.4544), tensor(11.7717), tensor(12.3371), tensor(13.3506),
tensor(12.5074), tensor(12.6687), tensor(12.0742), tensor(9.3188),
tensor(13.2697), tensor(12.5799), tensor(13.1056), tensor(11.9170),
tensor(14.1101), tensor(12.1777), tensor(13.1902), tensor(11.7522),
tensor(9.4562), tensor(10.3391), tensor(11.9515), tensor(10.7204),
tensor(12.3315), tensor(14.5878), tensor(13.0450), tensor(12.4909),
tensor(12.4534), tensor(13.1792)]
[tensor(15.6637), tensor(12.7850), tensor(16.1524), tensor(10.6112),
tensor(13.1950), tensor(12.0249), tensor(13.3535), tensor(14.4758),
tensor(12.2440), tensor(13.4998), tensor(12.8985), tensor(14.7975),
tensor(12.3283), tensor(13.8055), tensor(13.3213), tensor(14.6497),
```

```
tensor(14.2947), tensor(11.7041), tensor(11.7113), tensor(12.8995),
tensor(11.4007), tensor(13.2710), tensor(15.2711), tensor(14.2292),
tensor(13.4263), tensor(13.0066), tensor(13.3516), tensor(13.2354),
tensor(13.3737), tensor(13.3845), tensor(13.9637), tensor(11.4560),
tensor(14.0568), tensor(13.1091), tensor(14.2127), tensor(13.2175),
tensor(14.9608), tensor(13.3282), tensor(14.6428), tensor(11.9038),
tensor(11.7551), tensor(13.4993), tensor(12.1449), tensor(10.2044),
tensor(13.5414), tensor(14.9645), tensor(15.0170), tensor(12.2147),
tensor(12.9521), tensor(12.8008)]
[tensor(15.1370), tensor(12.8317), tensor(14.1847), tensor(9.8036),
tensor(14.9385), tensor(12.2620), tensor(12.6634), tensor(13.5871),
tensor(13.0450), tensor(13.5027), tensor(12.2453), tensor(13.7129),
tensor(12.1810), tensor(12.3293), tensor(11.3923), tensor(14.1269),
tensor(13.6959), tensor(11.9532), tensor(12.7881), tensor(11.8901),
tensor(11.2723), tensor(13.3644), tensor(14.5963), tensor(13.5686),
tensor(13.7860), tensor(12.5060), tensor(12.8594), tensor(13.2841),
tensor(13.0331), tensor(13.2333), tensor(12.8288), tensor(11.2058),
tensor(12.5879), tensor(13.1332), tensor(14.3712), tensor(13.4060),
tensor(15.3676), tensor(13.7467), tensor(14.5915), tensor(11.9577),
tensor(11.4865), tensor(12.7998), tensor(12.9609), tensor(11.1268),
tensor(14.0887), tensor(14.5117), tensor(14.2927), tensor(13.1737),
tensor(12.6460), tensor(13.6518)]
[tensor(14.6689), tensor(13.2973), tensor(14.8594), tensor(9.6305),
tensor(13.6049), tensor(11.9421), tensor(12.9747), tensor(12.7662),
tensor(12.1929), tensor(13.3541), tensor(10.5329), tensor(13.7853),
tensor(11.8657), tensor(12.8174), tensor(12.7295), tensor(15.1239),
tensor(13.9382), tensor(10.9261), tensor(12.1385), tensor(12.6762),
tensor(12.4997), tensor(12.6753), tensor(14.1304), tensor(12.2850),
tensor(13.6172), tensor(12.6393), tensor(12.5893), tensor(13.3451),
tensor(13.1087), tensor(13.7095), tensor(12.0841), tensor(10.4402),
tensor(13.6384), tensor(12.2322), tensor(14.2542), tensor(11.5069),
tensor(14.5245), tensor(12.6406), tensor(14.0981), tensor(10.9877),
tensor(10.4402), tensor(12.0251), tensor(11.3014), tensor(10.9037),
tensor(12.9453), tensor(16.1290), tensor(13.9477), tensor(12.6155),
tensor(12.6902), tensor(13.3988)]
[tensor(13.8771), tensor(13.5571), tensor(14.6774), tensor(9.7134),
tensor(13.4100), tensor(11.7561), tensor(11.4705), tensor(12.9236),
tensor(12.6562), tensor(14.1939), tensor(12.4778), tensor(13.2310),
tensor(12.1512), tensor(12.8056), tensor(11.6205), tensor(14.9848),
tensor(14.3300), tensor(13.1853), tensor(12.1774), tensor(11.7579),
tensor(12.4168), tensor(13.0155), tensor(13.1682), tensor(13.1702),
tensor(12.9649), tensor(13.4351), tensor(13.6320), tensor(13.0131),
tensor(13.2436), tensor(12.4156), tensor(12.3588), tensor(11.2748),
tensor(14.5473), tensor(12.9374), tensor(14.5716), tensor(11.8429),
tensor(14.2362), tensor(13.1734), tensor(13.9866), tensor(11.0469),
tensor(10.3175), tensor(12.5177), tensor(10.8129), tensor(11.8963),
tensor(12.8261), tensor(14.7868), tensor(13.7611), tensor(12.3776),
tensor(13.6330), tensor(14.2635)]
```

[tensor(13.6648), tensor(11.0483), tensor(12.3896), tensor(8.6156),
tensor(13.9551), tensor(10.7843), tensor(11.0804), tensor(11.2971),
tensor(10.9973), tensor(12.9914), tensor(10.7375), tensor(11.6376),
tensor(9.9860), tensor(11.6615), tensor(11.8230), tensor(12.8024),
tensor(11.6913), tensor(10.8812), tensor(11.4985), tensor(10.1330),
tensor(11.0589), tensor(11.3983), tensor(12.8697), tensor(11.0163),
tensor(12.8618), tensor(11.0989), tensor(12.1967), tensor(11.3539),
tensor(12.8830), tensor(11.7805), tensor(13.2005), tensor(9.5699),
tensor(13.0386), tensor(10.8626), tensor(13.2208), tensor(11.9005),
tensor(11.7970), tensor(12.3703), tensor(11.8960), tensor(9.6526),
tensor(10.8220), tensor(10.0757), tensor(10.9672), tensor(9.9277),
tensor(10.9559), tensor(13.3104), tensor(13.5093), tensor(11.7317),
tensor(11.1159), tensor(13.0873)]
[tensor(14.9947), tensor(13.1214), tensor(14.1809), tensor(10.4924),
tensor(14.2044), tensor(12.7865), tensor(11.4564), tensor(14.1185),
tensor(12.1221), tensor(13.3973), tensor(13.2007), tensor(14.7415),
tensor(11.7942), tensor(13.6549), tensor(11.5178), tensor(14.0237),
tensor(14.8011), tensor(12.0914), tensor(11.5932), tensor(12.3354),
tensor(11.9312), tensor(14.1766), tensor(14.1741), tensor(13.6027),
tensor(13.7914), tensor(13.2207), tensor(12.3793), tensor(13.9510),
tensor(13.7292), tensor(12.7453), tensor(13.5850), tensor(12.2669),
tensor(15.1495), tensor(13.5962), tensor(14.4871), tensor(13.4361),
tensor(14.4248), tensor(13.6355), tensor(13.7269), tensor(12.3293),
tensor(10.8785), tensor(12.7983), tensor(12.0362), tensor(11.5442),
tensor(13.8227), tensor(13.7912), tensor(14.6581), tensor(13.7719),
tensor(13.4634), tensor(14.3844)]
[tensor(13.2668), tensor(11.7773), tensor(13.0171), tensor(8.8681),
tensor(11.8985), tensor(9.3187), tensor(11.1914), tensor(12.7044),
tensor(10.3433), tensor(12.9458), tensor(10.3680), tensor(12.0366),
tensor(9.8781), tensor(10.8933), tensor(11.6070), tensor(12.1132),
tensor(13.1305), tensor(9.7628), tensor(11.6822), tensor(10.6433),
tensor(9.6546), tensor(11.2266), tensor(12.1434), tensor(11.3868),
tensor(11.0799), tensor(11.7702), tensor(11.5603), tensor(10.9133),
tensor(11.7058), tensor(11.9866), tensor(11.4555), tensor(9.5732),
tensor(11.8053), tensor(11.4314), tensor(12.4937), tensor(11.6670),
tensor(11.7682), tensor(11.5213), tensor(12.5271), tensor(9.7256),
tensor(9.0738), tensor(10.4744), tensor(11.7065), tensor(9.1841),
tensor(10.7468), tensor(11.9377), tensor(11.7381), tensor(12.2057),
tensor(10.9611), tensor(11.7642)]
[tensor(12.4566), tensor(10.4050), tensor(12.8094), tensor(7.6491),
tensor(11.2133), tensor(10.1644), tensor(9.3310), tensor(12.1228),
tensor(10.6836), tensor(11.6376), tensor(10.6743), tensor(11.5282),
tensor(9.6244), tensor(10.3808), tensor(11.0626), tensor(12.6336),
tensor(10.8868), tensor(9.6181), tensor(9.1764), tensor(10.4752),
tensor(8.5951), tensor(10.0469), tensor(10.2227), tensor(10.9782),
tensor(12.2645), tensor(9.7060), tensor(10.6407), tensor(10.7094),
tensor(10.2760), tensor(11.2547), tensor(10.5402), tensor(9.9844),
tensor(10.9679), tensor(10.2321), tensor(11.5022), tensor(10.9804),

```
tensor(11.4048), tensor(10.4143), tensor(11.9121), tensor(8.4510),
tensor(8.5934), tensor(9.0242), tensor(8.8211), tensor(8.9820), tensor(10.4721),
tensor(11.0765), tensor(11.0647), tensor(10.0546), tensor(10.9976),
tensor(10.6089)]
[tensor(14.2569), tensor(12.2304), tensor(15.7588), tensor(9.4254),
tensor(14.1553), tensor(11.9132), tensor(12.1261), tensor(13.5903),
tensor(13.1077), tensor(14.1796), tensor(12.3015), tensor(13.0127),
tensor(11.7459), tensor(11.9773), tensor(12.0863), tensor(14.3325),
tensor(14.3070), tensor(11.5782), tensor(12.1779), tensor(12.5099),
tensor(10.7188), tensor(14.0974), tensor(13.3550), tensor(13.4411),
tensor(14.4017), tensor(11.9774), tensor(12.6589), tensor(13.1985),
tensor(12.9629), tensor(13.6364), tensor(14.0592), tensor(10.7885),
tensor(14.0386), tensor(12.7061), tensor(13.9379), tensor(13.0078),
tensor(14.2198), tensor(12.1207), tensor(14.6571), tensor(11.3684),
tensor(11.2304), tensor(11.8175), tensor(11.4680), tensor(11.2539),
tensor(13.1046), tensor(15.3224), tensor(14.0725), tensor(11.3878),
tensor(13.6761), tensor(14.0830)]
[tensor(14.3698), tensor(12.3888), tensor(15.4352), tensor(10.0384),
tensor(14.8450), tensor(12.6347), tensor(12.5970), tensor(11.9900),
tensor(12.6944), tensor(12.5730), tensor(13.4321), tensor(13.2358),
tensor(12.7334), tensor(13.5547), tensor(12.5060), tensor(14.6850),
tensor(12.7517), tensor(11.6861), tensor(12.5646), tensor(11.5782),
tensor(11.6718), tensor(11.8788), tensor(13.9738), tensor(12.3542),
tensor(12.7317), tensor(11.7636), tensor(12.7026), tensor(13.0813),
tensor(11.9119), tensor(12.6436), tensor(13.6280), tensor(11.0547),
tensor(15.7223), tensor(12.1088), tensor(13.7588), tensor(12.1848),
tensor(13.9500), tensor(13.4133), tensor(13.9717), tensor(11.2857),
tensor(10.4784), tensor(12.4394), tensor(11.5106), tensor(10.2303),
tensor(13.4573), tensor(13.9759), tensor(14.0863), tensor(12.9965),
tensor(13.7143), tensor(13.7718)]
[tensor(14.5187), tensor(13.2816), tensor(15.1063), tensor(10.7799),
tensor(13.0782), tensor(12.4928), tensor(12.3055), tensor(13.3593),
tensor(13.1325), tensor(13.8009), tensor(13.6364), tensor(14.6574),
tensor(13.3228), tensor(13.1773), tensor(13.3231), tensor(15.3605),
tensor(13.9687), tensor(11.9648), tensor(12.9055), tensor(12.4565),
tensor(10.4582), tensor(12.8946), tensor(13.6179), tensor(12.9806),
tensor(14.8389), tensor(12.7000), tensor(12.7724), tensor(14.2783),
tensor(13.2402), tensor(13.4825), tensor(12.9349), tensor(12.0516),
tensor(13.9191), tensor(12.8074), tensor(13.5397), tensor(13.4679),
tensor(14.3829), tensor(12.7769), tensor(13.9759), tensor(11.5919),
tensor(11.3981), tensor(11.9428), tensor(11.9573), tensor(10.7222),
tensor(14.0651), tensor(14.5037), tensor(14.5172), tensor(13.3534),
tensor(14.0392), tensor(13.4217)]
[tensor(14.7928), tensor(12.8709), tensor(15.4796), tensor(9.7736),
tensor(12.8486), tensor(11.8903), tensor(12.8871), tensor(12.9546),
tensor(12.1657), tensor(13.8573), tensor(12.7094), tensor(13.2522),
tensor(11.4649), tensor(13.1179), tensor(12.3741), tensor(13.8844),
tensor(13.8419), tensor(11.8637), tensor(11.4979), tensor(12.0857),
```

```
tensor(12.0118), tensor(12.3010), tensor(14.1398), tensor(13.8004),
tensor(12.5719), tensor(12.1342), tensor(13.5200), tensor(13.1956),
tensor(12.3144), tensor(13.4397), tensor(12.8414), tensor(10.9135),
tensor(15.3228), tensor(13.1099), tensor(14.2341), tensor(13.4822),
tensor(14.1060), tensor(14.0071), tensor(14.9425), tensor(13.2261),
tensor(11.3172), tensor(13.2875), tensor(13.0241), tensor(10.2383),
tensor(13.0704), tensor(14.6951), tensor(14.0966), tensor(12.5428),
tensor(12.9736), tensor(14.3008)]
[tensor(16.1476), tensor(13.1603), tensor(18.2398), tensor(10.2597),
tensor(14.7415), tensor(13.2825), tensor(14.0639), tensor(15.1408),
tensor(14.2364), tensor(15.1264), tensor(14.4066), tensor(14.8658),
tensor(12.7661), tensor(13.4033), tensor(13.1096), tensor(15.5103),
tensor(15.5788), tensor(12.9398), tensor(13.2476), tensor(14.0307),
tensor(12.3295), tensor(14.1542), tensor(15.0992), tensor(14.7888),
tensor(14.8192), tensor(13.0540), tensor(13.0914), tensor(14.3510),
tensor(14.0100), tensor(15.9083), tensor(14.6737), tensor(11.8889),
tensor(15.7400), tensor(14.9491), tensor(15.8996), tensor(13.5610),
tensor(15.0685), tensor(14.8880), tensor(16.3862), tensor(13.3686),
tensor(13.3380), tensor(13.8232), tensor(14.5092), tensor(13.0631),
tensor(13.9151), tensor(16.4514), tensor(15.1258), tensor(13.7597),
tensor(14.4361), tensor(14.8783)]
[tensor(14.4410), tensor(13.7949), tensor(15.4193), tensor(10.5917),
tensor(14.8461), tensor(11.3345), tensor(11.8661), tensor(14.0827),
tensor(12.1504), tensor(13.4564), tensor(12.6375), tensor(13.8233),
tensor(12.3328), tensor(13.4888), tensor(13.4919), tensor(14.5882),
tensor(14.5812), tensor(12.3401), tensor(12.0509), tensor(12.5067),
tensor(10.8147), tensor(13.2406), tensor(14.4363), tensor(12.2065),
tensor(13.0435), tensor(13.7852), tensor(12.6618), tensor(13.4403),
tensor(12.0299), tensor(13.9857), tensor(12.4288), tensor(10.8200),
tensor(14.9896), tensor(13.2443), tensor(14.5380), tensor(12.4890),
tensor(14.4736), tensor(14.3295), tensor(14.2122), tensor(12.0605),
tensor(11.2170), tensor(11.7700), tensor(12.3017), tensor(11.0930),
tensor(12.6300), tensor(14.5572), tensor(13.2381), tensor(13.4402),
tensor(12.6850), tensor(13.4216)]
[tensor(11.6028), tensor(9.9120), tensor(12.0476), tensor(8.3440),
tensor(10.8031), tensor(8.7241), tensor(9.7109), tensor(9.5964),
tensor(10.6564), tensor(11.8390), tensor(9.8962), tensor(10.7421),
tensor(9.9988), tensor(10.6249), tensor(9.4196), tensor(11.9782),
tensor(10.0575), tensor(8.8646), tensor(10.8944), tensor(8.8764),
tensor(10.2431), tensor(9.7917), tensor(11.7317), tensor(9.9054),
tensor(11.2298), tensor(10.0975), tensor(9.8198), tensor(11.0108),
tensor(10.9193), tensor(10.6129), tensor(10.9908), tensor(8.2987),
tensor(11.2049), tensor(9.9902), tensor(11.5966), tensor(10.6214),
tensor(11.4616), tensor(10.3248), tensor(10.1223), tensor(8.8092),
tensor(8.9952), tensor(9.8336), tensor(10.0926), tensor(8.2536),
tensor(10.6079), tensor(12.6842), tensor(10.8948), tensor(10.0837),
tensor(10.6441), tensor(11.4524)]
[tensor(15.8052), tensor(12.6165), tensor(15.1364), tensor(10.1873),
```

```
tensor(13.9731), tensor(11.7477), tensor(11.5280), tensor(12.1791),
tensor(13.2030), tensor(12.9584), tensor(12.6273), tensor(12.8209),
tensor(12.4155), tensor(12.0620), tensor(11.3696), tensor(14.8724),
tensor(13.5264), tensor(11.4387), tensor(11.9853), tensor(11.2429),
tensor(11.7614), tensor(11.6846), tensor(13.9073), tensor(12.6511),
tensor(13.0054), tensor(11.6772), tensor(13.3703), tensor(13.1842),
tensor(11.9197), tensor(14.0849), tensor(12.3417), tensor(10.2796),
tensor(13.5944), tensor(12.1223), tensor(15.0163), tensor(13.3643),
tensor(14.3158), tensor(12.7028), tensor(14.2728), tensor(11.0394),
tensor(10.8170), tensor(12.7755), tensor(11.3897), tensor(11.0886),
tensor(14.3071), tensor(14.7542), tensor(14.0753), tensor(12.4889),
tensor(12.8711), tensor(13.8168)]
[tensor(14.5115), tensor(11.9914), tensor(13.3639), tensor(8.6086),
tensor(13.6249), tensor(11.9436), tensor(12.8045), tensor(12.5105),
tensor(11.4496), tensor(12.8841), tensor(10.5906), tensor(12.1058),
tensor(10.1174), tensor(13.0713), tensor(11.3954), tensor(13.6189),
tensor(12.7342), tensor(11.2349), tensor(12.4805), tensor(11.2825),
tensor(9.6172), tensor(12.4523), tensor(13.2098), tensor(12.6582),
tensor(12.7322), tensor(11.5085), tensor(12.8135), tensor(11.2398),
tensor(11.9694), tensor(12.8098), tensor(11.6906), tensor(9.8273),
tensor(11.7583), tensor(12.0453), tensor(13.1873), tensor(11.6100),
tensor(12.4415), tensor(12.2354), tensor(13.2557), tensor(9.5769),
tensor(10.7100), tensor(10.8145), tensor(10.7744), tensor(10.4568),
tensor(12.3809), tensor(14.6177), tensor(12.7783), tensor(11.9094),
tensor(12.0480), tensor(12.9438)]
[tensor(11.1013), tensor(10.5806), tensor(11.1624), tensor(6.9832),
tensor(10.6882), tensor(9.7242), tensor(9.8270), tensor(11.2663),
tensor(8.8811), tensor(10.5877), tensor(9.8950), tensor(11.0552),
tensor(8.4484), tensor(10.7846), tensor(10.3785), tensor(10.7991),
tensor(11.0783), tensor(9.2680), tensor(9.7452), tensor(9.5966), tensor(8.2340),
tensor(10.6134), tensor(10.8882), tensor(10.8862), tensor(10.8059),
tensor(10.2643), tensor(10.1459), tensor(10.0397), tensor(10.6071),
tensor(10.0048), tensor(9.6678), tensor(10.1423), tensor(11.0355),
tensor(9.9911), tensor(10.9039), tensor(9.8873), tensor(10.7717),
tensor(9.7182), tensor(11.2618), tensor(8.6258), tensor(7.5771), tensor(8.8842),
tensor(8.6536), tensor(7.6773), tensor(9.6801), tensor(10.4924),
tensor(10.7502), tensor(10.2211), tensor(9.9488), tensor(9.9550)]
[tensor(13.8893), tensor(11.7375), tensor(14.6223), tensor(8.7354),
tensor(11.4977), tensor(10.2237), tensor(11.4431), tensor(12.3301),
tensor(11.3355), tensor(13.1757), tensor(11.5865), tensor(12.8217),
tensor(11.8851), tensor(9.8433), tensor(12.6006), tensor(13.4983),
tensor(13.0352), tensor(10.8143), tensor(9.9088), tensor(10.7972),
tensor(10.9083), tensor(10.9446), tensor(11.2074), tensor(11.8554),
tensor(11.8105), tensor(10.9718), tensor(12.4322), tensor(12.0646),
tensor(11.3546), tensor(12.9375), tensor(11.7618), tensor(10.5316),
tensor(11.6851), tensor(11.1696), tensor(12.7383), tensor(11.1375),
tensor(13.7471), tensor(12.0689), tensor(14.2893), tensor(10.7887),
tensor(9.4996), tensor(11.2419), tensor(10.8584), tensor(11.3470),
```

```
tensor(12.6854), tensor(14.0435), tensor(13.1912), tensor(11.8506),
tensor(11.5720), tensor(11.9961)]
[tensor(11.8521), tensor(10.2922), tensor(12.1141), tensor(8.4951),
tensor(11.3835), tensor(10.7329), tensor(9.8260), tensor(10.9732),
tensor(10.4236), tensor(10.6413), tensor(10.9741), tensor(11.1046),
tensor(10.5310), tensor(11.3387), tensor(10.6692), tensor(11.2673),
tensor(11.3184), tensor(10.6392), tensor(10.1031), tensor(11.3484),
tensor(9.6538), tensor(11.1960), tensor(10.9821), tensor(9.9285),
tensor(11.1807), tensor(10.4933), tensor(10.0452), tensor(10.0203),
tensor(10.6335), tensor(10.8208), tensor(10.0951), tensor(9.0248),
tensor(11.5907), tensor(10.8752), tensor(11.7421), tensor(10.4912),
tensor(11.4390), tensor(11.2247), tensor(11.1198), tensor(8.7678),
tensor(9.8527), tensor(9.2424), tensor(10.6839), tensor(8.9493),
tensor(10.5630), tensor(12.0283), tensor(11.3024), tensor(11.5350),
tensor(11.2788), tensor(11.2265)]
[tensor(15.8732), tensor(14.9631), tensor(16.7747), tensor(11.0652),
tensor(15.9795), tensor(13.8742), tensor(12.7422), tensor(14.8768),
tensor(13.4760), tensor(14.5565), tensor(13.6812), tensor(15.0063),
tensor(14.2973), tensor(14.1192), tensor(13.6461), tensor(15.9562),
tensor(15.6738), tensor(12.6761), tensor(12.9256), tensor(13.5905),
tensor(14.0984), tensor(14.0446), tensor(16.0568), tensor(13.1514),
tensor(13.9096), tensor(14.2801), tensor(14.2769), tensor(13.8825),
tensor(13.7544), tensor(15.1395), tensor(14.4761), tensor(12.7133),
tensor(16.0431), tensor(13.3989), tensor(16.2670), tensor(13.3622),
tensor(15.5714), tensor(14.5903), tensor(15.3254), tensor(13.0572),
tensor(10.9969), tensor(12.5583), tensor(11.8890), tensor(12.5489),
tensor(14.3345), tensor(15.9185), tensor(15.0048), tensor(14.5806),
tensor(15.1637), tensor(14.6787)]
[tensor(16.4110), tensor(13.4188), tensor(15.5029), tensor(10.1578),
tensor(13.7600), tensor(13.1420), tensor(12.0175), tensor(14.1812),
tensor(13.4749), tensor(13.5469), tensor(13.0874), tensor(14.3385),
tensor(13.5004), tensor(11.6397), tensor(11.9473), tensor(15.9738),
tensor(14.0596), tensor(11.5253), tensor(13.1535), tensor(12.7911),
tensor(11.0907), tensor(12.7109), tensor(13.6220), tensor(12.5836),
tensor(14.7552), tensor(11.9278), tensor(13.0498), tensor(13.9899),
tensor(12.7169), tensor(14.7910), tensor(13.2997), tensor(12.4559),
tensor(13.3638), tensor(13.1473), tensor(15.5021), tensor(13.4780),
tensor(14.9898), tensor(13.5771), tensor(15.3142), tensor(10.9663),
tensor(10.8460), tensor(12.5188), tensor(11.7227), tensor(11.9803),
tensor(14.6724), tensor(14.7945), tensor(15.3625), tensor(13.6583),
tensor(13.4326), tensor(14.3585)]
[tensor(12.9884), tensor(12.0196), tensor(14.7092), tensor(9.2296),
tensor(13.4900), tensor(10.7504), tensor(11.2368), tensor(13.1155),
tensor(11.7285), tensor(13.1267), tensor(11.5995), tensor(11.9253),
tensor(11.4348), tensor(11.1301), tensor(12.4371), tensor(13.4904),
tensor(13.3713), tensor(9.7014), tensor(11.3441), tensor(11.2583),
tensor(11.7170), tensor(11.0744), tensor(13.4015), tensor(11.1887),
tensor(11.9194), tensor(12.2965), tensor(11.8240), tensor(12.1588),
```

```
tensor(12.6546), tensor(12.5406), tensor(12.0112), tensor(10.3371),
tensor(13.6307), tensor(11.4744), tensor(14.1444), tensor(11.4174),
tensor(12.0935), tensor(11.8884), tensor(12.3267), tensor(10.2379),
tensor(10.0091), tensor(10.7075), tensor(10.7198), tensor(10.9430),
tensor(10.2090), tensor(13.3211), tensor(12.9315), tensor(12.3681),
tensor(12.1011), tensor(12.0577)]
[tensor(14.7793), tensor(12.4282), tensor(14.5680), tensor(9.4706),
tensor(13.9155), tensor(11.2221), tensor(11.8694), tensor(13.1860),
tensor(11.6017), tensor(13.7470), tensor(11.6577), tensor(12.6162),
tensor(11.7073), tensor(11.0733), tensor(11.1466), tensor(14.3232),
tensor(12.6208), tensor(11.1426), tensor(12.0080), tensor(10.7191),
tensor(11.6846), tensor(12.3354), tensor(12.7824), tensor(11.6235),
tensor(13.9455), tensor(11.2999), tensor(11.9904), tensor(13.6287),
tensor(11.7212), tensor(12.5207), tensor(13.1295), tensor(10.9626),
tensor(13.7957), tensor(12.5411), tensor(13.7643), tensor(12.4498),
tensor(14.2457), tensor(12.9829), tensor(14.6118), tensor(11.3728),
tensor(10.1241), tensor(11.6780), tensor(11.9779), tensor(10.4341),
tensor(12.8447), tensor(14.8669), tensor(13.6213), tensor(11.8367),
tensor(11.6594), tensor(14.2520)]
[tensor(16.6272), tensor(14.8795), tensor(17.7359), tensor(11.4590),
tensor(15.4250), tensor(12.7531), tensor(14.9966), tensor(14.9057),
tensor(14.1683), tensor(16.3657), tensor(14.3271), tensor(15.4769),
tensor(14.0781), tensor(14.1438), tensor(14.8435), tensor(16.7202),
tensor(15.2243), tensor(13.2345), tensor(14.3094), tensor(14.4217),
tensor(13.8743), tensor(14.2666), tensor(15.0645), tensor(13.6416),
tensor(16.7378), tensor(14.7606), tensor(15.0348), tensor(14.8270),
tensor(14.9280), tensor(15.2719), tensor(14.8817), tensor(12.8759),
tensor(16.8349), tensor(14.9079), tensor(16.3173), tensor(14.1331),
tensor(16.9271), tensor(15.6614), tensor(15.9991), tensor(13.4880),
tensor(12.1885), tensor(13.7751), tensor(14.9001), tensor(11.8768),
tensor(14.3272), tensor(17.9746), tensor(16.4691), tensor(14.2760),
tensor(14.4107), tensor(16.4582)]
[tensor(12.5568), tensor(12.0665), tensor(12.3015), tensor(9.3800),
tensor(12.3331), tensor(10.1707), tensor(10.3339), tensor(10.9126),
tensor(11.0755), tensor(13.3414), tensor(10.2138), tensor(11.8728),
tensor(11.3231), tensor(10.9498), tensor(10.8451), tensor(13.0347),
tensor(12.4035), tensor(10.4656), tensor(10.1080), tensor(10.8310),
tensor(11.4391), tensor(11.0258), tensor(12.2335), tensor(9.8351),
tensor(11.9343), tensor(11.3395), tensor(10.8854), tensor(11.6304),
tensor(12.1017), tensor(11.2287), tensor(10.8363), tensor(9.0725),
tensor(12.2848), tensor(11.0889), tensor(13.1385), tensor(11.0796),
tensor(13.0644), tensor(11.5899), tensor(11.1382), tensor(9.8313),
tensor(10.7093), tensor(9.7383), tensor(11.2369), tensor(9.9017),
tensor(11.2232), tensor(13.4393), tensor(12.1628), tensor(11.0483),
tensor(11.1124), tensor(12.2920)]
[tensor(15.2602), tensor(12.4136), tensor(14.5436), tensor(9.8780),
tensor(12.7238), tensor(10.2299), tensor(12.0732), tensor(12.7482),
tensor(11.6104), tensor(13.9715), tensor(12.2118), tensor(13.7658),
```

```
tensor(11.9606), tensor(14.0974), tensor(12.6875), tensor(13.8353),
tensor(14.3787), tensor(11.7448), tensor(11.8181), tensor(12.5687),
tensor(10.8969), tensor(12.5950), tensor(13.3158), tensor(13.0239),
tensor(13.9839), tensor(12.3300), tensor(11.8648), tensor(12.5544),
tensor(13.5688), tensor(12.4603), tensor(12.8235), tensor(10.1397),
tensor(13.1626), tensor(12.6910), tensor(12.6187), tensor(13.4346),
tensor(14.3780), tensor(12.6788), tensor(13.3970), tensor(9.6310),
tensor(11.1104), tensor(11.4325), tensor(12.3933), tensor(10.6721),
tensor(12.6590), tensor(14.6665), tensor(12.7735), tensor(12.4249),
tensor(12.4848), tensor(12.7427)]
[tensor(15.3654), tensor(13.6711), tensor(15.7195), tensor(10.9145),
tensor(14.2823), tensor(11.7545), tensor(13.1468), tensor(13.8336),
tensor(13.1667), tensor(14.6824), tensor(13.1823), tensor(14.0676),
tensor(12.3996), tensor(13.4293), tensor(13.7246), tensor(14.5367),
tensor(15.1436), tensor(12.8434), tensor(12.6142), tensor(12.5013),
tensor(11.8909), tensor(14.1738), tensor(14.9895), tensor(13.2112),
tensor(14.3741), tensor(13.9613), tensor(13.2112), tensor(13.4256),
tensor(14.0222), tensor(12.9818), tensor(13.5300), tensor(11.8324),
tensor(15.5863), tensor(14.6785), tensor(15.6356), tensor(13.0356),
tensor(15.1561), tensor(14.6628), tensor(13.9779), tensor(12.3055),
tensor(11.6885), tensor(13.2872), tensor(12.9777), tensor(12.0882),
tensor(13.4492), tensor(15.3580), tensor(14.3028), tensor(13.2700),
tensor(14.0879), tensor(15.0539)]
[tensor(14.4826), tensor(12.4652), tensor(15.7018), tensor(10.0077),
tensor(12.7384), tensor(11.9718), tensor(13.0423), tensor(13.4206),
tensor(12.6557), tensor(13.2661), tensor(12.5793), tensor(12.5749),
tensor(12.8893), tensor(12.5925), tensor(12.2576), tensor(14.6563),
tensor(14.3978), tensor(11.7342), tensor(11.9864), tensor(12.4033),
tensor(11.8361), tensor(12.3805), tensor(14.0274), tensor(12.3188),
tensor(14.0818), tensor(13.0270), tensor(12.1998), tensor(13.6707),
tensor(13.8577), tensor(13.2832), tensor(12.7893), tensor(11.7085),
tensor(13.2023), tensor(12.6122), tensor(15.0358), tensor(12.9550),
tensor(14.8828), tensor(13.4260), tensor(14.4573), tensor(10.5026),
tensor(11.4390), tensor(13.2193), tensor(12.5878), tensor(11.4917),
tensor(11.6481), tensor(14.9130), tensor(14.0770), tensor(13.4150),
tensor(12.0157), tensor(13.6644)]
[tensor(14.1249), tensor(11.7307), tensor(13.0650), tensor(9.7905),
tensor(10.9430), tensor(10.0405), tensor(10.8326), tensor(11.8410),
tensor(11.3880), tensor(12.6739), tensor(11.1442), tensor(12.2388),
tensor(12.0883), tensor(11.7668), tensor(11.0711), tensor(12.8208),
tensor(12.3806), tensor(9.4584), tensor(11.1158), tensor(11.3304),
tensor(10.2292), tensor(10.8620), tensor(13.3411), tensor(11.2115),
tensor(11.2709), tensor(11.9056), tensor(11.3810), tensor(11.6733),
tensor(12.1284), tensor(12.3449), tensor(11.5931), tensor(9.2670),
tensor(11.8087), tensor(12.1913), tensor(13.1497), tensor(12.1069),
tensor(12.9179), tensor(11.7062), tensor(11.9224), tensor(9.8768),
tensor(9.6963), tensor(10.4407), tensor(11.4351), tensor(10.1455),
tensor(12.3254), tensor(12.9203), tensor(12.6450), tensor(12.6174),
```

```
tensor(11.4270), tensor(11.9815)]
[tensor(16.7728), tensor(14.0343), tensor(18.4943), tensor(10.4371),
tensor(15.5612), tensor(13.0626), tensor(13.8500), tensor(14.7422),
tensor(13.5695), tensor(15.7286), tensor(13.3841), tensor(15.1396),
tensor(12.8221), tensor(14.6676), tensor(15.3462), tensor(16.3722),
tensor(15.3709), tensor(13.5853), tensor(13.3828), tensor(14.6981),
tensor(14.1484), tensor(14.0638), tensor(15.6422), tensor(13.4552),
tensor(16.1234), tensor(13.4793), tensor(14.2341), tensor(14.7752),
tensor(14.9962), tensor(16.1121), tensor(14.8376), tensor(12.1550),
tensor(16.7642), tensor(14.2042), tensor(16.1339), tensor(15.0951),
tensor(16.8111), tensor(14.7811), tensor(16.6950), tensor(12.7892),
tensor(13.0399), tensor(12.6006), tensor(12.7791), tensor(12.3387),
tensor(14.1212), tensor(17.6842), tensor(15.5277), tensor(13.7381),
tensor(13.9690), tensor(14.3959)]
[tensor(14.1949), tensor(13.5865), tensor(15.0843), tensor(11.5527),
tensor(14.5971), tensor(12.0662), tensor(11.7364), tensor(14.0469),
tensor(12.8247), tensor(14.4419), tensor(13.6593), tensor(14.4803),
tensor(12.2982), tensor(14.2682), tensor(12.6068), tensor(14.7916),
tensor(14.2444), tensor(12.4248), tensor(11.9538), tensor(12.9431),
tensor(11.2321), tensor(13.6069), tensor(14.5732), tensor(12.7100),
tensor(14.3831), tensor(14.1778), tensor(12.9370), tensor(14.1678),
tensor(13.8166), tensor(12.8016), tensor(13.2780), tensor(11.2790),
tensor(15.5522), tensor(13.8657), tensor(14.6364), tensor(14.3473),
tensor(14.7248), tensor(13.5406), tensor(13.1164), tensor(11.1708),
tensor(12.2249), tensor(12.5447), tensor(12.8655), tensor(10.5531),
tensor(12.5954), tensor(13.8340), tensor(14.1643), tensor(12.9764),
tensor(13.4320), tensor(14.4690)]
[tensor(14.3306), tensor(11.3918), tensor(14.7087), tensor(10.1319),
tensor(14.5490), tensor(11.2805), tensor(11.4039), tensor(13.8750),
tensor(12.7541), tensor(14.0429), tensor(12.0211), tensor(13.1234),
tensor(12.0929), tensor(13.0734), tensor(12.5697), tensor(14.3680),
tensor(12.8104), tensor(10.8308), tensor(11.7368), tensor(12.0837),
tensor(10.6278), tensor(12.0504), tensor(13.0996), tensor(12.0371),
tensor(14.9772), tensor(12.7536), tensor(12.3816), tensor(12.6405),
tensor(13.2192), tensor(12.6814), tensor(12.9556), tensor(9.8581),
tensor(13.3863), tensor(12.6776), tensor(13.3144), tensor(13.6059),
tensor(13.7621), tensor(12.6312), tensor(12.4804), tensor(9.6983),
tensor(11.0826), tensor(10.2780), tensor(11.5179), tensor(9.7658),
tensor(11.8198), tensor(14.0571), tensor(13.2431), tensor(12.2147),
tensor(12.5906), tensor(12.7683)]
[tensor(12.9669), tensor(12.1649), tensor(13.2284), tensor(9.2000),
tensor(12.8129), tensor(10.7193), tensor(11.8119), tensor(12.1672),
tensor(11.2789), tensor(13.0369), tensor(12.2993), tensor(12.7424),
tensor(11.3932), tensor(13.3288), tensor(12.8753), tensor(12.5495),
tensor(12.5549), tensor(11.1329), tensor(10.3918), tensor(10.9308),
tensor(10.4315), tensor(12.8559), tensor(12.5801), tensor(11.5228),
tensor(12.8550), tensor(11.6454), tensor(12.0650), tensor(13.2080),
tensor(11.5538), tensor(11.6384), tensor(11.9060), tensor(10.4653),
```

```
tensor(13.7003), tensor(11.8878), tensor(12.5289), tensor(13.5650),
tensor(13.6741), tensor(13.0967), tensor(13.1083), tensor(11.6744),
tensor(10.6966), tensor(11.1152), tensor(12.2953), tensor(8.8953),
tensor(12.3929), tensor(13.1175), tensor(12.6129), tensor(12.5341),
tensor(12.3994), tensor(12.6226)]
[tensor(14.7194), tensor(14.6121), tensor(15.2824), tensor(11.1776),
tensor(14.0644), tensor(12.2127), tensor(12.8191), tensor(13.9789),
tensor(13.4647), tensor(14.4425), tensor(12.4541), tensor(14.6486),
tensor(13.6616), tensor(14.2891), tensor(12.8121), tensor(16.4526),
tensor(15.6169), tensor(12.0739), tensor(12.8745), tensor(12.5467),
tensor(12.9798), tensor(13.2336), tensor(14.4442), tensor(12.8327),
tensor(15.0648), tensor(13.5026), tensor(12.9466), tensor(14.6585),
tensor(14.0654), tensor(14.0715), tensor(12.4006), tensor(11.9315),
tensor(14.8090), tensor(13.9917), tensor(15.3129), tensor(12.8347),
tensor(14.3688), tensor(14.1639), tensor(13.9151), tensor(12.4927),
tensor(10.9887), tensor(12.8771), tensor(13.1898), tensor(12.3403),
tensor(13.4667), tensor(16.2856), tensor(14.4216), tensor(14.3483),
tensor(13.3555), tensor(14.3151)]
[tensor(16.6154), tensor(14.4277), tensor(17.1420), tensor(11.5514),
tensor(15.0163), tensor(12.7799), tensor(14.6334), tensor(13.8714),
tensor(13.6298), tensor(15.8464), tensor(14.5022), tensor(14.7909),
tensor(13.8577), tensor(13.8288), tensor(13.3846), tensor(15.6008),
tensor(14.5239), tensor(12.8536), tensor(14.6069), tensor(13.5557),
tensor(13.3106), tensor(13.6282), tensor(15.8328), tensor(13.6272),
tensor(15.4205), tensor(13.9805), tensor(14.5416), tensor(14.6405),
tensor(13.9689), tensor(16.3401), tensor(14.2243), tensor(11.5337),
tensor(15.9732), tensor(14.5462), tensor(15.6729), tensor(14.4702),
tensor(16.0399), tensor(14.9532), tensor(15.3220), tensor(13.7824),
tensor(11.9953), tensor(13.8224), tensor(14.6314), tensor(12.0603),
tensor(14.7182), tensor(17.4024), tensor(16.3015), tensor(14.7525),
tensor(14.8474), tensor(16.5001)]
[tensor(12.1466), tensor(10.9431), tensor(11.9203), tensor(11.6371),
tensor(12.3401), tensor(9.9788), tensor(12.9186), tensor(13.8307),
tensor(12.0337), tensor(12.3737), tensor(13.2624), tensor(12.2887),
tensor(12.2739), tensor(9.4825), tensor(11.6362), tensor(13.0959),
tensor(12.1006), tensor(12.7508), tensor(11.5228), tensor(13.8360),
tensor(11.2302), tensor(11.9852), tensor(12.9025), tensor(13.0892),
tensor(13.1067), tensor(12.3194), tensor(13.0382), tensor(12.3906),
tensor(10.7352), tensor(12.9994), tensor(11.1733), tensor(13.8793),
tensor(11.9240), tensor(12.5230), tensor(10.6713), tensor(14.5074),
tensor(11.1394), tensor(11.3898), tensor(11.6659), tensor(12.8739),
tensor(11.4609), tensor(11.9505), tensor(11.7503), tensor(12.1207),
tensor(13.0497), tensor(12.7085), tensor(12.3101), tensor(11.1426),
tensor(13.5195), tensor(11.5569)]
[tensor(14.6027), tensor(14.1041), tensor(14.2206), tensor(14.7638),
tensor(15.3673), tensor(12.3968), tensor(13.0187), tensor(18.7167),
tensor(15.5550), tensor(15.5457), tensor(17.3337), tensor(14.7447),
tensor(14.4606), tensor(12.3819), tensor(12.8969), tensor(15.0012),
```

```
tensor(14.7647), tensor(16.4335), tensor(14.2505), tensor(16.7493),
tensor(12.8330), tensor(16.6864), tensor(16.8516), tensor(13.3521),
tensor(14.8907), tensor(14.4271), tensor(15.3598), tensor(16.0521),
tensor(13.2711), tensor(15.1972), tensor(13.0987), tensor(15.5266),
tensor(15.2597), tensor(15.2717), tensor(14.2761), tensor(18.1444),
tensor(14.7764), tensor(15.6293), tensor(14.9002), tensor(14.9715),
tensor(13.8636), tensor(13.2380), tensor(14.3079), tensor(15.1301),
tensor(15.4769), tensor(14.2270), tensor(14.9309), tensor(14.2815),
tensor(15.2963), tensor(14.1711)]
[tensor(11.8776), tensor(11.8836), tensor(12.7622), tensor(12.3100),
tensor(13.3065), tensor(10.2528), tensor(12.0260), tensor(15.0928),
tensor(12.6339), tensor(13.4572), tensor(14.0640), tensor(12.0340),
tensor(12.0096), tensor(10.4327), tensor(11.1649), tensor(12.1754),
tensor(12.9852), tensor(12.6132), tensor(11.5559), tensor(14.6004),
tensor(10.9707), tensor(13.0350), tensor(13.8235), tensor(12.4317),
tensor(13.4081), tensor(12.2446), tensor(12.8934), tensor(12.5312),
tensor(12.9264), tensor(13.0784), tensor(11.6834), tensor(14.4665),
tensor(11.6983), tensor(12.4563), tensor(11.1813), tensor(14.5922),
tensor(12.1533), tensor(13.0672), tensor(13.3681), tensor(12.4716),
tensor(10.5139), tensor(11.5447), tensor(11.0852), tensor(11.7100),
tensor(13.2643), tensor(12.6237), tensor(12.6644), tensor(11.7326),
tensor(12.2595), tensor(11.4033)]
[tensor(11.2102), tensor(10.2577), tensor(12.6485), tensor(10.8905),
tensor(12.7904), tensor(9.8722), tensor(11.7763), tensor(13.8671),
tensor(11.7143), tensor(11.5724), tensor(13.2757), tensor(10.9962),
tensor(11.9837), tensor(10.4736), tensor(11.8502), tensor(11.9331),
tensor(12.6684), tensor(10.7135), tensor(10.7648), tensor(12.8718),
tensor(10.5349), tensor(12.9418), tensor(14.8521), tensor(10.0498),
tensor(13.1131), tensor(11.2133), tensor(11.3013), tensor(11.7592),
tensor(11.6260), tensor(11.7771), tensor(9.4030), tensor(12.2709),
tensor(11.7437), tensor(12.2951), tensor(10.9941), tensor(13.7602),
tensor(11.5797), tensor(11.6198), tensor(11.4235), tensor(12.5774),
tensor(11.4617), tensor(11.5843), tensor(11.4128), tensor(11.5030),
tensor(12.8566), tensor(11.2243), tensor(11.0470), tensor(11.3877),
tensor(12.7710), tensor(11.1720)]
[tensor(13.3395), tensor(12.7506), tensor(11.7021), tensor(12.5611),
tensor(13.1245), tensor(10.8893), tensor(13.0546), tensor(15.2942),
tensor(13.1701), tensor(13.3821), tensor(14.7132), tensor(13.0384),
tensor(12.2252), tensor(10.7639), tensor(13.3718), tensor(13.7849),
tensor(13.6055), tensor(14.5975), tensor(13.2449), tensor(13.6524),
tensor(11.8573), tensor(14.4219), tensor(14.6919), tensor(13.5416),
tensor(13.5039), tensor(12.1385), tensor(14.9188), tensor(15.1143),
tensor(13.5646), tensor(13.2543), tensor(11.6866), tensor(14.8047),
tensor(12.6631), tensor(13.8406), tensor(12.1066), tensor(16.0372),
tensor(12.8358), tensor(13.6877), tensor(13.1159), tensor(13.8030),
tensor(12.1464), tensor(11.4298), tensor(13.2728), tensor(12.7898),
tensor(12.8114), tensor(13.5293), tensor(13.2137), tensor(13.4417),
tensor(13.5771), tensor(12.5208)]
```

```
[tensor(9.8340), tensor(10.3210), tensor(12.4343), tensor(11.7095),
tensor(12.5016), tensor(9.1493), tensor(11.4847), tensor(14.3202),
tensor(12.0601), tensor(11.6813), tensor(13.0972), tensor(11.8450),
tensor(10.5275), tensor(9.2694), tensor(10.3576), tensor(11.5046),
tensor(11.1984), tensor(12.2034), tensor(11.7244), tensor(12.8497),
tensor(9.8527), tensor(12.9877), tensor(12.5728), tensor(11.3403),
tensor(10.9329), tensor(11.6098), tensor(12.1842), tensor(12.3918),
tensor(11.0286), tensor(11.9062), tensor(10.2948), tensor(12.9081),
tensor(11.6627), tensor(11.3688), tensor(11.5487), tensor(14.2958),
tensor(10.9950), tensor(12.0962), tensor(12.7011), tensor(12.2156),
tensor(10.7763), tensor(10.6060), tensor(12.0394), tensor(11.3773),
tensor(11.3967), tensor(13.1609), tensor(12.0191), tensor(10.2685),
tensor(11.3420), tensor(11.1554)]
[tensor(13.4799), tensor(11.1869), tensor(13.6629), tensor(12.6038),
tensor(13.3759), tensor(11.3484), tensor(12.9832), tensor(15.5948),
tensor(12.3861), tensor(14.9597), tensor(14.4014), tensor(13.3160),
tensor(13.2516), tensor(11.0912), tensor(12.1747), tensor(14.4370),
tensor(15.4137), tensor(13.1077), tensor(12.9253), tensor(15.2240),
tensor(12.6304), tensor(16.0830), tensor(14.2595), tensor(12.7537),
tensor(13.8217), tensor(12.8745), tensor(13.9426), tensor(14.9947),
tensor(13.6122), tensor(14.7868), tensor(13.3830), tensor(15.6391),
tensor(13.3770), tensor(14.6251), tensor(12.3534), tensor(16.4993),
tensor(13.2565), tensor(13.7955), tensor(14.9860), tensor(14.4326),
tensor(12.7105), tensor(12.8013), tensor(13.0772), tensor(13.4111),
tensor(13.8632), tensor(13.5575), tensor(12.6767), tensor(12.9339),
tensor(14.6689), tensor(12.9206)]
[tensor(12.7140), tensor(11.1191), tensor(12.0478), tensor(11.0391),
tensor(11.6720), tensor(11.0485), tensor(11.6135), tensor(15.0143),
tensor(12.4029), tensor(13.7599), tensor(13.6962), tensor(11.4279),
tensor(11.4645), tensor(10.2240), tensor(11.6703), tensor(12.3884),
tensor(12.7079), tensor(12.9732), tensor(11.6657), tensor(13.2509),
tensor(11.0534), tensor(12.4628), tensor(14.0916), tensor(11.9131),
tensor(12.2191), tensor(11.8853), tensor(13.4851), tensor(12.7008),
tensor(11.7897), tensor(12.3698), tensor(10.9295), tensor(14.3277),
tensor(12.7889), tensor(13.2414), tensor(10.5113), tensor(14.7191),
tensor(10.8803), tensor(12.4463), tensor(12.8150), tensor(13.0118),
tensor(11.5157), tensor(11.2474), tensor(11.7865), tensor(11.3848),
tensor(13.2196), tensor(12.3449), tensor(12.3574), tensor(12.6975),
tensor(12.1388), tensor(11.0403)]
[tensor(11.7844), tensor(11.4189), tensor(11.9690), tensor(11.1063),
tensor(12.5601), tensor(9.5261), tensor(11.7737), tensor(14.2040),
tensor(12.1458), tensor(12.0035), tensor(14.1453), tensor(12.8859),
tensor(11.1374), tensor(9.4677), tensor(12.4602), tensor(12.7856),
tensor(12.1067), tensor(12.2093), tensor(11.4377), tensor(14.4459),
tensor(10.0606), tensor(12.3322), tensor(14.8020), tensor(10.1759),
tensor(13.4514), tensor(11.8127), tensor(12.9440), tensor(13.4644),
tensor(11.7335), tensor(12.9354), tensor(11.4697), tensor(12.0719),
tensor(11.3822), tensor(11.7417), tensor(11.7503), tensor(13.6848),
```

```
tensor(12.2506), tensor(12.3617), tensor(13.1431), tensor(13.2938),
tensor(11.8883), tensor(11.9136), tensor(11.3193), tensor(9.9878),
tensor(11.7851), tensor(12.0576), tensor(11.8302), tensor(11.1684),
tensor(12.1156), tensor(11.8411)]
[tensor(14.6309), tensor(12.2440), tensor(12.4310), tensor(13.0964),
tensor(13.2536), tensor(10.9778), tensor(12.1865), tensor(15.8972),
tensor(13.0376), tensor(14.1721), tensor(14.5292), tensor(12.6157),
tensor(13.4032), tensor(11.2305), tensor(12.6153), tensor(13.7663),
tensor(13.4143), tensor(15.0602), tensor(12.8679), tensor(13.6562),
tensor(12.0280), tensor(15.5963), tensor(14.9624), tensor(12.3713),
tensor(13.2534), tensor(12.6712), tensor(14.1821), tensor(14.6036),
tensor(12.8902), tensor(13.3279), tensor(11.2407), tensor(14.2172),
tensor(12.5123), tensor(14.7776), tensor(11.5480), tensor(16.9998),
tensor(12.7570), tensor(13.5983), tensor(13.9927), tensor(13.7420),
tensor(12.7542), tensor(12.3136), tensor(12.2814), tensor(12.9831),
tensor(14.5089), tensor(13.2824), tensor(13.3990), tensor(13.2480),
tensor(13.8621), tensor(13.3929)]
[tensor(10.4020), tensor(10.2759), tensor(11.4853), tensor(10.0061),
tensor(11.0763), tensor(8.4954), tensor(11.0390), tensor(13.7344),
tensor(10.9116), tensor(12.4032), tensor(13.4176), tensor(10.7522),
tensor(10.5980), tensor(10.1831), tensor(10.4493), tensor(10.9962),
tensor(11.5464), tensor(11.4064), tensor(10.7979), tensor(11.5732),
tensor(9.9733), tensor(12.8045), tensor(13.2158), tensor(9.6342),
tensor(11.9044), tensor(12.0037), tensor(11.9905), tensor(11.6595),
tensor(11.2450), tensor(12.2369), tensor(10.2152), tensor(13.7156),
tensor(11.5293), tensor(11.7321), tensor(10.8761), tensor(14.7408),
tensor(11.4726), tensor(11.2555), tensor(11.9384), tensor(11.1573),
tensor(10.6588), tensor(11.1598), tensor(10.6484), tensor(10.8980),
tensor(12.2384), tensor(10.6029), tensor(11.7158), tensor(10.8803),
tensor(12.1063), tensor(10.3614)]
[tensor(13.8275), tensor(13.1506), tensor(13.2860), tensor(13.5323),
tensor(14.7131), tensor(10.9782), tensor(13.1929), tensor(16.6097),
tensor(13.5133), tensor(14.7890), tensor(15.9169), tensor(13.4525),
tensor(12.4999), tensor(13.0931), tensor(12.2287), tensor(14.3014),
tensor(14.9578), tensor(13.8728), tensor(13.3688), tensor(14.9262),
tensor(13.0278), tensor(16.6995), tensor(16.1189), tensor(12.7471),
tensor(14.2742), tensor(12.5197), tensor(14.3330), tensor(15.2233),
tensor(14.6284), tensor(14.4986), tensor(12.5122), tensor(15.4957),
tensor(14.0706), tensor(14.6027), tensor(12.9554), tensor(17.4448),
tensor(14.7823), tensor(15.4021), tensor(14.2423), tensor(14.3091),
tensor(13.4237), tensor(13.0780), tensor(14.1582), tensor(13.9569),
tensor(14.4233), tensor(14.0213), tensor(14.3115), tensor(14.3482),
tensor(14.0400), tensor(12.9896)]
[tensor(13.0490), tensor(11.4894), tensor(13.4138), tensor(11.9630),
tensor(11.9084), tensor(10.3814), tensor(12.1202), tensor(15.9817),
tensor(13.1341), tensor(12.8786), tensor(14.0188), tensor(12.4978),
tensor(11.8529), tensor(10.6737), tensor(12.9136), tensor(11.4730),
tensor(12.3935), tensor(13.5460), tensor(11.9616), tensor(12.3584),
```

```
tensor(11.4223), tensor(13.8786), tensor(15.3577), tensor(12.0487),
tensor(13.0466), tensor(12.8227), tensor(13.3991), tensor(14.4712),
tensor(13.9242), tensor(12.8122), tensor(10.4840), tensor(13.4863),
tensor(11.2029), tensor(13.0984), tensor(12.0160), tensor(16.4645),
tensor(12.3956), tensor(12.1377), tensor(13.4770), tensor(12.9169),
tensor(11.9544), tensor(11.4679), tensor(13.4277), tensor(12.2039),
tensor(12.5956), tensor(12.7412), tensor(13.3784), tensor(11.5110),
tensor(13.1439), tensor(12.5022)]
[tensor(10.9254), tensor(10.5517), tensor(12.5062), tensor(11.5342),
tensor(12.4054), tensor(8.8664), tensor(12.3634), tensor(14.3293),
tensor(12.0147), tensor(12.0763), tensor(13.1023), tensor(11.6698),
tensor(12.0940), tensor(9.4935), tensor(11.6919), tensor(12.2182),
tensor(13.6985), tensor(12.2193), tensor(10.6862), tensor(13.1579),
tensor(11.2374), tensor(13.9282), tensor(14.1619), tensor(10.5807),
tensor(13.1886), tensor(12.0352), tensor(12.7399), tensor(12.5774),
tensor(11.5907), tensor(13.2820), tensor(10.3559), tensor(13.3371),
tensor(11.5940), tensor(11.6716), tensor(11.9382), tensor(14.8241),
tensor(11.9625), tensor(12.0929), tensor(12.5052), tensor(12.6681),
tensor(11.2511), tensor(12.4973), tensor(11.8841), tensor(11.7607),
tensor(12.4952), tensor(11.7149), tensor(12.3294), tensor(10.9099),
tensor(12.1789), tensor(11.4049)]
[tensor(12.9646), tensor(13.7103), tensor(13.7410), tensor(12.6283),
tensor(13.7670), tensor(10.8278), tensor(13.9456), tensor(15.7061),
tensor(13.2746), tensor(15.9442), tensor(15.9204), tensor(13.0710),
tensor(13.3011), tensor(10.2341), tensor(12.7587), tensor(13.5694),
tensor(14.0840), tensor(14.4387), tensor(12.8567), tensor(14.2858),
tensor(12.5907), tensor(15.1070), tensor(14.6025), tensor(12.6549),
tensor(14.3497), tensor(13.2116), tensor(13.4342), tensor(14.6347),
tensor(13.9098), tensor(14.3656), tensor(13.4174), tensor(16.5174),
tensor(14.1986), tensor(13.4401), tensor(13.7264), tensor(16.1106),
tensor(14.3332), tensor(13.2067), tensor(15.1206), tensor(14.1012),
tensor(12.7432), tensor(13.6634), tensor(13.0535), tensor(13.0893),
tensor(13.8664), tensor(14.1530), tensor(13.0475), tensor(12.1284),
tensor(12.8538), tensor(11.6369)]
[tensor(13.2826), tensor(12.2407), tensor(12.8531), tensor(12.8992),
tensor(13.3859), tensor(11.0159), tensor(13.5591), tensor(16.6325),
tensor(13.2266), tensor(13.8445), tensor(16.4438), tensor(12.7801),
tensor(11.8789), tensor(11.0387), tensor(12.8510), tensor(13.2227),
tensor(13.2286), tensor(13.5949), tensor(12.0004), tensor(14.0665),
tensor(12.0246), tensor(14.1203), tensor(15.3843), tensor(11.5852),
tensor(13.3856), tensor(12.3756), tensor(14.2419), tensor(14.3196),
tensor(13.1502), tensor(13.7157), tensor(11.8327), tensor(15.4913),
tensor(13.7904), tensor(14.6788), tensor(11.6734), tensor(17.3472),
tensor(13.8617), tensor(12.8813), tensor(14.2542), tensor(13.6770),
tensor(12.5206), tensor(12.9695), tensor(13.5516), tensor(13.1698),
tensor(13.7374), tensor(13.6905), tensor(12.6968), tensor(13.6922),
tensor(12.4232), tensor(12.3777)]
[tensor(12.7103), tensor(12.7049), tensor(10.9807), tensor(13.5407),
```

```
tensor(13.7840), tensor(10.6190), tensor(13.0754), tensor(14.5211),
tensor(13.2895), tensor(13.6076), tensor(14.9771), tensor(13.3177),
tensor(12.5835), tensor(10.7714), tensor(11.6065), tensor(14.3256),
tensor(14.3328), tensor(13.7697), tensor(12.6972), tensor(14.1148),
tensor(12.3190), tensor(15.2549), tensor(14.4933), tensor(12.2856),
tensor(12.6623), tensor(12.9757), tensor(13.3390), tensor(14.2165),
tensor(13.4115), tensor(13.3227), tensor(12.2517), tensor(15.5522),
tensor(12.8113), tensor(14.4617), tensor(12.5074), tensor(15.5743),
tensor(13.1205), tensor(13.3539), tensor(13.4228), tensor(14.4444),
tensor(12.8164), tensor(12.1053), tensor(12.4613), tensor(12.8465),
tensor(12.5373), tensor(13.0082), tensor(12.7379), tensor(13.3357),
tensor(13.4895), tensor(13.3801)]
[tensor(12.7584), tensor(11.7427), tensor(13.2148), tensor(13.6898),
tensor(13.6888), tensor(10.7648), tensor(13.3688), tensor(16.9548),
tensor(13.9946), tensor(14.6563), tensor(15.0485), tensor(13.3988),
tensor(12.8109), tensor(11.8823), tensor(12.4359), tensor(13.0312),
tensor(14.5865), tensor(13.5153), tensor(12.8258), tensor(14.8218),
tensor(13.1677), tensor(15.8238), tensor(15.1904), tensor(12.5953),
tensor(14.2371), tensor(13.9551), tensor(12.0802), tensor(14.4611),
tensor(13.3182), tensor(14.5765), tensor(11.9726), tensor(14.2562),
tensor(13.3979), tensor(14.9430), tensor(13.1104), tensor(16.1609),
tensor(13.4730), tensor(14.0277), tensor(14.5587), tensor(13.7951),
tensor(14.3347), tensor(12.9464), tensor(13.6026), tensor(13.5135),
tensor(13.1463), tensor(12.9143), tensor(12.7219), tensor(12.8640),
tensor(14.3869), tensor(12.7473)]
[tensor(10.8632), tensor(11.8089), tensor(10.2186), tensor(11.5839),
tensor(11.2335), tensor(9.3470), tensor(10.5300), tensor(13.3773),
tensor(11.0389), tensor(12.1887), tensor(12.9123), tensor(11.2465),
tensor(12.4827), tensor(9.7679), tensor(11.0866), tensor(11.9379),
tensor(12.9346), tensor(10.9250), tensor(10.9255), tensor(12.7797),
tensor(10.9286), tensor(12.5975), tensor(14.0281), tensor(11.2823),
tensor(12.7127), tensor(11.6421), tensor(12.4816), tensor(12.4920),
tensor(10.4885), tensor(13.4698), tensor(10.4760), tensor(12.9787),
tensor(12.3927), tensor(12.4711), tensor(10.3524), tensor(13.1918),
tensor(11.3461), tensor(10.7481), tensor(11.4868), tensor(11.6020),
tensor(10.6576), tensor(9.9728), tensor(10.5705), tensor(11.3761),
tensor(11.3142), tensor(12.2272), tensor(10.8806), tensor(10.8398),
tensor(11.9184), tensor(11.4091)]
[tensor(13.9731), tensor(13.4279), tensor(13.5926), tensor(13.8616),
tensor(14.4148), tensor(12.1343), tensor(13.0762), tensor(17.3398),
tensor(14.2303), tensor(14.8757), tensor(15.9703), tensor(12.5702),
tensor(13.5546), tensor(11.1764), tensor(13.1596), tensor(13.0133),
tensor(14.4201), tensor(14.7977), tensor(13.0286), tensor(14.9213),
tensor(12.5236), tensor(14.9603), tensor(16.1369), tensor(12.9207),
tensor(14.7296), tensor(13.8130), tensor(14.7468), tensor(14.7799),
tensor(13.6261), tensor(14.4645), tensor(12.6776), tensor(15.2161),
tensor(13.7010), tensor(15.3794), tensor(12.7235), tensor(17.6477),
tensor(13.5118), tensor(13.9630), tensor(14.9351), tensor(14.2817),
```

```
tensor(11.9080), tensor(12.7532), tensor(12.9712), tensor(13.7164),
tensor(14.4892), tensor(14.0621), tensor(13.2045), tensor(13.6402),
tensor(13.4663), tensor(12.6066)]
[tensor(13.0168), tensor(11.1400), tensor(11.7969), tensor(12.1141),
tensor(11.3300), tensor(10.5265), tensor(12.3508), tensor(14.0562),
tensor(12.4571), tensor(14.6325), tensor(14.0317), tensor(10.9981),
tensor(12.8562), tensor(11.1223), tensor(11.6895), tensor(12.5982),
tensor(12.8260), tensor(12.8633), tensor(11.6399), tensor(13.4573),
tensor(11.3060), tensor(14.0696), tensor(14.8495), tensor(11.2638),
tensor(11.9270), tensor(10.9779), tensor(11.8645), tensor(13.6014),
tensor(12.0484), tensor(12.3680), tensor(11.2294), tensor(14.5193),
tensor(13.5434), tensor(12.9947), tensor(11.0428), tensor(14.3019),
tensor(11.3036), tensor(11.6115), tensor(12.8364), tensor(13.9674),
tensor(12.6256), tensor(11.2902), tensor(11.6907), tensor(12.5642),
tensor(13.4986), tensor(12.4088), tensor(11.6881), tensor(12.4528),
tensor(12.3722), tensor(11.1332)]
[tensor(13.5391), tensor(13.7319), tensor(13.3736), tensor(12.1247),
tensor(13.0642), tensor(10.8984), tensor(12.5922), tensor(16.1739),
tensor(13.8362), tensor(15.6868), tensor(16.1595), tensor(13.5288),
tensor(13.4782), tensor(11.6158), tensor(13.2341), tensor(13.7989),
tensor(14.4149), tensor(13.4545), tensor(13.1286), tensor(13.9234),
tensor(12.8911), tensor(15.2417), tensor(16.1476), tensor(12.7119),
tensor(14.4448), tensor(13.5246), tensor(14.1475), tensor(15.1380),
tensor(14.0143), tensor(14.7002), tensor(12.1471), tensor(15.6158),
tensor(14.5692), tensor(14.9030), tensor(13.1188), tensor(16.5465),
tensor(13.9804), tensor(12.9829), tensor(14.4846), tensor(13.6601),
tensor(12.1909), tensor(12.3126), tensor(13.6069), tensor(13.1090),
tensor(14.2799), tensor(15.0818), tensor(13.3855), tensor(13.3367),
tensor(13.9945), tensor(12.8816)]
[tensor(13.1456), tensor(12.3981), tensor(11.9412), tensor(12.8119),
tensor(13.6806), tensor(10.7708), tensor(12.9115), tensor(16.0972),
tensor(13.0700), tensor(14.0445), tensor(14.7611), tensor(13.1759),
tensor(12.8701), tensor(11.0036), tensor(13.2644), tensor(14.0356),
tensor(14.1430), tensor(15.2026), tensor(13.4300), tensor(14.1086),
tensor(12.5018), tensor(15.0222), tensor(14.5383), tensor(13.1227),
tensor(13.9949), tensor(13.6798), tensor(15.4357), tensor(14.5345),
tensor(13.9409), tensor(14.1130), tensor(11.7702), tensor(16.0750),
tensor(12.8737), tensor(14.6895), tensor(12.9278), tensor(17.3449),
tensor(13.4732), tensor(14.1569), tensor(13.9429), tensor(14.3886),
tensor(11.9829), tensor(12.1292), tensor(13.2856), tensor(13.1608),
tensor(13.8561), tensor(13.2726), tensor(13.4593), tensor(14.3143),
tensor(14.3771), tensor(13.6173)]
[tensor(12.0214), tensor(10.7625), tensor(11.7547), tensor(12.5341),
tensor(10.7161), tensor(9.7240), tensor(10.7252), tensor(13.9210),
tensor(11.5059), tensor(12.7549), tensor(13.0463), tensor(11.3439),
tensor(12.4089), tensor(9.8637), tensor(11.9293), tensor(11.6088),
tensor(12.0280), tensor(12.8206), tensor(11.5209), tensor(11.8858),
tensor(12.5562), tensor(14.2873), tensor(13.0525), tensor(12.6961),
```

```
tensor(12.1458), tensor(11.0144), tensor(12.7355), tensor(13.0102),
tensor(11.8152), tensor(12.9367), tensor(10.4772), tensor(12.9602),
tensor(12.3512), tensor(13.2170), tensor(11.3096), tensor(15.0682),
tensor(11.6702), tensor(11.5703), tensor(12.0943), tensor(12.4680),
tensor(10.9104), tensor(10.8690), tensor(12.6131), tensor(12.7580),
tensor(12.1275), tensor(12.8828), tensor(12.4575), tensor(11.4908),
tensor(10.7851), tensor(10.7727)]
[tensor(11.7110), tensor(11.3645), tensor(12.4066), tensor(12.6662),
tensor(12.9308), tensor(9.7021), tensor(11.3547), tensor(14.8319),
tensor(12.7936), tensor(13.4188), tensor(14.1926), tensor(11.5281),
tensor(10.8705), tensor(10.1887), tensor(11.6120), tensor(12.4283),
tensor(13.6127), tensor(12.9453), tensor(11.8572), tensor(14.2632),
tensor(11.5387), tensor(14.2969), tensor(14.1955), tensor(10.9695),
tensor(12.7261), tensor(10.9636), tensor(12.0723), tensor(13.1042),
tensor(12.1151), tensor(12.2408), tensor(11.8702), tensor(13.7617),
tensor(12.5057), tensor(12.8946), tensor(12.4252), tensor(15.2134),
tensor(12.2889), tensor(13.2069), tensor(12.0982), tensor(13.3007),
tensor(11.5549), tensor(11.8527), tensor(11.8776), tensor(12.5193),
tensor(13.5915), tensor(11.2816), tensor(12.1967), tensor(12.7422),
tensor(12.6485), tensor(11.8739)]
[tensor(13.2301), tensor(12.2507), tensor(12.6204), tensor(13.5628),
tensor(14.0408), tensor(11.9611), tensor(13.1822), tensor(15.3596),
tensor(12.2920), tensor(14.5943), tensor(14.7568), tensor(12.9367),
tensor(12.0263), tensor(10.4935), tensor(11.7366), tensor(13.0061),
tensor(12.4871), tensor(13.6420), tensor(12.2626), tensor(14.9048),
tensor(11.1508), tensor(14.0378), tensor(14.2685), tensor(11.3986),
tensor(13.1551), tensor(12.1778), tensor(13.6407), tensor(13.2014),
tensor(12.7132), tensor(13.6689), tensor(12.3456), tensor(15.2002),
tensor(13.4258), tensor(15.0593), tensor(11.4833), tensor(15.8638),
tensor(13.5246), tensor(13.1354), tensor(15.1552), tensor(14.3784),
tensor(11.8703), tensor(12.4458), tensor(13.0061), tensor(13.0268),
tensor(13.5273), tensor(13.6950), tensor(11.3056), tensor(12.3200),
tensor(12.4205), tensor(12.5810)]
[tensor(15.0292), tensor(13.0029), tensor(13.3591), tensor(14.4720),
tensor(14.5207), tensor(11.6835), tensor(13.1526), tensor(17.4701),
tensor(14.5168), tensor(14.7034), tensor(15.8592), tensor(13.5182),
tensor(14.4537), tensor(12.1636), tensor(13.9136), tensor(14.3808),
tensor(14.0534), tensor(15.2390), tensor(13.3866), tensor(16.1630),
tensor(12.1807), tensor(16.1122), tensor(17.7758), tensor(13.3717),
tensor(15.1103), tensor(13.5375), tensor(15.8439), tensor(15.6114),
tensor(14.3326), tensor(14.7811), tensor(12.7203), tensor(15.3520),
tensor(14.0770), tensor(15.5772), tensor(12.8251), tensor(18.2063),
tensor(14.0567), tensor(14.9235), tensor(15.2398), tensor(16.2237),
tensor(13.9255), tensor(13.0751), tensor(14.5613), tensor(13.5273),
tensor(15.6196), tensor(14.5223), tensor(14.4207), tensor(14.3713),
tensor(14.5726), tensor(13.6218)]
[tensor(11.4151), tensor(9.6569), tensor(11.9738), tensor(9.9507),
tensor(11.4411), tensor(8.8555), tensor(10.9079), tensor(12.6913),
```

```
tensor(11.8836), tensor(10.7152), tensor(11.2665), tensor(10.5578),
tensor(10.2951), tensor(9.0304), tensor(10.7503), tensor(10.4987),
tensor(11.7366), tensor(10.7432), tensor(10.3924), tensor(11.2763),
tensor(10.7898), tensor(11.4601), tensor(12.5621), tensor(10.6032),
tensor(11.0428), tensor(10.4548), tensor(11.0163), tensor(11.6158),
tensor(12.5448), tensor(11.9459), tensor(9.5892), tensor(12.6294),
tensor(9.5197), tensor(11.9114), tensor(10.3753), tensor(13.5389),
tensor(10.6435), tensor(11.2035), tensor(11.2378), tensor(11.8742),
tensor(10.3039), tensor(11.4057), tensor(11.4593), tensor(10.1137),
tensor(10.6452), tensor(10.7686), tensor(11.3930), tensor(9.9574),
tensor(11.6435), tensor(10.2306)]
[tensor(11.6236), tensor(12.6009), tensor(12.8085), tensor(13.5181),
tensor(13.1109), tensor(10.2350), tensor(12.6676), tensor(15.4532),
tensor(13.5817), tensor(12.7451), tensor(14.9638), tensor(13.0992),
tensor(12.5887), tensor(10.6990), tensor(12.5061), tensor(13.0421),
tensor(13.3908), tensor(14.1896), tensor(12.0283), tensor(14.1937),
tensor(12.5749), tensor(13.3081), tensor(14.8082), tensor(12.5799),
tensor(14.2739), tensor(12.1836), tensor(13.5597), tensor(13.5163),
tensor(12.7019), tensor(14.1328), tensor(10.8964), tensor(14.8644),
tensor(13.9376), tensor(14.3459), tensor(12.9823), tensor(16.2367),
tensor(13.1462), tensor(13.0193), tensor(12.1359), tensor(13.4587),
tensor(11.3324), tensor(11.7399), tensor(13.7894), tensor(14.0902),
tensor(12.7901), tensor(13.2190), tensor(12.5800), tensor(12.1536),
tensor(14.1590), tensor(12.7836)]
[tensor(12.3524), tensor(11.9057), tensor(14.4040), tensor(13.5022),
tensor(14.2938), tensor(11.2271), tensor(13.7901), tensor(15.5948),
tensor(14.0740), tensor(13.7534), tensor(15.7721), tensor(13.9796),
tensor(13.5789), tensor(11.6293), tensor(12.8296), tensor(13.8739),
tensor(13.7532), tensor(13.8697), tensor(12.6812), tensor(15.4428),
tensor(12.3747), tensor(15.1755), tensor(14.6625), tensor(12.7043),
tensor(13.1094), tensor(13.7014), tensor(13.0177), tensor(13.4588),
tensor(13.1010), tensor(13.5184), tensor(11.8263), tensor(15.5570),
tensor(12.6118), tensor(15.0177), tensor(12.7063), tensor(16.4784),
tensor(12.8065), tensor(12.9700), tensor(14.2615), tensor(14.0630),
tensor(12.3283), tensor(12.7587), tensor(12.3340), tensor(13.7410),
tensor(14.7298), tensor(14.0857), tensor(12.9853), tensor(13.4960),
tensor(13.4474), tensor(13.9951)]
[tensor(11.8257), tensor(10.6081), tensor(11.4255), tensor(11.6658),
tensor(11.3318), tensor(9.2186), tensor(11.2718), tensor(13.3737),
tensor(11.2275), tensor(12.6381), tensor(13.7944), tensor(11.8726),
tensor(12.1165), tensor(9.4717), tensor(11.0942), tensor(12.2176),
tensor(12.5618), tensor(11.7515), tensor(10.7533), tensor(13.1003),
tensor(11.3136), tensor(13.3854), tensor(12.9687), tensor(10.7656),
tensor(12.5515), tensor(11.4654), tensor(13.3093), tensor(12.6931),
tensor(12.2001), tensor(12.8032), tensor(11.9224), tensor(14.1947),
tensor(12.2658), tensor(12.1174), tensor(10.2614), tensor(14.5230),
tensor(12.4254), tensor(10.6453), tensor(12.5748), tensor(13.5359),
tensor(11.0915), tensor(12.4520), tensor(12.0424), tensor(11.0328),
```

```
tensor(10.6982), tensor(12.6112), tensor(11.1237), tensor(11.2495),
tensor(11.6146), tensor(11.4033)]
[tensor(12.3720), tensor(10.6155), tensor(12.8808), tensor(13.1886),
tensor(12.3792), tensor(10.0077), tensor(10.7970), tensor(14.7495),
tensor(12.8466), tensor(12.5619), tensor(13.6451), tensor(11.9996),
tensor(13.5783), tensor(10.3347), tensor(12.2953), tensor(13.3229),
tensor(13.2987), tensor(12.6774), tensor(11.1542), tensor(14.9720),
tensor(11.1580), tensor(13.7623), tensor(15.2981), tensor(11.9838),
tensor(13.4470), tensor(11.9244), tensor(14.1912), tensor(14.2699),
tensor(12.8590), tensor(13.6937), tensor(11.4001), tensor(13.2375),
tensor(12.5247), tensor(11.8181), tensor(12.1876), tensor(15.2265),
tensor(11.2288), tensor(12.1669), tensor(13.2090), tensor(14.1257),
tensor(11.2530), tensor(12.4639), tensor(12.0269), tensor(11.9906),
tensor(12.8432), tensor(13.3855), tensor(13.7072), tensor(12.8025),
tensor(13.0402), tensor(12.7037)]
[tensor(11.5827), tensor(11.6440), tensor(10.8815), tensor(12.8569),
tensor(12.5554), tensor(10.0693), tensor(11.5355), tensor(14.2078),
tensor(11.8417), tensor(13.1740), tensor(13.9625), tensor(11.5215),
tensor(11.6486), tensor(10.3369), tensor(10.4912), tensor(12.5428),
tensor(13.1407), tensor(11.4105), tensor(11.3482), tensor(14.0728),
tensor(10.6643), tensor(14.6233), tensor(14.3327), tensor(9.9360),
tensor(11.3620), tensor(11.4197), tensor(12.4739), tensor(14.0293),
tensor(12.6183), tensor(12.4824), tensor(11.6249), tensor(14.3414),
tensor(12.1908), tensor(12.8419), tensor(11.4421), tensor(14.3547),
tensor(12.6106), tensor(12.2109), tensor(13.0152), tensor(13.9527),
tensor(11.7861), tensor(10.7641), tensor(11.7799), tensor(12.3109),
tensor(12.3382), tensor(12.3575), tensor(12.0520), tensor(12.3189),
tensor(12.3344), tensor(12.6474)]
[tensor(11.9090), tensor(11.3656), tensor(12.2027), tensor(12.6011),
tensor(11.6123), tensor(10.6493), tensor(11.3715), tensor(14.8691),
tensor(12.8258), tensor(13.3685), tensor(14.8430), tensor(13.0082),
tensor(13.6002), tensor(10.3846), tensor(12.4057), tensor(14.2325),
tensor(13.8336), tensor(13.4800), tensor(11.1982), tensor(14.4401),
tensor(11.2216), tensor(14.1394), tensor(16.0757), tensor(11.6435),
tensor(13.9406), tensor(12.7107), tensor(13.8212), tensor(14.3373),
tensor(13.8976), tensor(14.2058), tensor(12.4720), tensor(14.1208),
tensor(13.5807), tensor(12.6906), tensor(12.6367), tensor(16.5231),
tensor(11.8817), tensor(12.3916), tensor(13.7049), tensor(12.9734),
tensor(11.8416), tensor(11.1319), tensor(12.0360), tensor(12.9288),
tensor(12.8378), tensor(12.5264), tensor(12.3563), tensor(13.4012),
tensor(13.4840), tensor(13.0535)]
[tensor(12.8486), tensor(11.6915), tensor(12.9796), tensor(11.7398),
tensor(12.4748), tensor(10.0062), tensor(11.9277), tensor(15.0493),
tensor(12.9538), tensor(13.0194), tensor(14.1800), tensor(12.6008),
tensor(12.0379), tensor(9.6261), tensor(13.0585), tensor(12.7946),
tensor(13.7099), tensor(13.0898), tensor(12.7413), tensor(14.4626),
tensor(10.9557), tensor(13.0391), tensor(15.4205), tensor(11.4115),
tensor(12.9777), tensor(12.2584), tensor(13.4731), tensor(13.2956),
```

```
tensor(12.3892), tensor(13.4338), tensor(11.9354), tensor(14.1114),
tensor(12.7007), tensor(13.0224), tensor(11.6777), tensor(15.2337),
tensor(11.5443), tensor(12.2811), tensor(13.3018), tensor(14.0959),
tensor(11.5998), tensor(12.2345), tensor(12.0111), tensor(11.0485),
tensor(13.4023), tensor(13.0063), tensor(11.9144), tensor(11.6393),
tensor(12.9170), tensor(12.8144)]
[tensor(12.6896), tensor(12.2218), tensor(11.8858), tensor(12.4418),
tensor(13.5887), tensor(10.9758), tensor(13.3274), tensor(15.8855),
tensor(13.4803), tensor(14.1772), tensor(14.5421), tensor(12.5761),
tensor(11.9628), tensor(10.6416), tensor(12.2628), tensor(12.6989),
tensor(14.9478), tensor(13.1603), tensor(12.4876), tensor(14.0198),
tensor(12.1993), tensor(13.8831), tensor(15.5597), tensor(12.5962),
tensor(13.2263), tensor(13.1041), tensor(13.7308), tensor(14.0898),
tensor(13.3491), tensor(14.3593), tensor(11.6855), tensor(15.8679),
tensor(13.8769), tensor(14.6771), tensor(12.8584), tensor(16.4694),
tensor(13.2455), tensor(13.4538), tensor(13.5310), tensor(13.7845),
tensor(13.2703), tensor(11.6510), tensor(13.4057), tensor(13.0962),
tensor(13.8715), tensor(12.5982), tensor(12.3677), tensor(12.5058),
tensor(13.9899), tensor(11.7464)]
[tensor(11.9487), tensor(11.6959), tensor(12.5032), tensor(11.8972),
tensor(12.6701), tensor(9.5678), tensor(12.4126), tensor(14.5271),
tensor(13.0633), tensor(11.7592), tensor(14.0148), tensor(13.0474),
tensor(12.6304), tensor(10.9407), tensor(12.7254), tensor(12.8297),
tensor(11.6736), tensor(12.6914), tensor(12.1243), tensor(13.3113),
tensor(11.5163), tensor(13.7910), tensor(14.2975), tensor(12.2847),
tensor(12.9053), tensor(13.0819), tensor(12.8767), tensor(12.2874),
tensor(12.1765), tensor(13.2612), tensor(10.2277), tensor(14.1255),
tensor(11.7397), tensor(14.1432), tensor(11.9708), tensor(14.8143),
tensor(12.6521), tensor(13.3404), tensor(12.3673), tensor(13.2348),
tensor(11.5865), tensor(12.4215), tensor(13.3665), tensor(12.0456),
tensor(12.3727), tensor(13.2108), tensor(12.6081), tensor(11.8050),
tensor(12.8087), tensor(12.4869)]
[tensor(11.5302), tensor(11.8295), tensor(10.7801), tensor(11.5240),
tensor(11.7477), tensor(9.6172), tensor(11.3287), tensor(13.8028),
tensor(11.9164), tensor(11.6597), tensor(13.5111), tensor(11.9000),
tensor(11.1788), tensor(9.8949), tensor(12.0190), tensor(12.6840),
tensor(12.5351), tensor(12.8197), tensor(11.7448), tensor(12.2780),
tensor(10.4559), tensor(14.2504), tensor(13.8065), tensor(11.0052),
tensor(12.0082), tensor(11.3121), tensor(12.3698), tensor(13.2333),
tensor(11.7613), tensor(11.7481), tensor(10.6601), tensor(13.6329),
tensor(11.2478), tensor(14.1963), tensor(11.3490), tensor(14.7317),
tensor(11.3796), tensor(12.2010), tensor(11.2227), tensor(13.2504),
tensor(10.8590), tensor(10.3439), tensor(12.9656), tensor(11.6526),
tensor(11.4416), tensor(11.9739), tensor(11.6775), tensor(11.6169),
tensor(11.8516), tensor(12.4831)]
[tensor(14.4543), tensor(12.6711), tensor(13.7501), tensor(13.6895),
tensor(13.3207), tensor(11.6521), tensor(13.1045), tensor(16.7289),
tensor(14.2455), tensor(15.5009), tensor(14.9362), tensor(15.1471),
```

```
tensor(14.3591), tensor(12.5991), tensor(14.0000), tensor(14.0073),
tensor(15.5939), tensor(15.3600), tensor(13.8045), tensor(15.5912),
tensor(13.3656), tensor(15.5041), tensor(16.6526), tensor(14.1743),
tensor(15.2099), tensor(14.4350), tensor(14.9735), tensor(15.1656),
tensor(14.4534), tensor(14.7978), tensor(12.3612), tensor(15.4641),
tensor(14.7169), tensor(14.7522), tensor(13.1213), tensor(17.3071),
tensor(13.6100), tensor(13.0806), tensor(14.0537), tensor(14.4165),
tensor(13.3808), tensor(12.4493), tensor(13.9211), tensor(13.8313),
tensor(14.0707), tensor(14.5408), tensor(14.1641), tensor(13.1122),
tensor(15.0127), tensor(14.4110)]
[tensor(12.7443), tensor(12.1324), tensor(11.2904), tensor(11.4328),
tensor(12.1647), tensor(10.1096), tensor(11.9226), tensor(14.5810),
tensor(12.0179), tensor(14.0687), tensor(15.0366), tensor(10.9300),
tensor(12.3489), tensor(10.3559), tensor(12.4460), tensor(13.2250),
tensor(12.7017), tensor(13.4977), tensor(11.7354), tensor(11.6901),
tensor(10.2346), tensor(13.1612), tensor(14.7735), tensor(11.0620),
tensor(12.4743), tensor(12.0090), tensor(14.3831), tensor(14.2806),
tensor(12.1902), tensor(13.3944), tensor(11.3633), tensor(15.4586),
tensor(13.1089), tensor(13.3983), tensor(12.2935), tensor(15.6170),
tensor(12.2040), tensor(12.0522), tensor(14.0905), tensor(13.0223),
tensor(11.5658), tensor(11.5269), tensor(11.7089), tensor(10.9486),
tensor(12.5958), tensor(13.7046), tensor(11.6776), tensor(12.3509),
tensor(12.2347), tensor(12.0273)]
[tensor(12.7502), tensor(12.0954), tensor(10.9362), tensor(11.7243),
tensor(12.1558), tensor(10.0687), tensor(12.1079), tensor(14.9770),
tensor(11.9760), tensor(12.9280), tensor(13.0357), tensor(11.8633),
tensor(11.2963), tensor(10.8876), tensor(11.8895), tensor(11.7323),
tensor(12.4379), tensor(13.5425), tensor(12.2613), tensor(12.4486),
tensor(10.8210), tensor(13.7990), tensor(14.3465), tensor(11.4642),
tensor(12.5960), tensor(11.5723), tensor(12.6558), tensor(13.1450),
tensor(11.4601), tensor(11.6985), tensor(9.4946), tensor(13.8710),
tensor(11.7824), tensor(13.2935), tensor(10.6548), tensor(14.0824),
tensor(12.4905), tensor(12.2735), tensor(11.1377), tensor(12.6527),
tensor(11.6931), tensor(10.5882), tensor(12.5814), tensor(12.3081),
tensor(12.6033), tensor(11.2481), tensor(11.8845), tensor(11.8571),
tensor(12.0192), tensor(12.1820)]
[tensor(14.3935), tensor(14.2294), tensor(12.0594), tensor(12.2980),
tensor(13.7523), tensor(11.7347), tensor(13.7627), tensor(16.4982),
tensor(13.0750), tensor(13.9301), tensor(15.5813), tensor(13.9638),
tensor(12.4442), tensor(11.6852), tensor(12.5172), tensor(14.2820),
tensor(14.3497), tensor(14.7470), tensor(14.9812), tensor(14.1276),
tensor(11.9510), tensor(15.4176), tensor(16.1810), tensor(12.2817),
tensor(13.1524), tensor(13.1416), tensor(15.1556), tensor(15.6344),
tensor(13.3635), tensor(14.8870), tensor(12.8844), tensor(15.4573),
tensor(13.0054), tensor(15.8589), tensor(11.9427), tensor(16.8089),
tensor(14.2657), tensor(14.7162), tensor(14.4381), tensor(15.0139),
tensor(12.7488), tensor(11.0544), tensor(13.5238), tensor(12.9156),
tensor(14.3767), tensor(13.4687), tensor(13.6528), tensor(12.8126),
```

```
tensor(13.8029), tensor(13.5258)]
[tensor(11.6583), tensor(12.4739), tensor(11.6610), tensor(11.4709),
tensor(12.5112), tensor(9.4320), tensor(12.2692), tensor(13.4948),
tensor(12.4520), tensor(13.5895), tensor(14.0501), tensor(11.4230),
tensor(11.7156), tensor(11.0841), tensor(10.8362), tensor(12.5808),
tensor(12.7836), tensor(13.1687), tensor(11.8496), tensor(13.1830),
tensor(10.7926), tensor(12.8830), tensor(13.5452), tensor(12.4536),
tensor(13.7393), tensor(10.7173), tensor(11.9083), tensor(13.1169),
tensor(12.3650), tensor(13.4191), tensor(11.1912), tensor(14.2717),
tensor(12.4096), tensor(12.2854), tensor(12.1129), tensor(14.3840),
tensor(12.4426), tensor(12.8522), tensor(11.7487), tensor(11.7785),
tensor(10.9022), tensor(10.6822), tensor(11.5449), tensor(12.4700),
tensor(13.3260), tensor(12.2270), tensor(11.7487), tensor(11.6675),
tensor(13.5742), tensor(10.7352)]
[tensor(11.5629), tensor(10.6551), tensor(12.1465), tensor(10.9872),
tensor(10.8944), tensor(8.4344), tensor(11.1888), tensor(12.5304),
tensor(11.1976), tensor(12.3801), tensor(12.1199), tensor(11.2111),
tensor(13.0458), tensor(8.6420), tensor(11.8474), tensor(12.1184),
tensor(12.0206), tensor(12.6564), tensor(10.1835), tensor(13.4715),
tensor(10.3870), tensor(12.5986), tensor(13.1423), tensor(10.8001),
tensor(12.7131), tensor(11.3767), tensor(10.6633), tensor(12.1933),
tensor(11.7294), tensor(11.7564), tensor(10.0574), tensor(13.1096),
tensor(11.2033), tensor(11.3122), tensor(11.2376), tensor(13.8243),
tensor(10.0901), tensor(9.9866), tensor(11.7706), tensor(13.2688),
tensor(10.0383), tensor(11.3142), tensor(11.5011), tensor(10.9295),
tensor(12.4034), tensor(11.4734), tensor(10.7216), tensor(11.0059),
tensor(11.7394), tensor(11.1862)]
[tensor(11.9226), tensor(11.1355), tensor(12.4132), tensor(11.2933),
tensor(12.2621), tensor(9.8457), tensor(12.8368), tensor(14.7555),
tensor(12.6133), tensor(12.7974), tensor(13.3571), tensor(13.0823),
tensor(11.4835), tensor(9.3386), tensor(12.5749), tensor(12.5687),
tensor(14.0249), tensor(13.5774), tensor(12.5606), tensor(13.2053),
tensor(11.7978), tensor(13.2116), tensor(13.1305), tensor(12.0164),
tensor(12.1407), tensor(13.0948), tensor(11.6337), tensor(13.1411),
tensor(13.1887), tensor(11.8700), tensor(10.5159), tensor(14.9981),
tensor(11.7060), tensor(13.3726), tensor(12.0466), tensor(15.2857),
tensor(11.2823), tensor(11.8566), tensor(12.8698), tensor(13.1920),
tensor(11.6461), tensor(11.3399), tensor(11.8786), tensor(11.1861),
tensor(12.4378), tensor(12.0981), tensor(11.3593), tensor(12.2772),
tensor(12.8922), tensor(12.3772)]
[tensor(13.1229), tensor(12.6480), tensor(12.8873), tensor(12.4487),
tensor(13.3879), tensor(10.2112), tensor(13.1808), tensor(15.6190),
tensor(13.4512), tensor(12.8218), tensor(17.0230), tensor(13.4265),
tensor(12.7219), tensor(11.3242), tensor(12.1918), tensor(13.6304),
tensor(14.4452), tensor(14.6360), tensor(11.8495), tensor(13.1577),
tensor(12.4987), tensor(14.8016), tensor(15.8718), tensor(11.3491),
tensor(13.1739), tensor(12.5727), tensor(14.0017), tensor(15.2372),
tensor(12.7219), tensor(12.5323), tensor(11.5960), tensor(14.3916),
```

```
tensor(12.5587), tensor(13.3770), tensor(11.6300), tensor(16.3268),
tensor(13.8082), tensor(13.2676), tensor(13.1156), tensor(13.4966),
tensor(13.4869), tensor(12.9024), tensor(12.2876), tensor(13.7021),
tensor(13.7300), tensor(12.3659), tensor(13.4385), tensor(13.3988),
tensor(14.5550), tensor(13.6916)]
[tensor(12.4107), tensor(10.2719), tensor(13.1313), tensor(13.3032),
tensor(13.0480), tensor(10.0106), tensor(11.5404), tensor(16.3151),
tensor(12.1672), tensor(13.4488), tensor(13.8654), tensor(12.4660),
tensor(11.9030), tensor(10.6737), tensor(11.7679), tensor(11.9642),
tensor(12.8438), tensor(12.8492), tensor(12.6265), tensor(14.7028),
tensor(11.5409), tensor(14.1599), tensor(13.8538), tensor(11.5618),
tensor(12.5794), tensor(12.6437), tensor(12.5980), tensor(12.6818),
tensor(12.9378), tensor(14.2218), tensor(11.6640), tensor(14.4619),
tensor(13.7410), tensor(13.1415), tensor(11.5837), tensor(16.9908),
tensor(12.3516), tensor(12.2602), tensor(14.4658), tensor(13.5713),
tensor(12.2592), tensor(13.2441), tensor(12.0332), tensor(11.9651),
tensor(12.8695), tensor(12.7835), tensor(11.9540), tensor(12.7933),
tensor(11.9014), tensor(11.8114)]
[tensor(11.2807), tensor(10.6302), tensor(11.8643), tensor(11.4993),
tensor(11.9023), tensor(10.0003), tensor(12.0066), tensor(14.1489),
tensor(11.8912), tensor(12.5193), tensor(13.0038), tensor(12.1956),
tensor(11.7739), tensor(9.7661), tensor(11.8899), tensor(12.2165),
tensor(13.5653), tensor(12.2500), tensor(11.1836), tensor(13.2175),
tensor(11.2412), tensor(12.3857), tensor(13.8687), tensor(11.0995),
tensor(13.5669), tensor(12.0395), tensor(12.1498), tensor(12.7792),
tensor(13.0189), tensor(12.7885), tensor(10.4135), tensor(13.2151),
tensor(11.9102), tensor(12.4312), tensor(11.2750), tensor(14.6342),
tensor(11.3826), tensor(11.3791), tensor(12.5988), tensor(12.5166),
tensor(10.5932), tensor(10.2073), tensor(11.8258), tensor(11.5652),
tensor(11.4620), tensor(12.2338), tensor(11.2420), tensor(11.2291),
tensor(13.1594), tensor(11.9412)]
[tensor(13.4003), tensor(12.1566), tensor(13.0927), tensor(12.7829),
tensor(12.1047), tensor(10.4968), tensor(10.9485), tensor(16.6247),
tensor(12.2158), tensor(14.5370), tensor(14.0831), tensor(12.7450),
tensor(12.4857), tensor(10.5726), tensor(12.4640), tensor(13.4149),
tensor(13.5719), tensor(14.2213), tensor(12.0092), tensor(14.5289),
tensor(11.2926), tensor(14.8499), tensor(14.8987), tensor(12.1671),
tensor(14.3959), tensor(12.5387), tensor(13.7180), tensor(14.3450),
tensor(12.5316), tensor(14.3115), tensor(12.6233), tensor(13.9387),
tensor(13.2389), tensor(13.0601), tensor(12.4268), tensor(16.2819),
tensor(12.7596), tensor(12.8927), tensor(14.2051), tensor(13.4653),
tensor(12.3820), tensor(12.4601), tensor(12.3206), tensor(11.5054),
tensor(12.9403), tensor(13.2101), tensor(12.0839), tensor(12.1365),
tensor(11.8876), tensor(12.4316)]
[tensor(11.8645), tensor(12.4082), tensor(12.1819), tensor(11.1931),
tensor(11.8240), tensor(9.3270), tensor(11.9643), tensor(15.2473),
tensor(11.6089), tensor(13.7819), tensor(15.2553), tensor(13.5723),
tensor(11.4900), tensor(10.5028), tensor(11.3464), tensor(12.4650),
```

```
tensor(13.8288), tensor(12.6559), tensor(11.5698), tensor(13.2233),
tensor(10.9444), tensor(14.3640), tensor(14.9592), tensor(11.2073),
tensor(13.7082), tensor(13.0650), tensor(13.0526), tensor(14.2435),
tensor(12.2312), tensor(12.6387), tensor(11.4800), tensor(14.0527),
tensor(13.0179), tensor(11.8993), tensor(11.2451), tensor(15.9963),
tensor(12.8444), tensor(11.1960), tensor(13.2613), tensor(12.4623),
tensor(11.9111), tensor(10.8570), tensor(12.1080), tensor(12.5777),
tensor(13.2667), tensor(13.1685), tensor(11.7892), tensor(11.2981),
tensor(13.4922), tensor(12.4427)]
[tensor(10.4005), tensor(11.8448), tensor(11.4662), tensor(10.8797),
tensor(12.8062), tensor(10.9606), tensor(11.9942), tensor(11.4981),
tensor(10.8708), tensor(12.0971), tensor(10.4839), tensor(14.0606),
tensor(12.1664), tensor(13.4828), tensor(11.8326), tensor(9.9833),
tensor(12.2755), tensor(9.8198), tensor(11.8286), tensor(12.2459),
tensor(11.6223), tensor(13.0127), tensor(12.0943), tensor(10.3491),
tensor(10.6214), tensor(12.7585), tensor(11.8872), tensor(12.6138),
tensor(11.2343), tensor(10.8090), tensor(13.1903), tensor(10.0194),
tensor(11.2926), tensor(13.0589), tensor(12.2159), tensor(11.8523),
tensor(11.1190), tensor(12.3382), tensor(11.9009), tensor(12.4843),
tensor(11.9594), tensor(11.3948), tensor(12.4358), tensor(9.7108),
tensor(10.4467), tensor(12.1676), tensor(10.9352), tensor(10.8316),
tensor(11.8114), tensor(10.8878)]
[tensor(11.6979), tensor(12.4197), tensor(11.7879), tensor(13.4477),
tensor(13.6366), tensor(12.3044), tensor(12.7364), tensor(13.4915),
tensor(13.2000), tensor(13.4383), tensor(11.6047), tensor(15.1640),
tensor(12.9652), tensor(12.9690), tensor(11.4157), tensor(11.6945),
tensor(12.4033), tensor(11.7738), tensor(13.7063), tensor(12.7952),
tensor(12.7330), tensor(12.9896), tensor(12.3512), tensor(10.9320),
tensor(10.6692), tensor(13.3211), tensor(11.6913), tensor(12.8988),
tensor(12.3152), tensor(11.0197), tensor(13.5950), tensor(10.2827),
tensor(10.7099), tensor(14.0580), tensor(13.9684), tensor(12.8766),
tensor(11.9982), tensor(12.1877), tensor(11.4168), tensor(12.1640),
tensor(13.4997), tensor(11.3373), tensor(12.7535), tensor(9.9923),
tensor(11.9068), tensor(12.9265), tensor(10.8013), tensor(12.6889),
tensor(12.6160), tensor(10.3181)]
[tensor(11.2150), tensor(12.7028), tensor(12.3171), tensor(13.7601),
tensor(14.5987), tensor(13.1525), tensor(14.3115), tensor(13.1120),
tensor(13.5713), tensor(13.2489), tensor(12.5058), tensor(15.3338),
tensor(12.9388), tensor(13.7240), tensor(12.6687), tensor(11.8226),
tensor(13.5456), tensor(11.7351), tensor(13.8066), tensor(13.5246),
tensor(12.8311), tensor(13.8435), tensor(13.3283), tensor(12.0890),
tensor(12.2634), tensor(14.6093), tensor(13.0087), tensor(14.1549),
tensor(12.9165), tensor(11.8667), tensor(14.6970), tensor(10.7417),
tensor(12.2343), tensor(14.7270), tensor(14.5352), tensor(12.3157),
tensor(12.4965), tensor(12.6599), tensor(12.8653), tensor(12.9659),
tensor(13.0429), tensor(12.5970), tensor(12.5686), tensor(10.9921),
tensor(13.0496), tensor(14.4427), tensor(11.5451), tensor(12.9321),
tensor(12.9491), tensor(11.1953)]
```

```
[tensor(13.3509), tensor(13.6984), tensor(12.3457), tensor(13.0298),
tensor(15.3064), tensor(13.1021), tensor(12.6123), tensor(12.4197),
tensor(13.5109), tensor(12.7938), tensor(12.0670), tensor(15.5795),
tensor(12.8829), tensor(13.0461), tensor(12.7302), tensor(12.1562),
tensor(14.8554), tensor(11.1330), tensor(11.8924), tensor(11.6945),
tensor(13.2458), tensor(14.1354), tensor(12.6308), tensor(12.7442),
tensor(12.9395), tensor(14.5050), tensor(11.8304), tensor(13.2360),
tensor(12.8781), tensor(11.6314), tensor(13.9984), tensor(11.9745),
tensor(12.0092), tensor(13.8202), tensor(14.0372), tensor(12.8227),
tensor(11.8452), tensor(13.0320), tensor(12.4638), tensor(12.5238),
tensor(13.0983), tensor(12.3302), tensor(13.1567), tensor(10.4313),
tensor(13.8597), tensor(13.3812), tensor(12.1383), tensor(13.4280),
tensor(13.7598), tensor(10.6614)]
[tensor(10.6668), tensor(11.3114), tensor(11.0647), tensor(11.7835),
tensor(14.0598), tensor(10.0438), tensor(10.7368), tensor(10.1633),
tensor(11.8011), tensor(11.0173), tensor(9.1685), tensor(13.0049),
tensor(12.1118), tensor(11.8963), tensor(11.1098), tensor(10.6237),
tensor(12.7070), tensor(9.9822), tensor(11.0240), tensor(11.0381),
tensor(11.9916), tensor(12.0350), tensor(11.3035), tensor(10.8911),
tensor(10.8242), tensor(13.0503), tensor(10.7514), tensor(11.2741),
tensor(11.3520), tensor(11.0494), tensor(12.9081), tensor(9.6915),
tensor(10.3000), tensor(11.3097), tensor(12.3750), tensor(11.2660),
tensor(9.4269), tensor(10.4710), tensor(10.7275), tensor(10.0317),
tensor(11.7899), tensor(11.1827), tensor(11.8292), tensor(10.4237),
tensor(11.8360), tensor(12.2885), tensor(10.0542), tensor(11.1863),
tensor(10.4230), tensor(10.3594)]
[tensor(11.1195), tensor(11.6522), tensor(12.0807), tensor(12.2793),
tensor(13.7474), tensor(12.0269), tensor(11.2631), tensor(11.6201),
tensor(11.5793), tensor(10.9542), tensor(10.1513), tensor(12.0014),
tensor(12.3842), tensor(12.4593), tensor(10.5801), tensor(10.4256),
tensor(11.7935), tensor(9.8570), tensor(11.7630), tensor(10.3157),
tensor(10.7774), tensor(12.9261), tensor(11.1186), tensor(10.0856),
tensor(11.4702), tensor(12.3897), tensor(10.6568), tensor(12.3587),
tensor(12.1467), tensor(10.3336), tensor(12.7766), tensor(9.1201),
tensor(10.6944), tensor(13.1399), tensor(12.6949), tensor(11.1813),
tensor(11.2228), tensor(11.4381), tensor(10.6344), tensor(11.3750),
tensor(12.0574), tensor(11.7782), tensor(12.5602), tensor(9.6802),
tensor(12.6313), tensor(12.4300), tensor(10.3636), tensor(12.1944),
tensor(11.9082), tensor(8.9943)]
[tensor(11.8294), tensor(11.9681), tensor(11.2727), tensor(11.1674),
tensor(14.1831), tensor(12.1194), tensor(11.1654), tensor(10.9082),
tensor(11.2952), tensor(11.6201), tensor(9.6156), tensor(13.8425),
tensor(12.8997), tensor(12.6213), tensor(12.0814), tensor(10.6877),
tensor(11.7990), tensor(10.3614), tensor(11.3904), tensor(10.0336),
tensor(11.4837), tensor(12.6635), tensor(11.1411), tensor(10.2610),
tensor(10.6340), tensor(13.6979), tensor(10.1707), tensor(11.7661),
tensor(11.9926), tensor(9.7141), tensor(11.8154), tensor(9.5435),
tensor(11.1269), tensor(12.3797), tensor(12.9708), tensor(11.2033),
```

```
tensor(10.4730), tensor(11.5240), tensor(11.2180), tensor(10.9492),
tensor(11.8248), tensor(10.6099), tensor(12.0329), tensor(10.4838),
tensor(11.5992), tensor(12.3217), tensor(10.6487), tensor(12.0202),
tensor(11.8910), tensor(9.7498)]
[tensor(11.6042), tensor(12.0113), tensor(12.6283), tensor(12.5853),
tensor(14.4637), tensor(11.8248), tensor(11.9952), tensor(11.6930),
tensor(12.8179), tensor(11.1154), tensor(11.1367), tensor(13.5325),
tensor(12.8539), tensor(12.6374), tensor(12.6819), tensor(10.5937),
tensor(12.5850), tensor(10.9092), tensor(11.6401), tensor(11.9629),
tensor(12.1456), tensor(12.7889), tensor(12.5482), tensor(11.4686),
tensor(11.1521), tensor(13.3734), tensor(12.0745), tensor(13.1254),
tensor(11.0659), tensor(10.0557), tensor(13.5860), tensor(9.9442),
tensor(11.7373), tensor(12.7577), tensor(12.6555), tensor(12.0264),
tensor(11.0654), tensor(11.1045), tensor(11.1420), tensor(10.8672),
tensor(12.5825), tensor(11.5856), tensor(11.3460), tensor(11.8863),
tensor(13.8972), tensor(12.1498), tensor(10.9999), tensor(12.4492),
tensor(12.9379), tensor(10.8391)]
[tensor(11.4539), tensor(11.2038), tensor(12.8342), tensor(12.2371),
tensor(14.6222), tensor(11.9261), tensor(12.4510), tensor(11.8373),
tensor(12.2724), tensor(12.0682), tensor(10.7315), tensor(13.3563),
tensor(11.6999), tensor(12.4285), tensor(12.3243), tensor(10.1523),
tensor(11.6750), tensor(11.1504), tensor(12.0971), tensor(10.9725),
tensor(11.1015), tensor(12.8441), tensor(11.8187), tensor(11.6911),
tensor(10.7320), tensor(13.8760), tensor(11.7652), tensor(12.4212),
tensor(12.8329), tensor(10.2491), tensor(12.7427), tensor(9.7264),
tensor(10.9102), tensor(13.8722), tensor(13.0271), tensor(11.8538),
tensor(10.7013), tensor(12.5202), tensor(11.3722), tensor(10.8770),
tensor(12.1766), tensor(11.7618), tensor(11.1242), tensor(10.8367),
tensor(12.0475), tensor(11.5623), tensor(9.7982), tensor(12.0564),
tensor(11.6751), tensor(10.6106)]
[tensor(11.8024), tensor(12.8149), tensor(12.3327), tensor(12.1832),
tensor(13.0675), tensor(11.9802), tensor(13.2690), tensor(12.4920),
tensor(12.2011), tensor(12.0855), tensor(11.4771), tensor(15.0447),
tensor(12.5070), tensor(12.0766), tensor(11.9215), tensor(11.3523),
tensor(12.7900), tensor(10.5839), tensor(12.4059), tensor(12.0475),
tensor(12.5939), tensor(13.4815), tensor(12.5664), tensor(12.2605),
tensor(11.4821), tensor(13.4233), tensor(11.5052), tensor(12.4284),
tensor(11.9796), tensor(10.0253), tensor(13.6039), tensor(9.7566),
tensor(10.9290), tensor(13.0394), tensor(11.9568), tensor(13.4132),
tensor(11.4759), tensor(12.5746), tensor(12.2883), tensor(11.4102),
tensor(12.9420), tensor(11.0799), tensor(11.3888), tensor(10.1800),
tensor(12.6357), tensor(11.5242), tensor(11.0350), tensor(11.3260),
tensor(12.6934), tensor(11.2411)]
[tensor(13.6514), tensor(13.9447), tensor(13.5825), tensor(15.7944),
tensor(15.7280), tensor(14.8983), tensor(13.9300), tensor(14.4349),
tensor(15.3123), tensor(14.8350), tensor(13.2785), tensor(15.5393),
tensor(15.4685), tensor(15.0571), tensor(13.7877), tensor(13.9548),
tensor(15.6975), tensor(12.8104), tensor(15.1860), tensor(14.7754),
```

```
tensor(15.0337), tensor(15.3177), tensor(14.4555), tensor(12.9820),
tensor(13.5272), tensor(15.8396), tensor(13.5355), tensor(15.2027),
tensor(13.3534), tensor(12.8483), tensor(15.7357), tensor(11.8015),
tensor(13.7518), tensor(15.8591), tensor(14.8501), tensor(14.2931),
tensor(13.7757), tensor(14.5088), tensor(13.6425), tensor(13.8480),
tensor(14.8211), tensor(13.7889), tensor(14.2420), tensor(11.8104),
tensor(13.8695), tensor(15.2235), tensor(13.5394), tensor(14.7023),
tensor(13.8968), tensor(12.8863)]
[tensor(11.9988), tensor(12.9381), tensor(12.1515), tensor(13.2383),
tensor(15.3128), tensor(13.2621), tensor(13.3305), tensor(13.1775),
tensor(13.7232), tensor(15.1089), tensor(10.8273), tensor(14.6940),
tensor(13.8413), tensor(13.7828), tensor(12.7699), tensor(11.7375),
tensor(14.3522), tensor(11.2121), tensor(13.9717), tensor(13.1583),
tensor(13.7495), tensor(14.2967), tensor(12.4623), tensor(12.1025),
tensor(11.3629), tensor(15.6691), tensor(12.7482), tensor(12.4916),
tensor(13.0980), tensor(13.1033), tensor(13.7608), tensor(11.5532),
tensor(12.4440), tensor(14.4950), tensor(14.8043), tensor(13.2065),
tensor(11.7524), tensor(12.8617), tensor(12.6618), tensor(12.0830),
tensor(13.2919), tensor(12.3293), tensor(14.0792), tensor(10.8172),
tensor(12.1879), tensor(14.0261), tensor(11.1629), tensor(12.7255),
tensor(12.0133), tensor(12.1241)]
[tensor(13.2503), tensor(13.1657), tensor(13.7587), tensor(15.5501),
tensor(16.7878), tensor(13.5150), tensor(12.4133), tensor(14.2926),
tensor(14.6921), tensor(13.1073), tensor(11.3793), tensor(14.9091),
tensor(14.7068), tensor(14.4031), tensor(13.1237), tensor(12.6732),
tensor(15.1092), tensor(10.8044), tensor(13.5885), tensor(12.1805),
tensor(12.3688), tensor(15.1070), tensor(14.2206), tensor(13.3377),
tensor(12.3503), tensor(16.1014), tensor(11.7847), tensor(13.1858),
tensor(13.3187), tensor(12.5608), tensor(14.4008), tensor(11.0505),
tensor(12.4503), tensor(15.0973), tensor(15.0016), tensor(12.4000),
tensor(11.8279), tensor(13.0720), tensor(11.9126), tensor(12.4423),
tensor(14.0444), tensor(13.2761), tensor(12.5110), tensor(12.1700),
tensor(14.1653), tensor(13.0095), tensor(12.5372), tensor(14.7526),
tensor(13.3044), tensor(12.1879)]
[tensor(11.5318), tensor(11.3437), tensor(8.9056), tensor(11.4714),
tensor(12.9879), tensor(12.2949), tensor(11.8768), tensor(10.5893),
tensor(10.8208), tensor(12.0141), tensor(10.2726), tensor(13.6026),
tensor(11.4349), tensor(11.9251), tensor(11.2202), tensor(11.1035),
tensor(12.5823), tensor(10.2664), tensor(10.6148), tensor(10.5008),
tensor(11.7589), tensor(11.7244), tensor(11.6857), tensor(10.0787),
tensor(11.6147), tensor(11.2354), tensor(11.4153), tensor(12.0051),
tensor(10.5108), tensor(10.9737), tensor(12.5683), tensor(9.7616),
tensor(10.3319), tensor(11.2625), tensor(12.2694), tensor(10.5316),
tensor(10.4995), tensor(11.3581), tensor(10.8166), tensor(11.7366),
tensor(11.6270), tensor(10.3959), tensor(11.2238), tensor(8.9874),
tensor(11.3176), tensor(12.8837), tensor(10.9563), tensor(12.2126),
tensor(12.5212), tensor(9.1265)]
[tensor(11.9090), tensor(11.9667), tensor(11.1475), tensor(11.7908),
```

```
tensor(14.2919), tensor(11.3408), tensor(11.2766), tensor(11.4848),
tensor(12.5947), tensor(11.1275), tensor(10.1188), tensor(13.6106),
tensor(12.2660), tensor(12.5335), tensor(12.4611), tensor(10.5577),
tensor(13.4130), tensor(10.8863), tensor(12.0976), tensor(10.8557),
tensor(12.0182), tensor(13.2310), tensor(12.3923), tensor(10.0788),
tensor(11.4875), tensor(13.1620), tensor(11.3987), tensor(12.5064),
tensor(11.2135), tensor(10.9554), tensor(13.0843), tensor(9.8163),
tensor(11.3205), tensor(12.9266), tensor(13.2126), tensor(11.3359),
tensor(10.8002), tensor(10.8554), tensor(11.6492), tensor(10.5911),
tensor(11.4891), tensor(11.8862), tensor(11.8264), tensor(9.8224),
tensor(12.9113), tensor(11.5467), tensor(11.0513), tensor(11.9763),
tensor(12.3794), tensor(9.6460)]
[tensor(11.7566), tensor(12.3178), tensor(12.4858), tensor(12.7430),
tensor(14.5096), tensor(12.0107), tensor(12.7772), tensor(11.7557),
tensor(12.3884), tensor(12.2050), tensor(11.5795), tensor(14.9988),
tensor(12.5463), tensor(12.9257), tensor(11.6285), tensor(12.8243),
tensor(12.6739), tensor(10.7174), tensor(11.9607), tensor(11.2896),
tensor(12.0157), tensor(13.2650), tensor(13.2405), tensor(11.9886),
tensor(11.2381), tensor(13.4176), tensor(11.4580), tensor(13.1737),
tensor(12.4950), tensor(11.7977), tensor(14.6174), tensor(9.3090),
tensor(10.2705), tensor(13.1433), tensor(12.9291), tensor(11.1461),
tensor(11.9066), tensor(11.9123), tensor(10.7662), tensor(11.6933),
tensor(12.5879), tensor(11.9912), tensor(13.0693), tensor(10.6419),
tensor(14.0714), tensor(12.6047), tensor(11.1526), tensor(12.8219),
tensor(12.8085), tensor(10.0408)]
[tensor(11.0711), tensor(11.1014), tensor(12.9176), tensor(11.9539),
tensor(14.1181), tensor(11.8022), tensor(11.8219), tensor(12.2299),
tensor(12.3489), tensor(12.3042), tensor(10.7738), tensor(13.7307),
tensor(12.3493), tensor(12.1040), tensor(11.5882), tensor(10.6086),
tensor(11.8203), tensor(10.3627), tensor(11.6889), tensor(11.4246),
tensor(12.0715), tensor(13.9758), tensor(12.3664), tensor(12.1239),
tensor(10.1494), tensor(13.3578), tensor(10.4311), tensor(11.2309),
tensor(11.5162), tensor(10.1381), tensor(12.0183), tensor(9.1756),
tensor(10.8269), tensor(13.1684), tensor(11.8065), tensor(12.1047),
tensor(10.9194), tensor(11.6183), tensor(10.3986), tensor(10.0173),
tensor(11.9018), tensor(10.7547), tensor(11.2629), tensor(10.8411),
tensor(11.7790), tensor(11.0188), tensor(10.2596), tensor(11.5047),
tensor(11.8897), tensor(9.9511)]
[tensor(9.6339), tensor(10.7734), tensor(9.9510), tensor(11.4164),
tensor(13.0862), tensor(10.6647), tensor(11.0131), tensor(10.8221),
tensor(9.7687), tensor(10.2067), tensor(7.7814), tensor(11.8155),
tensor(11.2790), tensor(11.5512), tensor(10.9528), tensor(9.9858),
tensor(11.8877), tensor(9.4041), tensor(11.1604), tensor(10.7356),
tensor(10.9591), tensor(11.4735), tensor(10.5984), tensor(8.6656),
tensor(10.3263), tensor(11.4587), tensor(9.3728), tensor(10.7053),
tensor(10.2848), tensor(10.6617), tensor(12.5054), tensor(7.8469),
tensor(10.2074), tensor(11.4072), tensor(11.2919), tensor(9.6355),
tensor(10.1382), tensor(9.2264), tensor(10.8686), tensor(10.0122),
```

```
tensor(11.1127), tensor(10.6479), tensor(10.1228), tensor(9.4322),
tensor(10.8174), tensor(13.0543), tensor(9.6636), tensor(10.3138),
tensor(11.0711), tensor(9.4738)]
[tensor(12.5088), tensor(12.2528), tensor(11.9142), tensor(13.5944),
tensor(14.3301), tensor(12.7408), tensor(11.7926), tensor(12.9475),
tensor(14.0782), tensor(13.0461), tensor(11.2927), tensor(14.5324),
tensor(14.3805), tensor(13.7306), tensor(11.0333), tensor(13.3391),
tensor(13.2290), tensor(10.7312), tensor(12.5955), tensor(11.8175),
tensor(12.7327), tensor(13.8406), tensor(12.5344), tensor(11.2284),
tensor(12.2534), tensor(14.4778), tensor(11.1905), tensor(14.3621),
tensor(12.6023), tensor(11.3819), tensor(14.1668), tensor(11.4448),
tensor(12.2858), tensor(13.7626), tensor(14.6334), tensor(13.0135),
tensor(12.1419), tensor(12.6654), tensor(11.4070), tensor(12.4018),
tensor(12.9633), tensor(12.3652), tensor(15.2946), tensor(9.2513),
tensor(14.0052), tensor(14.5872), tensor(11.3565), tensor(13.2020),
tensor(13.1416), tensor(10.2002)]
[tensor(10.6361), tensor(10.5965), tensor(9.6987), tensor(11.3459),
tensor(12.6371), tensor(10.8717), tensor(11.0136), tensor(12.0948),
tensor(12.1112), tensor(11.7481), tensor(10.2510), tensor(13.2390),
tensor(12.4165), tensor(12.2612), tensor(10.7694), tensor(10.6025),
tensor(12.4752), tensor(9.3018), tensor(11.2209), tensor(10.6966),
tensor(11.6620), tensor(12.6723), tensor(11.3467), tensor(8.9425),
tensor(10.5557), tensor(12.9470), tensor(10.5001), tensor(10.6987),
tensor(11.4415), tensor(11.2665), tensor(12.0079), tensor(9.4043),
tensor(10.1404), tensor(12.6071), tensor(12.6844), tensor(10.7318),
tensor(10.4645), tensor(10.9190), tensor(10.9984), tensor(10.7943),
tensor(11.7876), tensor(10.8255), tensor(11.3755), tensor(9.6568),
tensor(11.7386), tensor(12.3650), tensor(10.2656), tensor(11.5608),
tensor(12.3878), tensor(10.2716)]
[tensor(9.8426), tensor(11.8521), tensor(11.9526), tensor(12.4616),
tensor(13.2807), tensor(11.3933), tensor(12.1654), tensor(11.8477),
tensor(11.6334), tensor(10.5147), tensor(11.0576), tensor(13.0392),
tensor(11.6234), tensor(12.5965), tensor(11.6022), tensor(10.9577),
tensor(12.8083), tensor(10.4602), tensor(12.2078), tensor(12.2268),
tensor(11.4920), tensor(12.1055), tensor(12.0573), tensor(11.6115),
tensor(11.6212), tensor(13.4391), tensor(11.3055), tensor(12.6057),
tensor(10.4704), tensor(11.2037), tensor(13.6823), tensor(9.2687),
tensor(10.9841), tensor(13.2733), tensor(13.0609), tensor(11.3943),
tensor(11.8198), tensor(11.7561), tensor(12.0011), tensor(11.6902),
tensor(11.6468), tensor(12.0512), tensor(11.0617), tensor(10.5888),
tensor(13.1174), tensor(12.1385), tensor(11.2737), tensor(11.0379),
tensor(11.4074), tensor(10.4061)]
[tensor(11.3209), tensor(11.1683), tensor(12.2512), tensor(12.3964),
tensor(14.5923), tensor(13.4877), tensor(11.9277), tensor(12.6627),
tensor(13.1946), tensor(12.8468), tensor(12.1303), tensor(13.3665),
tensor(12.7852), tensor(12.7229), tensor(12.3924), tensor(10.3975),
tensor(13.1336), tensor(11.1143), tensor(12.7465), tensor(12.5700),
tensor(12.4262), tensor(12.8086), tensor(11.7972), tensor(11.4118),
```

```
tensor(11.4883), tensor(14.2976), tensor(12.1100), tensor(12.8494),
tensor(11.7966), tensor(12.2207), tensor(13.9664), tensor(10.0503),
tensor(12.2574), tensor(14.0853), tensor(13.9251), tensor(13.0927),
tensor(10.9957), tensor(11.8687), tensor(12.0783), tensor(10.9456),
tensor(13.4341), tensor(12.1529), tensor(11.4731), tensor(12.1492),
tensor(13.2768), tensor(13.0579), tensor(11.2660), tensor(13.2014),
tensor(12.4975), tensor(11.4987)]
[tensor(12.5069), tensor(13.0169), tensor(13.3003), tensor(13.2730),
tensor(15.4070), tensor(13.2067), tensor(12.6306), tensor(14.0235),
tensor(13.5599), tensor(12.8036), tensor(11.6720), tensor(16.0255),
tensor(15.5726), tensor(13.9390), tensor(13.5134), tensor(13.1445),
tensor(13.0261), tensor(10.8703), tensor(12.7621), tensor(11.5781),
tensor(12.8269), tensor(14.2856), tensor(13.8366), tensor(11.9852),
tensor(12.1470), tensor(15.9158), tensor(12.2599), tensor(13.9028),
tensor(13.3027), tensor(12.1525), tensor(14.5824), tensor(11.4071),
tensor(12.4916), tensor(13.8975), tensor(14.2247), tensor(12.8381),
tensor(12.7047), tensor(12.8057), tensor(13.5102), tensor(11.7302),
tensor(12.9975), tensor(12.7171), tensor(14.4796), tensor(10.5779),
tensor(13.9416), tensor(14.2667), tensor(12.4238), tensor(12.5517),
tensor(13.3936), tensor(11.3567)]
[tensor(13.8269), tensor(14.5870), tensor(12.9551), tensor(14.7871),
tensor(16.5657), tensor(14.3813), tensor(14.9057), tensor(14.1850),
tensor(15.2855), tensor(16.0564), tensor(13.4405), tensor(16.3761),
tensor(14.6889), tensor(15.1787), tensor(14.2450), tensor(12.9773),
tensor(16.9709), tensor(12.1668), tensor(15.7056), tensor(14.3199),
tensor(14.8534), tensor(15.7219), tensor(15.0527), tensor(13.2559),
tensor(13.2009), tensor(15.8784), tensor(13.9939), tensor(14.1453),
tensor(14.3545), tensor(13.6371), tensor(16.2224), tensor(12.5041),
tensor(13.4803), tensor(15.8737), tensor(15.7330), tensor(14.6912),
tensor(12.3764), tensor(14.7637), tensor(14.3758), tensor(13.6912),
tensor(14.9988), tensor(14.2632), tensor(13.3157), tensor(12.7492),
tensor(13.1754), tensor(14.4973), tensor(13.9876), tensor(14.9828),
tensor(14.1551), tensor(13.5314)]
[tensor(10.9307), tensor(12.2772), tensor(11.9501), tensor(10.8747),
tensor(13.5455), tensor(11.5019), tensor(11.1370), tensor(12.1352),
tensor(12.2005), tensor(11.2548), tensor(10.8000), tensor(14.3050),
tensor(12.0377), tensor(12.7727), tensor(11.8199), tensor(11.3108),
tensor(12.7242), tensor(9.8318), tensor(12.1556), tensor(11.9283),
tensor(12.6388), tensor(13.2130), tensor(12.0249), tensor(10.3404),
tensor(10.6513), tensor(13.6401), tensor(12.0188), tensor(11.9474),
tensor(11.3817), tensor(10.8616), tensor(13.2729), tensor(9.6999),
tensor(11.2149), tensor(12.2892), tensor(12.0559), tensor(12.4860),
tensor(11.8305), tensor(11.2089), tensor(11.2559), tensor(11.0813),
tensor(11.2998), tensor(11.5327), tensor(12.6422), tensor(10.1173),
tensor(12.2695), tensor(11.8628), tensor(10.4927), tensor(11.3541),
tensor(11.8880), tensor(9.8570)]
[tensor(11.1217), tensor(10.9116), tensor(11.3942), tensor(12.0613),
tensor(14.7841), tensor(12.0542), tensor(11.3582), tensor(10.7913),
```

```
tensor(11.6674), tensor(12.2532), tensor(10.1111), tensor(12.5278),
tensor(11.3520), tensor(12.0534), tensor(11.0505), tensor(10.3750),
tensor(13.1119), tensor(10.4687), tensor(12.0913), tensor(11.9801),
tensor(11.8837), tensor(12.0789), tensor(11.7848), tensor(11.3082),
tensor(10.6460), tensor(12.7576), tensor(10.7280), tensor(12.3247),
tensor(10.9432), tensor(10.3638), tensor(12.7299), tensor(9.6641),
tensor(11.4681), tensor(13.0771), tensor(12.3343), tensor(11.2830),
tensor(10.0932), tensor(11.3964), tensor(10.8625), tensor(10.5202),
tensor(12.2550), tensor(11.9716), tensor(10.9712), tensor(9.7139),
tensor(11.1818), tensor(12.6149), tensor(9.6196), tensor(11.6868),
tensor(10.9639), tensor(11.0634)]
[tensor(9.8635), tensor(11.0565), tensor(10.0764), tensor(10.5763),
tensor(12.9478), tensor(10.3781), tensor(11.6341), tensor(10.2855),
tensor(11.8674), tensor(11.9632), tensor(11.0911), tensor(12.7962),
tensor(11.9207), tensor(11.9946), tensor(10.6877), tensor(9.9408),
tensor(12.3597), tensor(9.0925), tensor(11.3808), tensor(11.4932),
tensor(11.9019), tensor(12.2828), tensor(11.7328), tensor(9.6221),
tensor(10.0144), tensor(12.0008), tensor(12.1878), tensor(11.0637),
tensor(11.6362), tensor(11.0689), tensor(12.4261), tensor(9.2693),
tensor(10.5524), tensor(12.5923), tensor(11.7807), tensor(10.6130),
tensor(9.7069), tensor(10.4575), tensor(10.3549), tensor(11.2672),
tensor(12.0756), tensor(10.5952), tensor(11.5793), tensor(10.6716),
tensor(10.2636), tensor(11.2329), tensor(9.6915), tensor(11.0299),
tensor(11.7572), tensor(10.0456)]
[tensor(11.8260), tensor(11.9460), tensor(13.0190), tensor(13.0780),
tensor(14.2034), tensor(12.8774), tensor(12.8610), tensor(12.9962),
tensor(12.9975), tensor(11.9050), tensor(11.8114), tensor(14.1901),
tensor(12.7139), tensor(13.0260), tensor(11.9857), tensor(11.4967),
tensor(13.2073), tensor(10.9583), tensor(12.7284), tensor(12.2291),
tensor(12.2033), tensor(13.3273), tensor(11.7129), tensor(12.4373),
tensor(12.1974), tensor(13.5670), tensor(11.3859), tensor(11.8477),
tensor(12.6267), tensor(12.4116), tensor(14.6418), tensor(9.9328),
tensor(11.6719), tensor(13.8318), tensor(13.3716), tensor(12.6547),
tensor(11.4578), tensor(12.3304), tensor(11.1426), tensor(12.2571),
tensor(13.1482), tensor(12.1701), tensor(11.9038), tensor(11.7620),
tensor(14.4033), tensor(12.4672), tensor(11.8873), tensor(12.5583),
tensor(12.3277), tensor(11.3130)]
[tensor(12.0370), tensor(12.7739), tensor(11.3193), tensor(12.3517),
tensor(14.3143), tensor(10.5492), tensor(11.3004), tensor(11.6887),
tensor(11.6314), tensor(10.8610), tensor(10.4129), tensor(13.8029),
tensor(12.8480), tensor(12.0943), tensor(11.9085), tensor(11.2316),
tensor(12.8106), tensor(9.5942), tensor(11.3092), tensor(10.0791),
tensor(12.0272), tensor(12.1901), tensor(11.9992), tensor(10.5447),
tensor(12.0610), tensor(12.5286), tensor(10.5483), tensor(11.8902),
tensor(12.1232), tensor(11.8098), tensor(13.3480), tensor(9.8123),
tensor(10.6551), tensor(12.3217), tensor(11.8401), tensor(11.0514),
tensor(10.3053), tensor(11.2088), tensor(11.7447), tensor(12.0267),
tensor(12.8956), tensor(11.4517), tensor(11.5760), tensor(10.1785),
```

```
tensor(11.7709), tensor(12.5420), tensor(11.4722), tensor(11.8810),
tensor(12.9294), tensor(9.5774)]
[tensor(12.5967), tensor(13.3227), tensor(13.7522), tensor(14.5407),
tensor(16.2860), tensor(14.1776), tensor(15.5438), tensor(13.4982),
tensor(13.9965), tensor(14.8620), tensor(13.0940), tensor(15.9080),
tensor(15.0214), tensor(14.2475), tensor(14.0530), tensor(13.5153),
tensor(13.9312), tensor(12.2940), tensor(14.5661), tensor(14.2264),
tensor(14.2804), tensor(15.3129), tensor(14.8971), tensor(11.8706),
tensor(12.9724), tensor(15.5702), tensor(12.9723), tensor(14.5939),
tensor(14.0771), tensor(13.1201), tensor(15.4191), tensor(10.6785),
tensor(13.3021), tensor(15.3621), tensor(16.0721), tensor(12.9180),
tensor(12.8859), tensor(14.0608), tensor(14.1988), tensor(13.1005),
tensor(14.7289), tensor(13.3101), tensor(14.3973), tensor(12.7100),
tensor(14.5308), tensor(14.7606), tensor(12.2086), tensor(14.1773),
tensor(14.6790), tensor(11.3953)]
[tensor(13.4877), tensor(13.9372), tensor(12.6679), tensor(13.8367),
tensor(15.2583), tensor(13.8501), tensor(12.8647), tensor(12.3073),
tensor(14.6717), tensor(13.8049), tensor(12.8975), tensor(15.8292),
tensor(14.0591), tensor(13.4044), tensor(13.6830), tensor(12.2497),
tensor(14.7967), tensor(12.2835), tensor(13.9072), tensor(14.0637),
tensor(15.1191), tensor(13.6489), tensor(13.3721), tensor(11.8523),
tensor(13.4929), tensor(15.3567), tensor(13.1427), tensor(14.4216),
tensor(12.9805), tensor(11.7688), tensor(15.5813), tensor(11.1479),
tensor(14.0166), tensor(15.0657), tensor(14.3356), tensor(13.5797),
tensor(12.1556), tensor(13.6284), tensor(12.8892), tensor(13.1707),
tensor(14.2860), tensor(13.1676), tensor(13.7300), tensor(11.8094),
tensor(13.7068), tensor(13.9298), tensor(13.0526), tensor(14.5672),
tensor(13.4619), tensor(12.3170)]
[tensor(11.6915), tensor(10.9732), tensor(12.9111), tensor(13.1114),
tensor(14.1650), tensor(11.9214), tensor(11.6680), tensor(12.0408),
tensor(13.4744), tensor(11.8144), tensor(11.2743), tensor(13.8858),
tensor(12.6560), tensor(12.9753), tensor(11.5315), tensor(11.0076),
tensor(12.6664), tensor(11.0714), tensor(11.7388), tensor(12.1001),
tensor(12.1186), tensor(13.8778), tensor(13.4755), tensor(11.9269),
tensor(11.6425), tensor(14.1647), tensor(11.7685), tensor(13.1339),
tensor(11.6922), tensor(11.3957), tensor(13.6340), tensor(9.1236),
tensor(11.3881), tensor(14.1590), tensor(12.7812), tensor(12.4903),
tensor(12.2902), tensor(13.1271), tensor(10.9871), tensor(11.3376),
tensor(11.9280), tensor(12.6062), tensor(11.9140), tensor(10.8366),
tensor(13.6709), tensor(12.6066), tensor(11.2603), tensor(12.7170),
tensor(12.9344), tensor(10.1398)]
[tensor(12.2046), tensor(12.3063), tensor(10.9108), tensor(11.9943),
tensor(14.8854), tensor(12.7795), tensor(11.4211), tensor(12.3877),
tensor(12.1839), tensor(11.7943), tensor(10.1079), tensor(14.2668),
tensor(13.4456), tensor(12.8376), tensor(11.0872), tensor(11.0568),
tensor(13.3928), tensor(10.1449), tensor(12.0275), tensor(10.8850),
tensor(12.1966), tensor(14.0920), tensor(12.8876), tensor(11.0058),
tensor(11.7408), tensor(13.9734), tensor(11.8011), tensor(11.9096),
```

```
tensor(12.4029), tensor(10.8502), tensor(13.3095), tensor(10.7619),
tensor(11.4119), tensor(13.3773), tensor(14.6657), tensor(12.4325),
tensor(11.3678), tensor(12.8109), tensor(10.6176), tensor(12.1950),
tensor(12.3428), tensor(12.0625), tensor(12.7766), tensor(10.4452),
tensor(11.7947), tensor(11.8745), tensor(10.8871), tensor(13.2240),
tensor(12.5871), tensor(10.0549)]
[tensor(12.0314), tensor(14.1011), tensor(13.1504), tensor(14.5056),
tensor(14.0810), tensor(11.1293), tensor(14.1188), tensor(13.3805),
tensor(13.3814), tensor(13.0153), tensor(11.4385), tensor(14.9389),
tensor(14.0162), tensor(12.8453), tensor(13.0059), tensor(11.6725),
tensor(14.1349), tensor(10.4581), tensor(13.7622), tensor(13.3159),
tensor(13.8969), tensor(13.4990), tensor(12.6221), tensor(11.8568),
tensor(12.3455), tensor(14.4927), tensor(12.0141), tensor(12.9520),
tensor(12.9584), tensor(12.9061), tensor(14.8500), tensor(11.1001),
tensor(11.0372), tensor(14.1827), tensor(13.4140), tensor(13.1924),
tensor(11.7417), tensor(11.9661), tensor(13.2723), tensor(12.4918),
tensor(14.1492), tensor(12.8759), tensor(12.8931), tensor(11.5546),
tensor(12.6820), tensor(14.6986), tensor(11.9869), tensor(12.4720),
tensor(13.0354), tensor(12.3983)]
[tensor(12.2425), tensor(12.8888), tensor(12.7104), tensor(13.2664),
tensor(14.7646), tensor(13.6691), tensor(12.7785), tensor(12.7112),
tensor(13.2823), tensor(12.4655), tensor(11.9925), tensor(15.9504),
tensor(13.8189), tensor(13.5495), tensor(13.5834), tensor(12.9161),
tensor(12.4155), tensor(11.0173), tensor(12.4908), tensor(12.1374),
tensor(12.9459), tensor(13.6946), tensor(12.5830), tensor(11.4775),
tensor(12.2538), tensor(15.3182), tensor(12.1680), tensor(14.0960),
tensor(13.0022), tensor(12.1488), tensor(14.3542), tensor(10.7749),
tensor(12.6193), tensor(13.4552), tensor(14.1231), tensor(12.5046),
tensor(12.4793), tensor(12.3331), tensor(12.4648), tensor(11.8730),
tensor(13.1732), tensor(12.2111), tensor(13.8560), tensor(11.5971),
tensor(14.7426), tensor(13.8751), tensor(11.7437), tensor(13.5830),
tensor(12.6551), tensor(11.4165)]
[tensor(10.4995), tensor(10.9303), tensor(10.0715), tensor(12.5646),
tensor(12.5299), tensor(11.1479), tensor(12.0311), tensor(11.5029),
tensor(11.1688), tensor(10.6762), tensor(9.3019), tensor(13.5097),
tensor(12.2083), tensor(11.4811), tensor(10.9428), tensor(10.8098),
tensor(11.6187), tensor(8.6757), tensor(10.6994), tensor(11.0778),
tensor(10.7222), tensor(12.2623), tensor(11.7184), tensor(10.7635),
tensor(10.7239), tensor(11.6646), tensor(10.6157), tensor(10.9618),
tensor(11.0804), tensor(10.6593), tensor(11.8820), tensor(8.5294),
tensor(9.9354), tensor(11.3338), tensor(11.1121), tensor(11.1473),
tensor(10.7527), tensor(10.9916), tensor(10.6988), tensor(10.1714),
tensor(10.9671), tensor(9.8948), tensor(10.1812), tensor(10.6955),
tensor(11.4897), tensor(11.6458), tensor(10.1521), tensor(11.1335),
tensor(11.2039), tensor(10.4634)]
[tensor(11.5555), tensor(11.0972), tensor(10.4956), tensor(11.3079),
tensor(12.8355), tensor(11.3401), tensor(11.9129), tensor(11.8408),
tensor(12.1176), tensor(12.6111), tensor(10.3403), tensor(13.7019),
```

```
tensor(12.3015), tensor(11.1746), tensor(12.1522), tensor(10.5116),
tensor(11.5513), tensor(9.9585), tensor(12.2481), tensor(11.4528),
tensor(11.8720), tensor(11.9668), tensor(11.4688), tensor(9.8652),
tensor(10.3537), tensor(13.3718), tensor(10.8237), tensor(11.5591),
tensor(10.8518), tensor(10.9031), tensor(11.9815), tensor(9.8419),
tensor(9.8287), tensor(12.2003), tensor(13.1257), tensor(11.4691),
tensor(9.7529), tensor(10.8172), tensor(11.2776), tensor(10.9128),
tensor(11.6762), tensor(10.3549), tensor(12.1634), tensor(9.7685),
tensor(11.0249), tensor(11.0937), tensor(10.0555), tensor(11.4133),
tensor(11.8919), tensor(10.0263)]
[tensor(10.2945), tensor(11.9444), tensor(10.7914), tensor(11.0799),
tensor(12.7418), tensor(11.2036), tensor(11.5126), tensor(11.7520),
tensor(11.2696), tensor(12.5648), tensor(11.4290), tensor(13.9207),
tensor(10.8011), tensor(11.5417), tensor(10.9599), tensor(10.7835),
tensor(11.5367), tensor(9.3198), tensor(11.0675), tensor(10.9295),
tensor(10.8890), tensor(11.7748), tensor(11.0328), tensor(11.8433),
tensor(9.8830), tensor(12.8731), tensor(10.6238), tensor(11.5624),
tensor(10.7444), tensor(10.6810), tensor(12.1427), tensor(8.6810),
tensor(9.8110), tensor(12.6965), tensor(11.4541), tensor(10.7674),
tensor(10.8035), tensor(11.4304), tensor(10.1508), tensor(10.6229),
tensor(11.2362), tensor(10.7666), tensor(11.2062), tensor(9.0739),
tensor(11.2307), tensor(10.1328), tensor(9.4284), tensor(11.4918),
tensor(10.8928), tensor(9.4040)]
[tensor(11.5164), tensor(10.8953), tensor(12.6278), tensor(12.1527),
tensor(14.5869), tensor(12.1867), tensor(11.8079), tensor(11.8047),
tensor(13.0591), tensor(11.8673), tensor(11.8322), tensor(14.4119),
tensor(11.7502), tensor(12.5785), tensor(12.3226), tensor(10.6691),
tensor(12.3408), tensor(10.9240), tensor(11.4093), tensor(11.3941),
tensor(11.4726), tensor(13.6837), tensor(12.3428), tensor(11.6131),
tensor(11.0642), tensor(14.2433), tensor(11.8354), tensor(11.7005),
tensor(12.2940), tensor(11.2433), tensor(12.7265), tensor(10.0698),
tensor(11.3795), tensor(14.5604), tensor(13.9982), tensor(11.7090),
tensor(11.4752), tensor(12.7823), tensor(11.2720), tensor(11.6324),
tensor(11.6864), tensor(12.5880), tensor(12.4843), tensor(10.9932),
tensor(13.2856), tensor(12.5158), tensor(10.1162), tensor(12.6714),
tensor(12.7361), tensor(10.2575)]
[tensor(11.0819), tensor(11.2555), tensor(11.9009), tensor(11.5451),
tensor(13.6301), tensor(11.3087), tensor(11.1900), tensor(11.4075),
tensor(12.3221), tensor(11.2421), tensor(9.9946), tensor(12.7166),
tensor(12.1396), tensor(12.8278), tensor(11.1459), tensor(10.2998),
tensor(12.9255), tensor(9.9361), tensor(11.0213), tensor(11.7052),
tensor(11.6283), tensor(13.0485), tensor(11.9590), tensor(10.7066),
tensor(11.4679), tensor(12.8062), tensor(10.6857), tensor(11.3213),
tensor(11.2882), tensor(10.8612), tensor(13.4377), tensor(9.1584),
tensor(11.4241), tensor(12.3426), tensor(12.9262), tensor(11.9431),
tensor(11.3476), tensor(11.6676), tensor(10.7652), tensor(11.5044),
tensor(11.8653), tensor(11.9056), tensor(11.8470), tensor(9.6864),
tensor(13.2653), tensor(12.2903), tensor(10.4221), tensor(11.8676),
```

```
tensor(12.2804), tensor(10.3503)]
[tensor(12.8188), tensor(13.4600), tensor(14.3435), tensor(14.9556),
tensor(16.4405), tensor(12.8705), tensor(13.9410), tensor(14.0521),
tensor(15.3273), tensor(14.2877), tensor(12.1703), tensor(15.1904),
tensor(14.5522), tensor(14.9209), tensor(13.8575), tensor(12.8458),
tensor(15.2913), tensor(11.9509), tensor(13.4811), tensor(13.9638),
tensor(14.1416), tensor(15.2024), tensor(14.0431), tensor(13.0782),
tensor(13.0784), tensor(16.3068), tensor(12.6852), tensor(13.7071),
tensor(14.5721), tensor(12.9531), tensor(15.8345), tensor(11.3095),
tensor(13.3392), tensor(16.2304), tensor(14.8410), tensor(13.5733),
tensor(13.2848), tensor(13.5835), tensor(13.4228), tensor(12.0664),
tensor(14.3223), tensor(13.2279), tensor(14.1863), tensor(12.0996),
tensor(14.7561), tensor(14.7624), tensor(11.9284), tensor(14.3244),
tensor(14.2624), tensor(12.3540)]
[tensor(10.2130), tensor(11.4684), tensor(11.3665), tensor(11.6297),
tensor(12.4230), tensor(11.0828), tensor(10.4192), tensor(12.8124),
tensor(12.6292), tensor(11.8403), tensor(9.9606), tensor(13.6015),
tensor(12.3143), tensor(11.3727), tensor(10.8773), tensor(10.7072),
tensor(11.4331), tensor(10.5290), tensor(11.2061), tensor(11.4170),
tensor(12.6377), tensor(12.8007), tensor(10.9270), tensor(10.0374),
tensor(11.8272), tensor(12.7289), tensor(10.2972), tensor(11.7637),
tensor(10.9368), tensor(10.9563), tensor(13.3895), tensor(8.8722),
tensor(11.0406), tensor(12.8662), tensor(12.1237), tensor(12.2499),
tensor(10.8882), tensor(10.6537), tensor(11.2188), tensor(10.2994),
tensor(11.3183), tensor(11.5007), tensor(11.5740), tensor(8.9035),
tensor(11.9141), tensor(14.0626), tensor(10.0236), tensor(11.5779),
tensor(11.6296), tensor(10.2435)]
[tensor(10.7139), tensor(11.8955), tensor(10.1311), tensor(10.7839),
tensor(13.2215), tensor(11.9716), tensor(11.4952), tensor(10.7613),
tensor(10.8269), tensor(11.2984), tensor(10.0219), tensor(13.0724),
tensor(12.3584), tensor(11.8229), tensor(11.5871), tensor(10.3762),
tensor(12.7791), tensor(9.9547), tensor(11.2515), tensor(10.2803),
tensor(11.1870), tensor(12.5122), tensor(11.8556), tensor(9.5461),
tensor(11.3984), tensor(12.2574), tensor(11.4203), tensor(11.9947),
tensor(11.3439), tensor(10.5506), tensor(13.9701), tensor(9.7592),
tensor(11.2317), tensor(11.3165), tensor(12.6288), tensor(11.1764),
tensor(9.6349), tensor(11.2587), tensor(11.3624), tensor(11.3592),
tensor(12.2394), tensor(10.8282), tensor(11.5375), tensor(10.2420),
tensor(11.5083), tensor(12.4011), tensor(10.7171), tensor(12.1197),
tensor(12.1595), tensor(10.3550)]
[tensor(13.6437), tensor(13.1248), tensor(14.1940), tensor(14.5742),
tensor(16.3242), tensor(13.5691), tensor(13.4582), tensor(13.5499),
tensor(15.6356), tensor(14.3212), tensor(13.3333), tensor(16.1236),
tensor(15.6776), tensor(14.8107), tensor(14.0991), tensor(12.7588),
tensor(14.7016), tensor(11.5569), tensor(13.1977), tensor(13.6765),
tensor(14.4683), tensor(15.4467), tensor(14.4797), tensor(12.7681),
tensor(12.6804), tensor(15.3151), tensor(12.9705), tensor(14.5179),
tensor(13.9656), tensor(12.3479), tensor(14.5829), tensor(12.6983),
```

```
tensor(12.9111), tensor(15.1286), tensor(14.9834), tensor(12.9181),
tensor(12.7597), tensor(12.4490), tensor(13.8381), tensor(11.7187),
tensor(14.2909), tensor(13.4228), tensor(14.2585), tensor(12.5396),
tensor(14.8179), tensor(15.2831), tensor(12.8554), tensor(13.6168),
tensor(13.2906), tensor(12.3479)]
[tensor(11.7937), tensor(13.2324), tensor(11.7309), tensor(13.1202),
tensor(14.1065), tensor(10.8977), tensor(12.2440), tensor(11.7392),
tensor(12.4889), tensor(12.0160), tensor(10.8569), tensor(14.7305),
tensor(13.2012), tensor(12.6617), tensor(12.4256), tensor(10.9846),
tensor(12.7295), tensor(10.5142), tensor(12.3833), tensor(11.9825),
tensor(12.0443), tensor(12.8349), tensor(12.7279), tensor(11.2948),
tensor(11.3607), tensor(14.0793), tensor(11.7016), tensor(13.0127),
tensor(12.4313), tensor(10.0843), tensor(13.8867), tensor(10.7739),
tensor(11.2428), tensor(14.0689), tensor(13.5355), tensor(11.4857),
tensor(10.3208), tensor(12.6153), tensor(11.4123), tensor(12.1937),
tensor(12.8205), tensor(11.5779), tensor(12.6098), tensor(11.0658),
tensor(11.3561), tensor(11.9660), tensor(10.5843), tensor(12.3155),
tensor(12.1197), tensor(11.1437)]
[tensor(8.2146), tensor(10.2527), tensor(11.3897), tensor(11.6419),
tensor(12.3200), tensor(11.0595), tensor(11.8822), tensor(11.9097),
tensor(10.0634), tensor(11.1359), tensor(10.4362), tensor(11.2250),
tensor(11.2318), tensor(11.8317), tensor(10.7325), tensor(8.9888),
tensor(11.0504), tensor(9.6436), tensor(11.4441), tensor(12.3059),
tensor(10.6175), tensor(11.2195), tensor(10.3476), tensor(10.2961),
tensor(10.4657), tensor(12.1574), tensor(10.5303), tensor(11.1825),
tensor(10.1065), tensor(10.7350), tensor(12.1368), tensor(8.1100),
tensor(11.1565), tensor(12.5707), tensor(11.6426), tensor(10.3662),
tensor(9.8620), tensor(10.4379), tensor(9.9822), tensor(10.3591),
tensor(11.2516), tensor(11.1819), tensor(10.5971), tensor(9.6590),
tensor(10.2812), tensor(11.6853), tensor(9.8741), tensor(11.0235),
tensor(10.6526), tensor(10.0745)]
[tensor(11.0722), tensor(11.9531), tensor(11.4046), tensor(12.8067),
tensor(13.8365), tensor(12.2371), tensor(12.0837), tensor(12.6579),
tensor(13.5759), tensor(12.3818), tensor(11.5534), tensor(14.3728),
tensor(12.0915), tensor(13.1093), tensor(11.5703), tensor(11.3462),
tensor(13.3995), tensor(11.1212), tensor(12.0811), tensor(12.3496),
tensor(12.5801), tensor(12.9333), tensor(12.0585), tensor(11.9926),
tensor(12.2669), tensor(12.4128), tensor(11.6979), tensor(12.4119),
tensor(11.6285), tensor(11.8586), tensor(14.0375), tensor(9.6187),
tensor(11.8407), tensor(13.3913), tensor(12.9565), tensor(12.1077),
tensor(12.1172), tensor(11.5901), tensor(11.8225), tensor(11.7638),
tensor(12.1830), tensor(11.9185), tensor(11.2385), tensor(10.0992),
tensor(13.4163), tensor(13.2091), tensor(11.3313), tensor(12.0533),
tensor(11.6599), tensor(10.7796)]
[tensor(14.1437), tensor(14.1004), tensor(13.3979), tensor(15.1553),
tensor(16.1032), tensor(14.9534), tensor(14.7846), tensor(14.4615),
tensor(15.3437), tensor(15.4164), tensor(13.1820), tensor(16.9093),
tensor(15.1106), tensor(14.9444), tensor(14.3238), tensor(12.9722),
```

```
tensor(16.1625), tensor(12.6669), tensor(14.6101), tensor(14.8658),
tensor(15.3795), tensor(16.3429), tensor(14.9151), tensor(12.5751),
tensor(14.0303), tensor(15.5790), tensor(14.2116), tensor(14.8252),
tensor(13.7237), tensor(14.3336), tensor(16.2049), tensor(13.6822),
tensor(13.5104), tensor(15.5655), tensor(16.8523), tensor(14.7058),
tensor(13.5084), tensor(15.2177), tensor(14.1027), tensor(14.0549),
tensor(15.6257), tensor(14.9417), tensor(14.8196), tensor(12.2965),
tensor(14.7484), tensor(16.1230), tensor(13.7230), tensor(14.8473),
tensor(14.6247), tensor(13.3656)]
[tensor(9.5730), tensor(10.2853), tensor(10.6673), tensor(11.2474),
tensor(11.4228), tensor(11.0243), tensor(11.9269), tensor(11.0902),
tensor(10.9153), tensor(11.3145), tensor(10.3240), tensor(12.7209),
tensor(11.5360), tensor(10.6318), tensor(10.7754), tensor(9.9005),
tensor(9.9362), tensor(9.2597), tensor(10.7266), tensor(10.5459),
tensor(10.0972), tensor(11.8634), tensor(11.6901), tensor(9.2472),
tensor(10.0855), tensor(12.2134), tensor(10.3559), tensor(11.1680),
tensor(10.9965), tensor(9.5689), tensor(11.7843), tensor(7.9314),
tensor(9.3477), tensor(12.0969), tensor(12.4535), tensor(10.0709),
tensor(10.0890), tensor(10.9502), tensor(10.0922), tensor(11.0851),
tensor(10.7787), tensor(9.9539), tensor(10.4037), tensor(9.3196),
tensor(10.2906), tensor(11.0746), tensor(9.3382), tensor(12.0088),
tensor(11.8066), tensor(9.3549)]
[tensor(12.3274), tensor(11.5912), tensor(11.7818), tensor(13.8226),
tensor(15.1524), tensor(12.6146), tensor(12.8385), tensor(13.2367),
tensor(13.2399), tensor(13.1180), tensor(10.7544), tensor(14.5579),
tensor(13.6899), tensor(13.3791), tensor(12.3561), tensor(11.2318),
tensor(13.4389), tensor(11.1129), tensor(11.5998), tensor(12.7118),
tensor(12.0124), tensor(13.7961), tensor(12.7104), tensor(12.7703),
tensor(11.5319), tensor(14.1539), tensor(12.1270), tensor(13.1731),
tensor(11.6964), tensor(12.3754), tensor(13.1602), tensor(10.9852),
tensor(11.2992), tensor(15.0871), tensor(14.7195), tensor(11.8041),
tensor(11.9986), tensor(13.0807), tensor(12.0611), tensor(11.7112),
tensor(12.7311), tensor(12.5896), tensor(12.3372), tensor(10.6925),
tensor(13.9268), tensor(13.4297), tensor(10.1848), tensor(11.7000),
tensor(13.0029), tensor(12.2453)]
[tensor(13.3430), tensor(12.5711), tensor(11.8512), tensor(8.5903),
tensor(12.2327), tensor(12.2536), tensor(11.4210), tensor(11.7880),
tensor(12.6943), tensor(11.1699), tensor(10.0642), tensor(12.2669),
tensor(12.2770), tensor(12.6659), tensor(11.6190), tensor(12.2366),
tensor(10.9577), tensor(10.0416), tensor(10.0992), tensor(11.0247),
tensor(13.8756), tensor(12.7284), tensor(12.2476), tensor(9.2997),
tensor(11.4349), tensor(10.3293), tensor(12.4878), tensor(9.6544),
tensor(10.9974), tensor(13.3016), tensor(9.9759), tensor(10.4239),
tensor(11.0165), tensor(12.5241), tensor(12.8331), tensor(9.5444),
tensor(13.6349), tensor(11.9243), tensor(10.0470), tensor(11.8408),
tensor(9.0807), tensor(12.5691), tensor(9.8472), tensor(12.0855),
tensor(12.0898), tensor(9.8391), tensor(12.2178), tensor(12.1276),
tensor(12.4170), tensor(11.4036)]
```

```
[tensor(13.6062), tensor(13.2110), tensor(13.0221), tensor(9.3493),
tensor(11.5850), tensor(13.4865), tensor(11.1991), tensor(12.1919),
tensor(13.7209), tensor(10.1979), tensor(9.9142), tensor(12.9091),
tensor(11.8693), tensor(12.6127), tensor(12.7607), tensor(12.1716),
tensor(11.7163), tensor(11.8356), tensor(10.2771), tensor(12.2415),
tensor(13.6975), tensor(12.7150), tensor(13.1663), tensor(8.6668),
tensor(11.2874), tensor(11.8447), tensor(13.6694), tensor(10.4852),
tensor(11.1774), tensor(14.3881), tensor(9.9726), tensor(11.6118),
tensor(10.8627), tensor(13.1884), tensor(12.8577), tensor(9.4415),
tensor(14.8498), tensor(12.3547), tensor(11.4542), tensor(10.6983),
tensor(11.0995), tensor(13.7124), tensor(11.1042), tensor(13.2140),
tensor(13.0875), tensor(10.5432), tensor(13.0793), tensor(13.2436),
tensor(13.0244), tensor(11.4646)]
[tensor(15.9494), tensor(15.1579), tensor(14.5903), tensor(10.6969),
tensor(14.2350), tensor(15.3019), tensor(13.7901), tensor(13.7410),
tensor(15.2302), tensor(12.2407), tensor(12.4029), tensor(14.3705),
tensor(14.2943), tensor(13.9886), tensor(14.0562), tensor(14.3996),
tensor(12.8137), tensor(12.2553), tensor(13.3368), tensor(14.7785),
tensor(15.7362), tensor(15.3027), tensor(13.0060), tensor(11.1986),
tensor(15.4445), tensor(13.5495), tensor(14.5209), tensor(12.2683),
tensor(14.7427), tensor(15.6861), tensor(11.9154), tensor(13.9284),
tensor(13.8694), tensor(15.0627), tensor(16.0445), tensor(11.5742),
tensor(18.9626), tensor(14.8020), tensor(12.6311), tensor(12.8659),
tensor(11.6001), tensor(15.4685), tensor(12.6124), tensor(14.7009),
tensor(15.8084), tensor(12.6498), tensor(15.7475), tensor(14.2121),
tensor(14.6516), tensor(13.4327)]
[tensor(12.4265), tensor(12.9645), tensor(10.7473), tensor(8.7022),
tensor(11.3640), tensor(11.8076), tensor(9.9348), tensor(11.2134),
tensor(11.0532), tensor(9.7395), tensor(9.2841), tensor(12.5404),
tensor(10.4719), tensor(10.8320), tensor(10.1318), tensor(12.1581),
tensor(10.0818), tensor(9.2898), tensor(11.1924), tensor(10.5583),
tensor(12.2177), tensor(12.3614), tensor(10.8227), tensor(8.2124),
tensor(11.6718), tensor(10.3677), tensor(11.7474), tensor(9.6172),
tensor(11.2610), tensor(12.7237), tensor(9.1466), tensor(10.1265),
tensor(10.5263), tensor(13.1143), tensor(12.1355), tensor(10.0358),
tensor(13.9706), tensor(11.5685), tensor(9.0267), tensor(11.9616),
tensor(8.1783), tensor(12.3793), tensor(9.2126), tensor(11.6838),
tensor(11.6336), tensor(10.4591), tensor(11.7465), tensor(11.8269),
tensor(11.1199), tensor(11.6042)]
[tensor(11.4990), tensor(12.6447), tensor(12.8590), tensor(9.4881),
tensor(11.0280), tensor(12.7968), tensor(10.4403), tensor(11.6498),
tensor(12.4123), tensor(9.5655), tensor(9.3734), tensor(13.8481),
tensor(9.8855), tensor(11.4674), tensor(11.9526), tensor(12.5129),
tensor(11.3101), tensor(11.2995), tensor(11.1181), tensor(11.6187),
tensor(13.2245), tensor(13.3520), tensor(10.7252), tensor(9.4051),
tensor(11.9966), tensor(10.9849), tensor(13.5515), tensor(11.0127),
tensor(11.7494), tensor(13.8351), tensor(9.8088), tensor(11.6005),
tensor(9.8422), tensor(13.1001), tensor(13.2097), tensor(9.4650),
```

```
tensor(14.5862), tensor(13.1830), tensor(10.8108), tensor(12.0388),
tensor(9.2184), tensor(12.7684), tensor(11.0061), tensor(12.2393),
tensor(12.0259), tensor(9.8064), tensor(12.2956), tensor(11.7442),
tensor(12.2250), tensor(11.5636)]
[tensor(12.9587), tensor(13.4572), tensor(12.7339), tensor(8.2269),
tensor(12.6643), tensor(13.2040), tensor(11.2187), tensor(11.3754),
tensor(12.5886), tensor(10.0428), tensor(10.4065), tensor(12.7489),
tensor(12.6637), tensor(12.4996), tensor(11.8068), tensor(12.3557),
tensor(10.6885), tensor(10.0784), tensor(10.4524), tensor(11.6727),
tensor(14.2602), tensor(13.0890), tensor(11.5834), tensor(8.6531),
tensor(11.9917), tensor(11.0680), tensor(12.6860), tensor(9.6260),
tensor(12.2303), tensor(12.6748), tensor(10.7088), tensor(11.5638),
tensor(11.2661), tensor(13.3260), tensor(12.0341), tensor(9.5151),
tensor(15.6964), tensor(12.1914), tensor(9.9590), tensor(10.3549),
tensor(10.7610), tensor(13.6035), tensor(10.8280), tensor(12.3878),
tensor(13.3408), tensor(10.5001), tensor(12.7494), tensor(12.3877),
tensor(11.6764), tensor(11.1702)]
[tensor(13.7000), tensor(12.2543), tensor(13.9188), tensor(10.0495),
tensor(11.7381), tensor(15.1459), tensor(12.9264), tensor(12.3990),
tensor(12.7287), tensor(10.1531), tensor(10.8002), tensor(13.8679),
tensor(11.8890), tensor(12.8587), tensor(12.5212), tensor(11.3799),
tensor(11.4739), tensor(12.0961), tensor(10.8261), tensor(12.4824),
tensor(14.3967), tensor(13.3285), tensor(12.8526), tensor(9.4792),
tensor(12.0096), tensor(11.7718), tensor(14.1470), tensor(11.5643),
tensor(11.9905), tensor(14.4201), tensor(9.7418), tensor(12.1724),
tensor(11.5616), tensor(13.5385), tensor(13.8954), tensor(10.0745),
tensor(14.4222), tensor(12.4027), tensor(11.4451), tensor(12.9969),
tensor(10.1585), tensor(13.6278), tensor(10.9096), tensor(12.9500),
tensor(11.6869), tensor(10.8530), tensor(13.2893), tensor(13.0930),
tensor(12.3659), tensor(11.3886)]
[tensor(16.1031), tensor(13.1983), tensor(14.3047), tensor(10.8557),
tensor(13.4438), tensor(16.3258), tensor(13.0122), tensor(13.2741),
tensor(13.3222), tensor(11.0875), tensor(11.7829), tensor(14.4516),
tensor(13.4788), tensor(13.3157), tensor(13.3988), tensor(14.2643),
tensor(12.8151), tensor(11.2914), tensor(12.3547), tensor(13.8535),
tensor(14.5638), tensor(14.1153), tensor(12.5443), tensor(10.1739),
tensor(13.4264), tensor(13.3520), tensor(14.0559), tensor(12.3005),
tensor(13.9995), tensor(15.2028), tensor(10.2472), tensor(13.1323),
tensor(11.8420), tensor(13.7934), tensor(13.8202), tensor(11.7501),
tensor(16.6070), tensor(15.0143), tensor(12.0942), tensor(14.0454),
tensor(10.2944), tensor(13.7125), tensor(11.8254), tensor(13.7571),
tensor(13.0554), tensor(12.3145), tensor(13.6273), tensor(14.7087),
tensor(13.4932), tensor(12.9300)]
[tensor(14.7697), tensor(13.1999), tensor(13.3380), tensor(10.3456),
tensor(12.0480), tensor(13.7640), tensor(12.1019), tensor(12.4227),
tensor(13.7869), tensor(11.2901), tensor(9.4660), tensor(12.9632),
tensor(13.0593), tensor(12.2762), tensor(12.4076), tensor(12.7731),
tensor(11.3137), tensor(11.3903), tensor(10.9647), tensor(12.1982),
```

```
tensor(14.3123), tensor(14.2542), tensor(12.0262), tensor(10.1898),
tensor(12.7011), tensor(11.2568), tensor(13.8079), tensor(12.2399),
tensor(12.7299), tensor(13.2999), tensor(10.6264), tensor(11.2530),
tensor(11.2624), tensor(13.9509), tensor(13.7508), tensor(10.4331),
tensor(16.3747), tensor(12.9425), tensor(10.5832), tensor(11.9285),
tensor(9.8527), tensor(14.0236), tensor(11.3572), tensor(12.7986),
tensor(12.7809), tensor(11.1319), tensor(14.2119), tensor(12.5551),
tensor(13.3084), tensor(12.2175)]
[tensor(13.6390), tensor(11.5228), tensor(13.0094), tensor(9.5698),
tensor(12.0929), tensor(12.9473), tensor(11.5036), tensor(11.1309),
tensor(11.3191), tensor(9.5168), tensor(10.4547), tensor(12.5037),
tensor(12.4582), tensor(11.6521), tensor(12.2021), tensor(12.7543),
tensor(9.9411), tensor(9.6324), tensor(10.4564), tensor(10.1925),
tensor(12.9793), tensor(12.3489), tensor(10.2217), tensor(8.8845),
tensor(11.4107), tensor(10.4598), tensor(12.2425), tensor(10.8423),
tensor(11.8216), tensor(13.1355), tensor(9.2614), tensor(10.5616),
tensor(11.2827), tensor(12.6419), tensor(12.5977), tensor(9.3321),
tensor(14.3073), tensor(12.9355), tensor(8.9786), tensor(12.6610),
tensor(9.1312), tensor(11.6752), tensor(10.2169), tensor(10.2736),
tensor(12.0729), tensor(11.7694), tensor(11.3318), tensor(11.3555),
tensor(12.6921), tensor(11.2770)]
[tensor(14.7831), tensor(12.4815), tensor(13.4364), tensor(10.7846),
tensor(12.5626), tensor(13.7854), tensor(11.6388), tensor(11.6928),
tensor(11.5504), tensor(10.9101), tensor(10.7937), tensor(11.9773),
tensor(11.8374), tensor(11.6672), tensor(11.9645), tensor(11.3591),
tensor(11.3576), tensor(10.3023), tensor(11.3293), tensor(10.3842),
tensor(13.2069), tensor(12.7991), tensor(11.7318), tensor(9.0245),
tensor(11.0175), tensor(11.8715), tensor(13.0589), tensor(10.1883),
tensor(12.9572), tensor(14.0615), tensor(9.9844), tensor(12.3576),
tensor(12.4798), tensor(14.3026), tensor(13.1233), tensor(10.7776),
tensor(14.8299), tensor(12.4854), tensor(11.3058), tensor(12.7813),
tensor(10.2963), tensor(12.6395), tensor(10.6315), tensor(11.5745),
tensor(12.7781), tensor(11.9964), tensor(12.5776), tensor(12.4926),
tensor(13.2136), tensor(11.6656)]
[tensor(14.8647), tensor(14.1955), tensor(13.8138), tensor(10.6126),
tensor(13.0602), tensor(14.9279), tensor(13.6262), tensor(13.5580),
tensor(14.6923), tensor(11.7378), tensor(11.7323), tensor(13.9730),
tensor(13.8199), tensor(13.6433), tensor(13.3751), tensor(13.3565),
tensor(12.9573), tensor(12.1369), tensor(12.2368), tensor(14.1639),
tensor(14.8457), tensor(14.3213), tensor(14.0266), tensor(10.2365),
tensor(13.9664), tensor(12.2940), tensor(14.0642), tensor(12.2052),
tensor(12.6067), tensor(15.6269), tensor(11.5196), tensor(12.6960),
tensor(12.2951), tensor(14.4233), tensor(14.1340), tensor(11.1189),
tensor(16.7934), tensor(15.0473), tensor(11.4778), tensor(12.3992),
tensor(11.8878), tensor(14.5862), tensor(11.8847), tensor(14.4565),
tensor(13.1586), tensor(12.2991), tensor(15.3696), tensor(14.7260),
tensor(14.4904), tensor(13.3720)]
[tensor(15.4576), tensor(14.9297), tensor(14.9785), tensor(10.4368),
```

```
tensor(13.0283), tensor(14.5624), tensor(13.9974), tensor(14.2311),
tensor(14.0968), tensor(12.3622), tensor(11.9467), tensor(14.1142),
tensor(13.4516), tensor(14.0705), tensor(14.0313), tensor(13.6125),
tensor(12.8122), tensor(12.1428), tensor(12.8506), tensor(13.4489),
tensor(15.1205), tensor(14.8622), tensor(14.4759), tensor(9.3711),
tensor(13.4951), tensor(12.8946), tensor(15.5889), tensor(11.5324),
tensor(13.4008), tensor(15.6434), tensor(11.1864), tensor(12.9704),
tensor(12.1230), tensor(14.4529), tensor(14.4225), tensor(11.7831),
tensor(16.5820), tensor(15.1466), tensor(12.3234), tensor(13.4874),
tensor(11.4304), tensor(15.1717), tensor(11.8647), tensor(14.6151),
tensor(13.8319), tensor(10.9849), tensor(14.9837), tensor(14.8174),
tensor(14.1080), tensor(13.6799)]
[tensor(13.6766), tensor(12.8152), tensor(12.5418), tensor(9.7939),
tensor(12.8743), tensor(13.4049), tensor(11.3067), tensor(12.0971),
tensor(13.9237), tensor(9.9936), tensor(10.3448), tensor(12.9324),
tensor(12.9227), tensor(12.1427), tensor(12.5344), tensor(12.4081),
tensor(11.8280), tensor(10.4131), tensor(10.1322), tensor(11.9497),
tensor(14.7541), tensor(13.1134), tensor(12.2254), tensor(9.7485),
tensor(11.5979), tensor(12.0227), tensor(13.4548), tensor(12.4859),
tensor(12.4947), tensor(14.1985), tensor(10.3941), tensor(11.3976),
tensor(11.5647), tensor(13.8999), tensor(13.6508), tensor(10.4585),
tensor(15.6183), tensor(12.0734), tensor(9.4869), tensor(11.2979),
tensor(10.1993), tensor(14.1235), tensor(12.0525), tensor(11.5682),
tensor(13.5603), tensor(11.6939), tensor(13.8873), tensor(13.3119),
tensor(14.0365), tensor(10.6644)]
[tensor(13.5149), tensor(13.1305), tensor(14.1619), tensor(9.8555),
tensor(13.0175), tensor(14.4752), tensor(11.6773), tensor(13.1277),
tensor(13.1785), tensor(10.5785), tensor(11.2431), tensor(14.0203),
tensor(11.7841), tensor(11.9414), tensor(13.2567), tensor(13.3084),
tensor(11.2688), tensor(11.0950), tensor(12.5197), tensor(12.4888),
tensor(14.0714), tensor(14.0025), tensor(11.7788), tensor(10.6966),
tensor(12.7036), tensor(12.0363), tensor(13.5659), tensor(12.9469),
tensor(13.4831), tensor(15.1203), tensor(9.6039), tensor(11.6305),
tensor(11.7136), tensor(13.3115), tensor(14.5554), tensor(10.8862),
tensor(16.4534), tensor(13.2899), tensor(10.7570), tensor(13.2519),
tensor(10.5750), tensor(12.6429), tensor(11.7150), tensor(12.7505),
tensor(13.3837), tensor(11.8871), tensor(13.1685), tensor(13.1562),
tensor(12.4457), tensor(11.6069)]
[tensor(14.8429), tensor(13.8571), tensor(15.5018), tensor(10.8623),
tensor(14.0260), tensor(14.9127), tensor(12.9359), tensor(13.1053),
tensor(14.4889), tensor(10.8127), tensor(11.7457), tensor(13.8504),
tensor(13.4962), tensor(14.6111), tensor(14.4056), tensor(13.4862),
tensor(12.6167), tensor(11.7078), tensor(11.9651), tensor(13.1250),
tensor(15.3871), tensor(14.6153), tensor(12.7003), tensor(11.3618),
tensor(12.8296), tensor(13.2815), tensor(13.7159), tensor(11.7942),
tensor(13.8696), tensor(15.9035), tensor(11.5085), tensor(13.1655),
tensor(11.5873), tensor(14.0504), tensor(14.7928), tensor(9.9929),
tensor(16.5560), tensor(16.1330), tensor(12.5420), tensor(12.6615),
```

```
tensor(11.1062), tensor(13.9128), tensor(13.5324), tensor(13.8928),
tensor(13.9884), tensor(11.9733), tensor(13.0147), tensor(14.0667),
tensor(14.5002), tensor(12.5714)]
[tensor(15.5132), tensor(15.2567), tensor(13.9752), tensor(10.2579),
tensor(14.1500), tensor(14.8297), tensor(13.3039), tensor(13.7719),
tensor(15.8674), tensor(12.4526), tensor(12.9063), tensor(13.9302),
tensor(14.2304), tensor(14.3369), tensor(13.9582), tensor(15.6745),
tensor(12.4167), tensor(11.2723), tensor(11.8569), tensor(12.8731),
tensor(15.9145), tensor(16.1191), tensor(13.1320), tensor(11.3081),
tensor(14.4013), tensor(12.4669), tensor(14.9960), tensor(12.0804),
tensor(14.4546), tensor(15.2588), tensor(12.7535), tensor(12.6015),
tensor(13.0326), tensor(15.4120), tensor(16.2314), tensor(10.7389),
tensor(18.4897), tensor(14.2331), tensor(10.9182), tensor(13.4050),
tensor(12.1767), tensor(15.4316), tensor(12.6371), tensor(13.8341),
tensor(15.8745), tensor(13.0296), tensor(15.2454), tensor(14.7941),
tensor(15.0340), tensor(13.5281)]
[tensor(11.8410), tensor(11.0731), tensor(12.0994), tensor(8.5556),
tensor(10.6256), tensor(11.9156), tensor(10.8770), tensor(10.7589),
tensor(10.7400), tensor(8.9175), tensor(9.8484), tensor(10.6755),
tensor(10.0235), tensor(11.4242), tensor(12.3593), tensor(10.7081),
tensor(10.0794), tensor(8.5032), tensor(10.5071), tensor(9.8170),
tensor(12.2829), tensor(11.4869), tensor(11.7725), tensor(7.9629),
tensor(9.8261), tensor(9.2906), tensor(12.0295), tensor(9.6867),
tensor(11.1946), tensor(13.1927), tensor(8.7548), tensor(10.4610),
tensor(10.4585), tensor(11.5874), tensor(11.6242), tensor(9.2151),
tensor(13.2942), tensor(12.4581), tensor(10.2615), tensor(11.2828),
tensor(10.5870), tensor(12.2755), tensor(10.5547), tensor(10.5976),
tensor(11.2297), tensor(9.5708), tensor(11.9201), tensor(11.0357),
tensor(12.3653), tensor(11.1282)]
[tensor(16.5900), tensor(13.9272), tensor(15.6541), tensor(10.8125),
tensor(13.8239), tensor(15.9341), tensor(13.6465), tensor(14.3689),
tensor(15.4679), tensor(11.6817), tensor(12.2928), tensor(14.0702),
tensor(14.7498), tensor(14.5193), tensor(14.6520), tensor(14.9146),
tensor(12.9393), tensor(11.7074), tensor(11.4547), tensor(13.3256),
tensor(16.1580), tensor(15.3379), tensor(14.7226), tensor(10.9590),
tensor(11.9886), tensor(13.4167), tensor(15.7268), tensor(12.1226),
tensor(14.3753), tensor(16.4871), tensor(12.4576), tensor(13.0148),
tensor(12.5417), tensor(16.1215), tensor(16.6094), tensor(10.9286),
tensor(16.7891), tensor(15.6065), tensor(12.1857), tensor(15.1181),
tensor(12.1639), tensor(16.0965), tensor(12.6885), tensor(14.6789),
tensor(15.2894), tensor(12.7835), tensor(14.5900), tensor(14.6921),
tensor(16.0947), tensor(12.8564)]
[tensor(14.4563), tensor(14.3707), tensor(15.0505), tensor(11.3192),
tensor(14.2226), tensor(15.2325), tensor(12.4100), tensor(13.9918),
tensor(14.2861), tensor(11.7432), tensor(12.1967), tensor(15.1667),
tensor(12.8957), tensor(14.5567), tensor(14.7319), tensor(15.0200),
tensor(13.0344), tensor(11.6637), tensor(13.2221), tensor(13.5860),
tensor(15.5136), tensor(13.9835), tensor(12.8163), tensor(11.2626),
```

```
tensor(13.4655), tensor(12.6569), tensor(15.3520), tensor(13.8950),
tensor(14.1562), tensor(16.8864), tensor(10.5765), tensor(13.2280),
tensor(13.7077), tensor(14.7613), tensor(15.4406), tensor(11.6513),
tensor(17.4453), tensor(14.4298), tensor(12.1925), tensor(14.0485),
tensor(11.6224), tensor(14.8666), tensor(12.9489), tensor(12.9640),
tensor(14.7857), tensor(13.2117), tensor(14.6841), tensor(14.1334),
tensor(14.8610), tensor(12.9171)]
[tensor(12.1019), tensor(10.6171), tensor(11.1336), tensor(8.4384),
tensor(9.2620), tensor(12.1606), tensor(10.3332), tensor(11.7478),
tensor(11.2354), tensor(9.8341), tensor(9.3679), tensor(11.1181),
tensor(10.1790), tensor(10.5410), tensor(10.0212), tensor(9.7656),
tensor(9.3786), tensor(9.4070), tensor(7.6584), tensor(10.5614),
tensor(11.1755), tensor(11.6578), tensor(11.0928), tensor(8.3991),
tensor(10.3682), tensor(9.6036), tensor(12.0423), tensor(8.7701),
tensor(10.5237), tensor(10.9476), tensor(8.9868), tensor(9.1582),
tensor(9.0536), tensor(11.7652), tensor(12.7641), tensor(8.2883),
tensor(12.5980), tensor(10.4711), tensor(8.7888), tensor(10.4055),
tensor(8.9194), tensor(11.7565), tensor(9.1830), tensor(11.2121),
tensor(10.8298), tensor(9.1807), tensor(10.4089), tensor(11.6171),
tensor(10.9536), tensor(9.6207)]
[tensor(12.0901), tensor(12.1704), tensor(11.1179), tensor(8.7317),
tensor(10.0351), tensor(10.9881), tensor(9.7318), tensor(10.2154),
tensor(11.4744), tensor(9.9711), tensor(7.9770), tensor(11.5172),
tensor(10.8620), tensor(10.3026), tensor(10.5872), tensor(10.5008),
tensor(10.1561), tensor(10.2500), tensor(9.7599), tensor(10.4027),
tensor(12.1635), tensor(12.2851), tensor(11.3818), tensor(7.6117),
tensor(11.6367), tensor(10.1796), tensor(12.0418), tensor(9.6282),
tensor(10.6997), tensor(12.6880), tensor(8.8886), tensor(11.1217),
tensor(9.7537), tensor(12.0526), tensor(12.1026), tensor(9.0173),
tensor(13.5641), tensor(12.2122), tensor(9.2510), tensor(10.1064),
tensor(8.4972), tensor(11.4513), tensor(10.1072), tensor(11.2754),
tensor(11.2871), tensor(9.5652), tensor(11.1231), tensor(12.0074),
tensor(11.9609), tensor(11.4388)]
[tensor(15.7612), tensor(15.1320), tensor(14.9102), tensor(10.6177),
tensor(14.9610), tensor(14.9556), tensor(13.0111), tensor(13.3935),
tensor(14.4622), tensor(11.3848), tensor(11.6360), tensor(14.2168),
tensor(14.1496), tensor(13.6220), tensor(13.7552), tensor(14.0399),
tensor(12.7170), tensor(11.6836), tensor(13.0227), tensor(12.6443),
tensor(16.1293), tensor(15.1412), tensor(13.2960), tensor(11.6531),
tensor(13.2334), tensor(12.8725), tensor(14.5139), tensor(12.6816),
tensor(14.7777), tensor(16.2941), tensor(12.0967), tensor(12.8056),
tensor(13.2945), tensor(15.3942), tensor(15.4604), tensor(11.4388),
tensor(17.8447), tensor(14.7490), tensor(11.6626), tensor(14.5566),
tensor(12.0259), tensor(14.9036), tensor(11.9944), tensor(14.0034),
tensor(14.6681), tensor(12.9183), tensor(14.4319), tensor(13.7884),
tensor(14.6020), tensor(13.0802)]
[tensor(11.5800), tensor(11.7718), tensor(12.5178), tensor(9.1066),
tensor(11.0042), tensor(12.6108), tensor(9.7688), tensor(10.2978),
```

```
tensor(11.2326), tensor(8.8179), tensor(9.2906), tensor(12.9962),
tensor(9.8882), tensor(10.1536), tensor(11.6845), tensor(10.9238),
tensor(10.8222), tensor(10.0471), tensor(10.6939), tensor(10.9687),
tensor(12.0123), tensor(11.8774), tensor(10.0668), tensor(8.8097),
tensor(11.8059), tensor(10.1852), tensor(11.9650), tensor(10.0486),
tensor(11.0381), tensor(13.9981), tensor(8.4829), tensor(11.2822),
tensor(10.6074), tensor(11.9255), tensor(12.1377), tensor(9.2861),
tensor(14.2655), tensor(13.0855), tensor(9.6504), tensor(11.4322),
tensor(9.1865), tensor(10.4785), tensor(10.3892), tensor(10.7895),
tensor(11.6070), tensor(10.9337), tensor(11.5323), tensor(11.0802),
tensor(11.1483), tensor(10.5412)]
[tensor(15.9881), tensor(14.1757), tensor(16.0821), tensor(11.0402),
tensor(14.3343), tensor(15.9611), tensor(13.6490), tensor(13.7449),
tensor(14.7666), tensor(11.4139), tensor(12.5453), tensor(14.8924),
tensor(13.5282), tensor(14.5707), tensor(14.2776), tensor(14.2909),
tensor(12.9106), tensor(12.9111), tensor(11.8935), tensor(14.4577),
tensor(15.4473), tensor(15.1292), tensor(14.8494), tensor(11.1546),
tensor(13.2201), tensor(13.6810), tensor(14.9623), tensor(13.4287),
tensor(14.4528), tensor(16.2960), tensor(12.2250), tensor(13.5702),
tensor(13.0765), tensor(15.9282), tensor(16.7934), tensor(10.7930),
tensor(16.5681), tensor(15.0163), tensor(12.5284), tensor(14.3483),
tensor(10.8906), tensor(15.9651), tensor(12.3451), tensor(14.9533),
tensor(14.6436), tensor(12.9113), tensor(13.9399), tensor(14.7032),
tensor(15.0419), tensor(12.7887)]
[tensor(14.2642), tensor(13.2611), tensor(13.8601), tensor(11.1167),
tensor(12.6919), tensor(14.1503), tensor(11.4128), tensor(13.7856),
tensor(13.7976), tensor(11.9857), tensor(10.5069), tensor(13.9365),
tensor(12.2076), tensor(12.5520), tensor(12.0889), tensor(13.3435),
tensor(12.2043), tensor(12.5075), tensor(11.4321), tensor(11.7368),
tensor(14.1587), tensor(14.3804), tensor(12.9536), tensor(10.4718),
tensor(12.2706), tensor(12.8708), tensor(15.1733), tensor(12.7382),
tensor(12.6968), tensor(14.9446), tensor(10.0954), tensor(12.0705),
tensor(11.9085), tensor(14.3965), tensor(14.7553), tensor(11.7893),
tensor(16.0024), tensor(12.8606), tensor(10.5967), tensor(12.5068),
tensor(11.1616), tensor(13.1446), tensor(12.3807), tensor(13.4066),
tensor(13.8980), tensor(11.6252), tensor(14.0769), tensor(13.8294),
tensor(13.9016), tensor(11.7145)]
[tensor(14.9036), tensor(13.4416), tensor(14.7799), tensor(10.0999),
tensor(13.4241), tensor(14.5038), tensor(13.0782), tensor(12.3659),
tensor(13.0067), tensor(10.2611), tensor(11.3621), tensor(12.8844),
tensor(12.5630), tensor(13.2924), tensor(13.0458), tensor(13.4181),
tensor(10.8734), tensor(11.7947), tensor(12.2919), tensor(11.4121),
tensor(14.5334), tensor(14.0218), tensor(12.6312), tensor(10.1375),
tensor(11.6363), tensor(13.1687), tensor(14.0255), tensor(10.8859),
tensor(12.8323), tensor(15.1917), tensor(10.6679), tensor(12.6603),
tensor(12.2114), tensor(14.9855), tensor(14.6271), tensor(9.9762),
tensor(14.8513), tensor(14.1182), tensor(12.0679), tensor(14.0179),
tensor(9.6915), tensor(13.7148), tensor(10.8327), tensor(14.5939),
```

```
tensor(13.1926), tensor(10.5033), tensor(12.3026), tensor(12.3704),
tensor(13.5397), tensor(12.6293)]
[tensor(15.0919), tensor(15.6734), tensor(16.7017), tensor(12.0244),
tensor(14.8489), tensor(16.4541), tensor(13.6065), tensor(14.6482),
tensor(15.8344), tensor(11.5235), tensor(12.1378), tensor(15.2127),
tensor(13.8116), tensor(15.6657), tensor(15.1253), tensor(14.5258),
tensor(12.8400), tensor(13.4628), tensor(13.0999), tensor(14.3702),
tensor(16.8261), tensor(17.1369), tensor(14.5494), tensor(11.9047),
tensor(13.8444), tensor(14.6501), tensor(16.1964), tensor(13.2128),
tensor(15.1334), tensor(17.4742), tensor(12.3846), tensor(14.8265),
tensor(12.6485), tensor(15.6893), tensor(16.9812), tensor(12.0017),
tensor(17.6636), tensor(15.8802), tensor(12.5389), tensor(14.3481),
tensor(11.6160), tensor(15.9902), tensor(14.3943), tensor(15.3609),
tensor(15.3513), tensor(13.5135), tensor(15.3700), tensor(14.8069),
tensor(15.7063), tensor(13.8987)]
[tensor(13.3150), tensor(13.3833), tensor(14.4127), tensor(10.5511),
tensor(13.0322), tensor(14.1973), tensor(12.9972), tensor(12.9767),
tensor(13.7032), tensor(11.4412), tensor(12.2824), tensor(13.5970),
tensor(12.6468), tensor(13.7792), tensor(12.9508), tensor(12.6134),
tensor(12.4571), tensor(11.1159), tensor(11.5213), tensor(12.9656),
tensor(13.7971), tensor(14.0172), tensor(14.1796), tensor(9.8928),
tensor(12.2983), tensor(11.5225), tensor(14.5504), tensor(11.6023),
tensor(13.7669), tensor(15.1381), tensor(10.8380), tensor(12.3550),
tensor(12.6480), tensor(14.1666), tensor(14.4889), tensor(11.1446),
tensor(15.7376), tensor(14.6648), tensor(11.3947), tensor(13.4909),
tensor(11.0821), tensor(13.6162), tensor(11.5147), tensor(13.1061),
tensor(13.2339), tensor(12.6281), tensor(13.8624), tensor(13.5629),
tensor(13.5151), tensor(12.4366)]
[tensor(13.3791), tensor(13.3182), tensor(13.8246), tensor(10.3347),
tensor(12.1335), tensor(13.6193), tensor(13.0925), tensor(12.8955),
tensor(12.7574), tensor(9.8947), tensor(10.7419), tensor(14.3300),
tensor(12.2836), tensor(12.4783), tensor(13.3363), tensor(12.5965),
tensor(10.9872), tensor(11.3811), tensor(11.2981), tensor(12.7050),
tensor(14.1636), tensor(13.8639), tensor(12.8995), tensor(10.3329),
tensor(11.6422), tensor(11.4016), tensor(13.9326), tensor(12.5852),
tensor(13.4981), tensor(15.2701), tensor(10.5503), tensor(12.4564),
tensor(12.6537), tensor(13.3212), tensor(14.2930), tensor(10.0855),
tensor(15.7201), tensor(13.5021), tensor(10.4193), tensor(12.4262),
tensor(11.1007), tensor(13.5549), tensor(11.4448), tensor(12.5510),
tensor(12.5090), tensor(11.4517), tensor(13.8888), tensor(12.6520),
tensor(14.0955), tensor(11.4870)]
[tensor(11.2496), tensor(10.9115), tensor(11.3717), tensor(8.6234),
tensor(10.6254), tensor(10.6162), tensor(9.3483), tensor(9.8484),
tensor(11.1174), tensor(8.2080), tensor(8.6489), tensor(10.6787),
tensor(10.1464), tensor(10.7313), tensor(10.9061), tensor(10.4714),
tensor(8.8989), tensor(8.8982), tensor(9.8975), tensor(9.7278), tensor(11.4439),
tensor(11.6748), tensor(10.7585), tensor(8.4493), tensor(9.3296),
tensor(9.2043), tensor(11.3520), tensor(9.8336), tensor(10.5596),
```

```
tensor(12.2128), tensor(8.8332), tensor(10.5022), tensor(9.8942),
tensor(12.0213), tensor(12.3306), tensor(8.5389), tensor(12.7567),
tensor(11.4702), tensor(9.1551), tensor(11.4849), tensor(8.4941),
tensor(11.4984), tensor(9.2246), tensor(10.3141), tensor(11.4039),
tensor(10.4383), tensor(11.2846), tensor(9.8682), tensor(12.1869),
tensor(10.4110)]
[tensor(13.9681), tensor(13.2937), tensor(14.8140), tensor(10.4639),
tensor(12.4097), tensor(14.3901), tensor(11.9806), tensor(13.7503),
tensor(13.1812), tensor(11.4924), tensor(11.8701), tensor(12.8251),
tensor(11.8167), tensor(13.7811), tensor(13.3003), tensor(12.9754),
tensor(12.1160), tensor(10.4519), tensor(12.5097), tensor(12.0987),
tensor(13.4018), tensor(14.3733), tensor(12.9394), tensor(9.7638),
tensor(13.0775), tensor(11.7887), tensor(15.4182), tensor(11.9610),
tensor(13.2806), tensor(14.5184), tensor(10.4528), tensor(11.6513),
tensor(12.0086), tensor(15.4651), tensor(14.8130), tensor(11.7545),
tensor(17.2912), tensor(14.0426), tensor(11.2897), tensor(13.8242),
tensor(11.4874), tensor(14.4511), tensor(12.0623), tensor(13.6390),
tensor(15.2661), tensor(12.0979), tensor(13.9013), tensor(12.9068),
tensor(13.8276), tensor(12.2874)]
[tensor(13.9166), tensor(13.0670), tensor(13.9459), tensor(9.7689),
tensor(12.5881), tensor(14.1022), tensor(12.4271), tensor(12.3869),
tensor(13.5612), tensor(10.9990), tensor(11.0030), tensor(14.0180),
tensor(12.6541), tensor(12.3971), tensor(13.5452), tensor(13.0219),
tensor(11.8423), tensor(11.2926), tensor(10.8061), tensor(13.1349),
tensor(14.7407), tensor(13.9131), tensor(12.9314), tensor(9.2608),
tensor(12.6760), tensor(11.7513), tensor(14.0788), tensor(11.3079),
tensor(12.8973), tensor(15.0087), tensor(10.8099), tensor(11.6538),
tensor(11.8662), tensor(13.0923), tensor(13.7641), tensor(10.3764),
tensor(15.5274), tensor(13.6790), tensor(11.4326), tensor(12.2451),
tensor(10.5091), tensor(13.2303), tensor(11.7989), tensor(12.4820),
tensor(13.3405), tensor(11.7062), tensor(13.2516), tensor(13.7113),
tensor(14.2978), tensor(12.6386)]
[tensor(15.2028), tensor(14.9105), tensor(15.5843), tensor(12.1183),
tensor(12.4313), tensor(14.2321), tensor(12.8076), tensor(14.0494),
tensor(13.9104), tensor(12.4160), tensor(11.3516), tensor(14.5891),
tensor(12.3658), tensor(14.6371), tensor(13.1250), tensor(13.1395),
tensor(12.8435), tensor(11.6597), tensor(11.8257), tensor(12.6196),
tensor(14.7906), tensor(14.4175), tensor(13.7128), tensor(10.6683),
tensor(14.2531), tensor(12.1658), tensor(14.9147), tensor(11.8950),
tensor(14.3793), tensor(15.5164), tensor(10.9396), tensor(12.2234),
tensor(12.8038), tensor(14.8339), tensor(15.2054), tensor(11.3475),
tensor(17.5851), tensor(15.2738), tensor(12.0238), tensor(14.2049),
tensor(10.9032), tensor(14.4779), tensor(12.2258), tensor(14.3365),
tensor(13.4380), tensor(12.5353), tensor(13.1146), tensor(13.9210),
tensor(15.0087), tensor(14.5846)]
[tensor(12.2659), tensor(13.6517), tensor(11.7319), tensor(8.3417),
tensor(11.8323), tensor(13.4565), tensor(10.3108), tensor(12.8689),
tensor(11.8824), tensor(9.7634), tensor(9.8079), tensor(13.0413),
```

```
tensor(11.5414), tensor(11.4699), tensor(11.9384), tensor(10.9919),
tensor(11.3146), tensor(10.6289), tensor(10.9506), tensor(10.6992),
tensor(12.7511), tensor(12.0616), tensor(12.1248), tensor(9.5432),
tensor(12.6008), tensor(10.8973), tensor(11.9161), tensor(10.7806),
tensor(11.8872), tensor(13.0460), tensor(9.0729), tensor(10.7479),
tensor(11.1037), tensor(13.3215), tensor(13.2724), tensor(9.7992),
tensor(15.0476), tensor(12.1503), tensor(10.7789), tensor(10.6304),
tensor(11.3092), tensor(13.4275), tensor(10.7771), tensor(12.3946),
tensor(13.2984), tensor(10.5452), tensor(12.8161), tensor(12.7665),
tensor(12.3162), tensor(11.6322)]
[tensor(15.9427), tensor(13.9925), tensor(14.1946), tensor(10.7397),
tensor(14.4062), tensor(15.2048), tensor(12.9893), tensor(13.6789),
tensor(14.5094), tensor(10.4373), tensor(12.4495), tensor(13.5491),
tensor(14.0529), tensor(13.8911), tensor(13.9306), tensor(12.6829),
tensor(11.7674), tensor(12.4257), tensor(11.1013), tensor(11.8508),
tensor(15.4143), tensor(14.4300), tensor(13.1174), tensor(11.6318),
tensor(11.8234), tensor(14.3017), tensor(13.5092), tensor(12.7964),
tensor(14.3094), tensor(15.2895), tensor(10.4102), tensor(13.6151),
tensor(13.3442), tensor(15.3436), tensor(15.1289), tensor(11.0096),
tensor(15.9642), tensor(13.1540), tensor(11.3756), tensor(12.9159),
tensor(11.7455), tensor(15.2630), tensor(11.1297), tensor(13.9196),
tensor(14.0562), tensor(12.2019), tensor(14.0697), tensor(13.7510),
tensor(13.9042), tensor(12.2642)]
[tensor(11.3908), tensor(10.4254), tensor(11.4897), tensor(9.2033),
tensor(10.7157), tensor(11.2894), tensor(9.9069), tensor(10.3620),
tensor(11.5845), tensor(9.4577), tensor(9.0618), tensor(11.5667),
tensor(10.5960), tensor(11.3963), tensor(10.4234), tensor(10.4418),
tensor(10.1915), tensor(8.6912), tensor(9.0818), tensor(10.2612),
tensor(10.2966), tensor(12.1275), tensor(10.7497), tensor(9.5827),
tensor(10.2170), tensor(10.1644), tensor(11.3191), tensor(9.9559),
tensor(11.0804), tensor(12.2558), tensor(8.6219), tensor(10.6845),
tensor(10.6270), tensor(12.5589), tensor(12.4833), tensor(8.5459),
tensor(14.9321), tensor(12.8394), tensor(8.8401), tensor(11.2474),
tensor(9.6157), tensor(10.9833), tensor(9.4731), tensor(11.3616),
tensor(11.8237), tensor(11.5330), tensor(12.5255), tensor(11.0204),
tensor(11.4238), tensor(10.1954)]
[tensor(14.5071), tensor(11.9794), tensor(13.4203), tensor(10.9935),
tensor(12.1392), tensor(13.1993), tensor(11.3266), tensor(12.9513),
tensor(13.9506), tensor(10.6283), tensor(11.2322), tensor(12.7405),
tensor(12.3404), tensor(13.7905), tensor(12.6369), tensor(13.2071),
tensor(12.6599), tensor(10.4402), tensor(9.9748), tensor(12.6274),
tensor(13.5267), tensor(13.7416), tensor(12.7968), tensor(10.2618),
tensor(11.4345), tensor(11.6601), tensor(14.1686), tensor(11.8170),
tensor(12.6014), tensor(14.8029), tensor(10.1185), tensor(11.7277),
tensor(11.0738), tensor(13.8521), tensor(13.9805), tensor(10.4067),
tensor(15.8617), tensor(13.9626), tensor(10.0696), tensor(13.3286),
tensor(10.9574), tensor(14.1728), tensor(11.2855), tensor(13.2294),
tensor(12.4204), tensor(11.2394), tensor(13.2192), tensor(13.4610),
```

```
tensor(13.2437), tensor(11.3475)]
[tensor(16.1888), tensor(15.1709), tensor(14.9660), tensor(11.2880),
tensor(14.2621), tensor(16.6059), tensor(12.8720), tensor(12.8777),
tensor(15.0246), tensor(10.4903), tensor(10.9549), tensor(14.0668),
tensor(14.0860), tensor(13.0576), tensor(14.2119), tensor(13.8330),
tensor(13.3562), tensor(11.9915), tensor(12.1489), tensor(12.0758),
tensor(16.0310), tensor(15.2788), tensor(13.2297), tensor(11.1640),
tensor(12.5161), tensor(13.4218), tensor(15.0138), tensor(12.0801),
tensor(14.1535), tensor(15.9426), tensor(11.8619), tensor(14.3436),
tensor(11.6600), tensor(15.6320), tensor(14.9782), tensor(11.0374),
tensor(17.0235), tensor(13.9993), tensor(11.2419), tensor(14.0589),
tensor(11.4206), tensor(15.0834), tensor(12.0553), tensor(13.7912),
tensor(13.9764), tensor(12.4305), tensor(13.8883), tensor(14.2804),
tensor(14.1798), tensor(12.3605)]
[tensor(14.0580), tensor(12.4707), tensor(12.9577), tensor(10.4376),
tensor(11.3972), tensor(12.5045), tensor(11.6755), tensor(11.2158),
tensor(12.2935), tensor(10.8708), tensor(10.6136), tensor(12.9598),
tensor(11.3544), tensor(13.0250), tensor(12.5861), tensor(12.3956),
tensor(11.0935), tensor(9.9584), tensor(10.6812), tensor(12.5129),
tensor(13.4747), tensor(13.8150), tensor(11.3425), tensor(9.8274),
tensor(12.2056), tensor(11.6106), tensor(12.7388), tensor(9.5587),
tensor(11.9125), tensor(13.6566), tensor(10.4374), tensor(11.4682),
tensor(10.8507), tensor(12.6399), tensor(13.2616), tensor(8.7656),
tensor(15.6287), tensor(14.3717), tensor(10.9720), tensor(12.3485),
tensor(8.9618), tensor(12.3381), tensor(10.6937), tensor(13.4844),
tensor(12.0059), tensor(10.2876), tensor(12.1722), tensor(12.2848),
tensor(12.5160), tensor(12.9496)]
[tensor(16.2533), tensor(15.8330), tensor(16.5076), tensor(12.2466),
tensor(13.8243), tensor(17.8329), tensor(14.6426), tensor(14.4082),
tensor(15.1880), tensor(12.5279), tensor(12.5019), tensor(16.3034),
tensor(14.4930), tensor(15.2772), tensor(15.2914), tensor(15.0696),
tensor(13.8139), tensor(13.0955), tensor(13.3967), tensor(14.2069),
tensor(15.7329), tensor(16.4494), tensor(15.6183), tensor(10.9849),
tensor(14.7153), tensor(13.3278), tensor(16.8887), tensor(13.6660),
tensor(15.0442), tensor(16.8362), tensor(11.8425), tensor(15.0259),
tensor(13.4894), tensor(16.8373), tensor(16.6882), tensor(11.9868),
tensor(18.5695), tensor(16.4765), tensor(12.9755), tensor(15.4402),
tensor(12.8212), tensor(15.7570), tensor(13.5986), tensor(15.3790),
tensor(15.7234), tensor(14.4884), tensor(15.8895), tensor(15.7239),
tensor(15.3068), tensor(14.7492)]
[tensor(15.7968), tensor(13.7599), tensor(14.8760), tensor(11.2879),
tensor(13.0707), tensor(14.7069), tensor(12.4962), tensor(13.5374),
tensor(13.7668), tensor(11.6481), tensor(12.1680), tensor(13.5223),
tensor(13.0861), tensor(13.9115), tensor(13.7203), tensor(14.0284),
tensor(12.6064), tensor(11.2198), tensor(11.3216), tensor(11.9338),
tensor(13.8507), tensor(14.2555), tensor(13.0731), tensor(10.4711),
tensor(12.7995), tensor(12.1363), tensor(14.5820), tensor(12.4990),
tensor(13.9204), tensor(15.6963), tensor(10.7449), tensor(12.4342),
```

```
tensor(12.4095), tensor(15.1148), tensor(15.0808), tensor(10.6112),
tensor(16.3191), tensor(14.7844), tensor(11.8122), tensor(14.0209),
tensor(10.8244), tensor(14.2743), tensor(11.7687), tensor(13.2026),
tensor(13.7004), tensor(12.4398), tensor(13.3147), tensor(14.4187),
tensor(14.3953), tensor(12.7293)]
[tensor(14.2001), tensor(12.3499), tensor(13.6994), tensor(9.2356),
tensor(11.7879), tensor(12.7662), tensor(11.5246), tensor(12.2883),
tensor(12.8802), tensor(10.9791), tensor(11.2452), tensor(12.2645),
tensor(10.0181), tensor(12.6405), tensor(12.2882), tensor(13.1244),
tensor(11.7542), tensor(10.5272), tensor(10.0742), tensor(12.1444),
tensor(12.9772), tensor(13.4055), tensor(12.3144), tensor(9.7339),
tensor(12.3306), tensor(11.4835), tensor(12.3891), tensor(11.1816),
tensor(12.3077), tensor(13.1524), tensor(10.1285), tensor(10.9663),
tensor(10.8940), tensor(13.7324), tensor(14.3291), tensor(10.2432),
tensor(15.4935), tensor(13.0227), tensor(9.8812), tensor(12.3013),
tensor(9.7506), tensor(13.4338), tensor(10.1088), tensor(12.6818),
tensor(13.1761), tensor(10.6212), tensor(11.9503), tensor(13.0199),
tensor(13.1336), tensor(11.6861)]
[tensor(16.8259), tensor(14.9165), tensor(15.4528), tensor(12.4164),
tensor(14.5644), tensor(16.2436), tensor(13.3959), tensor(14.6374),
tensor(16.0451), tensor(12.7641), tensor(12.7654), tensor(15.2456),
tensor(15.6412), tensor(15.5522), tensor(14.9027), tensor(15.2617),
tensor(13.0510), tensor(11.8748), tensor(12.2748), tensor(14.6433),
tensor(15.8644), tensor(16.9593), tensor(14.1078), tensor(13.1676),
tensor(14.3788), tensor(13.8727), tensor(15.7878), tensor(14.0302),
tensor(15.9262), tensor(16.3714), tensor(12.8289), tensor(14.1946),
tensor(13.6860), tensor(17.2088), tensor(17.8122), tensor(11.8902),
tensor(19.5306), tensor(16.2864), tensor(12.5739), tensor(14.6507),
tensor(12.2179), tensor(15.9232), tensor(14.0173), tensor(15.5952),
tensor(15.8953), tensor(13.6087), tensor(16.0169), tensor(15.3101),
tensor(15.4740), tensor(13.4481)]
[tensor(13.6303), tensor(11.5893), tensor(12.3586), tensor(10.2224),
tensor(10.3415), tensor(13.1055), tensor(11.8703), tensor(11.4678),
tensor(12.5561), tensor(10.7270), tensor(10.4659), tensor(12.6689),
tensor(11.3413), tensor(12.2234), tensor(12.5779), tensor(11.2574),
tensor(12.0109), tensor(9.9784), tensor(9.5890), tensor(11.9779),
tensor(13.3778), tensor(13.3732), tensor(11.9802), tensor(9.1965),
tensor(11.9031), tensor(10.3091), tensor(13.6261), tensor(9.7909),
tensor(11.4035), tensor(12.9927), tensor(10.0939), tensor(11.5099),
tensor(9.9565), tensor(12.3238), tensor(12.7492), tensor(9.1710),
tensor(15.1872), tensor(13.4875), tensor(10.2183), tensor(11.9544),
tensor(10.0425), tensor(12.5019), tensor(10.9064), tensor(12.3780),
tensor(11.9125), tensor(10.1541), tensor(12.5155), tensor(13.0960),
tensor(12.6228), tensor(11.5549)]
[tensor(13.1968), tensor(12.8609), tensor(13.1323), tensor(9.5770),
tensor(12.8736), tensor(13.8929), tensor(12.1525), tensor(12.4011),
tensor(13.8123), tensor(10.5525), tensor(10.8128), tensor(12.7136),
tensor(11.9187), tensor(12.7440), tensor(12.6251), tensor(12.1868),
```

```
tensor(12.0969), tensor(10.1771), tensor(11.4374), tensor(13.2868),
tensor(13.5152), tensor(13.8419), tensor(11.8105), tensor(9.8383),
tensor(13.0320), tensor(11.3746), tensor(13.5352), tensor(9.5185),
tensor(12.3173), tensor(14.7589), tensor(11.0394), tensor(12.1054),
tensor(11.3681), tensor(14.0710), tensor(15.2012), tensor(9.4557),
tensor(16.1117), tensor(13.7427), tensor(11.5034), tensor(13.2638),
tensor(10.0904), tensor(12.8775), tensor(10.6903), tensor(13.8706),
tensor(13.5728), tensor(11.5866), tensor(13.7016), tensor(12.8709),
tensor(12.5223), tensor(11.7081)]
[tensor(12.8366), tensor(12.0355), tensor(14.0603), tensor(9.7250),
tensor(11.5146), tensor(13.4741), tensor(11.4479), tensor(10.9197),
tensor(13.9526), tensor(10.4654), tensor(10.5292), tensor(12.4507),
tensor(12.2857), tensor(13.5560), tensor(12.6707), tensor(12.2315),
tensor(11.1189), tensor(9.9509), tensor(10.2728), tensor(12.5807),
tensor(13.1348), tensor(13.0011), tensor(12.6653), tensor(9.5103),
tensor(11.9542), tensor(10.3363), tensor(13.9240), tensor(11.4232),
tensor(11.8101), tensor(14.1450), tensor(10.0220), tensor(11.1768),
tensor(11.2570), tensor(13.9834), tensor(13.8200), tensor(9.8225),
tensor(16.4567), tensor(13.1079), tensor(11.4390), tensor(11.6165),
tensor(10.3915), tensor(12.9371), tensor(12.0695), tensor(12.6949),
tensor(13.5146), tensor(11.3021), tensor(13.5600), tensor(12.1630),
tensor(13.1553), tensor(10.4785)]
[tensor(15.0797), tensor(13.8429), tensor(13.7998), tensor(9.4998),
tensor(13.1231), tensor(14.1218), tensor(11.7226), tensor(12.4334),
tensor(14.4458), tensor(10.7171), tensor(10.2494), tensor(12.5463),
tensor(12.4059), tensor(12.9272), tensor(13.2578), tensor(12.2693),
tensor(11.4431), tensor(11.7168), tensor(10.4447), tensor(12.7601),
tensor(14.5151), tensor(13.7133), tensor(13.1755), tensor(10.2784),
tensor(12.5548), tensor(11.8733), tensor(13.1543), tensor(12.6962),
tensor(13.1527), tensor(14.4583), tensor(10.7427), tensor(11.8065),
tensor(11.9514), tensor(14.0946), tensor(14.6066), tensor(10.2193),
tensor(15.9255), tensor(12.0749), tensor(11.5385), tensor(11.7860),
tensor(10.4512), tensor(15.6697), tensor(10.8605), tensor(12.8707),
tensor(13.0313), tensor(11.4779), tensor(13.7324), tensor(12.8173),
tensor(13.4257), tensor(12.0506)]
[tensor(13.7297), tensor(11.9976), tensor(13.5169), tensor(10.4901),
tensor(11.7956), tensor(14.0400), tensor(12.6080), tensor(11.3547),
tensor(12.1605), tensor(9.3369), tensor(10.8036), tensor(12.5309),
tensor(11.3853), tensor(12.2877), tensor(12.6115), tensor(11.1236),
tensor(10.4654), tensor(10.7545), tensor(11.0288), tensor(11.1284),
tensor(13.1265), tensor(13.5737), tensor(11.8892), tensor(9.3908),
tensor(10.5301), tensor(11.6849), tensor(12.6020), tensor(11.3790),
tensor(12.2845), tensor(13.5607), tensor(10.0629), tensor(11.8881),
tensor(10.6598), tensor(12.6454), tensor(13.3224), tensor(9.6150),
tensor(13.4974), tensor(12.2533), tensor(10.9958), tensor(13.0826),
tensor(9.1134), tensor(12.1101), tensor(10.4117), tensor(11.6210),
tensor(10.9143), tensor(10.9330), tensor(12.2090), tensor(11.9766),
tensor(13.1065), tensor(11.5597)]
```

```
[tensor(16.4916), tensor(14.6626), tensor(15.5034), tensor(10.7487),
tensor(14.0245), tensor(15.5243), tensor(13.3349), tensor(14.9082),
tensor(14.4851), tensor(12.7149), tensor(12.2066), tensor(15.5307),
tensor(13.0239), tensor(14.4300), tensor(15.4137), tensor(15.3674),
tensor(13.3108), tensor(12.1858), tensor(12.3668), tensor(14.1664),
tensor(15.6097), tensor(16.4110), tensor(14.0889), tensor(11.0196),
tensor(13.8237), tensor(13.9668), tensor(15.0331), tensor(13.2127),
tensor(14.1314), tensor(15.7197), tensor(11.0149), tensor(13.0191),
tensor(12.2792), tensor(15.5476), tensor(15.4322), tensor(11.3327),
tensor(17.8169), tensor(15.8720), tensor(11.4848), tensor(14.0636),
tensor(12.0881), tensor(14.9069), tensor(12.6578), tensor(15.6576),
tensor(15.9489), tensor(12.1694), tensor(14.0839), tensor(14.8195),
tensor(15.0040), tensor(14.6338)]
[tensor(16.5174), tensor(14.2041), tensor(13.8406), tensor(14.3545),
tensor(13.7647), tensor(13.6623), tensor(14.8141), tensor(12.1675),
tensor(14.9342), tensor(13.9186), tensor(16.4232), tensor(15.3982),
tensor(11.3516), tensor(14.3230), tensor(11.5099), tensor(12.1710),
tensor(14.2554), tensor(13.6921), tensor(10.2343), tensor(13.6990),
tensor(13.2174), tensor(15.2513), tensor(13.7134), tensor(14.0406),
tensor(15.6384), tensor(14.6701), tensor(13.4447), tensor(11.6970),
tensor(14.3376), tensor(13.8046), tensor(15.2290), tensor(14.4083),
tensor(14.5943), tensor(11.6891), tensor(11.8035), tensor(14.8930),
tensor(13.8531), tensor(16.5139), tensor(15.1142), tensor(13.6101),
tensor(12.1808), tensor(12.2308), tensor(12.6413), tensor(13.4319),
tensor(15.5442), tensor(13.9880), tensor(12.9515), tensor(16.3657),
tensor(12.9508), tensor(13.1567)]
[tensor(14.2767), tensor(12.2229), tensor(10.7961), tensor(12.1613),
tensor(12.1067), tensor(11.7874), tensor(12.2728), tensor(10.7725),
tensor(12.4507), tensor(11.4176), tensor(14.2980), tensor(13.2877),
tensor(10.8726), tensor(12.2756), tensor(10.3619), tensor(11.5585),
tensor(12.2696), tensor(11.5728), tensor(10.3458), tensor(10.5398),
tensor(11.4081), tensor(13.5559), tensor(12.9619), tensor(11.9585),
tensor(12.4653), tensor(12.8966), tensor(12.0081), tensor(11.0078),
tensor(12.3463), tensor(12.9079), tensor(12.6165), tensor(12.2297),
tensor(12.6639), tensor(10.1112), tensor(8.7685), tensor(13.7875),
tensor(11.6749), tensor(13.5911), tensor(12.6020), tensor(11.8539),
tensor(11.0929), tensor(11.3105), tensor(12.0645), tensor(10.9617),
tensor(12.6646), tensor(12.9727), tensor(12.4533), tensor(15.1846),
tensor(10.7290), tensor(12.4534)]
[tensor(13.0564), tensor(13.6418), tensor(13.2387), tensor(13.0786),
tensor(11.7731), tensor(11.0920), tensor(12.8252), tensor(10.5051),
tensor(12.8770), tensor(12.2423), tensor(13.4707), tensor(12.5631),
tensor(10.9489), tensor(12.6370), tensor(10.5100), tensor(10.1823),
tensor(11.7988), tensor(10.8068), tensor(10.3312), tensor(11.4615),
tensor(10.8378), tensor(12.0635), tensor(12.6803), tensor(11.4262),
tensor(11.2902), tensor(12.4960), tensor(12.1139), tensor(11.5303),
tensor(10.3326), tensor(11.3210), tensor(14.2280), tensor(11.9073),
tensor(11.8465), tensor(11.2245), tensor(11.0699), tensor(13.2650),
```

```
tensor(12.5538), tensor(13.6276), tensor(11.6856), tensor(11.6617),
tensor(11.1161), tensor(11.6343), tensor(12.5759), tensor(10.2636),
tensor(12.2075), tensor(12.8210), tensor(12.1028), tensor(14.0789),
tensor(12.4099), tensor(12.4019)]
[tensor(14.9839), tensor(13.0287), tensor(12.6390), tensor(13.7064),
tensor(12.1955), tensor(12.2591), tensor(13.7638), tensor(10.8510),
tensor(12.9500), tensor(12.5089), tensor(13.4315), tensor(13.5989),
tensor(10.2854), tensor(13.1228), tensor(10.3278), tensor(10.6014),
tensor(12.3840), tensor(11.8987), tensor(9.9961), tensor(11.8109),
tensor(10.9636), tensor(13.7671), tensor(12.7637), tensor(12.4427),
tensor(13.3763), tensor(12.3214), tensor(12.6284), tensor(12.2281),
tensor(12.7354), tensor(13.2330), tensor(13.7339), tensor(12.9742),
tensor(13.9149), tensor(11.2421), tensor(11.3528), tensor(13.3657),
tensor(13.3110), tensor(13.8482), tensor(13.3414), tensor(13.1248),
tensor(11.3372), tensor(11.6057), tensor(10.4538), tensor(11.4802),
tensor(13.9303), tensor(12.5783), tensor(12.6367), tensor(14.8246),
tensor(11.2528), tensor(12.1757)]
[tensor(14.2245), tensor(13.2360), tensor(12.0156), tensor(13.6251),
tensor(11.9691), tensor(12.6615), tensor(13.3208), tensor(10.8395),
tensor(12.5019), tensor(12.0884), tensor(13.4769), tensor(14.2871),
tensor(9.8712), tensor(13.2149), tensor(10.5234), tensor(10.4476),
tensor(11.9985), tensor(11.7859), tensor(9.6022), tensor(11.5671),
tensor(11.2685), tensor(13.5902), tensor(12.8455), tensor(12.1687),
tensor(12.1391), tensor(11.6843), tensor(12.1421), tensor(10.6020),
tensor(11.7108), tensor(12.7050), tensor(14.4727), tensor(11.9458),
tensor(12.5722), tensor(11.3061), tensor(10.4545), tensor(12.7255),
tensor(11.5005), tensor(13.4600), tensor(12.7115), tensor(11.1513),
tensor(9.8971), tensor(11.1334), tensor(11.5929), tensor(9.8326),
tensor(12.6907), tensor(12.0213), tensor(12.8304), tensor(15.3598),
tensor(11.2016), tensor(12.2530)]
[tensor(12.9912), tensor(10.6474), tensor(10.5888), tensor(12.1340),
tensor(11.4119), tensor(10.5981), tensor(11.8578), tensor(10.6057),
tensor(12.0235), tensor(10.7074), tensor(12.3203), tensor(12.1102),
tensor(9.8216), tensor(11.6135), tensor(9.3118), tensor(9.9350),
tensor(11.0195), tensor(10.3490), tensor(9.4008), tensor(9.8981),
tensor(10.6100), tensor(11.5588), tensor(11.9069), tensor(10.7221),
tensor(12.6254), tensor(11.5840), tensor(10.7658), tensor(9.8354),
tensor(10.4006), tensor(10.8733), tensor(12.4592), tensor(11.1891),
tensor(10.9542), tensor(9.1392), tensor(8.7038), tensor(11.7747),
tensor(10.5202), tensor(13.3688), tensor(10.9847), tensor(10.5370),
tensor(9.7944), tensor(10.3875), tensor(11.1136), tensor(8.9926),
tensor(11.4429), tensor(12.2120), tensor(10.0421), tensor(13.4184),
tensor(10.1159), tensor(10.8435)]
[tensor(13.9243), tensor(12.4219), tensor(12.3992), tensor(13.9101),
tensor(13.2163), tensor(13.6705), tensor(13.2454), tensor(11.1066),
tensor(14.9498), tensor(14.0568), tensor(14.9596), tensor(13.8660),
tensor(11.7039), tensor(14.0897), tensor(11.6329), tensor(11.0058),
tensor(13.1380), tensor(12.2455), tensor(10.3597), tensor(12.0609),
```

```
tensor(12.7485), tensor(12.7598), tensor(13.4022), tensor(12.7830),
tensor(15.5632), tensor(12.8849), tensor(13.3043), tensor(11.4418),
tensor(11.5067), tensor(12.4660), tensor(14.1209), tensor(13.7705),
tensor(12.9759), tensor(12.2673), tensor(12.3817), tensor(14.1071),
tensor(13.1102), tensor(15.4075), tensor(14.0333), tensor(13.3238),
tensor(11.8329), tensor(12.4714), tensor(13.3163), tensor(11.4983),
tensor(13.0706), tensor(14.3244), tensor(12.5835), tensor(15.0522),
tensor(12.8964), tensor(13.0959)]
[tensor(14.8903), tensor(13.4163), tensor(13.2059), tensor(13.4437),
tensor(13.6375), tensor(13.7050), tensor(14.9724), tensor(11.7670),
tensor(13.9354), tensor(13.5359), tensor(14.9070), tensor(14.6369),
tensor(10.8093), tensor(14.1902), tensor(11.5813), tensor(12.0342),
tensor(12.7458), tensor(11.9052), tensor(10.5758), tensor(13.0930),
tensor(12.0373), tensor(14.4855), tensor(13.0939), tensor(13.0025),
tensor(13.8972), tensor(13.8069), tensor(12.8515), tensor(12.4924),
tensor(12.0938), tensor(12.7157), tensor(14.0668), tensor(13.5356),
tensor(13.4844), tensor(11.8650), tensor(11.4388), tensor(14.0116),
tensor(13.1207), tensor(15.2640), tensor(13.3336), tensor(11.1791),
tensor(11.0042), tensor(11.9227), tensor(11.9520), tensor(10.4240),
tensor(14.1143), tensor(14.0368), tensor(13.2582), tensor(15.4850),
tensor(12.7535), tensor(12.1085)]
[tensor(12.8599), tensor(10.9554), tensor(10.6011), tensor(10.4040),
tensor(11.0179), tensor(12.1761), tensor(11.4296), tensor(9.7749),
tensor(12.1077), tensor(11.7589), tensor(13.1971), tensor(11.2204),
tensor(8.8501), tensor(11.4364), tensor(10.2701), tensor(9.7878),
tensor(12.6172), tensor(9.9055), tensor(8.4370), tensor(11.4798),
tensor(10.6177), tensor(11.9740), tensor(10.8229), tensor(11.2056),
tensor(12.5624), tensor(11.2823), tensor(11.7724), tensor(9.8793),
tensor(10.2207), tensor(11.3578), tensor(11.7702), tensor(11.0548),
tensor(12.1964), tensor(9.1549), tensor(9.3736), tensor(11.7473),
tensor(11.9716), tensor(13.7657), tensor(11.4347), tensor(10.9386),
tensor(10.2733), tensor(10.6065), tensor(11.5616), tensor(9.9757),
tensor(12.1016), tensor(11.4230), tensor(11.0085), tensor(12.8993),
tensor(11.0158), tensor(10.3338)]
[tensor(14.1835), tensor(12.2000), tensor(11.9036), tensor(11.3543),
tensor(11.9136), tensor(12.6990), tensor(12.7746), tensor(10.4846),
tensor(12.8141), tensor(13.4081), tensor(14.1211), tensor(12.6747),
tensor(10.9953), tensor(12.8899), tensor(10.3489), tensor(11.0901),
tensor(12.0793), tensor(11.0493), tensor(9.8235), tensor(12.5396),
tensor(11.2733), tensor(13.7494), tensor(11.5922), tensor(12.8159),
tensor(12.6419), tensor(12.1742), tensor(12.1970), tensor(11.5124),
tensor(11.2120), tensor(12.9535), tensor(13.2406), tensor(12.7546),
tensor(12.2680), tensor(10.6806), tensor(10.2730), tensor(13.9522),
tensor(12.5116), tensor(14.7079), tensor(12.1957), tensor(11.7878),
tensor(11.4880), tensor(10.7919), tensor(12.2278), tensor(10.7656),
tensor(13.0924), tensor(13.2685), tensor(12.6473), tensor(14.4166),
tensor(12.0701), tensor(11.2584)]
[tensor(15.8563), tensor(13.9415), tensor(14.3214), tensor(15.6302),
```

```
tensor(13.8970), tensor(14.0628), tensor(15.0707), tensor(13.0109),
tensor(15.7181), tensor(14.4032), tensor(15.8683), tensor(15.4694),
tensor(13.0675), tensor(16.5127), tensor(11.2908), tensor(12.8315),
tensor(13.3473), tensor(15.1405), tensor(10.9967), tensor(13.6624),
tensor(13.4617), tensor(15.9203), tensor(15.4191), tensor(13.9244),
tensor(16.0284), tensor(14.2250), tensor(15.1602), tensor(13.3938),
tensor(13.6278), tensor(14.7869), tensor(16.5853), tensor(15.4523),
tensor(15.4680), tensor(13.4401), tensor(13.0121), tensor(14.6310),
tensor(15.4830), tensor(17.2616), tensor(15.5251), tensor(14.5584),
tensor(13.3056), tensor(12.8495), tensor(13.8016), tensor(12.1109),
tensor(15.5162), tensor(14.6821), tensor(13.5483), tensor(16.4244),
tensor(14.0796), tensor(13.5262)]
[tensor(15.8247), tensor(13.4827), tensor(13.6871), tensor(14.0508),
tensor(13.3361), tensor(13.1951), tensor(14.0677), tensor(11.7430),
tensor(15.0267), tensor(13.9779), tensor(15.3660), tensor(15.4091),
tensor(11.4754), tensor(14.7483), tensor(10.9263), tensor(12.3305),
tensor(13.6352), tensor(14.0338), tensor(10.8918), tensor(12.9667),
tensor(12.6132), tensor(14.8713), tensor(13.4491), tensor(13.6420),
tensor(14.2850), tensor(13.5730), tensor(14.1253), tensor(12.2004),
tensor(12.0653), tensor(14.3437), tensor(15.1678), tensor(13.9218),
tensor(13.6244), tensor(12.1679), tensor(12.2086), tensor(14.1388),
tensor(14.5340), tensor(16.0345), tensor(14.2205), tensor(13.9200),
tensor(11.8800), tensor(12.9518), tensor(12.7908), tensor(12.6705),
tensor(14.7048), tensor(14.6817), tensor(13.0657), tensor(15.7616),
tensor(13.1852), tensor(12.7159)]
[tensor(15.3371), tensor(14.0879), tensor(13.5740), tensor(15.3269),
tensor(13.2196), tensor(13.2531), tensor(14.2226), tensor(11.8109),
tensor(14.9968), tensor(13.7303), tensor(15.3576), tensor(15.2911),
tensor(11.8687), tensor(14.2398), tensor(13.2958), tensor(11.6212),
tensor(13.8007), tensor(12.6901), tensor(11.1319), tensor(12.6621),
tensor(12.1834), tensor(15.3675), tensor(14.0919), tensor(13.3446),
tensor(14.8969), tensor(13.7130), tensor(14.1507), tensor(11.4173),
tensor(12.8792), tensor(13.9962), tensor(15.5338), tensor(14.0855),
tensor(13.1034), tensor(12.9681), tensor(11.9254), tensor(13.9106),
tensor(13.9580), tensor(16.1177), tensor(13.8784), tensor(14.5460),
tensor(11.1390), tensor(12.4334), tensor(12.3783), tensor(12.1913),
tensor(14.9657), tensor(14.5546), tensor(12.7407), tensor(16.9430),
tensor(12.7963), tensor(13.3865)]
[tensor(14.8088), tensor(12.5109), tensor(11.7910), tensor(12.3106),
tensor(12.5678), tensor(13.8160), tensor(12.6043), tensor(10.5276),
tensor(12.4970), tensor(13.2342), tensor(14.1214), tensor(12.8624),
tensor(12.2831), tensor(13.7245), tensor(10.9730), tensor(11.7024),
tensor(12.6896), tensor(11.9817), tensor(10.8143), tensor(12.9419),
tensor(11.1117), tensor(14.6524), tensor(12.6477), tensor(13.6748),
tensor(13.7157), tensor(12.8792), tensor(13.3596), tensor(12.0722),
tensor(11.8044), tensor(12.5294), tensor(12.9209), tensor(13.1304),
tensor(12.7032), tensor(11.1540), tensor(10.9140), tensor(13.9031),
tensor(13.5123), tensor(14.9962), tensor(12.6545), tensor(13.5434),
```

```
tensor(12.5839), tensor(11.8690), tensor(11.5463), tensor(11.2888),
tensor(13.6214), tensor(13.7957), tensor(12.7603), tensor(15.2732),
tensor(12.2877), tensor(11.9635)]
[tensor(16.2045), tensor(13.2564), tensor(13.4701), tensor(14.3253),
tensor(12.4734), tensor(13.0238), tensor(14.5294), tensor(12.3187),
tensor(14.2665), tensor(14.2361), tensor(16.4489), tensor(13.6450),
tensor(13.6555), tensor(13.4439), tensor(12.5094), tensor(11.4427),
tensor(13.7730), tensor(12.1982), tensor(11.3578), tensor(13.2004),
tensor(11.7336), tensor(14.8190), tensor(13.8760), tensor(14.3245),
tensor(15.2423), tensor(13.0566), tensor(15.3521), tensor(11.6521),
tensor(13.2034), tensor(13.8959), tensor(15.5627), tensor(13.6607),
tensor(13.3573), tensor(11.9569), tensor(12.2738), tensor(13.8359),
tensor(14.5430), tensor(16.8302), tensor(12.7063), tensor(14.7641),
tensor(12.5658), tensor(12.6086), tensor(12.1691), tensor(12.1531),
tensor(15.7750), tensor(15.8433), tensor(12.4185), tensor(16.0940),
tensor(12.3530), tensor(13.8376)]
[tensor(15.6954), tensor(13.1978), tensor(13.4531), tensor(14.1329),
tensor(13.8503), tensor(12.8943), tensor(14.3461), tensor(11.7773),
tensor(14.1522), tensor(13.8752), tensor(15.4861), tensor(13.4937),
tensor(11.9102), tensor(13.7663), tensor(12.4294), tensor(12.4554),
tensor(13.5607), tensor(12.2621), tensor(10.1295), tensor(13.7699),
tensor(12.5314), tensor(15.1138), tensor(14.1216), tensor(14.5595),
tensor(13.1897), tensor(13.3663), tensor(13.8700), tensor(12.0715),
tensor(12.2811), tensor(12.9297), tensor(15.7351), tensor(13.7992),
tensor(13.7951), tensor(12.5503), tensor(11.7939), tensor(14.3867),
tensor(12.8842), tensor(15.5060), tensor(14.2977), tensor(13.2994),
tensor(12.0691), tensor(13.0897), tensor(13.3556), tensor(12.0113),
tensor(14.0723), tensor(14.5994), tensor(13.6276), tensor(15.4574),
tensor(13.9677), tensor(12.8813)]
[tensor(13.0829), tensor(11.0489), tensor(11.5548), tensor(11.3083),
tensor(11.4010), tensor(11.3872), tensor(11.2988), tensor(9.1618),
tensor(12.4948), tensor(10.9541), tensor(12.7362), tensor(11.9053),
tensor(10.0838), tensor(11.8263), tensor(10.4174), tensor(9.4454),
tensor(11.8903), tensor(11.0214), tensor(9.2099), tensor(12.1511),
tensor(10.8845), tensor(12.8685), tensor(11.7629), tensor(11.5164),
tensor(12.3441), tensor(12.0880), tensor(10.5890), tensor(10.1072),
tensor(10.4902), tensor(11.3572), tensor(12.2806), tensor(11.7791),
tensor(11.3645), tensor(10.3463), tensor(10.9318), tensor(12.0161),
tensor(11.0353), tensor(14.7792), tensor(12.3576), tensor(12.0832),
tensor(10.1842), tensor(10.2513), tensor(10.9309), tensor(9.7152),
tensor(12.8084), tensor(11.2734), tensor(11.5815), tensor(13.4398),
tensor(11.6540), tensor(10.9191)]
[tensor(12.7284), tensor(12.6115), tensor(12.2316), tensor(12.3618),
tensor(11.4706), tensor(12.6800), tensor(12.2048), tensor(10.9695),
tensor(13.8435), tensor(13.3948), tensor(13.5542), tensor(14.0105),
tensor(10.7346), tensor(13.7088), tensor(11.3668), tensor(10.7843),
tensor(13.0350), tensor(12.2548), tensor(9.3585), tensor(12.1091),
tensor(11.5153), tensor(14.6608), tensor(11.2914), tensor(12.8896),
```

```
tensor(13.3900), tensor(12.3187), tensor(12.4376), tensor(10.6382),
tensor(10.4614), tensor(12.4583), tensor(14.6030), tensor(12.3333),
tensor(12.2260), tensor(11.9683), tensor(9.4550), tensor(13.5232),
tensor(12.0988), tensor(15.2492), tensor(12.6696), tensor(11.4176),
tensor(10.3240), tensor(10.2287), tensor(11.9372), tensor(10.3575),
tensor(13.8828), tensor(12.1128), tensor(12.4739), tensor(14.4376),
tensor(12.6703), tensor(11.0535)]
[tensor(15.9118), tensor(14.2838), tensor(13.5814), tensor(13.2926),
tensor(13.5150), tensor(14.2749), tensor(14.8194), tensor(11.4478),
tensor(14.2357), tensor(13.9533), tensor(15.9547), tensor(13.5465),
tensor(12.9771), tensor(15.0349), tensor(11.9364), tensor(12.7437),
tensor(13.5230), tensor(13.0184), tensor(10.6456), tensor(13.6708),
tensor(12.4696), tensor(15.3785), tensor(13.2418), tensor(14.4867),
tensor(15.1540), tensor(13.8550), tensor(13.6422), tensor(12.7330),
tensor(13.4003), tensor(14.2914), tensor(15.2688), tensor(13.8782),
tensor(14.1339), tensor(11.8079), tensor(13.0252), tensor(14.9381),
tensor(15.0367), tensor(16.1964), tensor(14.9848), tensor(14.3902),
tensor(14.0364), tensor(12.1536), tensor(13.0227), tensor(12.3291),
tensor(15.2029), tensor(15.8275), tensor(13.7550), tensor(15.8121),
tensor(13.4990), tensor(13.3433)]
[tensor(14.6145), tensor(12.2714), tensor(11.5287), tensor(12.8371),
tensor(11.2139), tensor(11.1621), tensor(11.8204), tensor(10.0347),
tensor(12.8074), tensor(11.6316), tensor(13.4489), tensor(12.5807),
tensor(9.7478), tensor(12.4605), tensor(10.0031), tensor(9.8612),
tensor(11.2909), tensor(11.7292), tensor(8.6399), tensor(11.4687),
tensor(11.0115), tensor(13.9015), tensor(12.9409), tensor(11.3880),
tensor(12.6885), tensor(12.0877), tensor(11.5740), tensor(10.5994),
tensor(12.3485), tensor(12.6365), tensor(12.9209), tensor(11.5869),
tensor(13.4388), tensor(11.0626), tensor(10.6376), tensor(13.0312),
tensor(11.5954), tensor(14.0839), tensor(12.6660), tensor(12.1555),
tensor(10.2768), tensor(11.2382), tensor(10.8803), tensor(10.1460),
tensor(11.9894), tensor(11.6687), tensor(11.6229), tensor(14.8619),
tensor(10.7253), tensor(10.9248)]
[tensor(13.9036), tensor(13.4026), tensor(12.5166), tensor(12.8216),
tensor(12.1922), tensor(13.2170), tensor(13.1673), tensor(11.6055),
tensor(14.6975), tensor(12.8078), tensor(14.5295), tensor(14.5132),
tensor(9.6695), tensor(13.1165), tensor(10.8565), tensor(12.1398),
tensor(12.7031), tensor(13.0642), tensor(11.0711), tensor(11.1819),
tensor(12.8171), tensor(13.7546), tensor(13.5907), tensor(14.0451),
tensor(13.6122), tensor(12.7083), tensor(12.5006), tensor(11.1045),
tensor(12.1150), tensor(12.7044), tensor(14.5641), tensor(13.0499),
tensor(13.2504), tensor(12.0778), tensor(11.0837), tensor(13.4661),
tensor(13.5315), tensor(14.9832), tensor(15.1464), tensor(12.0994),
tensor(11.2510), tensor(12.1145), tensor(13.0495), tensor(11.3407),
tensor(13.6220), tensor(13.2927), tensor(13.9161), tensor(14.6021),
tensor(12.1921), tensor(11.8085)]
[tensor(16.0850), tensor(13.6089), tensor(13.2958), tensor(14.8260),
tensor(15.6162), tensor(13.9555), tensor(14.1388), tensor(12.4717),
```

```
tensor(14.9180), tensor(15.3993), tensor(15.0342), tensor(15.4797),
tensor(12.4987), tensor(15.0202), tensor(12.5880), tensor(11.5242),
tensor(13.9902), tensor(12.3349), tensor(12.0178), tensor(14.4068),
tensor(13.1454), tensor(15.9048), tensor(13.6065), tensor(14.7400),
tensor(15.2525), tensor(14.1350), tensor(13.8932), tensor(12.5841),
tensor(12.5586), tensor(14.0475), tensor(16.4470), tensor(15.1676),
tensor(12.9404), tensor(12.7538), tensor(12.3291), tensor(15.1941),
tensor(13.0617), tensor(16.8325), tensor(13.9722), tensor(13.0757),
tensor(11.6238), tensor(12.7604), tensor(13.8059), tensor(11.6938),
tensor(15.0023), tensor(15.5754), tensor(13.7194), tensor(17.0418),
tensor(13.5559), tensor(13.8011)]
[tensor(11.4677), tensor(11.1116), tensor(10.6470), tensor(10.4223),
tensor(10.3187), tensor(10.0883), tensor(10.9293), tensor(9.6775),
tensor(11.4428), tensor(11.7090), tensor(12.2565), tensor(10.3340),
tensor(9.3927), tensor(10.0219), tensor(9.4532), tensor(9.4306),
tensor(10.6187), tensor(9.5953), tensor(8.0241), tensor(10.5633),
tensor(8.7792), tensor(11.3270), tensor(9.9443), tensor(10.5674),
tensor(11.1412), tensor(9.8741), tensor(11.3233), tensor(9.4572),
tensor(9.6398), tensor(10.6517), tensor(12.3582), tensor(9.5692),
tensor(10.7711), tensor(10.1813), tensor(9.2264), tensor(11.8256),
tensor(11.0685), tensor(12.3421), tensor(10.0859), tensor(10.5813),
tensor(9.7670), tensor(9.3570), tensor(9.5346), tensor(9.4976), tensor(11.8926),
tensor(11.4274), tensor(10.9799), tensor(11.6904), tensor(10.6725),
tensor(9.5861)]
[tensor(18.2652), tensor(15.1445), tensor(14.3455), tensor(15.8218),
tensor(15.0227), tensor(15.8975), tensor(16.1228), tensor(12.9451),
tensor(16.4961), tensor(15.5640), tensor(17.4355), tensor(17.5795),
tensor(13.9048), tensor(16.9388), tensor(12.8828), tensor(13.4890),
tensor(15.9922), tensor(15.4371), tensor(11.2223), tensor(14.8538),
tensor(14.9211), tensor(17.1563), tensor(16.1020), tensor(15.8833),
tensor(16.6831), tensor(16.1695), tensor(16.0711), tensor(13.6197),
tensor(15.0511), tensor(16.3540), tensor(17.1189), tensor(15.3205),
tensor(16.0099), tensor(13.8652), tensor(12.9782), tensor(16.9245),
tensor(15.1542), tensor(18.5516), tensor(15.4763), tensor(15.4769),
tensor(14.2531), tensor(14.9479), tensor(14.7774), tensor(14.2012),
tensor(16.8032), tensor(16.1898), tensor(15.3037), tensor(18.2360),
tensor(14.3352), tensor(15.3130)]
[tensor(14.1622), tensor(12.4036), tensor(12.2776), tensor(11.0950),
tensor(11.4012), tensor(10.8296), tensor(12.9135), tensor(10.3094),
tensor(13.2490), tensor(11.6763), tensor(13.7470), tensor(13.6280),
tensor(10.6146), tensor(11.6593), tensor(10.5161), tensor(10.5940),
tensor(12.5449), tensor(11.9657), tensor(9.4220), tensor(12.5558),
tensor(11.4567), tensor(12.2800), tensor(12.9281), tensor(12.3672),
tensor(12.3624), tensor(12.0241), tensor(12.4728), tensor(9.3237),
tensor(10.5871), tensor(12.0419), tensor(14.3635), tensor(11.6599),
tensor(11.5107), tensor(10.5494), tensor(10.0408), tensor(11.7869),
tensor(12.1284), tensor(15.5497), tensor(11.7159), tensor(11.1852),
tensor(10.5892), tensor(11.3122), tensor(10.8417), tensor(10.3902),
```

```
tensor(13.5269), tensor(12.8684), tensor(11.2443), tensor(13.4308),
tensor(11.2036), tensor(11.4264)]
[tensor(14.6456), tensor(12.2122), tensor(11.7661), tensor(13.2936),
tensor(12.0475), tensor(11.6552), tensor(13.1921), tensor(10.2073),
tensor(13.3173), tensor(12.8293), tensor(13.0830), tensor(13.1625),
tensor(11.6641), tensor(11.9978), tensor(9.6074), tensor(10.2395),
tensor(12.2696), tensor(11.9389), tensor(10.4070), tensor(12.0232),
tensor(11.7552), tensor(13.3094), tensor(13.7084), tensor(12.0950),
tensor(13.2592), tensor(11.8283), tensor(12.5876), tensor(11.1704),
tensor(12.4694), tensor(13.2595), tensor(13.8882), tensor(12.2188),
tensor(11.9381), tensor(10.8944), tensor(10.4752), tensor(13.2223),
tensor(12.9431), tensor(14.6221), tensor(12.8849), tensor(12.1524),
tensor(10.9899), tensor(11.1848), tensor(11.4804), tensor(10.5638),
tensor(12.8950), tensor(12.6663), tensor(11.6108), tensor(14.5731),
tensor(9.7291), tensor(12.9460)]
[tensor(13.7846), tensor(12.5183), tensor(12.3856), tensor(13.5260),
tensor(12.4326), tensor(12.9062), tensor(14.0159), tensor(12.0708),
tensor(12.9360), tensor(12.5581), tensor(14.8046), tensor(13.6303),
tensor(11.0755), tensor(14.2909), tensor(11.1427), tensor(11.8874),
tensor(12.8630), tensor(11.8250), tensor(10.0952), tensor(11.3082),
tensor(10.9619), tensor(13.9715), tensor(12.9547), tensor(12.7542),
tensor(13.1893), tensor(12.6517), tensor(12.9918), tensor(12.0363),
tensor(10.9199), tensor(13.3775), tensor(14.3938), tensor(13.0473),
tensor(12.7769), tensor(10.9110), tensor(11.6800), tensor(13.9045),
tensor(12.8510), tensor(14.5842), tensor(13.0493), tensor(12.7859),
tensor(11.4355), tensor(11.3528), tensor(12.0511), tensor(9.9323),
tensor(13.3722), tensor(13.5187), tensor(13.1158), tensor(15.1485),
tensor(12.6380), tensor(13.4496)]
[tensor(15.2726), tensor(11.8966), tensor(11.0945), tensor(13.5836),
tensor(12.6388), tensor(12.0784), tensor(14.1943), tensor(9.8487),
tensor(13.0986), tensor(12.1783), tensor(13.7827), tensor(12.5126),
tensor(10.4046), tensor(12.5997), tensor(9.5459), tensor(9.9049),
tensor(12.2736), tensor(12.3800), tensor(10.0412), tensor(12.3023),
tensor(11.5643), tensor(13.0472), tensor(12.7246), tensor(12.3002),
tensor(13.5832), tensor(11.9860), tensor(12.0368), tensor(10.6403),
tensor(12.1677), tensor(12.9515), tensor(13.6658), tensor(12.8849),
tensor(12.2590), tensor(11.0228), tensor(12.0012), tensor(12.7530),
tensor(12.4190), tensor(14.8343), tensor(12.4015), tensor(13.3198),
tensor(10.8847), tensor(11.8349), tensor(12.6750), tensor(9.7551),
tensor(12.3382), tensor(13.1571), tensor(12.3043), tensor(14.2055),
tensor(10.6728), tensor(12.7857)]
[tensor(14.7620), tensor(12.5745), tensor(11.8470), tensor(12.5933),
tensor(12.8804), tensor(13.5477), tensor(12.7635), tensor(10.6820),
tensor(14.4405), tensor(14.3959), tensor(14.4414), tensor(12.4286),
tensor(10.9789), tensor(13.8422), tensor(11.3494), tensor(12.5728),
tensor(12.7267), tensor(13.0961), tensor(10.5934), tensor(12.9608),
tensor(11.6614), tensor(14.2678), tensor(13.0028), tensor(12.2996),
tensor(13.5192), tensor(12.4218), tensor(12.7864), tensor(12.4174),
```

```
tensor(11.0165), tensor(13.2705), tensor(14.7095), tensor(14.0720),
tensor(12.4685), tensor(11.2009), tensor(11.3563), tensor(14.0063),
tensor(14.2444), tensor(14.8033), tensor(13.5502), tensor(14.3013),
tensor(12.2550), tensor(11.5053), tensor(12.8127), tensor(11.2148),
tensor(13.8466), tensor(13.8389), tensor(12.8408), tensor(13.7780),
tensor(12.3747), tensor(11.3410)]
[tensor(12.2930), tensor(11.7409), tensor(11.6244), tensor(12.3240),
tensor(11.6550), tensor(11.3714), tensor(12.1277), tensor(9.8432),
tensor(13.3948), tensor(12.9263), tensor(14.2945), tensor(11.8816),
tensor(12.0876), tensor(12.3597), tensor(9.9209), tensor(9.6784),
tensor(12.2622), tensor(12.2363), tensor(9.5058), tensor(11.1197),
tensor(10.4200), tensor(12.7232), tensor(11.7360), tensor(11.9327),
tensor(13.5374), tensor(12.3036), tensor(12.9299), tensor(11.0615),
tensor(11.3050), tensor(13.0540), tensor(13.6938), tensor(13.0926),
tensor(12.5174), tensor(11.4317), tensor(10.4778), tensor(12.4547),
tensor(13.2551), tensor(14.3466), tensor(12.0423), tensor(11.8342),
tensor(10.8152), tensor(10.2073), tensor(10.6031), tensor(10.7343),
tensor(13.1785), tensor(11.7846), tensor(11.4947), tensor(14.5647),
tensor(11.2602), tensor(12.4067)]
[tensor(15.5356), tensor(13.8592), tensor(12.6551), tensor(13.5488),
tensor(12.5827), tensor(13.2429), tensor(14.1169), tensor(11.9190),
tensor(14.7978), tensor(14.2352), tensor(14.9324), tensor(13.7215),
tensor(12.1516), tensor(13.1954), tensor(11.9478), tensor(11.5407),
tensor(14.6037), tensor(11.8541), tensor(10.0681), tensor(12.4875),
tensor(10.3638), tensor(14.6759), tensor(12.7131), tensor(14.1781),
tensor(14.0901), tensor(14.2252), tensor(13.8015), tensor(12.3526),
tensor(14.0837), tensor(13.0359), tensor(14.2927), tensor(13.3897),
tensor(14.7142), tensor(12.7663), tensor(11.9077), tensor(14.9040),
tensor(14.8172), tensor(15.4608), tensor(13.3109), tensor(14.3196),
tensor(12.0276), tensor(12.2825), tensor(12.0887), tensor(13.4535),
tensor(15.2982), tensor(14.3850), tensor(13.1826), tensor(15.1358),
tensor(12.5632), tensor(12.9689)]
[tensor(14.2855), tensor(13.3773), tensor(12.6031), tensor(12.8502),
tensor(12.8705), tensor(12.5245), tensor(13.7837), tensor(11.8918),
tensor(13.2821), tensor(13.4843), tensor(15.3381), tensor(13.6707),
tensor(12.0092), tensor(13.6172), tensor(10.6305), tensor(11.1581),
tensor(12.6371), tensor(12.4870), tensor(10.1934), tensor(12.7152),
tensor(11.5847), tensor(14.0831), tensor(12.1384), tensor(13.7756),
tensor(13.7160), tensor(13.4201), tensor(13.7999), tensor(10.8396),
tensor(12.4611), tensor(12.6723), tensor(14.0487), tensor(14.1081),
tensor(13.2739), tensor(11.0996), tensor(11.6323), tensor(14.1304),
tensor(13.5452), tensor(15.4425), tensor(12.9896), tensor(12.8204),
tensor(11.3895), tensor(11.2541), tensor(12.2484), tensor(11.9121),
tensor(14.9909), tensor(14.7452), tensor(12.5211), tensor(14.8511),
tensor(12.6081), tensor(12.6296)]
[tensor(13.7825), tensor(10.9608), tensor(11.2128), tensor(12.6585),
tensor(11.5030), tensor(10.8268), tensor(11.4788), tensor(9.7854),
tensor(11.9040), tensor(11.9082), tensor(12.3779), tensor(12.3491),
```

```
tensor(9.9111), tensor(12.3997), tensor(10.1658), tensor(9.8713),
tensor(11.5158), tensor(11.9928), tensor(9.2523), tensor(11.1147),
tensor(11.6891), tensor(13.3279), tensor(11.5196), tensor(12.3929),
tensor(12.5182), tensor(11.2554), tensor(11.4545), tensor(9.4686),
tensor(11.0812), tensor(11.4543), tensor(12.5138), tensor(12.1832),
tensor(12.2959), tensor(10.1445), tensor(8.7575), tensor(12.0564),
tensor(10.6147), tensor(13.7718), tensor(11.8533), tensor(11.2882),
tensor(10.3399), tensor(10.9568), tensor(11.8463), tensor(9.7051),
tensor(11.5954), tensor(12.5115), tensor(11.2718), tensor(14.9566),
tensor(10.1526), tensor(11.5805)]
[tensor(17.2889), tensor(14.1213), tensor(14.7033), tensor(16.0282),
tensor(14.8970), tensor(14.3501), tensor(15.2286), tensor(13.3948),
tensor(15.6945), tensor(15.0095), tensor(16.5334), tensor(15.6829),
tensor(13.2985), tensor(14.8714), tensor(12.9636), tensor(12.9965),
tensor(15.3871), tensor(13.7250), tensor(11.9557), tensor(14.0117),
tensor(14.1559), tensor(16.6775), tensor(14.6347), tensor(16.0235),
tensor(15.9149), tensor(15.4792), tensor(14.1873), tensor(13.3177),
tensor(14.3312), tensor(15.4059), tensor(15.8029), tensor(15.0806),
tensor(15.6113), tensor(13.9761), tensor(12.9627), tensor(16.4805),
tensor(14.9115), tensor(17.2568), tensor(16.4567), tensor(14.9765),
tensor(12.2471), tensor(13.8776), tensor(15.0168), tensor(12.7476),
tensor(15.3397), tensor(15.5174), tensor(15.0888), tensor(18.2898),
tensor(13.2266), tensor(15.5292)]
[tensor(14.7820), tensor(13.2523), tensor(12.5287), tensor(12.9600),
tensor(12.2677), tensor(12.7928), tensor(14.5428), tensor(10.7549),
tensor(13.8258), tensor(13.9042), tensor(15.5342), tensor(13.7577),
tensor(11.6945), tensor(13.7434), tensor(12.2851), tensor(11.5340),
tensor(13.6069), tensor(12.2453), tensor(9.9284), tensor(13.1838),
tensor(12.0509), tensor(14.3239), tensor(12.8831), tensor(13.6557),
tensor(15.5266), tensor(12.5791), tensor(13.7155), tensor(11.1183),
tensor(12.7214), tensor(12.2770), tensor(14.9088), tensor(14.2054),
tensor(13.1060), tensor(11.7873), tensor(12.1084), tensor(14.4052),
tensor(13.4381), tensor(16.0138), tensor(14.7047), tensor(12.7814),
tensor(12.5675), tensor(12.0692), tensor(12.6607), tensor(11.3577),
tensor(13.4380), tensor(14.1848), tensor(12.7294), tensor(16.0178),
tensor(11.6679), tensor(12.9199)]
[tensor(11.7508), tensor(11.0280), tensor(11.9034), tensor(12.5346),
tensor(11.0990), tensor(11.0987), tensor(10.7992), tensor(10.0819),
tensor(12.8588), tensor(11.5357), tensor(13.1135), tensor(12.8874),
tensor(9.8478), tensor(11.3133), tensor(10.4115), tensor(10.0090),
tensor(10.9011), tensor(10.5558), tensor(8.4002), tensor(10.8606),
tensor(10.7657), tensor(11.9602), tensor(11.9301), tensor(11.5444),
tensor(13.5720), tensor(11.2518), tensor(11.5412), tensor(10.4526),
tensor(10.2752), tensor(10.8563), tensor(12.9407), tensor(11.3681),
tensor(11.8327), tensor(10.1759), tensor(9.6149), tensor(11.9744),
tensor(11.8300), tensor(13.3025), tensor(12.6161), tensor(11.0953),
tensor(9.7076), tensor(10.4976), tensor(10.6286), tensor(10.0295),
tensor(11.2242), tensor(11.0156), tensor(10.7763), tensor(13.4700),
```

```
tensor(10.6624), tensor(10.3343)]
[tensor(14.7106), tensor(13.1458), tensor(12.8251), tensor(14.3160),
tensor(12.2909), tensor(12.7723), tensor(12.8047), tensor(10.3235),
tensor(13.4801), tensor(13.0505), tensor(15.0776), tensor(12.7266),
tensor(10.5990), tensor(13.4703), tensor(11.1766), tensor(10.8189),
tensor(12.2449), tensor(11.9421), tensor(10.0089), tensor(12.8033),
tensor(11.2927), tensor(13.7194), tensor(13.2596), tensor(12.1734),
tensor(13.0712), tensor(12.0609), tensor(12.9268), tensor(12.1870),
tensor(10.4847), tensor(13.4412), tensor(14.5637), tensor(13.1270),
tensor(12.3276), tensor(11.6334), tensor(11.0226), tensor(13.3234),
tensor(12.6899), tensor(14.7726), tensor(12.7719), tensor(13.4487),
tensor(11.1814), tensor(10.9253), tensor(11.7093), tensor(10.2089),
tensor(12.9610), tensor(12.8487), tensor(12.4675), tensor(15.4257),
tensor(12.1308), tensor(13.3550)]
[tensor(16.0979), tensor(13.5739), tensor(12.7416), tensor(13.4280),
tensor(13.7512), tensor(15.2169), tensor(14.9764), tensor(13.5826),
tensor(15.6239), tensor(15.2541), tensor(16.8271), tensor(14.9804),
tensor(13.0887), tensor(15.7802), tensor(12.3266), tensor(12.8400),
tensor(15.6143), tensor(14.0691), tensor(11.1939), tensor(12.7893),
tensor(13.2908), tensor(16.0996), tensor(14.6193), tensor(15.4387),
tensor(15.9710), tensor(14.3925), tensor(15.1702), tensor(12.3099),
tensor(13.4751), tensor(15.7632), tensor(17.0815), tensor(14.4359),
tensor(14.6426), tensor(12.8252), tensor(12.8675), tensor(15.5077),
tensor(14.9392), tensor(17.4408), tensor(14.7826), tensor(14.4344),
tensor(13.6138), tensor(13.1210), tensor(14.3153), tensor(12.8536),
tensor(16.9167), tensor(16.0205), tensor(14.8597), tensor(16.1377),
tensor(14.4144), tensor(14.2540)]
[tensor(15.4610), tensor(13.4675), tensor(12.7542), tensor(13.2846),
tensor(12.2592), tensor(13.4387), tensor(13.4130), tensor(11.3223),
tensor(14.4436), tensor(14.0488), tensor(14.3958), tensor(13.8991),
tensor(11.4278), tensor(13.2099), tensor(11.6706), tensor(10.0352),
tensor(13.6906), tensor(11.4338), tensor(10.2234), tensor(12.5647),
tensor(11.4052), tensor(14.0556), tensor(12.9834), tensor(13.0217),
tensor(14.5420), tensor(14.0161), tensor(13.0800), tensor(10.6256),
tensor(13.1320), tensor(12.8255), tensor(13.4244), tensor(12.3714),
tensor(13.3437), tensor(11.8798), tensor(11.8128), tensor(14.3966),
tensor(13.3025), tensor(15.3963), tensor(13.0080), tensor(13.7220),
tensor(10.9695), tensor(12.5263), tensor(11.7874), tensor(11.7571),
tensor(13.9893), tensor(14.5509), tensor(11.7801), tensor(15.1492),
tensor(10.8618), tensor(12.4186)]
[tensor(15.5569), tensor(13.8282), tensor(13.0920), tensor(14.2277),
tensor(12.7701), tensor(14.5931), tensor(14.6796), tensor(10.8424),
tensor(14.5237), tensor(13.7109), tensor(14.9721), tensor(13.6484),
tensor(11.2851), tensor(14.7472), tensor(12.3131), tensor(11.5939),
tensor(14.3179), tensor(13.1940), tensor(11.3586), tensor(13.4303),
tensor(12.5445), tensor(14.6089), tensor(13.3903), tensor(14.5323),
tensor(14.3264), tensor(12.6911), tensor(13.7917), tensor(11.8876),
tensor(12.5796), tensor(13.2343), tensor(13.9970), tensor(13.9354),
```

```
tensor(13.9424), tensor(12.5166), tensor(11.5155), tensor(13.9882),
tensor(14.2665), tensor(17.1523), tensor(13.6770), tensor(14.5048),
tensor(11.9356), tensor(12.4301), tensor(12.6417), tensor(12.1012),
tensor(15.2494), tensor(13.7022), tensor(13.9652), tensor(16.4902),
tensor(12.8936), tensor(13.6312)]
[tensor(14.6964), tensor(12.4394), tensor(13.7857), tensor(13.2866),
tensor(12.5183), tensor(12.1471), tensor(13.1123), tensor(11.6888),
tensor(14.3686), tensor(13.0711), tensor(15.4207), tensor(13.7118),
tensor(12.2714), tensor(13.1184), tensor(11.4642), tensor(10.7682),
tensor(12.8947), tensor(12.4728), tensor(9.8958), tensor(13.3952),
tensor(12.3615), tensor(14.4889), tensor(14.0242), tensor(12.3772),
tensor(14.4129), tensor(13.8965), tensor(12.9536), tensor(11.8927),
tensor(12.5800), tensor(13.2770), tensor(14.2796), tensor(12.8752),
tensor(14.2773), tensor(11.3907), tensor(11.2013), tensor(14.8113),
tensor(12.5103), tensor(16.6710), tensor(13.6720), tensor(13.5296),
tensor(12.2265), tensor(12.2396), tensor(12.5012), tensor(10.8474),
tensor(14.1963), tensor(13.3067), tensor(12.9042), tensor(16.6096),
tensor(12.9677), tensor(12.4683)]
[tensor(12.8790), tensor(10.7261), tensor(11.0999), tensor(12.1008),
tensor(11.0937), tensor(10.8171), tensor(12.1326), tensor(9.5200),
tensor(11.7922), tensor(11.2784), tensor(12.6070), tensor(11.3086),
tensor(10.3325), tensor(10.8581), tensor(8.3333), tensor(10.0326),
tensor(10.3346), tensor(10.3080), tensor(7.8331), tensor(11.0297),
tensor(10.3843), tensor(11.7109), tensor(11.6179), tensor(10.8882),
tensor(11.8020), tensor(10.9631), tensor(10.4805), tensor(10.6526),
tensor(10.6877), tensor(12.5087), tensor(12.4756), tensor(12.1456),
tensor(11.6448), tensor(9.2742), tensor(9.7561), tensor(11.7208),
tensor(10.5203), tensor(12.8536), tensor(12.0738), tensor(10.6760),
tensor(9.5705), tensor(9.1908), tensor(9.6838), tensor(10.4029),
tensor(12.0337), tensor(11.1782), tensor(10.6076), tensor(12.6893),
tensor(9.9810), tensor(11.0370)]
[tensor(14.6101), tensor(13.4549), tensor(13.3845), tensor(14.8426),
tensor(12.9925), tensor(13.8713), tensor(14.0367), tensor(11.5095),
tensor(14.9659), tensor(14.2777), tensor(14.9440), tensor(13.7232),
tensor(11.4289), tensor(13.8627), tensor(12.1172), tensor(10.4577),
tensor(13.6072), tensor(12.3741), tensor(11.0910), tensor(12.1097),
tensor(13.4574), tensor(14.3817), tensor(14.1339), tensor(13.6371),
tensor(15.3037), tensor(13.1028), tensor(13.5188), tensor(11.4503),
tensor(12.3621), tensor(14.1814), tensor(15.5832), tensor(14.1774),
tensor(13.0773), tensor(12.3449), tensor(12.3491), tensor(13.9046),
tensor(13.0906), tensor(15.6070), tensor(14.9902), tensor(13.0114),
tensor(11.0552), tensor(11.9950), tensor(12.4408), tensor(11.3870),
tensor(14.5344), tensor(13.5474), tensor(12.9187), tensor(16.1539),
tensor(12.6194), tensor(13.2720)]
[tensor(14.8833), tensor(14.2298), tensor(14.0824), tensor(14.0563),
tensor(13.1409), tensor(12.4134), tensor(13.6536), tensor(11.7273),
tensor(14.0342), tensor(13.2442), tensor(14.2929), tensor(13.9458),
tensor(10.4990), tensor(13.7324), tensor(11.1667), tensor(10.4867),
```

```
tensor(12.5662), tensor(12.0707), tensor(9.8645), tensor(13.1832),
tensor(12.4884), tensor(14.0960), tensor(12.5368), tensor(13.4436),
tensor(14.1306), tensor(13.7161), tensor(12.9324), tensor(11.7956),
tensor(11.7680), tensor(12.2205), tensor(14.7176), tensor(13.4892),
tensor(13.8037), tensor(12.8210), tensor(10.9806), tensor(13.5293),
tensor(13.1775), tensor(14.5849), tensor(14.6466), tensor(11.3481),
tensor(10.2918), tensor(11.6083), tensor(12.0302), tensor(11.4141),
tensor(13.5352), tensor(13.2465), tensor(12.5811), tensor(15.4031),
tensor(12.9524), tensor(11.4375)]
[tensor(12.7644), tensor(10.7551), tensor(10.3864), tensor(10.4522),
tensor(10.4526), tensor(10.8873), tensor(10.6460), tensor(8.6474),
tensor(12.1606), tensor(11.2009), tensor(12.1074), tensor(11.0582),
tensor(8.9601), tensor(10.1385), tensor(10.0034), tensor(10.2998),
tensor(10.8636), tensor(9.7782), tensor(9.5864), tensor(10.8168),
tensor(9.0627), tensor(11.2580), tensor(10.7047), tensor(10.6906),
tensor(11.5553), tensor(11.0502), tensor(10.6309), tensor(10.1299),
tensor(9.8014), tensor(10.9432), tensor(11.4261), tensor(11.3923),
tensor(10.1108), tensor(10.0267), tensor(8.0392), tensor(11.4098),
tensor(11.6948), tensor(12.9569), tensor(10.4243), tensor(10.8633),
tensor(9.5760), tensor(9.9412), tensor(10.1252), tensor(10.1556),
tensor(12.2393), tensor(11.5228), tensor(10.6846), tensor(12.6019),
tensor(9.1305), tensor(10.2784)]
[tensor(14.8488), tensor(13.0919), tensor(13.4244), tensor(14.0998),
tensor(13.5445), tensor(13.5733), tensor(13.7351), tensor(11.0899),
tensor(13.9095), tensor(14.0788), tensor(14.2118), tensor(14.0158),
tensor(11.4743), tensor(14.2283), tensor(11.5447), tensor(10.3368),
tensor(12.9536), tensor(12.6967), tensor(10.7574), tensor(13.1014),
tensor(12.3980), tensor(14.8689), tensor(12.7082), tensor(13.6969),
tensor(13.2593), tensor(12.4778), tensor(13.2238), tensor(11.8056),
tensor(11.4031), tensor(13.3512), tensor(14.6191), tensor(12.4434),
tensor(11.9041), tensor(11.5694), tensor(12.0416), tensor(13.8882),
tensor(13.3694), tensor(15.2743), tensor(13.8687), tensor(12.0761),
tensor(10.6263), tensor(12.0433), tensor(12.1323), tensor(10.1010),
tensor(13.9513), tensor(14.0828), tensor(13.7853), tensor(16.4010),
tensor(12.8185), tensor(11.9806)]
[tensor(13.7867), tensor(11.9719), tensor(11.4413), tensor(12.4699),
tensor(12.1259), tensor(10.9791), tensor(12.2737), tensor(9.3221),
tensor(12.7361), tensor(12.7075), tensor(11.7221), tensor(11.2992),
tensor(10.3348), tensor(11.6273), tensor(9.9302), tensor(9.4311),
tensor(11.4949), tensor(10.8092), tensor(9.9714), tensor(11.1398),
tensor(10.4315), tensor(13.1453), tensor(9.9508), tensor(11.3405),
tensor(11.7711), tensor(11.5995), tensor(11.0984), tensor(10.6958),
tensor(11.1009), tensor(11.4088), tensor(11.8036), tensor(11.9085),
tensor(10.5998), tensor(10.0602), tensor(8.4689), tensor(12.1304),
tensor(11.9968), tensor(12.7719), tensor(11.4637), tensor(11.8777),
tensor(9.1474), tensor(10.0486), tensor(10.4937), tensor(10.7433),
tensor(11.8768), tensor(12.6661), tensor(10.8812), tensor(13.7509),
tensor(10.3942), tensor(10.6264)]
```

[tensor(12.3191), tensor(10.5642), tensor(10.7342), tensor(10.3372),
tensor(10.0803), tensor(10.9283), tensor(11.1306), tensor(8.7071),
tensor(11.9338), tensor(11.9010), tensor(11.5497), tensor(10.8023),
tensor(9.6878), tensor(11.1981), tensor(9.7840), tensor(9.2129),
tensor(10.9175), tensor(10.6052), tensor(8.2989), tensor(10.7909),
tensor(8.9655), tensor(12.2618), tensor(9.9869), tensor(10.4068),
tensor(11.3618), tensor(9.7789), tensor(10.8245), tensor(9.5329),
tensor(9.7693), tensor(10.9535), tensor(12.5967), tensor(11.1309),
tensor(10.3163), tensor(9.9899), tensor(8.8745), tensor(11.5414),
tensor(11.0532), tensor(12.1205), tensor(11.7836), tensor(10.3427),
tensor(9.7028), tensor(8.5723), tensor(9.9656), tensor(8.3822), tensor(11.6206),
tensor(11.3691), tensor(10.8113), tensor(13.2660), tensor(9.9146),
tensor(10.1767)]
[tensor(12.4456), tensor(10.7933), tensor(10.6890), tensor(11.4557),
tensor(11.3454), tensor(12.4741), tensor(10.4070), tensor(10.3182),
tensor(12.5803), tensor(11.9779), tensor(14.1119), tensor(13.1073),
tensor(10.2027), tensor(11.9273), tensor(9.6058), tensor(10.7280),
tensor(11.8880), tensor(10.8934), tensor(8.8528), tensor(11.3458),
tensor(9.9638), tensor(12.5401), tensor(12.5896), tensor(12.1264),
tensor(12.7276), tensor(11.0135), tensor(12.2085), tensor(10.2122),
tensor(10.6378), tensor(11.9886), tensor(13.5029), tensor(11.1902),
tensor(12.4068), tensor(10.4636), tensor(10.7231), tensor(12.4801),
tensor(11.8376), tensor(13.7957), tensor(12.5241), tensor(11.5745),
tensor(11.7240), tensor(10.5239), tensor(11.4257), tensor(9.8731),
tensor(12.2999), tensor(12.2551), tensor(11.5674), tensor(13.6044),
tensor(11.9254), tensor(11.1179)]
[tensor(14.3095), tensor(13.2668), tensor(13.4050), tensor(14.2335),
tensor(11.7405), tensor(11.8136), tensor(13.1655), tensor(11.8463),
tensor(14.3341), tensor(13.0499), tensor(15.1388), tensor(12.6615),
tensor(11.8818), tensor(13.2018), tensor(11.0563), tensor(10.9240),
tensor(12.2605), tensor(11.9853), tensor(10.6182), tensor(10.9808),
tensor(11.8441), tensor(13.2193), tensor(13.4945), tensor(13.7530),
tensor(13.4983), tensor(13.0739), tensor(13.4487), tensor(11.1159),
tensor(12.3555), tensor(13.5272), tensor(15.3447), tensor(13.2922),
tensor(13.3156), tensor(11.9039), tensor(11.1382), tensor(13.5168),
tensor(13.5838), tensor(15.2005), tensor(12.8525), tensor(13.2879),
tensor(11.7141), tensor(11.8053), tensor(11.8086), tensor(11.8607),
tensor(15.0812), tensor(14.3270), tensor(12.0000), tensor(14.8056),
tensor(12.0630), tensor(13.3863)]
[tensor(12.7068), tensor(12.7932), tensor(13.0635), tensor(15.3237),
tensor(13.5432), tensor(12.0063), tensor(11.8404), tensor(12.2378),
tensor(15.4152), tensor(12.9836), tensor(13.9336), tensor(14.3434),
tensor(11.1473), tensor(11.5105), tensor(14.1469), tensor(10.9270),
tensor(14.1371), tensor(14.3452), tensor(12.8727), tensor(15.0533),
tensor(12.8602), tensor(14.8111), tensor(13.8624), tensor(12.9317),
tensor(14.7258), tensor(14.3571), tensor(13.5404), tensor(13.1404),
tensor(14.9883), tensor(12.7689), tensor(13.6191), tensor(12.7208),
tensor(12.9285), tensor(12.8816), tensor(15.3701), tensor(11.7203),

```
tensor(11.1455), tensor(11.6663), tensor(13.2500), tensor(12.5480),
tensor(13.5058), tensor(12.6893), tensor(13.4932), tensor(12.8683),
tensor(15.1585), tensor(14.9174), tensor(12.5777), tensor(12.0835),
tensor(16.2377), tensor(13.6105)]
[tensor(13.0410), tensor(13.5933), tensor(14.5541), tensor(15.9765),
tensor(15.0800), tensor(12.7624), tensor(13.9957), tensor(13.4438),
tensor(15.5453), tensor(13.9324), tensor(14.7481), tensor(15.6883),
tensor(12.8476), tensor(12.4098), tensor(13.6582), tensor(12.2234),
tensor(14.3090), tensor(14.5970), tensor(15.0890), tensor(15.6802),
tensor(15.3062), tensor(17.5622), tensor(14.5343), tensor(13.8254),
tensor(15.1349), tensor(14.3531), tensor(13.6616), tensor(14.7158),
tensor(16.9026), tensor(14.0426), tensor(13.5560), tensor(14.2547),
tensor(14.8353), tensor(14.1631), tensor(16.3173), tensor(14.4271),
tensor(12.3633), tensor(13.0288), tensor(13.3908), tensor(13.0493),
tensor(14.7789), tensor(13.4149), tensor(14.3592), tensor(14.1510),
tensor(15.1864), tensor(15.5597), tensor(11.3715), tensor(12.5112),
tensor(17.1624), tensor(14.7918)]
[tensor(10.8475), tensor(11.3115), tensor(11.3956), tensor(11.7546),
tensor(11.9124), tensor(10.4672), tensor(11.0251), tensor(10.2098),
tensor(11.6060), tensor(10.6357), tensor(12.0502), tensor(13.2019),
tensor(9.5194), tensor(9.7102), tensor(11.1460), tensor(9.1983),
tensor(12.7450), tensor(12.0670), tensor(10.2402), tensor(11.0638),
tensor(10.9035), tensor(12.0959), tensor(11.3957), tensor(10.1097),
tensor(12.4141), tensor(12.4080), tensor(11.8187), tensor(10.4313),
tensor(13.2876), tensor(10.9135), tensor(11.1201), tensor(10.6778),
tensor(10.3748), tensor(10.0148), tensor(12.2980), tensor(11.4966),
tensor(10.9785), tensor(9.5937), tensor(11.2683), tensor(10.5123),
tensor(11.8793), tensor(11.3773), tensor(10.8407), tensor(10.4615),
tensor(12.4562), tensor(11.4391), tensor(10.4278), tensor(9.6153),
tensor(13.9098), tensor(10.9506)]
[tensor(11.7672), tensor(11.6859), tensor(11.0061), tensor(12.6031),
tensor(12.5239), tensor(10.1596), tensor(11.0596), tensor(12.0414),
tensor(13.5810), tensor(11.2588), tensor(11.9458), tensor(13.5961),
tensor(11.0650), tensor(12.0906), tensor(11.5729), tensor(10.7765),
tensor(13.1255), tensor(13.2218), tensor(12.2241), tensor(12.9180),
tensor(12.8626), tensor(13.1618), tensor(12.5794), tensor(11.0733),
tensor(12.8216), tensor(12.2002), tensor(12.3768), tensor(12.0959),
tensor(14.5541), tensor(12.5393), tensor(12.8348), tensor(12.0394),
tensor(12.9714), tensor(11.9369), tensor(13.3397), tensor(11.4738),
tensor(10.6503), tensor(10.5153), tensor(13.1675), tensor(10.0239),
tensor(12.6039), tensor(11.8643), tensor(11.6890), tensor(12.4692),
tensor(11.7659), tensor(12.1255), tensor(11.4170), tensor(11.5061),
tensor(14.2801), tensor(12.6544)]
[tensor(11.7414), tensor(12.4512), tensor(12.6886), tensor(13.0630),
tensor(12.2397), tensor(10.8061), tensor(11.1264), tensor(12.2621),
tensor(12.2857), tensor(12.0011), tensor(11.9624), tensor(13.4309),
tensor(12.6829), tensor(10.5174), tensor(11.4031), tensor(10.3488),
tensor(13.0490), tensor(13.4585), tensor(11.5821), tensor(13.1980),
```

```
tensor(12.9283), tensor(13.6401), tensor(12.7174), tensor(11.2057),
tensor(12.3741), tensor(12.5358), tensor(13.1776), tensor(11.2287),
tensor(14.7829), tensor(11.4337), tensor(13.3523), tensor(12.3065),
tensor(13.0778), tensor(11.8234), tensor(14.2983), tensor(12.8558),
tensor(12.6737), tensor(10.8669), tensor(12.5395), tensor(11.6418),
tensor(14.0981), tensor(11.1875), tensor(11.4563), tensor(12.2391),
tensor(13.4681), tensor(12.1112), tensor(11.3910), tensor(10.9480),
tensor(14.4984), tensor(12.9140)]
[tensor(12.1341), tensor(12.3820), tensor(12.8939), tensor(13.6245),
tensor(12.3298), tensor(12.5245), tensor(13.0431), tensor(11.9198),
tensor(14.3427), tensor(12.1050), tensor(13.1617), tensor(14.5041),
tensor(10.8836), tensor(11.0092), tensor(12.5236), tensor(10.5199),
tensor(12.9068), tensor(12.2172), tensor(12.4454), tensor(13.4999),
tensor(11.5370), tensor(14.5962), tensor(13.2731), tensor(13.0864),
tensor(13.4016), tensor(14.5130), tensor(12.7215), tensor(13.1209),
tensor(14.1800), tensor(12.7159), tensor(11.9853), tensor(12.5801),
tensor(12.5521), tensor(13.2348), tensor(13.6041), tensor(12.0142),
tensor(10.3877), tensor(12.2446), tensor(13.0312), tensor(12.4851),
tensor(12.0646), tensor(12.3507), tensor(12.7198), tensor(11.7666),
tensor(13.9018), tensor(13.0876), tensor(11.1199), tensor(11.4335),
tensor(15.5209), tensor(12.3673)]
[tensor(11.3094), tensor(12.5976), tensor(11.9390), tensor(14.7397),
tensor(13.1277), tensor(12.5278), tensor(11.7994), tensor(11.9642),
tensor(14.4364), tensor(11.8966), tensor(13.7481), tensor(14.9253),
tensor(10.4602), tensor(11.7716), tensor(13.1611), tensor(10.5754),
tensor(13.0699), tensor(13.1679), tensor(12.7313), tensor(13.9514),
tensor(13.1343), tensor(14.3022), tensor(13.3876), tensor(12.1387),
tensor(14.2214), tensor(12.9768), tensor(12.0324), tensor(12.9251),
tensor(14.9556), tensor(13.3995), tensor(12.6488), tensor(11.3930),
tensor(13.3721), tensor(13.2479), tensor(14.7854), tensor(12.1859),
tensor(10.9232), tensor(10.7215), tensor(12.2506), tensor(11.8389),
tensor(12.8701), tensor(12.0751), tensor(12.1457), tensor(11.7446),
tensor(13.7355), tensor(13.5804), tensor(11.9934), tensor(11.4625),
tensor(15.8522), tensor(13.0722)]
[tensor(11.4367), tensor(11.7429), tensor(11.2063), tensor(12.1816),
tensor(11.2752), tensor(10.6537), tensor(11.2979), tensor(10.5228),
tensor(11.2274), tensor(10.5228), tensor(12.9134), tensor(13.8008),
tensor(9.7437), tensor(10.1847), tensor(12.2270), tensor(9.3958),
tensor(11.5026), tensor(12.0566), tensor(10.5804), tensor(11.9130),
tensor(10.9553), tensor(13.2146), tensor(11.9535), tensor(10.5312),
tensor(11.7567), tensor(11.9390), tensor(11.0644), tensor(10.9159),
tensor(13.3269), tensor(11.5868), tensor(11.6821), tensor(10.9403),
tensor(11.4809), tensor(11.4948), tensor(12.6193), tensor(11.2855),
tensor(9.5671), tensor(8.5947), tensor(11.7299), tensor(10.4157),
tensor(11.3367), tensor(11.8984), tensor(11.3796), tensor(10.5405),
tensor(11.7718), tensor(11.9741), tensor(11.0514), tensor(11.6605),
tensor(14.9193), tensor(11.5751)]
[tensor(12.8789), tensor(14.1631), tensor(12.6724), tensor(13.8092),
```

```
tensor(14.3892), tensor(13.4909), tensor(13.1799), tensor(12.0657),
tensor(15.0508), tensor(14.6675), tensor(12.8517), tensor(15.7981),
tensor(11.1080), tensor(12.9942), tensor(13.4445), tensor(11.6948),
tensor(15.6114), tensor(14.3759), tensor(13.6346), tensor(14.6182),
tensor(14.0843), tensor(14.8361), tensor(14.5225), tensor(13.0967),
tensor(14.5314), tensor(14.8713), tensor(13.6736), tensor(11.6807),
tensor(15.2877), tensor(13.1645), tensor(14.4676), tensor(12.7215),
tensor(14.5294), tensor(13.8000), tensor(16.4623), tensor(14.0875),
tensor(12.9394), tensor(11.0568), tensor(14.0720), tensor(12.9503),
tensor(14.2402), tensor(13.6725), tensor(13.8289), tensor(12.5009),
tensor(14.3517), tensor(14.6224), tensor(11.7483), tensor(12.4998),
tensor(16.9612), tensor(13.8527)]
[tensor(12.4159), tensor(12.6690), tensor(12.4316), tensor(12.9649),
tensor(12.6610), tensor(11.3401), tensor(11.8642), tensor(11.6798),
tensor(14.5850), tensor(12.1650), tensor(12.3358), tensor(13.6924),
tensor(10.3243), tensor(11.5324), tensor(12.8571), tensor(11.6355),
tensor(14.0291), tensor(13.0208), tensor(11.9761), tensor(13.1131),
tensor(12.8158), tensor(13.3524), tensor(12.8039), tensor(11.0699),
tensor(13.1847), tensor(13.1214), tensor(12.8678), tensor(12.3738),
tensor(14.6027), tensor(11.7977), tensor(12.8076), tensor(11.4518),
tensor(12.3723), tensor(12.4277), tensor(13.4307), tensor(11.6845),
tensor(11.3212), tensor(11.2719), tensor(13.1799), tensor(10.9963),
tensor(13.4480), tensor(12.6302), tensor(12.6347), tensor(12.1718),
tensor(13.3101), tensor(13.4012), tensor(11.3702), tensor(10.8620),
tensor(15.2312), tensor(12.2923)]
[tensor(12.1607), tensor(13.0468), tensor(12.8641), tensor(11.3929),
tensor(12.6823), tensor(12.1404), tensor(12.3003), tensor(12.6513),
tensor(12.4396), tensor(12.1243), tensor(12.3628), tensor(14.6344),
tensor(10.4292), tensor(10.8287), tensor(11.8328), tensor(11.3690),
tensor(12.8343), tensor(12.5402), tensor(12.0794), tensor(12.6126),
tensor(12.2215), tensor(14.3710), tensor(12.3333), tensor(11.2432),
tensor(12.4641), tensor(13.7661), tensor(12.6852), tensor(12.8169),
tensor(14.7184), tensor(11.0174), tensor(11.9748), tensor(12.2613),
tensor(12.1193), tensor(11.4852), tensor(13.6535), tensor(12.2371),
tensor(11.8074), tensor(11.7585), tensor(12.8958), tensor(10.3730),
tensor(12.2090), tensor(12.7100), tensor(12.2384), tensor(12.0601),
tensor(12.0190), tensor(12.4859), tensor(10.6814), tensor(10.9784),
tensor(14.4477), tensor(12.1443)]
[tensor(12.9078), tensor(14.1990), tensor(12.6742), tensor(14.1980),
tensor(14.4394), tensor(12.0463), tensor(13.5584), tensor(13.1322),
tensor(14.1432), tensor(12.8912), tensor(13.8312), tensor(15.8187),
tensor(12.1181), tensor(13.1555), tensor(13.5086), tensor(12.2186),
tensor(14.6059), tensor(14.4845), tensor(13.5898), tensor(14.2062),
tensor(14.0396), tensor(14.7515), tensor(15.2912), tensor(12.5175),
tensor(14.9007), tensor(14.1938), tensor(15.0923), tensor(13.7436),
tensor(16.8920), tensor(14.0706), tensor(13.8773), tensor(13.6418),
tensor(14.0437), tensor(13.5563), tensor(14.8120), tensor(13.8624),
tensor(13.1360), tensor(12.2704), tensor(14.3754), tensor(13.8163),
```

```
tensor(15.0883), tensor(12.5662), tensor(13.2081), tensor(13.5327),
tensor(14.9973), tensor(13.6769), tensor(12.2944), tensor(12.0547),
tensor(16.8997), tensor(13.7334)]
[tensor(11.7604), tensor(12.7028), tensor(12.7147), tensor(12.5241),
tensor(13.1429), tensor(11.5337), tensor(11.1674), tensor(11.6092),
tensor(13.3122), tensor(12.1089), tensor(12.3592), tensor(13.9388),
tensor(11.2391), tensor(11.4763), tensor(12.6822), tensor(11.5442),
tensor(12.9708), tensor(13.4354), tensor(11.8804), tensor(12.6571),
tensor(12.6572), tensor(13.8676), tensor(13.4031), tensor(11.7602),
tensor(13.6670), tensor(12.9286), tensor(12.4043), tensor(12.9538),
tensor(15.4062), tensor(12.1341), tensor(13.8680), tensor(11.2924),
tensor(11.9890), tensor(11.4819), tensor(13.7453), tensor(12.6319),
tensor(12.4154), tensor(10.9645), tensor(12.3083), tensor(11.0777),
tensor(13.1689), tensor(10.8915), tensor(10.9754), tensor(12.0302),
tensor(13.0857), tensor(12.1632), tensor(10.7717), tensor(10.1852),
tensor(14.5428), tensor(12.2934)]
[tensor(12.9393), tensor(13.6495), tensor(12.6494), tensor(13.8450),
tensor(12.6367), tensor(12.5875), tensor(12.0542), tensor(11.7006),
tensor(13.7505), tensor(12.2836), tensor(13.7505), tensor(14.1683),
tensor(11.7502), tensor(11.7847), tensor(12.6075), tensor(10.2163),
tensor(13.4431), tensor(14.0824), tensor(13.5613), tensor(14.6062),
tensor(13.4881), tensor(14.7168), tensor(14.0313), tensor(13.2908),
tensor(13.7327), tensor(12.6392), tensor(13.5487), tensor(12.0872),
tensor(14.4216), tensor(13.0570), tensor(13.0936), tensor(13.4391),
tensor(13.2904), tensor(13.0549), tensor(14.6954), tensor(12.9956),
tensor(11.3278), tensor(11.6283), tensor(13.4011), tensor(12.6783),
tensor(13.8277), tensor(13.3062), tensor(13.3123), tensor(11.6816),
tensor(15.0519), tensor(13.9594), tensor(12.6634), tensor(12.6644),
tensor(16.0740), tensor(13.2189)]
[tensor(10.6679), tensor(12.0155), tensor(11.1303), tensor(13.0381),
tensor(12.2791), tensor(12.3412), tensor(10.7199), tensor(11.7488),
tensor(13.9372), tensor(11.9382), tensor(12.4460), tensor(13.6986),
tensor(9.9221), tensor(11.2958), tensor(11.9649), tensor(9.9643),
tensor(12.2911), tensor(11.8095), tensor(12.6703), tensor(13.8014),
tensor(12.0219), tensor(13.3740), tensor(13.2836), tensor(11.7156),
tensor(13.7519), tensor(11.8630), tensor(13.4212), tensor(11.2745),
tensor(14.1919), tensor(11.7227), tensor(12.5125), tensor(10.7754),
tensor(13.8512), tensor(12.0238), tensor(13.7944), tensor(12.3759),
tensor(10.8931), tensor(9.5587), tensor(12.2130), tensor(11.9509),
tensor(12.8011), tensor(11.0140), tensor(11.7091), tensor(11.2955),
tensor(13.5830), tensor(13.2713), tensor(11.5434), tensor(10.6739),
tensor(14.3329), tensor(12.6437)]
[tensor(12.2552), tensor(13.3418), tensor(13.1416), tensor(13.7682),
tensor(12.4550), tensor(11.6095), tensor(12.5686), tensor(11.4940),
tensor(15.0499), tensor(12.7503), tensor(12.1091), tensor(15.5085),
tensor(11.8342), tensor(12.6019), tensor(12.0044), tensor(10.9900),
tensor(13.7629), tensor(13.4226), tensor(12.2331), tensor(13.1485),
tensor(13.8855), tensor(14.7747), tensor(13.7153), tensor(13.0630),
```

```
tensor(14.2230), tensor(14.0703), tensor(14.6227), tensor(12.5212),
tensor(14.2223), tensor(13.5434), tensor(14.2075), tensor(12.6253),
tensor(12.0321), tensor(12.8142), tensor(14.2119), tensor(12.2052),
tensor(12.3966), tensor(11.9194), tensor(13.3476), tensor(12.6013),
tensor(13.8194), tensor(13.0900), tensor(13.4096), tensor(12.0910),
tensor(14.7703), tensor(13.3400), tensor(11.6169), tensor(10.6660),
tensor(14.6527), tensor(12.6948)]
[tensor(12.5065), tensor(12.0334), tensor(11.6678), tensor(13.1825),
tensor(12.7671), tensor(11.5113), tensor(12.4828), tensor(11.8497),
tensor(14.2332), tensor(11.7634), tensor(12.1489), tensor(14.2853),
tensor(10.4626), tensor(11.0797), tensor(11.9592), tensor(10.5265),
tensor(14.1422), tensor(13.0113), tensor(12.5303), tensor(13.4232),
tensor(12.2158), tensor(14.4533), tensor(13.1597), tensor(11.7360),
tensor(13.3382), tensor(14.2587), tensor(13.1534), tensor(11.9658),
tensor(13.6529), tensor(12.0337), tensor(13.1192), tensor(11.7412),
tensor(13.0319), tensor(12.8629), tensor(13.9411), tensor(12.8448),
tensor(10.0887), tensor(11.0324), tensor(13.4847), tensor(11.7084),
tensor(13.3398), tensor(12.7112), tensor(13.6409), tensor(11.9490),
tensor(13.8000), tensor(12.9642), tensor(12.4340), tensor(11.4781),
tensor(14.6134), tensor(13.5196)]
[tensor(11.6853), tensor(12.8350), tensor(12.6850), tensor(12.9194),
tensor(12.3584), tensor(11.0661), tensor(11.5639), tensor(11.7409),
tensor(12.8461), tensor(12.2862), tensor(12.7606), tensor(13.2823),
tensor(9.8005), tensor(9.8355), tensor(12.1485), tensor(8.5284),
tensor(13.5245), tensor(13.1721), tensor(11.7604), tensor(13.5115),
tensor(11.6877), tensor(12.8859), tensor(12.7705), tensor(11.3992),
tensor(13.0866), tensor(11.7221), tensor(11.5455), tensor(10.1703),
tensor(13.7206), tensor(12.1920), tensor(12.2559), tensor(10.5388),
tensor(12.0561), tensor(11.5617), tensor(13.8889), tensor(12.2797),
tensor(10.8068), tensor(10.4361), tensor(11.2775), tensor(11.3859),
tensor(12.3640), tensor(11.0605), tensor(11.9472), tensor(11.2306),
tensor(12.9513), tensor(13.0059), tensor(9.8615), tensor(10.5572),
tensor(14.4746), tensor(12.1934)]
[tensor(11.5262), tensor(12.5069), tensor(12.0326), tensor(13.1533),
tensor(12.4103), tensor(11.1647), tensor(13.5171), tensor(11.5651),
tensor(12.6604), tensor(12.1352), tensor(11.3829), tensor(14.4745),
tensor(9.7907), tensor(10.8061), tensor(12.7564), tensor(11.2289),
tensor(13.7082), tensor(12.1070), tensor(11.7316), tensor(12.6425),
tensor(11.6784), tensor(13.9359), tensor(11.7732), tensor(10.9235),
tensor(13.0636), tensor(12.8729), tensor(11.3930), tensor(12.2300),
tensor(14.5590), tensor(12.4949), tensor(12.7395), tensor(11.0205),
tensor(13.3474), tensor(11.9879), tensor(13.6490), tensor(12.1663),
tensor(11.3237), tensor(11.7790), tensor(12.3104), tensor(10.7800),
tensor(12.4482), tensor(12.4843), tensor(12.6191), tensor(11.7383),
tensor(12.1597), tensor(12.2362), tensor(9.1897), tensor(11.3945),
tensor(15.5968), tensor(12.8268)]
[tensor(11.7889), tensor(11.0621), tensor(11.8537), tensor(12.3622),
tensor(12.4769), tensor(11.9884), tensor(11.6317), tensor(10.2395),
```

```
tensor(12.7774), tensor(10.9844), tensor(11.5802), tensor(13.0483),
tensor(10.1433), tensor(12.3453), tensor(12.8773), tensor(10.7168),
tensor(12.0240), tensor(11.9763), tensor(12.0748), tensor(12.7885),
tensor(10.8083), tensor(13.2173), tensor(11.2693), tensor(11.8663),
tensor(12.6077), tensor(13.0334), tensor(12.0231), tensor(11.7650),
tensor(14.1396), tensor(11.5582), tensor(12.3197), tensor(12.1560),
tensor(11.6154), tensor(10.3165), tensor(12.9841), tensor(12.1846),
tensor(11.1779), tensor(10.0611), tensor(13.2232), tensor(10.4846),
tensor(11.9023), tensor(11.7909), tensor(11.2243), tensor(10.8516),
tensor(13.2774), tensor(12.7938), tensor(11.9155), tensor(11.4545),
tensor(15.7030), tensor(10.8686)]
[tensor(11.3764), tensor(12.2725), tensor(12.4228), tensor(13.0009),
tensor(11.1954), tensor(12.1986), tensor(11.8280), tensor(11.4398),
tensor(14.0143), tensor(13.1997), tensor(12.3697), tensor(14.7320),
tensor(10.3952), tensor(12.1588), tensor(12.8537), tensor(10.2581),
tensor(12.7244), tensor(13.7025), tensor(13.1169), tensor(13.6132),
tensor(12.6767), tensor(13.9407), tensor(13.0215), tensor(12.7469),
tensor(12.6780), tensor(12.7137), tensor(12.6811), tensor(11.0673),
tensor(13.6091), tensor(13.4487), tensor(13.2860), tensor(12.1870),
tensor(12.1707), tensor(11.9184), tensor(14.3842), tensor(11.9858),
tensor(12.1767), tensor(11.1931), tensor(14.0589), tensor(12.6416),
tensor(12.3195), tensor(13.1392), tensor(13.9559), tensor(11.6488),
tensor(15.5606), tensor(13.9705), tensor(11.2378), tensor(11.1804),
tensor(15.9133), tensor(13.0954)]
[tensor(11.3454), tensor(11.6744), tensor(11.9173), tensor(12.4012),
tensor(12.4008), tensor(10.7087), tensor(10.9299), tensor(10.4245),
tensor(11.5791), tensor(11.3431), tensor(10.9470), tensor(12.2352),
tensor(11.0282), tensor(10.6136), tensor(10.9930), tensor(9.8968),
tensor(12.8615), tensor(11.6323), tensor(11.4602), tensor(12.2588),
tensor(11.3029), tensor(12.9235), tensor(11.6955), tensor(11.8055),
tensor(11.6736), tensor(13.0558), tensor(11.4257), tensor(12.3440),
tensor(13.5558), tensor(10.7331), tensor(12.7035), tensor(11.3896),
tensor(11.1862), tensor(12.0532), tensor(12.5396), tensor(11.6726),
tensor(11.4444), tensor(11.4398), tensor(11.9798), tensor(10.7122),
tensor(12.3356), tensor(11.3751), tensor(11.6776), tensor(11.5678),
tensor(11.7921), tensor(11.9400), tensor(10.5998), tensor(10.8913),
tensor(14.6693), tensor(12.0727)]
[tensor(12.5377), tensor(13.2926), tensor(13.5900), tensor(14.1725),
tensor(12.0534), tensor(11.6474), tensor(13.3459), tensor(11.7862),
tensor(14.1552), tensor(12.8810), tensor(12.4351), tensor(15.2925),
tensor(11.1428), tensor(11.8514), tensor(13.5035), tensor(10.4766),
tensor(14.4060), tensor(14.7272), tensor(12.4120), tensor(14.1631),
tensor(13.1557), tensor(14.9897), tensor(14.7070), tensor(13.3362),
tensor(14.4176), tensor(13.9326), tensor(12.6681), tensor(12.0451),
tensor(15.0610), tensor(13.3320), tensor(14.2974), tensor(11.7239),
tensor(12.2782), tensor(12.5058), tensor(16.3279), tensor(12.8265),
tensor(12.0718), tensor(11.8481), tensor(14.6160), tensor(11.8009),
tensor(13.5125), tensor(13.9210), tensor(13.9326), tensor(12.7586),
```

```
tensor(15.1730), tensor(13.3049), tensor(12.3380), tensor(12.2542),
tensor(16.0085), tensor(13.4436)]
[tensor(11.0707), tensor(10.4755), tensor(11.0862), tensor(11.8929),
tensor(10.4618), tensor(11.2325), tensor(10.7120), tensor(11.9423),
tensor(13.1334), tensor(11.8222), tensor(10.7980), tensor(13.6463),
tensor(10.1049), tensor(10.3963), tensor(12.4764), tensor(10.2377),
tensor(11.9630), tensor(11.1462), tensor(11.6780), tensor(13.5138),
tensor(11.3936), tensor(13.5194), tensor(12.5180), tensor(11.4456),
tensor(12.5892), tensor(12.6449), tensor(12.5467), tensor(11.0522),
tensor(13.9646), tensor(11.1323), tensor(12.6089), tensor(11.9430),
tensor(11.4155), tensor(10.6664), tensor(12.1530), tensor(11.3137),
tensor(10.1530), tensor(9.9804), tensor(12.1166), tensor(11.4003),
tensor(11.7702), tensor(11.3395), tensor(12.3610), tensor(10.3835),
tensor(12.7526), tensor(12.7807), tensor(10.0903), tensor(9.2839),
tensor(13.5950), tensor(12.0107)]
[tensor(11.3726), tensor(11.3799), tensor(11.7728), tensor(12.0183),
tensor(12.1116), tensor(11.2392), tensor(12.3260), tensor(11.3553),
tensor(12.9396), tensor(11.3192), tensor(12.6045), tensor(13.6650),
tensor(10.2687), tensor(10.4520), tensor(11.6738), tensor(8.9218),
tensor(12.2648), tensor(11.3597), tensor(11.8555), tensor(12.6658),
tensor(12.1542), tensor(12.5739), tensor(12.6911), tensor(10.3008),
tensor(12.2159), tensor(12.3709), tensor(11.6171), tensor(12.4561),
tensor(14.3305), tensor(12.3978), tensor(11.1205), tensor(11.5460),
tensor(13.0181), tensor(12.3161), tensor(13.5062), tensor(11.5247),
tensor(9.6000), tensor(10.2327), tensor(12.9681), tensor(12.0733),
tensor(11.6041), tensor(13.0866), tensor(12.6931), tensor(11.9563),
tensor(12.4844), tensor(12.7720), tensor(11.3172), tensor(11.5231),
tensor(13.9542), tensor(11.9729)]
[tensor(11.7989), tensor(13.8369), tensor(12.2008), tensor(13.1495),
tensor(12.8108), tensor(11.8797), tensor(12.5225), tensor(10.6962),
tensor(13.6377), tensor(11.2885), tensor(12.2014), tensor(14.7276),
tensor(9.5962), tensor(11.4085), tensor(12.1856), tensor(10.2481),
tensor(14.4388), tensor(11.6444), tensor(11.7663), tensor(12.7392),
tensor(12.4761), tensor(13.7091), tensor(12.8715), tensor(11.5676),
tensor(12.8465), tensor(12.9502), tensor(13.1158), tensor(11.7798),
tensor(13.0441), tensor(12.0882), tensor(12.4738), tensor(11.5573),
tensor(13.3041), tensor(12.9256), tensor(13.6798), tensor(11.3685),
tensor(10.8563), tensor(10.5504), tensor(12.5457), tensor(10.1426),
tensor(12.1634), tensor(12.4950), tensor(12.0370), tensor(11.8772),
tensor(13.4220), tensor(12.9401), tensor(11.4066), tensor(11.3370),
tensor(14.7712), tensor(12.7815)]
[tensor(11.2881), tensor(11.8695), tensor(11.9562), tensor(13.5103),
tensor(11.7170), tensor(11.9778), tensor(11.8034), tensor(12.6291),
tensor(14.2419), tensor(11.9836), tensor(11.8558), tensor(14.2959),
tensor(9.6837), tensor(11.5844), tensor(12.4119), tensor(10.5850),
tensor(14.3488), tensor(11.5803), tensor(11.4020), tensor(12.6724),
tensor(11.6763), tensor(14.1411), tensor(12.6197), tensor(12.8090),
tensor(13.7994), tensor(13.3757), tensor(12.7278), tensor(12.7434),
```

```
tensor(14.4055), tensor(12.0222), tensor(12.8583), tensor(11.2294),
tensor(11.5819), tensor(12.2505), tensor(13.1284), tensor(12.3938),
tensor(11.2110), tensor(10.9031), tensor(11.9136), tensor(11.8848),
tensor(12.9579), tensor(11.5467), tensor(12.6909), tensor(11.4051),
tensor(13.2244), tensor(12.7421), tensor(11.1597), tensor(9.8795),
tensor(15.0107), tensor(13.0230)]
[tensor(11.9871), tensor(13.5189), tensor(12.6245), tensor(13.6205),
tensor(11.8302), tensor(12.0763), tensor(12.9802), tensor(12.5946),
tensor(14.1430), tensor(13.4070), tensor(13.0731), tensor(15.7775),
tensor(11.1904), tensor(11.3160), tensor(13.3465), tensor(11.1992),
tensor(13.6652), tensor(14.4289), tensor(12.6383), tensor(13.5491),
tensor(14.0482), tensor(14.7071), tensor(14.8609), tensor(12.7017),
tensor(13.7491), tensor(13.5953), tensor(13.7130), tensor(11.4123),
tensor(15.6461), tensor(13.4935), tensor(13.2149), tensor(12.2151),
tensor(12.9593), tensor(12.8063), tensor(15.8084), tensor(13.7020),
tensor(12.6272), tensor(12.2037), tensor(13.1115), tensor(13.4114),
tensor(14.0058), tensor(12.1402), tensor(13.3083), tensor(12.0942),
tensor(15.8390), tensor(13.5979), tensor(10.7343), tensor(10.5968),
tensor(15.9876), tensor(14.2928)]
[tensor(12.7268), tensor(12.3229), tensor(11.7006), tensor(13.0705),
tensor(11.9028), tensor(10.6482), tensor(11.6516), tensor(11.9348),
tensor(13.7570), tensor(12.1554), tensor(12.2924), tensor(14.5205),
tensor(11.0930), tensor(10.8376), tensor(13.4396), tensor(10.7272),
tensor(12.8731), tensor(13.2207), tensor(13.0585), tensor(14.3554),
tensor(12.4747), tensor(13.4922), tensor(14.0816), tensor(11.5948),
tensor(12.9990), tensor(13.2324), tensor(12.9723), tensor(11.4586),
tensor(14.8169), tensor(12.2505), tensor(12.4106), tensor(12.6774),
tensor(12.6916), tensor(12.7033), tensor(13.9951), tensor(11.1860),
tensor(10.3747), tensor(11.1924), tensor(14.2080), tensor(12.1068),
tensor(12.6018), tensor(12.6210), tensor(13.7469), tensor(11.7918),
tensor(14.0571), tensor(14.0826), tensor(11.7507), tensor(11.6829),
tensor(15.3498), tensor(12.8528)]
[tensor(12.4125), tensor(11.9584), tensor(12.6638), tensor(13.8240),
tensor(12.0430), tensor(11.3918), tensor(12.6385), tensor(12.1647),
tensor(14.1561), tensor(12.1976), tensor(12.6499), tensor(14.1979),
tensor(11.7441), tensor(11.0386), tensor(11.8945), tensor(9.7715),
tensor(14.1905), tensor(13.3157), tensor(13.2354), tensor(13.3811),
tensor(12.7486), tensor(13.4815), tensor(14.0143), tensor(12.3237),
tensor(13.3558), tensor(13.4045), tensor(12.9994), tensor(12.7647),
tensor(15.1924), tensor(13.3004), tensor(12.6924), tensor(13.0696),
tensor(12.8180), tensor(12.1481), tensor(14.0864), tensor(12.8922),
tensor(11.7818), tensor(11.3720), tensor(14.4847), tensor(11.9530),
tensor(13.2141), tensor(13.5856), tensor(12.6998), tensor(12.7974),
tensor(14.0964), tensor(12.3490), tensor(11.8145), tensor(11.4597),
tensor(15.4373), tensor(13.1411)]
[tensor(11.0549), tensor(10.5586), tensor(11.0613), tensor(12.1731),
tensor(11.0905), tensor(11.0400), tensor(10.9949), tensor(10.9976),
tensor(14.7724), tensor(12.1267), tensor(10.5265), tensor(12.7021),
```

```
tensor(10.4069), tensor(11.5128), tensor(11.5973), tensor(10.2032),
tensor(12.1457), tensor(11.6520), tensor(13.7703), tensor(13.1181),
tensor(12.2803), tensor(12.5823), tensor(11.9317), tensor(11.2822),
tensor(13.2507), tensor(12.5267), tensor(12.8278), tensor(11.4032),
tensor(13.3429), tensor(12.2106), tensor(13.5510), tensor(11.9381),
tensor(12.2981), tensor(10.9537), tensor(12.6121), tensor(12.2430),
tensor(10.9326), tensor(10.8528), tensor(13.1747), tensor(10.9512),
tensor(12.5210), tensor(11.0679), tensor(12.5679), tensor(12.0530),
tensor(13.8234), tensor(13.1806), tensor(9.5500), tensor(10.0122),
tensor(13.3416), tensor(11.7587)]
[tensor(13.0347), tensor(13.1945), tensor(12.6238), tensor(13.7561),
tensor(12.2116), tensor(12.9060), tensor(13.3693), tensor(12.6736),
tensor(14.6317), tensor(12.5770), tensor(13.9503), tensor(15.1002),
tensor(10.8036), tensor(12.0077), tensor(12.7538), tensor(11.1670),
tensor(15.0811), tensor(12.9974), tensor(12.6697), tensor(13.4379),
tensor(11.3558), tensor(14.1769), tensor(12.8204), tensor(11.6210),
tensor(12.9766), tensor(13.1930), tensor(11.8348), tensor(12.2675),
tensor(14.6518), tensor(12.8371), tensor(12.8340), tensor(12.8161),
tensor(13.0789), tensor(13.3056), tensor(13.3853), tensor(13.4816),
tensor(10.9932), tensor(11.4366), tensor(13.3610), tensor(10.4757),
tensor(13.1088), tensor(13.0809), tensor(12.3359), tensor(12.1754),
tensor(13.5536), tensor(13.1729), tensor(11.6249), tensor(12.5258),
tensor(17.2533), tensor(12.3283)]
[tensor(11.1573), tensor(11.2289), tensor(12.0332), tensor(12.0129),
tensor(11.5257), tensor(10.1028), tensor(11.3945), tensor(9.7301),
tensor(12.0097), tensor(9.8854), tensor(11.2346), tensor(13.1796),
tensor(10.4851), tensor(10.1771), tensor(11.4818), tensor(10.0568),
tensor(11.8359), tensor(11.8037), tensor(11.4615), tensor(12.3485),
tensor(11.6482), tensor(12.9943), tensor(12.6930), tensor(11.7531),
tensor(11.0998), tensor(12.3819), tensor(11.9752), tensor(11.3089),
tensor(13.0496), tensor(10.6381), tensor(11.6332), tensor(10.4593),
tensor(10.9722), tensor(11.1960), tensor(11.7526), tensor(10.7235),
tensor(10.9076), tensor(10.0312), tensor(12.4890), tensor(9.9857),
tensor(11.4466), tensor(12.1789), tensor(11.0173), tensor(11.3568),
tensor(12.9454), tensor(12.3032), tensor(11.2271), tensor(10.4464),
tensor(13.5603), tensor(11.4236)]
[tensor(13.7450), tensor(14.0177), tensor(14.9516), tensor(15.6694),
tensor(15.3862), tensor(14.1028), tensor(13.9570), tensor(13.4725),
tensor(15.9635), tensor(14.0300), tensor(14.3857), tensor(16.0424),
tensor(13.0964), tensor(13.4412), tensor(14.1509), tensor(12.5625),
tensor(15.2037), tensor(14.8085), tensor(14.2495), tensor(15.3021),
tensor(14.6951), tensor(17.1016), tensor(14.3914), tensor(14.4605),
tensor(15.6511), tensor(15.7730), tensor(15.6344), tensor(14.1926),
tensor(16.2770), tensor(14.5789), tensor(15.4694), tensor(14.4718),
tensor(14.9697), tensor(12.9838), tensor(15.9210), tensor(14.4489),
tensor(13.8374), tensor(11.9840), tensor(14.9754), tensor(13.6252),
tensor(14.5652), tensor(14.7211), tensor(14.1082), tensor(13.8632),
tensor(15.8284), tensor(14.8166), tensor(13.2437), tensor(12.9848),
```

```
tensor(16.8570), tensor(14.7199)]
[tensor(11.9000), tensor(11.8103), tensor(11.9283), tensor(12.3222),
tensor(11.2326), tensor(10.4998), tensor(10.8404), tensor(10.3434),
tensor(12.4086), tensor(10.9846), tensor(12.1710), tensor(12.2971),
tensor(10.2491), tensor(9.7850), tensor(12.2915), tensor(8.7137),
tensor(11.8626), tensor(11.8838), tensor(11.7519), tensor(12.7894),
tensor(12.2426), tensor(13.3688), tensor(13.0345), tensor(11.2632),
tensor(11.9024), tensor(11.8290), tensor(11.5146), tensor(11.8066),
tensor(12.8997), tensor(10.8338), tensor(12.0168), tensor(11.2421),
tensor(11.1048), tensor(10.8703), tensor(13.7618), tensor(11.8141),
tensor(9.8045), tensor(9.3973), tensor(10.9556), tensor(9.6507),
tensor(11.8061), tensor(11.8471), tensor(11.1512), tensor(11.4369),
tensor(12.2841), tensor(12.5131), tensor(10.4774), tensor(10.6322),
tensor(13.7447), tensor(11.9362)]
[tensor(11.8304), tensor(11.3346), tensor(11.5925), tensor(11.5092),
tensor(12.3024), tensor(12.1684), tensor(12.8123), tensor(11.1929),
tensor(13.8402), tensor(13.4291), tensor(12.1107), tensor(13.9610),
tensor(11.1544), tensor(10.9028), tensor(13.1997), tensor(11.0883),
tensor(11.7904), tensor(12.4722), tensor(12.6066), tensor(13.5037),
tensor(12.9702), tensor(13.4345), tensor(13.1089), tensor(11.4515),
tensor(12.9769), tensor(14.0174), tensor(12.1999), tensor(12.6661),
tensor(14.7288), tensor(12.7510), tensor(12.6174), tensor(11.7200),
tensor(12.8276), tensor(11.7479), tensor(13.3923), tensor(12.3849),
tensor(11.5880), tensor(10.6881), tensor(13.5966), tensor(12.9784),
tensor(12.9315), tensor(12.0788), tensor(12.5821), tensor(12.0158),
tensor(12.9335), tensor(12.5686), tensor(10.4500), tensor(11.4098),
tensor(14.4808), tensor(11.8313)]
[tensor(11.8630), tensor(12.6686), tensor(12.5417), tensor(13.8345),
tensor(12.5844), tensor(11.4733), tensor(11.2502), tensor(11.7012),
tensor(14.4484), tensor(12.0398), tensor(12.4355), tensor(13.6417),
tensor(10.2488), tensor(11.1405), tensor(12.1122), tensor(10.4724),
tensor(13.2784), tensor(12.2633), tensor(12.1575), tensor(12.7577),
tensor(11.7723), tensor(14.3091), tensor(12.0903), tensor(12.2313),
tensor(12.8994), tensor(12.7244), tensor(12.8543), tensor(11.1607),
tensor(13.7646), tensor(11.9691), tensor(12.9180), tensor(10.7752),
tensor(12.1329), tensor(12.3247), tensor(13.6161), tensor(12.4273),
tensor(10.6825), tensor(10.5716), tensor(11.8862), tensor(11.3491),
tensor(12.7773), tensor(10.7787), tensor(12.4542), tensor(11.8270),
tensor(13.8520), tensor(13.4682), tensor(11.0966), tensor(11.0568),
tensor(14.4324), tensor(13.0896)]
[tensor(14.2148), tensor(14.6338), tensor(14.2217), tensor(16.7697),
tensor(16.1232), tensor(15.1694), tensor(14.2025), tensor(14.6411),
tensor(17.7130), tensor(15.4624), tensor(15.7547), tensor(17.4676),
tensor(12.7434), tensor(15.2669), tensor(16.0915), tensor(12.6873),
tensor(16.1206), tensor(15.9853), tensor(15.4424), tensor(17.2710),
tensor(15.5689), tensor(16.7285), tensor(16.5872), tensor(14.7200),
tensor(17.2770), tensor(15.7632), tensor(16.2660), tensor(14.7692),
tensor(18.5265), tensor(15.4947), tensor(15.9051), tensor(14.8503),
```

```
tensor(15.3404), tensor(14.7968), tensor(16.8118), tensor(15.9339),
tensor(13.1434), tensor(11.9853), tensor(15.0642), tensor(15.7657),
tensor(16.9507), tensor(14.8857), tensor(15.6422), tensor(13.2361),
tensor(17.4522), tensor(17.4435), tensor(14.2336), tensor(13.5632),
tensor(19.4681), tensor(15.1403)]
[tensor(13.1924), tensor(12.6102), tensor(12.5053), tensor(13.4299),
tensor(13.5110), tensor(13.6847), tensor(13.9413), tensor(12.4787),
tensor(13.6053), tensor(13.2357), tensor(13.3903), tensor(14.7082),
tensor(11.1568), tensor(12.2586), tensor(14.1197), tensor(10.0673),
tensor(14.5725), tensor(13.4676), tensor(13.1799), tensor(15.2138),
tensor(13.2983), tensor(14.4717), tensor(14.2606), tensor(12.7074),
tensor(13.9649), tensor(14.6641), tensor(11.8572), tensor(13.7332),
tensor(16.9592), tensor(12.7976), tensor(15.0095), tensor(12.5339),
tensor(13.5552), tensor(13.1163), tensor(14.5065), tensor(14.6572),
tensor(12.8027), tensor(11.8080), tensor(15.2055), tensor(12.4863),
tensor(14.1391), tensor(13.3964), tensor(13.8961), tensor(12.6647),
tensor(13.3230), tensor(14.8309), tensor(12.7570), tensor(13.0129),
tensor(16.2781), tensor(13.9828)]
[tensor(11.6398), tensor(13.1536), tensor(12.3474), tensor(13.4087),
tensor(13.4800), tensor(11.7930), tensor(12.9803), tensor(11.9748),
tensor(14.1524), tensor(13.5422), tensor(12.8181), tensor(14.8927),
tensor(10.3646), tensor(11.8875), tensor(13.2543), tensor(10.1739),
tensor(13.0655), tensor(14.4765), tensor(12.7388), tensor(13.7060),
tensor(12.9664), tensor(13.7552), tensor(13.7255), tensor(11.4457),
tensor(14.2023), tensor(13.4669), tensor(13.7707), tensor(11.7108),
tensor(14.5115), tensor(13.9426), tensor(12.7922), tensor(11.3831),
tensor(13.1463), tensor(12.6829), tensor(15.3360), tensor(12.6682),
tensor(11.6175), tensor(11.4936), tensor(13.6179), tensor(14.5174),
tensor(13.0440), tensor(12.6939), tensor(14.2050), tensor(11.6674),
tensor(15.3827), tensor(14.3708), tensor(11.5657), tensor(11.2395),
tensor(15.8505), tensor(12.9541)]
[tensor(12.7225), tensor(12.4016), tensor(13.0202), tensor(14.2441),
tensor(13.2614), tensor(13.2398), tensor(11.1128), tensor(13.1959),
tensor(14.9408), tensor(11.8259), tensor(13.7595), tensor(14.5410),
tensor(11.4284), tensor(12.0091), tensor(12.7957), tensor(11.0713),
tensor(13.9342), tensor(12.8181), tensor(12.7510), tensor(15.2000),
tensor(13.6086), tensor(14.1580), tensor(13.8695), tensor(13.4987),
tensor(13.7597), tensor(13.5915), tensor(13.9188), tensor(13.8993),
tensor(16.0103), tensor(12.7000), tensor(13.1538), tensor(13.5470),
tensor(12.7873), tensor(12.4542), tensor(13.9905), tensor(12.7351),
tensor(11.2320), tensor(11.3531), tensor(13.8419), tensor(12.3418),
tensor(13.5929), tensor(12.6395), tensor(13.1844), tensor(12.9428),
tensor(13.9658), tensor(14.1256), tensor(12.8033), tensor(11.9389),
tensor(16.5646), tensor(13.0359)]
[tensor(13.4963), tensor(13.4076), tensor(13.2473), tensor(14.1808),
tensor(12.7062), tensor(12.7830), tensor(13.5825), tensor(13.4563),
tensor(14.8414), tensor(13.0900), tensor(13.8034), tensor(14.9359),
tensor(12.4305), tensor(12.8281), tensor(13.6090), tensor(10.1536),
```

```
tensor(14.0085), tensor(14.3979), tensor(14.1403), tensor(14.5527),
tensor(13.0442), tensor(15.5679), tensor(14.3123), tensor(13.1963),
tensor(15.0135), tensor(14.3260), tensor(14.5189), tensor(13.7141),
tensor(16.2010), tensor(14.2157), tensor(15.3324), tensor(13.3215),
tensor(13.4740), tensor(12.4863), tensor(14.9972), tensor(15.7192),
tensor(12.4215), tensor(11.5647), tensor(14.3931), tensor(12.3278),
tensor(14.3676), tensor(13.4302), tensor(13.7363), tensor(13.2667),
tensor(15.1270), tensor(14.5641), tensor(12.3957), tensor(12.3727),
tensor(16.4772), tensor(13.9423)]
[tensor(14.1094), tensor(13.7015), tensor(14.5466), tensor(15.0655),
tensor(13.0609), tensor(13.2551), tensor(13.6229), tensor(13.8292),
tensor(15.8692), tensor(14.0103), tensor(13.7546), tensor(16.2736),
tensor(12.2844), tensor(13.4308), tensor(14.1726), tensor(11.8241),
tensor(15.7006), tensor(14.8214), tensor(14.1151), tensor(14.0006),
tensor(13.6682), tensor(16.3263), tensor(14.4964), tensor(13.6757),
tensor(14.4218), tensor(15.4370), tensor(14.0909), tensor(14.2605),
tensor(16.5108), tensor(13.9559), tensor(16.0772), tensor(13.9592),
tensor(13.1310), tensor(13.7022), tensor(16.0555), tensor(15.0156),
tensor(12.9571), tensor(12.4784), tensor(16.0251), tensor(12.8562),
tensor(14.6897), tensor(15.0897), tensor(16.0552), tensor(13.7168),
tensor(16.1427), tensor(15.3671), tensor(13.3229), tensor(12.6736),
tensor(18.1325), tensor(15.3696)]
[tensor(12.2643), tensor(13.4323), tensor(13.0972), tensor(14.4628),
tensor(14.1415), tensor(12.2548), tensor(12.2101), tensor(13.4580),
tensor(15.2319), tensor(14.3635), tensor(13.5091), tensor(15.2317),
tensor(10.6800), tensor(12.2747), tensor(13.8041), tensor(11.1383),
tensor(14.1969), tensor(14.7369), tensor(14.4915), tensor(14.9018),
tensor(13.1075), tensor(13.9816), tensor(14.9373), tensor(12.2876),
tensor(15.4005), tensor(13.4096), tensor(13.6854), tensor(12.4774),
tensor(16.5260), tensor(14.3690), tensor(14.2981), tensor(12.4987),
tensor(14.1063), tensor(12.7157), tensor(16.5236), tensor(13.4676),
tensor(12.0766), tensor(10.9125), tensor(14.6767), tensor(13.7834),
tensor(13.8849), tensor(13.0716), tensor(14.1198), tensor(13.0470),
tensor(15.0614), tensor(14.3766), tensor(12.0107), tensor(12.4461),
tensor(16.9760), tensor(14.0090)]
[tensor(12.1437), tensor(13.2794), tensor(13.2879), tensor(12.0177),
tensor(11.8283), tensor(11.3066), tensor(12.8247), tensor(12.2103),
tensor(12.7528), tensor(12.6227), tensor(12.8072), tensor(14.5557),
tensor(11.2671), tensor(10.9709), tensor(12.2633), tensor(10.6239),
tensor(13.3970), tensor(13.1868), tensor(12.6882), tensor(13.1112),
tensor(11.4607), tensor(13.3258), tensor(12.8502), tensor(11.8011),
tensor(12.4260), tensor(12.7275), tensor(12.4541), tensor(11.3255),
tensor(14.0747), tensor(12.4111), tensor(12.6295), tensor(12.3992),
tensor(12.0226), tensor(12.0566), tensor(13.8921), tensor(12.9669),
tensor(11.7818), tensor(10.7462), tensor(13.2001), tensor(11.1221),
tensor(13.1599), tensor(12.0072), tensor(12.2555), tensor(12.1282),
tensor(14.4784), tensor(12.6838), tensor(11.5670), tensor(12.1856),
tensor(15.7862), tensor(11.6800)]
```

```
[tensor(13.9086), tensor(14.1174), tensor(13.4351), tensor(15.2532),
tensor(13.3392), tensor(13.3771), tensor(13.1700), tensor(13.1328),
tensor(14.1561), tensor(12.3673), tensor(13.9700), tensor(14.6867),
tensor(12.8109), tensor(12.7123), tensor(14.3089), tensor(11.2445),
tensor(14.9432), tensor(15.1232), tensor(13.0751), tensor(15.3067),
tensor(14.6059), tensor(16.7290), tensor(14.7122), tensor(13.7573),
tensor(14.3163), tensor(14.4069), tensor(13.2862), tensor(13.8516),
tensor(15.9331), tensor(13.1837), tensor(15.1202), tensor(13.6206),
tensor(13.9678), tensor(13.1439), tensor(15.2235), tensor(14.0696),
tensor(13.0410), tensor(12.1823), tensor(14.0678), tensor(11.4186),
tensor(14.8120), tensor(13.6256), tensor(12.8380), tensor(13.9719),
tensor(15.0369), tensor(14.0713), tensor(13.2380), tensor(12.9914),
tensor(16.2523), tensor(14.9270)]
[tensor(12.5693), tensor(12.5067), tensor(11.7966), tensor(12.6243),
tensor(12.4857), tensor(12.3967), tensor(12.0205), tensor(10.5609),
tensor(13.9302), tensor(11.2287), tensor(13.0470), tensor(14.3382),
tensor(10.2643), tensor(12.3129), tensor(13.2476), tensor(11.0957),
tensor(12.7121), tensor(13.2567), tensor(12.0087), tensor(13.3317),
tensor(13.1674), tensor(13.6424), tensor(13.1612), tensor(11.9923),
tensor(13.4570), tensor(13.4794), tensor(12.5724), tensor(11.4654),
tensor(13.9235), tensor(12.3133), tensor(12.6611), tensor(12.0257),
tensor(12.3427), tensor(12.5038), tensor(13.5717), tensor(12.1012),
tensor(10.6256), tensor(10.3571), tensor(13.0119), tensor(11.1893),
tensor(13.1516), tensor(12.4915), tensor(11.9777), tensor(10.9238),
tensor(13.6882), tensor(13.5121), tensor(12.3850), tensor(11.5438),
tensor(15.5320), tensor(11.8410)]
[tensor(12.3182), tensor(12.5262), tensor(13.0564), tensor(12.5805),
tensor(12.2035), tensor(12.6093), tensor(12.1419), tensor(11.6184),
tensor(12.8600), tensor(12.1682), tensor(12.9576), tensor(13.6340),
tensor(10.5846), tensor(10.7421), tensor(11.8891), tensor(8.3925),
tensor(12.6580), tensor(12.9148), tensor(11.5071), tensor(12.3348),
tensor(11.5110), tensor(14.2381), tensor(12.6945), tensor(11.9694),
tensor(12.5414), tensor(12.8551), tensor(11.1033), tensor(12.8355),
tensor(14.3173), tensor(12.3584), tensor(12.5265), tensor(10.5950),
tensor(11.7013), tensor(12.1286), tensor(13.5472), tensor(13.9852),
tensor(10.7451), tensor(9.4400), tensor(12.3492), tensor(12.4512),
tensor(12.5545), tensor(13.1594), tensor(12.6749), tensor(10.8243),
tensor(12.5940), tensor(13.3435), tensor(11.4876), tensor(10.6880),
tensor(15.5857), tensor(12.2894)]
[tensor(9.5444), tensor(10.3730), tensor(9.9892), tensor(11.4314),
tensor(10.7131), tensor(10.5431), tensor(9.4910), tensor(10.3995),
tensor(12.1815), tensor(11.2608), tensor(10.6667), tensor(11.5017),
tensor(9.5568), tensor(9.7112), tensor(11.4183), tensor(9.1719),
tensor(10.6228), tensor(11.3700), tensor(9.7751), tensor(11.9955),
tensor(10.7759), tensor(12.2885), tensor(11.9972), tensor(10.4608),
tensor(12.1516), tensor(11.6203), tensor(10.3280), tensor(11.2150),
tensor(12.9571), tensor(9.6337), tensor(11.1940), tensor(9.5914),
tensor(10.5399), tensor(11.0522), tensor(11.9064), tensor(9.4757),
```

```
tensor(10.0427), tensor(9.8267), tensor(10.7823), tensor(10.9089),
tensor(10.9368), tensor(10.1036), tensor(10.4028), tensor(9.3390),
tensor(11.3347), tensor(12.0637), tensor(10.2828), tensor(8.6257),
tensor(12.5234), tensor(10.8592)]
[tensor(12.3360), tensor(12.6790), tensor(12.2737), tensor(13.1858),
tensor(12.4614), tensor(11.1954), tensor(12.1442), tensor(12.2167),
tensor(13.3834), tensor(11.7222), tensor(12.6878), tensor(14.4442),
tensor(11.3502), tensor(11.1856), tensor(12.3473), tensor(10.5475),
tensor(12.6868), tensor(12.2227), tensor(12.7840), tensor(12.6041),
tensor(12.2041), tensor(14.5162), tensor(13.0249), tensor(11.7817),
tensor(13.0730), tensor(13.1029), tensor(13.6704), tensor(12.8888),
tensor(14.3439), tensor(12.4688), tensor(13.0261), tensor(11.5609),
tensor(12.5931), tensor(12.5354), tensor(13.0086), tensor(12.3390),
tensor(11.4883), tensor(10.8387), tensor(13.1853), tensor(12.6233),
tensor(12.7845), tensor(12.8410), tensor(13.6266), tensor(11.8820),
tensor(13.3720), tensor(13.2566), tensor(12.0655), tensor(11.2068),
tensor(14.2980), tensor(13.1253)]
[tensor(12.6011), tensor(14.3965), tensor(12.4160), tensor(13.2361),
tensor(14.9486), tensor(12.9570), tensor(12.2650), tensor(14.3559),
tensor(12.1648), tensor(13.7357), tensor(14.7440), tensor(12.0834),
tensor(16.1714), tensor(16.0110), tensor(12.1360), tensor(12.3958),
tensor(12.1824), tensor(12.0430), tensor(16.7118), tensor(15.1831),
tensor(11.5689), tensor(14.7121), tensor(13.9648), tensor(14.5991),
tensor(14.1311), tensor(13.3737), tensor(13.5874), tensor(13.9079),
tensor(16.6411), tensor(13.1859), tensor(15.2285), tensor(14.3361),
tensor(11.5895), tensor(14.1588), tensor(14.5012), tensor(15.9088),
tensor(12.9846), tensor(11.7398), tensor(15.4341), tensor(14.6004),
tensor(14.3732), tensor(13.9113), tensor(11.1392), tensor(13.1156),
tensor(10.6014), tensor(15.5237), tensor(12.9311), tensor(13.7324),
tensor(12.6219), tensor(16.2051)]
[tensor(11.4378), tensor(11.4375), tensor(11.5363), tensor(11.2582),
tensor(11.8567), tensor(11.1998), tensor(10.9279), tensor(11.5447),
tensor(9.7522), tensor(11.5900), tensor(12.6293), tensor(10.2229),
tensor(13.8559), tensor(12.2788), tensor(10.4843), tensor(11.2715),
tensor(10.4006), tensor(10.1356), tensor(12.6473), tensor(11.5633),
tensor(8.9460), tensor(12.7344), tensor(10.8207), tensor(11.8350),
tensor(11.9726), tensor(11.1890), tensor(11.1379), tensor(11.1185),
tensor(14.4708), tensor(10.5606), tensor(12.9108), tensor(12.5644),
tensor(12.1460), tensor(11.3752), tensor(12.9271), tensor(13.3964),
tensor(11.5376), tensor(10.7324), tensor(12.7326), tensor(12.0536),
tensor(10.7820), tensor(12.6977), tensor(9.7949), tensor(11.2169),
tensor(9.7961), tensor(13.1635), tensor(10.6874), tensor(10.2807),
tensor(11.0407), tensor(12.4836)]
[tensor(13.6961), tensor(14.9259), tensor(14.5589), tensor(13.0368),
tensor(15.0489), tensor(12.5324), tensor(14.2066), tensor(12.1921),
tensor(12.4578), tensor(14.2089), tensor(13.3016), tensor(13.3199),
tensor(17.2245), tensor(14.9983), tensor(13.0668), tensor(13.0750),
tensor(12.9540), tensor(11.9754), tensor(15.3915), tensor(14.8229),
```

```
tensor(12.1121), tensor(15.2836), tensor(13.6952), tensor(15.4198),
tensor(14.8952), tensor(14.0333), tensor(12.6580), tensor(13.7405),
tensor(17.2925), tensor(14.1189), tensor(16.9723), tensor(15.3497),
tensor(11.2730), tensor(12.9161), tensor(15.5704), tensor(16.4024),
tensor(12.3849), tensor(13.6024), tensor(15.5401), tensor(14.1685),
tensor(13.8540), tensor(15.0396), tensor(11.9188), tensor(13.2629),
tensor(11.7807), tensor(15.1814), tensor(13.5947), tensor(14.7668),
tensor(12.4442), tensor(15.1339)]
[tensor(11.5912), tensor(12.3421), tensor(11.1270), tensor(10.8079),
tensor(12.1464), tensor(11.0930), tensor(11.3558), tensor(11.5202),
tensor(10.8376), tensor(11.5434), tensor(11.5672), tensor(10.9267),
tensor(13.4912), tensor(13.7846), tensor(10.3171), tensor(11.4845),
tensor(10.9001), tensor(10.1191), tensor(13.3263), tensor(12.9291),
tensor(9.8033), tensor(13.7057), tensor(11.2804), tensor(13.1030),
tensor(11.6930), tensor(12.1391), tensor(12.2569), tensor(12.0832),
tensor(14.1404), tensor(12.2390), tensor(14.8418), tensor(12.5468),
tensor(9.6500), tensor(10.7276), tensor(12.1576), tensor(14.1907),
tensor(11.4788), tensor(10.3227), tensor(12.5808), tensor(11.8032),
tensor(12.2870), tensor(12.3077), tensor(9.6863), tensor(11.6878),
tensor(8.9882), tensor(13.0075), tensor(10.8477), tensor(11.8460),
tensor(10.4808), tensor(13.4667)]
[tensor(12.1403), tensor(12.9545), tensor(11.9423), tensor(11.9102),
tensor(13.1055), tensor(10.5161), tensor(12.6107), tensor(12.0301),
tensor(11.8486), tensor(12.9774), tensor(13.0326), tensor(11.3593),
tensor(15.3160), tensor(14.0179), tensor(11.0095), tensor(11.4745),
tensor(11.3875), tensor(12.0406), tensor(14.6366), tensor(14.9532),
tensor(10.6203), tensor(14.5450), tensor(12.1054), tensor(14.2774),
tensor(11.9966), tensor(13.2351), tensor(12.3338), tensor(12.9168),
tensor(15.2504), tensor(11.8742), tensor(14.5896), tensor(13.2196),
tensor(10.8684), tensor(12.2038), tensor(12.6273), tensor(14.8450),
tensor(13.1534), tensor(11.6892), tensor(13.4702), tensor(13.4107),
tensor(12.1308), tensor(13.9084), tensor(11.0597), tensor(11.3605),
tensor(12.0229), tensor(12.7563), tensor(10.7620), tensor(13.1440),
tensor(12.5548), tensor(14.1388)]
[tensor(8.5809), tensor(10.3477), tensor(10.1742), tensor(9.3039),
tensor(10.2660), tensor(8.3282), tensor(9.1907), tensor(8.8615), tensor(7.5761),
tensor(8.6456), tensor(9.6609), tensor(8.5485), tensor(10.7485), tensor(9.9754),
tensor(9.1482), tensor(8.9193), tensor(8.3258), tensor(7.7492), tensor(10.1535),
tensor(10.7907), tensor(8.3675), tensor(9.7497), tensor(9.3310),
tensor(10.0199), tensor(9.0608), tensor(8.4817), tensor(9.5261), tensor(9.3589),
tensor(11.6811), tensor(9.2815), tensor(11.3863), tensor(10.5878),
tensor(8.8841), tensor(8.8224), tensor(11.0759), tensor(11.3017),
tensor(8.7623), tensor(9.0640), tensor(10.6030), tensor(10.9029),
tensor(9.4980), tensor(10.5171), tensor(8.9808), tensor(9.5543), tensor(7.9472),
tensor(10.9127), tensor(9.0578), tensor(9.8469), tensor(8.6793), tensor(9.2972)]
[tensor(12.2474), tensor(12.7884), tensor(12.3044), tensor(10.8226),
tensor(11.9090), tensor(9.4018), tensor(12.1202), tensor(11.1001),
tensor(11.6247), tensor(11.7395), tensor(12.1830), tensor(10.9840),
```

```
tensor(13.9620), tensor(13.6370), tensor(11.2756), tensor(11.5279),
tensor(11.2778), tensor(10.9766), tensor(13.0610), tensor(13.2986),
tensor(9.8295), tensor(12.8035), tensor(11.5135), tensor(13.3994),
tensor(11.5160), tensor(12.3443), tensor(11.8390), tensor(12.6382),
tensor(15.3251), tensor(11.4081), tensor(14.2208), tensor(13.4787),
tensor(9.7532), tensor(10.6638), tensor(12.6835), tensor(15.1030),
tensor(11.3481), tensor(11.1396), tensor(14.0540), tensor(12.2718),
tensor(11.3980), tensor(13.0372), tensor(10.0941), tensor(11.6323),
tensor(9.4343), tensor(13.4204), tensor(11.4331), tensor(12.9262),
tensor(10.5345), tensor(13.7315)]
[tensor(11.7999), tensor(13.1820), tensor(13.1456), tensor(12.2830),
tensor(13.0496), tensor(11.3864), tensor(11.8812), tensor(12.2555),
tensor(11.5937), tensor(11.7298), tensor(11.8565), tensor(12.0049),
tensor(15.0399), tensor(14.3098), tensor(11.2296), tensor(11.9760),
tensor(12.3322), tensor(11.4277), tensor(13.1721), tensor(14.2119),
tensor(11.9200), tensor(14.4778), tensor(12.6493), tensor(13.8392),
tensor(12.8815), tensor(12.3838), tensor(12.1695), tensor(11.0505),
tensor(15.4064), tensor(12.1745), tensor(15.6088), tensor(13.3735),
tensor(11.2004), tensor(13.0576), tensor(13.4910), tensor(14.5021),
tensor(12.6975), tensor(11.9446), tensor(14.0222), tensor(13.4390),
tensor(12.1468), tensor(13.1948), tensor(11.7346), tensor(12.7316),
tensor(11.8499), tensor(13.2785), tensor(11.3865), tensor(12.2829),
tensor(11.5750), tensor(14.3405)]
[tensor(12.0774), tensor(14.2278), tensor(13.5404), tensor(12.0733),
tensor(14.7061), tensor(12.7121), tensor(12.3299), tensor(13.5232),
tensor(11.8055), tensor(13.7037), tensor(13.7402), tensor(13.0328),
tensor(15.8633), tensor(15.5985), tensor(12.6577), tensor(12.9143),
tensor(12.5467), tensor(12.4975), tensor(15.0527), tensor(15.2902),
tensor(12.0759), tensor(15.2621), tensor(12.9992), tensor(15.1326),
tensor(13.7049), tensor(13.6036), tensor(12.9850), tensor(13.5870),
tensor(17.5174), tensor(13.2804), tensor(16.6145), tensor(14.4807),
tensor(11.6212), tensor(13.9797), tensor(14.4407), tensor(15.8250),
tensor(12.6758), tensor(12.7577), tensor(14.3320), tensor(15.1949),
tensor(13.9834), tensor(14.2578), tensor(11.4754), tensor(12.4617),
tensor(11.7103), tensor(14.1101), tensor(12.0794), tensor(13.2676),
tensor(13.1170), tensor(15.1710)]
[tensor(12.0168), tensor(15.0576), tensor(11.7684), tensor(13.2458),
tensor(14.8188), tensor(12.4159), tensor(13.3581), tensor(12.9868),
tensor(12.2058), tensor(12.8671), tensor(13.3232), tensor(12.1401),
tensor(17.1912), tensor(15.1674), tensor(11.7321), tensor(12.0619),
tensor(11.7914), tensor(12.2372), tensor(15.2407), tensor(14.9791),
tensor(11.0016), tensor(14.0686), tensor(11.8712), tensor(15.6492),
tensor(13.7444), tensor(12.9554), tensor(13.0731), tensor(13.8457),
tensor(15.9829), tensor(13.4070), tensor(14.7713), tensor(14.0698),
tensor(11.6121), tensor(13.4432), tensor(14.2889), tensor(15.3481),
tensor(12.8443), tensor(12.3807), tensor(14.8032), tensor(14.5805),
tensor(13.9162), tensor(14.7197), tensor(11.7537), tensor(12.8153),
tensor(10.8958), tensor(15.2210), tensor(11.6303), tensor(14.2078),
```

tensor(13.2042), tensor(14.9616)]
[tensor(11.4001), tensor(13.8670), tensor(11.5339), tensor(10.6538),
tensor(11.7290), tensor(10.4666), tensor(11.5419), tensor(10.3485),
tensor(10.9648), tensor(11.4531), tensor(12.1160), tensor(10.6102),
tensor(15.1416), tensor(13.2506), tensor(11.1085), tensor(12.6452),
tensor(11.9756), tensor(11.0041), tensor(11.7692), tensor(12.8167),
tensor(9.4844), tensor(12.8169), tensor(11.9783), tensor(13.1584),
tensor(12.7531), tensor(11.2391), tensor(11.9495), tensor(11.5109),
tensor(14.7845), tensor(11.4230), tensor(14.4928), tensor(12.5198),
tensor(10.6867), tensor(10.8024), tensor(11.8187), tensor(13.7642),
tensor(12.2069), tensor(11.2522), tensor(12.1813), tensor(12.6063),
tensor(11.0097), tensor(13.0850), tensor(10.9563), tensor(12.6509),
tensor(9.8375), tensor(12.3890), tensor(10.0864), tensor(10.8534),
tensor(10.3785), tensor(12.6840)]
[tensor(11.4607), tensor(13.6654), tensor(10.4688), tensor(11.7464),
tensor(12.3521), tensor(10.0526), tensor(11.3544), tensor(10.7242),
tensor(10.8512), tensor(11.8575), tensor(11.9904), tensor(9.7185),
tensor(14.6202), tensor(13.4771), tensor(10.9780), tensor(11.5367),
tensor(11.8886), tensor(11.7398), tensor(12.6738), tensor(12.9156),
tensor(10.4976), tensor(12.4981), tensor(11.9931), tensor(12.4101),
tensor(12.0019), tensor(11.5159), tensor(10.6848), tensor(11.0110),
tensor(14.7022), tensor(11.5183), tensor(14.2478), tensor(12.4426),
tensor(10.1373), tensor(10.5956), tensor(13.2111), tensor(14.9097),
tensor(12.1040), tensor(10.5780), tensor(12.8032), tensor(11.9688),
tensor(11.6422), tensor(12.4867), tensor(10.5109), tensor(12.6770),
tensor(10.4020), tensor(12.6793), tensor(10.5523), tensor(11.8056),
tensor(10.3360), tensor(13.0476)]
[tensor(12.0100), tensor(12.1451), tensor(11.9650), tensor(11.5376),
tensor(11.5118), tensor(9.8788), tensor(12.3620), tensor(11.9429),
tensor(11.6506), tensor(12.3836), tensor(12.8181), tensor(11.9905),
tensor(14.7776), tensor(12.7749), tensor(11.4813), tensor(10.8692),
tensor(10.7186), tensor(10.7788), tensor(13.8018), tensor(13.4743),
tensor(10.5711), tensor(13.1994), tensor(11.2764), tensor(13.5121),
tensor(12.8101), tensor(12.1842), tensor(12.2768), tensor(12.2280),
tensor(14.3913), tensor(12.2676), tensor(14.1043), tensor(12.3945),
tensor(10.2580), tensor(11.8015), tensor(12.3594), tensor(14.0269),
tensor(11.5382), tensor(12.3423), tensor(14.6485), tensor(12.4166),
tensor(12.1520), tensor(13.3413), tensor(10.9054), tensor(12.0143),
tensor(10.3478), tensor(12.9836), tensor(11.2213), tensor(13.1336),
tensor(11.3547), tensor(14.4206)]
[tensor(12.8036), tensor(13.5556), tensor(13.0062), tensor(12.3378),
tensor(12.1902), tensor(10.8730), tensor(11.3982), tensor(11.6525),
tensor(11.8147), tensor(12.8439), tensor(11.6630), tensor(11.3660),
tensor(14.5832), tensor(14.4373), tensor(11.2619), tensor(12.5909),
tensor(11.1054), tensor(11.9504), tensor(13.4457), tensor(13.5776),
tensor(10.2574), tensor(14.0826), tensor(11.9538), tensor(13.6720),
tensor(12.2820), tensor(12.8426), tensor(11.5736), tensor(12.0119),
tensor(15.4699), tensor(11.3287), tensor(14.7745), tensor(13.4863),

```
tensor(10.4856), tensor(11.1537), tensor(12.5723), tensor(14.9548),
tensor(11.5275), tensor(12.2869), tensor(14.2529), tensor(11.8544),
tensor(12.0145), tensor(12.8608), tensor(10.2654), tensor(12.7394),
tensor(10.1512), tensor(12.6236), tensor(11.6185), tensor(12.3449),
tensor(11.0761), tensor(13.3243)]
[tensor(12.0941), tensor(11.9641), tensor(12.1900), tensor(12.2938),
tensor(12.2862), tensor(11.5302), tensor(12.0996), tensor(12.4077),
tensor(10.7424), tensor(13.0422), tensor(12.5513), tensor(10.8346),
tensor(15.4590), tensor(13.6071), tensor(10.9235), tensor(12.1613),
tensor(12.2104), tensor(10.6331), tensor(14.5427), tensor(13.3958),
tensor(11.1753), tensor(13.9697), tensor(12.3623), tensor(13.1279),
tensor(12.4888), tensor(12.9367), tensor(12.3838), tensor(12.4920),
tensor(14.9115), tensor(12.3828), tensor(15.3974), tensor(13.2404),
tensor(11.0518), tensor(12.0010), tensor(13.5118), tensor(14.6702),
tensor(12.3740), tensor(12.1519), tensor(13.2691), tensor(14.1512),
tensor(11.6547), tensor(14.3983), tensor(11.2770), tensor(11.9754),
tensor(11.6857), tensor(13.2878), tensor(10.7402), tensor(12.8028),
tensor(12.0523), tensor(13.8972)]
[tensor(10.2712), tensor(11.7205), tensor(10.4025), tensor(10.9058),
tensor(12.2677), tensor(9.5763), tensor(10.9625), tensor(11.6897),
tensor(10.2185), tensor(11.6039), tensor(10.7788), tensor(9.7968),
tensor(14.8517), tensor(12.9713), tensor(10.7428), tensor(11.8270),
tensor(11.3565), tensor(10.1522), tensor(14.1000), tensor(12.6855),
tensor(10.2396), tensor(13.3485), tensor(10.8922), tensor(12.1864),
tensor(11.3226), tensor(10.9094), tensor(10.9309), tensor(11.5745),
tensor(14.7102), tensor(11.7883), tensor(13.2834), tensor(11.9018),
tensor(9.5733), tensor(11.0141), tensor(12.6847), tensor(13.7792),
tensor(11.0878), tensor(9.9786), tensor(12.8796), tensor(11.5486),
tensor(11.2961), tensor(12.7824), tensor(10.1990), tensor(11.1358),
tensor(9.9062), tensor(11.9032), tensor(10.7983), tensor(11.9711),
tensor(10.6383), tensor(13.5434)]
[tensor(10.1464), tensor(11.3754), tensor(9.4620), tensor(10.4303),
tensor(11.0783), tensor(9.1726), tensor(10.2250), tensor(10.6987),
tensor(9.5499), tensor(10.8665), tensor(10.8177), tensor(9.6837),
tensor(13.2115), tensor(12.2133), tensor(9.2314), tensor(11.0926),
tensor(9.6645), tensor(10.1883), tensor(11.9401), tensor(12.2782),
tensor(9.0248), tensor(12.3840), tensor(10.7039), tensor(12.3561),
tensor(10.1931), tensor(10.5372), tensor(10.5161), tensor(11.5769),
tensor(13.7540), tensor(10.8404), tensor(11.9460), tensor(11.3765),
tensor(9.4702), tensor(10.0005), tensor(10.8666), tensor(12.4165),
tensor(10.4740), tensor(9.2418), tensor(11.4736), tensor(11.2113),
tensor(10.6330), tensor(11.0901), tensor(9.6072), tensor(10.0723),
tensor(9.0606), tensor(11.5241), tensor(9.5737), tensor(10.9053),
tensor(9.8302), tensor(11.8186)]
[tensor(11.6281), tensor(11.2802), tensor(12.3275), tensor(10.2388),
tensor(11.2570), tensor(9.7880), tensor(11.3899), tensor(9.7631),
tensor(9.9935), tensor(10.9743), tensor(11.3563), tensor(10.9774),
tensor(13.7025), tensor(12.2720), tensor(10.0546), tensor(11.4701),
```

```
tensor(11.3607), tensor(9.4847), tensor(11.9506), tensor(12.5207),
tensor(9.7167), tensor(13.3077), tensor(11.4178), tensor(12.3140),
tensor(12.1402), tensor(11.7066), tensor(12.2875), tensor(12.1512),
tensor(13.8139), tensor(11.3372), tensor(13.9665), tensor(12.4838),
tensor(9.5471), tensor(9.3211), tensor(12.8735), tensor(13.6790),
tensor(10.7880), tensor(10.3188), tensor(12.1156), tensor(11.6408),
tensor(10.3742), tensor(12.8815), tensor(9.2935), tensor(10.7339),
tensor(9.8491), tensor(11.9527), tensor(10.9559), tensor(11.0148),
tensor(9.2906), tensor(12.0814)]
[tensor(11.2412), tensor(11.3554), tensor(11.0134), tensor(10.9829),
tensor(12.9226), tensor(10.1244), tensor(10.9484), tensor(11.6380),
tensor(10.0883), tensor(9.9973), tensor(12.1783), tensor(9.8828),
tensor(13.2991), tensor(12.8405), tensor(10.1280), tensor(11.1337),
tensor(10.6049), tensor(10.1283), tensor(12.6028), tensor(12.3070),
tensor(9.2074), tensor(12.7405), tensor(11.0019), tensor(11.8553),
tensor(11.5074), tensor(11.1347), tensor(11.9728), tensor(10.3912),
tensor(13.8336), tensor(10.0501), tensor(13.1080), tensor(11.8807),
tensor(10.3202), tensor(10.9593), tensor(12.9598), tensor(12.3977),
tensor(11.8787), tensor(9.9663), tensor(13.2511), tensor(12.3131),
tensor(11.6511), tensor(12.9144), tensor(9.0782), tensor(10.9951),
tensor(8.6208), tensor(12.3455), tensor(10.1449), tensor(11.3399),
tensor(10.1025), tensor(12.2902)]
[tensor(12.6284), tensor(12.2385), tensor(11.5020), tensor(11.1791),
tensor(13.1200), tensor(12.0582), tensor(12.2121), tensor(12.2027),
tensor(10.8650), tensor(12.7592), tensor(13.0912), tensor(11.3250),
tensor(15.0985), tensor(13.0913), tensor(11.7189), tensor(11.1330),
tensor(11.6863), tensor(11.3329), tensor(13.3534), tensor(13.2041),
tensor(10.5408), tensor(12.6860), tensor(12.4301), tensor(13.5309),
tensor(12.7384), tensor(12.1245), tensor(12.8111), tensor(11.4991),
tensor(14.7413), tensor(12.0069), tensor(14.7132), tensor(12.7687),
tensor(10.5372), tensor(12.6959), tensor(13.2340), tensor(13.7793),
tensor(12.6886), tensor(12.1044), tensor(14.2235), tensor(13.8277),
tensor(11.4507), tensor(13.2449), tensor(10.8683), tensor(11.4628),
tensor(9.5728), tensor(14.1520), tensor(10.3623), tensor(11.8018),
tensor(11.8072), tensor(14.0696)]
[tensor(12.1473), tensor(13.6511), tensor(12.9934), tensor(12.7420),
tensor(13.1618), tensor(12.4599), tensor(11.9781), tensor(12.3153),
tensor(11.4435), tensor(13.4860), tensor(13.9843), tensor(11.7971),
tensor(16.0700), tensor(14.1960), tensor(11.8270), tensor(11.8657),
tensor(11.6200), tensor(12.5309), tensor(14.5078), tensor(13.8673),
tensor(11.1343), tensor(14.8311), tensor(12.9041), tensor(13.7166),
tensor(13.7098), tensor(13.5423), tensor(11.9939), tensor(13.4008),
tensor(15.8868), tensor(12.7856), tensor(15.4654), tensor(13.4919),
tensor(11.4520), tensor(12.8334), tensor(13.9430), tensor(15.4845),
tensor(12.3426), tensor(12.7954), tensor(14.6561), tensor(13.9053),
tensor(13.6659), tensor(15.0197), tensor(11.4718), tensor(12.4112),
tensor(11.5947), tensor(13.7192), tensor(12.9974), tensor(13.4784),
tensor(12.3632), tensor(13.8631)]
```

```
[tensor(12.7537), tensor(13.0038), tensor(12.1358), tensor(12.6818),
tensor(12.4726), tensor(11.8353), tensor(12.3159), tensor(12.0654),
tensor(10.4864), tensor(13.7624), tensor(13.0899), tensor(10.7943),
tensor(15.4351), tensor(14.2169), tensor(10.9599), tensor(12.2808),
tensor(10.9359), tensor(10.9614), tensor(14.4354), tensor(13.8011),
tensor(9.8505), tensor(13.8673), tensor(12.2696), tensor(13.5476),
tensor(12.6277), tensor(12.6472), tensor(12.1306), tensor(12.6754),
tensor(15.7939), tensor(13.0202), tensor(15.6715), tensor(13.0379),
tensor(11.5699), tensor(12.2243), tensor(12.5707), tensor(14.5435),
tensor(11.6051), tensor(11.9811), tensor(14.3040), tensor(13.5513),
tensor(12.8480), tensor(13.8651), tensor(11.1263), tensor(13.3713),
tensor(10.2977), tensor(14.6507), tensor(11.6140), tensor(13.3929),
tensor(11.6128), tensor(13.1813)]
[tensor(12.8517), tensor(15.7759), tensor(13.6004), tensor(13.9410),
tensor(15.1976), tensor(12.7129), tensor(14.2591), tensor(13.1195),
tensor(12.2908), tensor(14.0405), tensor(13.4629), tensor(13.0572),
tensor(18.4197), tensor(16.2883), tensor(13.6297), tensor(14.3290),
tensor(13.0617), tensor(12.3891), tensor(15.8019), tensor(16.3366),
tensor(12.8241), tensor(16.3100), tensor(14.4006), tensor(16.3562),
tensor(14.1853), tensor(13.7125), tensor(13.2074), tensor(15.0414),
tensor(18.3693), tensor(13.6550), tensor(16.5463), tensor(15.1888),
tensor(11.7899), tensor(13.9016), tensor(15.6817), tensor(16.8158),
tensor(13.7347), tensor(13.1220), tensor(15.1843), tensor(14.5743),
tensor(14.0601), tensor(16.1371), tensor(12.8219), tensor(12.9557),
tensor(13.1618), tensor(15.2431), tensor(13.8064), tensor(14.9515),
tensor(13.1964), tensor(16.1714)]
[tensor(12.5175), tensor(14.0030), tensor(13.0544), tensor(12.3477),
tensor(14.5260), tensor(12.1850), tensor(12.5579), tensor(12.5548),
tensor(12.6236), tensor(13.4510), tensor(13.2762), tensor(11.7003),
tensor(15.7629), tensor(15.0652), tensor(12.5638), tensor(12.4736),
tensor(12.2126), tensor(12.1865), tensor(15.0110), tensor(14.6044),
tensor(10.5537), tensor(14.0489), tensor(13.6816), tensor(14.6152),
tensor(13.8675), tensor(12.5594), tensor(12.7120), tensor(12.7189),
tensor(17.1483), tensor(13.1007), tensor(16.1328), tensor(14.0709),
tensor(10.4614), tensor(12.3771), tensor(14.1012), tensor(15.6982),
tensor(12.3233), tensor(12.0870), tensor(14.9979), tensor(13.5881),
tensor(12.8940), tensor(13.2371), tensor(10.9755), tensor(12.0972),
tensor(10.9449), tensor(14.6445), tensor(12.2671), tensor(13.6618),
tensor(11.6144), tensor(14.6509)]
[tensor(12.5017), tensor(14.0672), tensor(12.7081), tensor(11.4379),
tensor(12.6475), tensor(11.7038), tensor(12.6657), tensor(11.6911),
tensor(11.0836), tensor(12.7574), tensor(13.0981), tensor(13.6640),
tensor(16.5935), tensor(14.3540), tensor(11.8516), tensor(12.9120),
tensor(12.5469), tensor(11.8136), tensor(13.4342), tensor(15.2614),
tensor(11.3562), tensor(14.3114), tensor(13.1686), tensor(14.7858),
tensor(14.3859), tensor(12.3961), tensor(12.8734), tensor(13.3478),
tensor(15.5836), tensor(11.9057), tensor(14.7716), tensor(13.1954),
tensor(11.8878), tensor(12.7507), tensor(13.7421), tensor(14.4188),
```

```
tensor(12.9943), tensor(12.2494), tensor(13.5690), tensor(14.9033),
tensor(12.0165), tensor(14.1061), tensor(11.7199), tensor(12.5920),
tensor(11.8435), tensor(13.2894), tensor(11.6542), tensor(12.2373),
tensor(11.4980), tensor(14.7496)]
[tensor(10.4773), tensor(11.2907), tensor(11.6831), tensor(10.6439),
tensor(12.3882), tensor(9.7367), tensor(11.5642), tensor(10.2204),
tensor(9.8630), tensor(10.7780), tensor(11.6685), tensor(11.4496),
tensor(14.1653), tensor(12.3248), tensor(10.5643), tensor(11.0786),
tensor(10.8722), tensor(8.7623), tensor(11.6183), tensor(13.6357),
tensor(10.1677), tensor(12.3194), tensor(11.7922), tensor(13.1420),
tensor(11.8155), tensor(11.2814), tensor(11.0953), tensor(11.6898),
tensor(13.8905), tensor(11.9663), tensor(13.4972), tensor(12.3600),
tensor(9.6468), tensor(11.2281), tensor(12.1517), tensor(13.9980),
tensor(10.5748), tensor(11.2310), tensor(12.1587), tensor(12.4154),
tensor(11.7238), tensor(12.3695), tensor(9.7917), tensor(11.1907),
tensor(10.0614), tensor(12.6518), tensor(11.4941), tensor(11.3107),
tensor(9.2976), tensor(13.0682)]
[tensor(13.4230), tensor(13.2992), tensor(13.1654), tensor(12.5604),
tensor(13.5915), tensor(12.4514), tensor(11.8845), tensor(12.4224),
tensor(12.1549), tensor(12.9512), tensor(13.2930), tensor(11.5711),
tensor(14.9437), tensor(14.5764), tensor(11.8453), tensor(12.4431),
tensor(12.6694), tensor(11.8353), tensor(14.0501), tensor(13.4264),
tensor(9.5156), tensor(14.0603), tensor(12.5425), tensor(13.4706),
tensor(13.9540), tensor(13.6051), tensor(12.7148), tensor(12.8627),
tensor(15.9382), tensor(11.2279), tensor(15.8354), tensor(13.5536),
tensor(11.2581), tensor(12.1190), tensor(12.6129), tensor(14.8277),
tensor(11.8668), tensor(12.1912), tensor(14.4843), tensor(13.3029),
tensor(12.0147), tensor(13.2584), tensor(10.1001), tensor(12.2249),
tensor(10.0539), tensor(13.8382), tensor(11.2149), tensor(12.5960),
tensor(11.4231), tensor(14.1869)]
[tensor(12.9944), tensor(13.2129), tensor(13.0043), tensor(13.3837),
tensor(13.2320), tensor(10.3794), tensor(13.1590), tensor(12.8093),
tensor(12.2329), tensor(11.2983), tensor(12.5083), tensor(11.8012),
tensor(15.0546), tensor(14.0613), tensor(11.8618), tensor(13.1302),
tensor(11.1334), tensor(10.8147), tensor(11.9920), tensor(13.5205),
tensor(10.4449), tensor(14.5538), tensor(13.5052), tensor(14.9262),
tensor(12.4218), tensor(12.4739), tensor(11.8969), tensor(12.6137),
tensor(15.4689), tensor(11.2056), tensor(13.9438), tensor(12.6939),
tensor(11.0878), tensor(11.9717), tensor(13.2211), tensor(14.5880),
tensor(12.9853), tensor(11.4225), tensor(14.4501), tensor(12.3745),
tensor(13.0295), tensor(13.8263), tensor(10.5760), tensor(13.4131),
tensor(10.4269), tensor(13.2478), tensor(11.9267), tensor(12.6530),
tensor(9.6097), tensor(13.6465)]
[tensor(13.4041), tensor(13.6301), tensor(13.6737), tensor(14.1459),
tensor(14.9109), tensor(13.1821), tensor(12.9992), tensor(13.1639),
tensor(12.4690), tensor(13.6729), tensor(13.6798), tensor(13.0463),
tensor(16.8859), tensor(15.7263), tensor(12.6653), tensor(13.8234),
tensor(12.6431), tensor(11.8038), tensor(15.2448), tensor(14.7881),
```

```
tensor(12.7717), tensor(15.1968), tensor(14.2240), tensor(15.7262),
tensor(14.4418), tensor(13.6793), tensor(14.6741), tensor(12.7842),
tensor(17.1115), tensor(13.7897), tensor(16.1387), tensor(15.1424),
tensor(12.3543), tensor(14.1155), tensor(15.6106), tensor(16.6716),
tensor(13.3335), tensor(12.5149), tensor(15.3038), tensor(13.7769),
tensor(13.9113), tensor(14.6292), tensor(11.2503), tensor(13.6618),
tensor(11.6847), tensor(14.9221), tensor(12.9199), tensor(13.4127),
tensor(12.7044), tensor(15.9001)]
[tensor(11.7909), tensor(12.7386), tensor(11.8512), tensor(13.0717),
tensor(14.1329), tensor(11.3545), tensor(12.6151), tensor(13.4888),
tensor(11.8277), tensor(13.7671), tensor(12.4571), tensor(12.5287),
tensor(17.1376), tensor(15.1586), tensor(11.8349), tensor(12.5463),
tensor(12.2396), tensor(11.5840), tensor(15.6730), tensor(14.5698),
tensor(11.2823), tensor(14.7442), tensor(13.3267), tensor(15.5756),
tensor(13.1508), tensor(12.4013), tensor(12.5040), tensor(11.9862),
tensor(16.7426), tensor(14.4885), tensor(14.6732), tensor(15.2869),
tensor(10.3674), tensor(12.5959), tensor(14.5449), tensor(15.3488),
tensor(13.3947), tensor(10.7568), tensor(14.3013), tensor(13.2444),
tensor(13.7173), tensor(13.7128), tensor(11.3333), tensor(13.6864),
tensor(11.1327), tensor(14.7783), tensor(12.2256), tensor(13.8252),
tensor(11.5236), tensor(16.3637)]
[tensor(10.2285), tensor(10.6601), tensor(10.2195), tensor(10.7549),
tensor(10.9801), tensor(9.1657), tensor(11.3759), tensor(10.1219),
tensor(8.4945), tensor(10.4653), tensor(11.1070), tensor(10.4293),
tensor(13.5995), tensor(11.4191), tensor(9.5818), tensor(10.5115),
tensor(8.8871), tensor(9.5426), tensor(12.1166), tensor(11.8424),
tensor(8.8033), tensor(12.5055), tensor(10.5698), tensor(12.1714),
tensor(9.5384), tensor(10.6934), tensor(9.7304), tensor(9.7093),
tensor(13.1987), tensor(10.3200), tensor(12.1365), tensor(11.2286),
tensor(10.5681), tensor(10.1899), tensor(12.5113), tensor(12.4312),
tensor(11.5549), tensor(9.9220), tensor(11.7667), tensor(10.9263),
tensor(10.2729), tensor(12.7697), tensor(8.9252), tensor(10.4875),
tensor(9.6328), tensor(11.4439), tensor(10.4162), tensor(10.1932),
tensor(10.3214), tensor(11.6809)]
[tensor(12.5310), tensor(14.1587), tensor(13.1869), tensor(12.9577),
tensor(14.1662), tensor(12.2049), tensor(13.7468), tensor(13.0656),
tensor(11.3576), tensor(14.0634), tensor(13.1258), tensor(12.2977),
tensor(17.3199), tensor(14.2881), tensor(13.0866), tensor(13.4164),
tensor(13.0944), tensor(11.5167), tensor(16.0716), tensor(14.2752),
tensor(12.4981), tensor(14.8513), tensor(13.4170), tensor(14.1609),
tensor(14.0360), tensor(12.5351), tensor(12.8896), tensor(12.8480),
tensor(17.2245), tensor(14.2348), tensor(16.3113), tensor(14.4538),
tensor(12.5008), tensor(12.8264), tensor(15.4806), tensor(15.9451),
tensor(12.6950), tensor(13.2716), tensor(15.6729), tensor(14.3068),
tensor(12.8541), tensor(15.1237), tensor(12.0258), tensor(13.3487),
tensor(11.7792), tensor(14.7767), tensor(13.1237), tensor(13.8213),
tensor(12.6728), tensor(14.7201)]
[tensor(11.4361), tensor(11.7851), tensor(11.8483), tensor(12.0463),
```

```
tensor(12.3143), tensor(10.8872), tensor(12.0525), tensor(11.5031),
tensor(10.9676), tensor(12.2215), tensor(12.7865), tensor(10.8730),
tensor(15.6575), tensor(13.7879), tensor(10.8087), tensor(10.9681),
tensor(11.1243), tensor(11.5190), tensor(13.6674), tensor(12.7960),
tensor(9.8294), tensor(14.0408), tensor(12.0300), tensor(13.7799),
tensor(12.4316), tensor(13.3784), tensor(11.8731), tensor(11.0469),
tensor(15.1164), tensor(12.1583), tensor(14.6198), tensor(12.6621),
tensor(11.4225), tensor(12.3299), tensor(11.9705), tensor(13.2442),
tensor(12.7102), tensor(11.8909), tensor(13.4334), tensor(12.6981),
tensor(11.8594), tensor(13.8489), tensor(10.4379), tensor(12.1790),
tensor(10.6001), tensor(11.9415), tensor(10.3329), tensor(12.1687),
tensor(11.9684), tensor(13.3577)]
[tensor(10.0179), tensor(10.5446), tensor(10.2699), tensor(10.2999),
tensor(11.6772), tensor(10.1852), tensor(10.6882), tensor(10.2862),
tensor(10.7648), tensor(10.6973), tensor(10.9148), tensor(10.0077),
tensor(13.2389), tensor(12.8224), tensor(9.9886), tensor(10.4579),
tensor(10.2324), tensor(10.0092), tensor(12.5148), tensor(12.4156),
tensor(9.0930), tensor(12.0604), tensor(10.2413), tensor(12.2175),
tensor(11.5035), tensor(10.0803), tensor(10.6916), tensor(10.3761),
tensor(13.2475), tensor(10.6952), tensor(13.0755), tensor(11.5687),
tensor(9.2745), tensor(10.3470), tensor(11.0929), tensor(12.6140),
tensor(10.6314), tensor(10.3205), tensor(11.7452), tensor(12.0202),
tensor(10.4934), tensor(11.7993), tensor(9.0602), tensor(10.1748),
tensor(9.3487), tensor(12.1567), tensor(9.6913), tensor(10.4658),
tensor(10.3801), tensor(11.9646)]
[tensor(10.1894), tensor(9.8300), tensor(10.4988), tensor(9.9514),
tensor(11.8496), tensor(9.6717), tensor(9.8868), tensor(10.6634),
tensor(9.2424), tensor(9.0367), tensor(10.9334), tensor(9.7912),
tensor(10.5489), tensor(12.7346), tensor(8.8870), tensor(9.2824),
tensor(9.2124), tensor(9.2761), tensor(12.0221), tensor(12.4497),
tensor(8.4489), tensor(11.6596), tensor(10.5369), tensor(10.9874),
tensor(9.8370), tensor(11.3905), tensor(10.4299), tensor(10.4222),
tensor(13.0361), tensor(10.0326), tensor(12.9369), tensor(10.8389),
tensor(9.1913), tensor(10.5112), tensor(10.7245), tensor(11.5938),
tensor(10.6671), tensor(9.3252), tensor(11.0628), tensor(11.1969),
tensor(11.3594), tensor(10.5235), tensor(8.0111), tensor(9.5678),
tensor(8.2265), tensor(11.3740), tensor(9.6049), tensor(10.1865),
tensor(8.7598), tensor(10.8566)]
[tensor(9.3219), tensor(11.1399), tensor(9.7723), tensor(9.3140),
tensor(9.9587), tensor(10.6587), tensor(9.1595), tensor(11.3078),
tensor(9.0913), tensor(11.4621), tensor(11.1139), tensor(8.9230),
tensor(12.6709), tensor(11.5265), tensor(9.5622), tensor(9.6379),
tensor(9.8847), tensor(9.4336), tensor(12.7440), tensor(10.6362),
tensor(9.0034), tensor(12.1661), tensor(9.8721), tensor(9.9664),
tensor(10.5408), tensor(10.1699), tensor(10.5008), tensor(10.3917),
tensor(12.5407), tensor(10.7408), tensor(13.3908), tensor(10.8896),
tensor(9.2983), tensor(10.0114), tensor(11.6989), tensor(12.6875),
tensor(10.0725), tensor(9.9505), tensor(11.7852), tensor(10.9829),
```

```
tensor(10.8758), tensor(11.6126), tensor(9.0785), tensor(10.1362),
tensor(8.2736), tensor(11.7110), tensor(10.2091), tensor(10.3514),
tensor(10.1675), tensor(12.4447)]
[tensor(12.2011), tensor(13.8299), tensor(11.9862), tensor(12.0951),
tensor(12.6274), tensor(11.0232), tensor(11.3987), tensor(10.7141),
tensor(10.6006), tensor(12.3015), tensor(11.8269), tensor(11.5516),
tensor(14.9647), tensor(13.4343), tensor(10.5627), tensor(12.0046),
tensor(10.7425), tensor(10.6670), tensor(13.2593), tensor(13.4928),
tensor(9.1519), tensor(13.4765), tensor(11.6357), tensor(13.2582),
tensor(12.8299), tensor(12.1571), tensor(11.7975), tensor(12.3769),
tensor(15.0011), tensor(11.8356), tensor(13.3973), tensor(12.3848),
tensor(10.6695), tensor(11.2702), tensor(12.9703), tensor(13.9641),
tensor(11.5073), tensor(10.5269), tensor(13.0535), tensor(12.0906),
tensor(12.3005), tensor(12.5885), tensor(10.2154), tensor(12.6552),
tensor(9.8718), tensor(12.4757), tensor(11.7934), tensor(12.4988),
tensor(9.9467), tensor(13.6711)]
[tensor(10.7774), tensor(12.2862), tensor(10.8518), tensor(12.1786),
tensor(12.5352), tensor(11.7431), tensor(11.3329), tensor(13.0129),
tensor(11.5660), tensor(12.4969), tensor(12.2686), tensor(10.1802),
tensor(14.4579), tensor(12.3412), tensor(10.7516), tensor(10.5200),
tensor(10.0992), tensor(10.1276), tensor(14.0122), tensor(12.6464),
tensor(10.4367), tensor(13.3368), tensor(11.8234), tensor(13.0133),
tensor(12.4311), tensor(11.9220), tensor(11.7349), tensor(12.4001),
tensor(14.0509), tensor(12.0261), tensor(13.6522), tensor(11.6969),
tensor(10.3832), tensor(12.2883), tensor(12.5148), tensor(13.9211),
tensor(11.7128), tensor(11.9713), tensor(12.9442), tensor(13.6830),
tensor(12.3236), tensor(12.9738), tensor(10.9122), tensor(10.9710),
tensor(10.9803), tensor(13.2950), tensor(10.3719), tensor(12.8400),
tensor(12.1976), tensor(13.5683)]
[tensor(10.9007), tensor(12.3316), tensor(10.1771), tensor(10.7722),
tensor(11.2307), tensor(10.7292), tensor(10.4490), tensor(9.7167),
tensor(9.3252), tensor(11.5952), tensor(12.1495), tensor(9.5926),
tensor(13.1871), tensor(12.6495), tensor(10.7975), tensor(9.7794),
tensor(9.6331), tensor(10.5869), tensor(12.1339), tensor(12.8923),
tensor(10.1494), tensor(12.6104), tensor(12.1341), tensor(12.0230),
tensor(11.5774), tensor(10.9670), tensor(12.1049), tensor(11.5172),
tensor(13.7754), tensor(11.1896), tensor(13.0077), tensor(11.1405),
tensor(10.3638), tensor(10.9103), tensor(12.1404), tensor(12.8845),
tensor(11.0229), tensor(10.5573), tensor(12.7813), tensor(11.2559),
tensor(11.2619), tensor(11.5677), tensor(10.0361), tensor(10.7537),
tensor(8.8061), tensor(12.3641), tensor(10.8515), tensor(11.6500),
tensor(10.4213), tensor(11.8240)]
[tensor(13.4441), tensor(15.2359), tensor(12.7244), tensor(12.9802),
tensor(14.4337), tensor(12.1523), tensor(13.3811), tensor(12.8522),
tensor(11.5380), tensor(14.2289), tensor(14.4588), tensor(12.8351),
tensor(17.5173), tensor(16.0862), tensor(11.9930), tensor(13.3997),
tensor(13.1616), tensor(12.4662), tensor(16.2840), tensor(16.2003),
tensor(12.4450), tensor(15.3995), tensor(13.8813), tensor(15.2570),
```

```
tensor(14.6306), tensor(13.9978), tensor(13.6993), tensor(14.4926),
tensor(16.0964), tensor(13.3218), tensor(16.7160), tensor(14.4970),
tensor(11.0647), tensor(13.0657), tensor(14.3753), tensor(16.8216),
tensor(12.7690), tensor(12.3793), tensor(15.7579), tensor(14.5063),
tensor(14.2303), tensor(15.3827), tensor(11.1909), tensor(14.1033),
tensor(11.2799), tensor(14.7433), tensor(13.1313), tensor(13.6294),
tensor(12.2261), tensor(15.7614)]
[tensor(12.2753), tensor(12.9014), tensor(13.0212), tensor(11.4361),
tensor(12.9101), tensor(11.4115), tensor(12.2043), tensor(12.6052),
tensor(11.5249), tensor(12.8136), tensor(12.5768), tensor(12.2544),
tensor(16.0978), tensor(12.9842), tensor(11.2734), tensor(12.0647),
tensor(11.7682), tensor(10.4641), tensor(14.5736), tensor(14.0146),
tensor(11.3298), tensor(15.0658), tensor(12.7820), tensor(13.5268),
tensor(13.0979), tensor(13.4235), tensor(13.3551), tensor(13.0825),
tensor(15.5372), tensor(12.6987), tensor(15.8765), tensor(15.0261),
tensor(11.4073), tensor(11.7619), tensor(14.1015), tensor(14.6334),
tensor(12.5637), tensor(13.3690), tensor(13.5938), tensor(14.1666),
tensor(12.8889), tensor(14.1872), tensor(11.7068), tensor(11.7369),
tensor(11.2964), tensor(13.7650), tensor(11.8942), tensor(12.8452),
tensor(12.0465), tensor(14.9776)]
[tensor(10.1740), tensor(11.7316), tensor(10.9713), tensor(10.9367),
tensor(11.0309), tensor(10.1246), tensor(10.4769), tensor(9.6629),
tensor(9.2941), tensor(11.0383), tensor(10.7726), tensor(10.3122),
tensor(14.3957), tensor(13.3219), tensor(10.5384), tensor(11.1504),
tensor(10.8933), tensor(10.2294), tensor(11.5045), tensor(11.2132),
tensor(9.6833), tensor(12.5620), tensor(11.7488), tensor(12.4611),
tensor(12.8230), tensor(11.3789), tensor(10.7689), tensor(11.0917),
tensor(13.9826), tensor(10.5390), tensor(12.7440), tensor(12.2786),
tensor(8.8876), tensor(10.4914), tensor(12.2635), tensor(12.4281),
tensor(10.3279), tensor(10.3682), tensor(12.3322), tensor(10.5045),
tensor(10.2908), tensor(12.5613), tensor(9.2189), tensor(10.7457),
tensor(9.0489), tensor(11.2615), tensor(10.1959), tensor(11.1912),
tensor(9.5053), tensor(12.5046)]
[tensor(11.3348), tensor(12.5515), tensor(10.8978), tensor(10.1501),
tensor(11.7794), tensor(10.8090), tensor(11.4553), tensor(11.5911),
tensor(10.5449), tensor(11.4112), tensor(12.8250), tensor(11.3255),
tensor(14.9648), tensor(13.2203), tensor(10.4071), tensor(11.4645),
tensor(11.0253), tensor(11.0420), tensor(12.2355), tensor(13.3286),
tensor(8.4444), tensor(13.0561), tensor(11.8225), tensor(13.4222),
tensor(12.0322), tensor(11.6684), tensor(12.4095), tensor(12.2645),
tensor(14.3183), tensor(11.2036), tensor(14.1264), tensor(12.2599),
tensor(10.1779), tensor(10.5404), tensor(12.0510), tensor(13.4394),
tensor(11.9753), tensor(10.6279), tensor(12.8833), tensor(12.4443),
tensor(11.6615), tensor(12.8122), tensor(9.9488), tensor(10.9437),
tensor(8.7428), tensor(13.1342), tensor(10.8690), tensor(11.2900),
tensor(10.3939), tensor(13.3884)]
[tensor(11.8794), tensor(12.2856), tensor(12.4625), tensor(11.2770),
tensor(12.9046), tensor(10.2390), tensor(10.8550), tensor(11.1979),
```

```
tensor(9.8822), tensor(11.9434), tensor(11.4965), tensor(10.4775),
tensor(14.6543), tensor(12.9021), tensor(10.8832), tensor(12.3353),
tensor(10.6323), tensor(10.3352), tensor(13.5687), tensor(13.1332),
tensor(9.8539), tensor(12.4902), tensor(12.2687), tensor(12.8365),
tensor(12.9446), tensor(10.4338), tensor(12.2899), tensor(11.0161),
tensor(15.1013), tensor(11.9371), tensor(13.0178), tensor(12.2687),
tensor(10.3273), tensor(10.4987), tensor(13.9962), tensor(14.0349),
tensor(11.0417), tensor(10.4902), tensor(14.6011), tensor(11.2202),
tensor(11.1681), tensor(12.6030), tensor(10.4283), tensor(12.0424),
tensor(9.2867), tensor(12.3857), tensor(11.4908), tensor(11.4160),
tensor(10.0566), tensor(13.0036)]
[tensor(11.4397), tensor(11.5090), tensor(11.3373), tensor(11.8689),
tensor(12.0796), tensor(10.2551), tensor(11.3587), tensor(11.5676),
tensor(11.5566), tensor(12.3064), tensor(11.9487), tensor(10.7733),
tensor(14.3606), tensor(13.5918), tensor(10.7205), tensor(10.4792),
tensor(10.6480), tensor(11.0172), tensor(13.8070), tensor(13.5995),
tensor(11.8946), tensor(14.2559), tensor(12.2297), tensor(13.0389),
tensor(11.8577), tensor(12.9706), tensor(11.8256), tensor(11.8672),
tensor(15.4226), tensor(11.7112), tensor(14.1125), tensor(13.6391),
tensor(10.7347), tensor(11.8786), tensor(12.3134), tensor(13.9624),
tensor(12.0009), tensor(11.8274), tensor(12.9078), tensor(11.8863),
tensor(11.4443), tensor(12.4212), tensor(10.0868), tensor(10.8643),
tensor(11.4568), tensor(12.8657), tensor(10.6419), tensor(11.4508),
tensor(11.3387), tensor(14.1639)]
[tensor(12.3758), tensor(13.9438), tensor(13.6491), tensor(12.1810),
tensor(12.5221), tensor(10.0590), tensor(12.0486), tensor(11.6288),
tensor(11.5068), tensor(12.8755), tensor(13.7302), tensor(12.4008),
tensor(15.5290), tensor(14.9339), tensor(11.5677), tensor(11.7516),
tensor(11.2024), tensor(12.3279), tensor(14.2794), tensor(15.5181),
tensor(11.4517), tensor(14.9474), tensor(13.6228), tensor(15.0145),
tensor(12.9092), tensor(14.5708), tensor(12.7109), tensor(13.7141),
tensor(16.2988), tensor(12.6237), tensor(14.7761), tensor(13.9477),
tensor(11.2668), tensor(12.2221), tensor(13.5436), tensor(15.0643),
tensor(12.4647), tensor(12.2786), tensor(14.2189), tensor(13.4474),
tensor(13.4198), tensor(14.1771), tensor(11.1867), tensor(12.6505),
tensor(11.0481), tensor(12.6297), tensor(12.1534), tensor(13.5210),
tensor(11.6190), tensor(14.6027)]
[tensor(13.4488), tensor(14.1563), tensor(12.8713), tensor(13.6639),
tensor(13.4595), tensor(13.3902), tensor(13.3545), tensor(11.8651),
tensor(11.2633), tensor(13.6902), tensor(13.8098), tensor(12.6006),
tensor(16.7458), tensor(15.8006), tensor(12.4840), tensor(12.5166),
tensor(12.7626), tensor(12.1936), tensor(14.9607), tensor(15.3658),
tensor(12.2956), tensor(15.9868), tensor(14.3137), tensor(14.8458),
tensor(14.0289), tensor(14.4969), tensor(13.7174), tensor(14.2607),
tensor(16.3734), tensor(13.1960), tensor(16.2058), tensor(14.4320),
tensor(12.6969), tensor(14.1554), tensor(13.8969), tensor(14.8830),
tensor(13.5913), tensor(13.5582), tensor(14.3451), tensor(14.8025),
tensor(13.1506), tensor(14.3364), tensor(13.0204), tensor(13.0600),
```

```
tensor(11.3962), tensor(14.7568), tensor(12.8343), tensor(14.0673),
tensor(12.9023), tensor(15.2887)]
[tensor(11.7482), tensor(13.4092), tensor(11.9763), tensor(13.0397),
tensor(12.6446), tensor(11.4162), tensor(12.2241), tensor(10.9930),
tensor(10.4448), tensor(13.6269), tensor(11.3052), tensor(11.8027),
tensor(14.8828), tensor(14.5136), tensor(11.6110), tensor(12.5039),
tensor(10.3012), tensor(11.2111), tensor(13.4623), tensor(13.2628),
tensor(10.8409), tensor(13.6758), tensor(11.3827), tensor(13.9411),
tensor(12.8977), tensor(11.4609), tensor(11.3153), tensor(12.2638),
tensor(15.5407), tensor(11.5993), tensor(13.6212), tensor(13.8777),
tensor(10.2476), tensor(12.4371), tensor(13.6693), tensor(14.5393),
tensor(11.7688), tensor(11.6366), tensor(13.4011), tensor(12.9256),
tensor(11.7993), tensor(13.6138), tensor(10.8903), tensor(11.9775),
tensor(10.5003), tensor(13.0933), tensor(12.5664), tensor(14.0307),
tensor(11.9813), tensor(13.6429)]
[tensor(10.7379), tensor(13.0867), tensor(11.0705), tensor(11.1214),
tensor(12.0153), tensor(9.4939), tensor(10.8481), tensor(12.3047),
tensor(11.4317), tensor(11.8020), tensor(11.9944), tensor(10.8326),
tensor(14.4433), tensor(13.2448), tensor(10.0371), tensor(11.4992),
tensor(10.5274), tensor(10.2671), tensor(13.3097), tensor(13.1927),
tensor(10.1978), tensor(12.5142), tensor(11.1565), tensor(13.3967),
tensor(11.4460), tensor(10.7829), tensor(11.5043), tensor(12.0762),
tensor(14.7438), tensor(12.0104), tensor(13.2673), tensor(14.0629),
tensor(9.3070), tensor(11.0285), tensor(12.7138), tensor(14.1499),
tensor(11.7398), tensor(10.1473), tensor(12.6254), tensor(13.3789),
tensor(12.5295), tensor(13.5301), tensor(10.8992), tensor(12.2439),
tensor(10.1203), tensor(14.2289), tensor(10.9878), tensor(13.0853),
tensor(10.5659), tensor(13.9541)]
[tensor(10.1431), tensor(12.6888), tensor(12.5297), tensor(11.2572),
tensor(12.3599), tensor(10.3687), tensor(11.0446), tensor(11.4745),
tensor(10.7100), tensor(11.6137), tensor(12.7421), tensor(10.6188),
tensor(14.3107), tensor(14.6165), tensor(11.1619), tensor(11.8008),
tensor(11.0694), tensor(10.3034), tensor(13.6399), tensor(13.7004),
tensor(10.0461), tensor(13.5932), tensor(12.5027), tensor(12.8103),
tensor(13.1017), tensor(11.1140), tensor(12.3759), tensor(12.1803),
tensor(15.5040), tensor(11.7167), tensor(14.1761), tensor(13.4671),
tensor(10.5218), tensor(11.2914), tensor(12.4668), tensor(13.9158),
tensor(11.7252), tensor(11.4942), tensor(13.2754), tensor(13.2860),
tensor(11.5288), tensor(13.3819), tensor(10.7599), tensor(12.2316),
tensor(9.8948), tensor(13.1601), tensor(11.8925), tensor(11.7143),
tensor(10.7778), tensor(12.9073)]
[tensor(11.0292), tensor(14.1634), tensor(12.0461), tensor(13.2936),
tensor(13.3616), tensor(12.1342), tensor(15.0527), tensor(12.1084),
tensor(11.2188), tensor(11.9725), tensor(12.7552), tensor(11.1632),
tensor(10.3530), tensor(11.8918), tensor(12.4510), tensor(12.8128),
tensor(11.5776), tensor(13.3886), tensor(10.7189), tensor(10.9251),
tensor(9.6012), tensor(10.8924), tensor(13.5555), tensor(11.7667),
tensor(10.7995), tensor(11.9429), tensor(11.2560), tensor(12.1426),
```

```
tensor(10.7214), tensor(13.0507), tensor(12.1351), tensor(12.5918),
tensor(10.9536), tensor(12.1775), tensor(12.6528), tensor(11.7080),
tensor(11.8185), tensor(12.6493), tensor(13.2578), tensor(11.9201),
tensor(13.2749), tensor(13.8880), tensor(11.9957), tensor(12.9292),
tensor(15.7590), tensor(14.6245), tensor(13.4023), tensor(13.2492),
tensor(13.3194), tensor(12.1300)]
[tensor(11.9710), tensor(14.2240), tensor(12.0155), tensor(12.9224),
tensor(14.8808), tensor(11.5085), tensor(13.8776), tensor(11.5024),
tensor(11.3252), tensor(13.7608), tensor(12.5676), tensor(12.9661),
tensor(10.5918), tensor(12.0119), tensor(14.1152), tensor(12.2008),
tensor(11.7891), tensor(13.3364), tensor(9.6166), tensor(12.6115),
tensor(10.7365), tensor(12.0681), tensor(13.0389), tensor(13.5038),
tensor(11.5201), tensor(11.3966), tensor(12.7755), tensor(11.1638),
tensor(11.7953), tensor(12.6419), tensor(12.5507), tensor(12.1594),
tensor(12.5887), tensor(11.3366), tensor(13.1369), tensor(12.6465),
tensor(12.1776), tensor(14.0706), tensor(12.2966), tensor(11.7188),
tensor(12.3430), tensor(13.1934), tensor(12.7264), tensor(12.9143),
tensor(14.2455), tensor(13.8684), tensor(13.8172), tensor(12.8469),
tensor(12.7080), tensor(13.5572)]
[tensor(8.7722), tensor(10.9820), tensor(10.2637), tensor(11.4130),
tensor(11.8675), tensor(8.2228), tensor(12.2894), tensor(10.0047),
tensor(9.8292), tensor(12.4106), tensor(10.7985), tensor(11.1585),
tensor(8.4872), tensor(10.8064), tensor(10.7456), tensor(10.5356),
tensor(8.8242), tensor(9.9556), tensor(7.9843), tensor(9.5123), tensor(8.2349),
tensor(9.1284), tensor(9.7584), tensor(11.3886), tensor(9.6784), tensor(9.6398),
tensor(9.8277), tensor(9.4462), tensor(10.0682), tensor(10.1651),
tensor(9.1865), tensor(10.3343), tensor(10.1157), tensor(8.5460),
tensor(10.2297), tensor(9.0126), tensor(9.4710), tensor(11.3233),
tensor(10.4922), tensor(10.0785), tensor(9.7539), tensor(10.8030),
tensor(10.2659), tensor(9.4793), tensor(11.2846), tensor(10.6715),
tensor(11.2768), tensor(10.4736), tensor(11.1791), tensor(10.2480)]
[tensor(12.1742), tensor(13.6488), tensor(12.4168), tensor(11.8723),
tensor(13.0674), tensor(11.3224), tensor(14.8303), tensor(13.3168),
tensor(12.6229), tensor(12.9845), tensor(12.8807), tensor(12.5787),
tensor(10.4997), tensor(11.4882), tensor(11.8342), tensor(12.3098),
tensor(10.3755), tensor(12.0322), tensor(10.9618), tensor(11.5854),
tensor(10.9355), tensor(11.6842), tensor(12.1393), tensor(13.4155),
tensor(11.8425), tensor(11.9583), tensor(11.1991), tensor(12.2182),
tensor(10.4506), tensor(12.7088), tensor(13.5198), tensor(11.6331),
tensor(11.7079), tensor(11.5513), tensor(13.6961), tensor(11.6870),
tensor(11.2268), tensor(13.5254), tensor(13.7966), tensor(11.8416),
tensor(11.8371), tensor(13.5592), tensor(12.5808), tensor(12.3137),
tensor(14.0887), tensor(13.5013), tensor(14.7332), tensor(12.9371),
tensor(12.0049), tensor(13.5634)]
[tensor(13.3020), tensor(14.8671), tensor(12.2462), tensor(13.2703),
tensor(13.6366), tensor(12.6505), tensor(15.0932), tensor(12.1620),
tensor(12.6720), tensor(14.0610), tensor(13.8976), tensor(14.1581),
tensor(10.2411), tensor(13.3011), tensor(13.1773), tensor(14.2491),
```

```
tensor(12.0439), tensor(12.5384), tensor(10.5479), tensor(12.4015),
tensor(11.2873), tensor(12.1425), tensor(14.3068), tensor(13.6797),
tensor(12.0406), tensor(12.6539), tensor(12.4058), tensor(12.0638),
tensor(11.6919), tensor(13.5212), tensor(12.5020), tensor(13.8967),
tensor(12.3266), tensor(11.3632), tensor(12.8804), tensor(12.5045),
tensor(12.8261), tensor(14.0243), tensor(13.7004), tensor(13.2588),
tensor(12.3830), tensor(13.4154), tensor(13.2130), tensor(13.6454),
tensor(15.6004), tensor(13.7381), tensor(15.1136), tensor(14.2509),
tensor(14.0241), tensor(14.1613)]
[tensor(11.1766), tensor(13.8675), tensor(11.7494), tensor(12.6192),
tensor(14.7114), tensor(11.3480), tensor(12.9839), tensor(12.8755),
tensor(12.2632), tensor(12.3918), tensor(13.4614), tensor(12.9381),
tensor(10.5890), tensor(12.5401), tensor(13.2066), tensor(12.2055),
tensor(11.8490), tensor(13.9721), tensor(11.5911), tensor(10.9335),
tensor(10.7496), tensor(11.3102), tensor(12.9569), tensor(13.5235),
tensor(11.9117), tensor(13.3210), tensor(12.5915), tensor(11.9563),
tensor(11.8840), tensor(13.1560), tensor(13.3045), tensor(13.6484),
tensor(12.1485), tensor(13.1060), tensor(13.7060), tensor(12.6444),
tensor(12.9480), tensor(13.3390), tensor(13.2866), tensor(11.9990),
tensor(13.5226), tensor(13.9728), tensor(12.3301), tensor(12.9114),
tensor(16.6349), tensor(13.3597), tensor(13.7784), tensor(14.3982),
tensor(13.8647), tensor(13.2462)]
[tensor(13.4997), tensor(15.2559), tensor(13.0011), tensor(13.4867),
tensor(14.4704), tensor(13.0512), tensor(16.0648), tensor(13.3366),
tensor(12.8421), tensor(15.0043), tensor(14.1513), tensor(14.1589),
tensor(11.4437), tensor(13.7930), tensor(13.5976), tensor(13.9706),
tensor(13.6806), tensor(13.8648), tensor(12.0580), tensor(12.4994),
tensor(11.0792), tensor(11.7772), tensor(13.8463), tensor(13.2557),
tensor(12.3147), tensor(12.9745), tensor(12.2279), tensor(13.0065),
tensor(12.7203), tensor(14.2885), tensor(12.3244), tensor(14.0799),
tensor(13.4074), tensor(12.1332), tensor(13.8723), tensor(12.0504),
tensor(13.3921), tensor(13.8303), tensor(13.5541), tensor(13.5185),
tensor(12.7186), tensor(13.9268), tensor(12.7687), tensor(13.5596),
tensor(15.3496), tensor(13.9156), tensor(14.1138), tensor(14.6210),
tensor(13.6904), tensor(14.4885)]
[tensor(10.1921), tensor(10.9207), tensor(9.0471), tensor(11.3460),
tensor(10.5542), tensor(9.3684), tensor(10.7936), tensor(9.3509),
tensor(9.5704), tensor(10.5848), tensor(10.6158), tensor(10.0531),
tensor(7.2518), tensor(8.9833), tensor(9.9184), tensor(9.9701), tensor(9.0375),
tensor(9.1075), tensor(7.6880), tensor(9.4138), tensor(8.3700), tensor(10.2602),
tensor(11.1337), tensor(10.5486), tensor(9.7193), tensor(10.4920),
tensor(9.7720), tensor(9.3384), tensor(9.0269), tensor(8.5992), tensor(9.1047),
tensor(10.9009), tensor(9.4612), tensor(7.5771), tensor(10.6530),
tensor(9.9590), tensor(10.9567), tensor(11.5990), tensor(11.7065),
tensor(9.5728), tensor(10.5436), tensor(9.6350), tensor(10.3792),
tensor(10.7024), tensor(11.8976), tensor(11.1019), tensor(11.7665),
tensor(11.0226), tensor(10.9408), tensor(11.2883)]
[tensor(10.4181), tensor(12.5567), tensor(9.3941), tensor(11.4597),
```

```
tensor(12.8194), tensor(10.2855), tensor(12.0297), tensor(11.4368),
tensor(10.7723), tensor(12.7144), tensor(11.7988), tensor(12.5320),
tensor(7.4995), tensor(11.5969), tensor(10.7256), tensor(11.0136),
tensor(10.1251), tensor(12.4694), tensor(8.6676), tensor(10.6457),
tensor(9.4506), tensor(10.5797), tensor(11.7891), tensor(13.0409),
tensor(10.7270), tensor(10.8656), tensor(11.0907), tensor(10.5605),
tensor(10.1175), tensor(11.8193), tensor(10.5944), tensor(11.7110),
tensor(11.1016), tensor(10.3468), tensor(11.1192), tensor(10.3013),
tensor(10.6463), tensor(12.0716), tensor(10.7950), tensor(11.3407),
tensor(10.8150), tensor(11.4165), tensor(10.0620), tensor(10.9861),
tensor(12.9114), tensor(12.1055), tensor(12.3470), tensor(11.8582),
tensor(12.3792), tensor(10.9981)]
[tensor(11.0113), tensor(13.1714), tensor(11.5779), tensor(13.4998),
tensor(13.5889), tensor(10.0065), tensor(13.1014), tensor(11.6607),
tensor(11.0236), tensor(12.5285), tensor(12.7202), tensor(11.8488),
tensor(9.9917), tensor(12.3415), tensor(11.8242), tensor(11.0681),
tensor(11.1896), tensor(11.9664), tensor(10.3349), tensor(11.8642),
tensor(9.7772), tensor(10.7563), tensor(11.5401), tensor(12.4832),
tensor(10.9506), tensor(12.0197), tensor(11.3291), tensor(10.7591),
tensor(11.4555), tensor(11.8355), tensor(12.0266), tensor(13.9015),
tensor(11.8706), tensor(10.3070), tensor(10.9022), tensor(11.5119),
tensor(12.2285), tensor(12.9612), tensor(12.9357), tensor(11.2957),
tensor(11.9358), tensor(12.2480), tensor(10.7632), tensor(11.5483),
tensor(14.8791), tensor(12.8851), tensor(12.2078), tensor(12.2875),
tensor(13.3232), tensor(12.8964)]
[tensor(12.2267), tensor(14.2254), tensor(10.9504), tensor(12.7370),
tensor(12.6203), tensor(11.2299), tensor(13.9294), tensor(11.9885),
tensor(12.7106), tensor(12.8919), tensor(13.6723), tensor(12.3735),
tensor(10.2747), tensor(11.9765), tensor(10.6341), tensor(12.0188),
tensor(10.6397), tensor(11.5716), tensor(9.4344), tensor(10.6570),
tensor(9.8183), tensor(11.4737), tensor(13.5437), tensor(12.0377),
tensor(12.1721), tensor(12.3026), tensor(11.5275), tensor(10.5117),
tensor(10.5357), tensor(13.4643), tensor(12.5113), tensor(12.0363),
tensor(12.8192), tensor(10.2257), tensor(12.0116), tensor(12.3110),
tensor(11.6073), tensor(13.7581), tensor(13.8802), tensor(11.8942),
tensor(12.0158), tensor(14.0343), tensor(11.0878), tensor(11.4091),
tensor(13.1850), tensor(12.7559), tensor(13.3682), tensor(13.0103),
tensor(12.7389), tensor(12.6828)]
[tensor(11.9016), tensor(13.5927), tensor(10.6738), tensor(11.4513),
tensor(14.1690), tensor(13.5965), tensor(13.4249), tensor(11.7346),
tensor(11.3833), tensor(13.3782), tensor(12.6176), tensor(12.7717),
tensor(8.6179), tensor(12.7834), tensor(12.8828), tensor(13.1942),
tensor(10.9765), tensor(12.8056), tensor(10.5477), tensor(11.3798),
tensor(10.3525), tensor(10.9334), tensor(14.6909), tensor(13.5164),
tensor(11.7314), tensor(12.1009), tensor(11.6729), tensor(11.6395),
tensor(11.1373), tensor(12.8650), tensor(12.1111), tensor(13.2359),
tensor(11.1119), tensor(10.9182), tensor(12.5654), tensor(11.1912),
tensor(11.3785), tensor(13.9215), tensor(12.2586), tensor(11.1460),
```

```
tensor(13.2542), tensor(12.3826), tensor(12.5811), tensor(12.4317),
tensor(15.0167), tensor(12.1492), tensor(13.5953), tensor(13.4168),
tensor(13.2926), tensor(12.4680)]
[tensor(11.3918), tensor(13.4154), tensor(11.0828), tensor(12.6404),
tensor(12.7438), tensor(11.7218), tensor(13.5640), tensor(11.4894),
tensor(12.6044), tensor(12.1535), tensor(13.0958), tensor(11.9207),
tensor(10.3711), tensor(11.8811), tensor(12.2323), tensor(12.0233),
tensor(10.9285), tensor(12.5052), tensor(11.3612), tensor(11.1379),
tensor(9.8189), tensor(10.0922), tensor(13.3210), tensor(13.6784),
tensor(12.0965), tensor(12.9956), tensor(10.5975), tensor(11.7343),
tensor(10.9407), tensor(12.6343), tensor(12.0740), tensor(12.9611),
tensor(10.2750), tensor(10.9801), tensor(13.2067), tensor(11.6223),
tensor(11.4817), tensor(11.7822), tensor(13.6299), tensor(12.3311),
tensor(13.1152), tensor(13.5711), tensor(12.2479), tensor(12.5030),
tensor(15.7988), tensor(14.3307), tensor(13.4399), tensor(12.9264),
tensor(13.0592), tensor(12.3355)]
[tensor(10.5938), tensor(12.3398), tensor(10.7857), tensor(12.3411),
tensor(13.3895), tensor(9.5426), tensor(12.0714), tensor(10.7088),
tensor(11.2293), tensor(11.3269), tensor(11.3937), tensor(12.8693),
tensor(9.6004), tensor(11.5480), tensor(12.1872), tensor(11.5621),
tensor(9.7865), tensor(11.0006), tensor(8.6239), tensor(10.5863),
tensor(9.6235), tensor(10.3821), tensor(11.6560), tensor(10.7112),
tensor(10.8534), tensor(9.7873), tensor(11.0126), tensor(10.0199),
tensor(10.5702), tensor(11.5742), tensor(11.1640), tensor(12.3130),
tensor(11.2978), tensor(9.0083), tensor(10.0675), tensor(11.1837),
tensor(10.8817), tensor(11.7164), tensor(11.3706), tensor(11.1064),
tensor(10.1228), tensor(12.1728), tensor(11.3599), tensor(10.7283),
tensor(12.6747), tensor(12.2506), tensor(10.7421), tensor(12.1787),
tensor(12.1357), tensor(11.9943)]
[tensor(11.4096), tensor(13.7473), tensor(12.1844), tensor(13.8758),
tensor(15.0281), tensor(11.2695), tensor(13.9078), tensor(13.5430),
tensor(12.4536), tensor(14.2170), tensor(14.1908), tensor(14.1515),
tensor(10.6511), tensor(12.2133), tensor(14.1990), tensor(13.2938),
tensor(12.5622), tensor(12.8049), tensor(11.0904), tensor(12.8745),
tensor(11.1832), tensor(11.6153), tensor(12.9292), tensor(15.2103),
tensor(12.0221), tensor(13.1713), tensor(13.9735), tensor(12.4581),
tensor(13.4655), tensor(12.8368), tensor(11.4065), tensor(15.4351),
tensor(10.9662), tensor(11.8307), tensor(13.4042), tensor(12.5029),
tensor(12.4351), tensor(14.4556), tensor(14.6807), tensor(12.0029),
tensor(13.9330), tensor(13.8905), tensor(13.7758), tensor(14.2109),
tensor(16.4213), tensor(14.0454), tensor(14.2382), tensor(13.7752),
tensor(14.6518), tensor(13.8786)]
[tensor(11.1965), tensor(14.0076), tensor(11.3274), tensor(13.2960),
tensor(13.5237), tensor(12.1420), tensor(13.9792), tensor(13.0292),
tensor(11.4196), tensor(13.5194), tensor(12.6046), tensor(12.1074),
tensor(9.4106), tensor(12.0872), tensor(10.5071), tensor(12.8713),
tensor(10.5076), tensor(11.8749), tensor(10.6863), tensor(10.3711),
tensor(10.3344), tensor(11.4087), tensor(12.8007), tensor(13.6314),
```

```
tensor(10.7688), tensor(12.8698), tensor(11.7163), tensor(11.8738),
tensor(11.1834), tensor(13.2972), tensor(11.0726), tensor(13.9292),
tensor(11.6864), tensor(10.4950), tensor(12.8408), tensor(11.4215),
tensor(10.4553), tensor(13.1759), tensor(13.0759), tensor(12.0857),
tensor(12.3332), tensor(13.0995), tensor(11.7582), tensor(11.0155),
tensor(14.0158), tensor(12.4344), tensor(13.4463), tensor(13.0666),
tensor(13.4360), tensor(12.3797)]
[tensor(10.6428), tensor(12.4237), tensor(10.3351), tensor(12.2094),
tensor(12.5814), tensor(11.0301), tensor(13.1554), tensor(10.5792),
tensor(12.4302), tensor(12.7132), tensor(13.0588), tensor(12.8386),
tensor(8.4854), tensor(11.4556), tensor(11.4988), tensor(11.9360),
tensor(10.5911), tensor(11.9552), tensor(9.8078), tensor(10.8257),
tensor(9.3273), tensor(10.8509), tensor(12.0916), tensor(13.9712),
tensor(11.3425), tensor(13.0170), tensor(11.9059), tensor(11.2294),
tensor(9.5257), tensor(11.8096), tensor(11.4582), tensor(11.8401),
tensor(11.2095), tensor(11.1054), tensor(12.1672), tensor(10.4748),
tensor(11.0409), tensor(13.0027), tensor(12.1782), tensor(11.9252),
tensor(12.2838), tensor(12.0927), tensor(11.4317), tensor(12.0117),
tensor(14.4410), tensor(14.0547), tensor(13.7213), tensor(12.1551),
tensor(12.9560), tensor(11.4942)]
[tensor(12.1515), tensor(14.5769), tensor(13.0237), tensor(14.1616),
tensor(15.8297), tensor(12.0635), tensor(15.5786), tensor(13.3895),
tensor(12.6540), tensor(14.7477), tensor(14.0870), tensor(13.9149),
tensor(11.6434), tensor(12.9684), tensor(13.3682), tensor(13.3053),
tensor(12.0826), tensor(13.1111), tensor(11.8481), tensor(13.6182),
tensor(11.9462), tensor(12.1822), tensor(13.3615), tensor(15.5701),
tensor(12.7472), tensor(13.6898), tensor(12.8462), tensor(12.8058),
tensor(12.1846), tensor(14.2434), tensor(13.5379), tensor(15.2714),
tensor(13.1950), tensor(10.6595), tensor(13.9331), tensor(11.8851),
tensor(11.9364), tensor(14.3286), tensor(13.8113), tensor(12.8365),
tensor(12.5110), tensor(14.0512), tensor(13.4154), tensor(13.5586),
tensor(15.4736), tensor(14.6881), tensor(14.9920), tensor(13.7493),
tensor(14.2368), tensor(13.8219)]
[tensor(10.8935), tensor(15.6256), tensor(11.4547), tensor(13.7365),
tensor(14.9629), tensor(10.6828), tensor(14.6455), tensor(13.9577),
tensor(12.0196), tensor(14.7679), tensor(14.5661), tensor(14.2049),
tensor(10.7030), tensor(13.1505), tensor(13.3331), tensor(13.0112),
tensor(12.1052), tensor(13.3332), tensor(10.6335), tensor(13.0181),
tensor(11.1146), tensor(11.6011), tensor(12.8004), tensor(14.3350),
tensor(12.3985), tensor(12.7028), tensor(12.7740), tensor(12.7389),
tensor(13.0355), tensor(13.2202), tensor(12.2229), tensor(14.8809),
tensor(12.6969), tensor(11.5567), tensor(12.9696), tensor(12.1571),
tensor(12.2470), tensor(13.9874), tensor(13.9596), tensor(12.8839),
tensor(13.0249), tensor(14.2903), tensor(13.0039), tensor(13.6457),
tensor(14.7359), tensor(13.0433), tensor(14.6207), tensor(13.6297),
tensor(14.1870), tensor(13.9239)]
[tensor(9.6702), tensor(13.0485), tensor(9.8668), tensor(11.9769),
tensor(12.8177), tensor(9.7498), tensor(12.2043), tensor(10.2518),
```

```
tensor(11.4815), tensor(12.9766), tensor(12.3294), tensor(11.4679),
tensor(9.2410), tensor(11.1947), tensor(11.6858), tensor(11.6760),
tensor(10.6762), tensor(11.4715), tensor(9.2102), tensor(10.7428),
tensor(8.7515), tensor(9.8761), tensor(11.4946), tensor(12.2072),
tensor(11.3088), tensor(12.8848), tensor(11.6059), tensor(9.9849),
tensor(10.9149), tensor(11.8845), tensor(9.7357), tensor(12.5354),
tensor(10.1362), tensor(9.5556), tensor(11.6623), tensor(10.9959),
tensor(11.0762), tensor(12.6722), tensor(12.1186), tensor(10.5145),
tensor(11.9922), tensor(11.2830), tensor(11.4543), tensor(11.1471),
tensor(12.5736), tensor(10.9143), tensor(11.8839), tensor(11.4449),
tensor(13.2822), tensor(11.8235)]
[tensor(9.1053), tensor(12.3739), tensor(9.9241), tensor(11.1692),
tensor(10.9614), tensor(9.6650), tensor(11.0393), tensor(10.3496),
tensor(9.3471), tensor(10.1670), tensor(11.2975), tensor(10.5629),
tensor(8.3108), tensor(9.9123), tensor(10.0199), tensor(10.7702),
tensor(9.1421), tensor(10.2678), tensor(9.2931), tensor(8.8268), tensor(9.5408),
tensor(8.9290), tensor(11.4584), tensor(11.5316), tensor(9.8078),
tensor(9.6189), tensor(9.7928), tensor(10.4460), tensor(9.9516),
tensor(11.3640), tensor(10.0424), tensor(11.9653), tensor(9.8478),
tensor(9.4574), tensor(9.7090), tensor(9.2473), tensor(9.2419), tensor(11.4583),
tensor(12.1141), tensor(10.0676), tensor(10.4118), tensor(10.3976),
tensor(10.0339), tensor(10.5599), tensor(13.6990), tensor(11.3839),
tensor(12.1498), tensor(10.8551), tensor(11.1906), tensor(10.0328)]
[tensor(11.8962), tensor(13.4372), tensor(11.4565), tensor(13.1332),
tensor(13.9643), tensor(12.2496), tensor(14.6170), tensor(13.2888),
tensor(11.6847), tensor(13.6719), tensor(13.1750), tensor(12.7760),
tensor(9.8825), tensor(12.4978), tensor(12.4420), tensor(12.8523),
tensor(12.4951), tensor(13.2813), tensor(11.3564), tensor(10.8658),
tensor(10.3209), tensor(11.1951), tensor(13.6446), tensor(13.8049),
tensor(11.5131), tensor(13.2276), tensor(11.8097), tensor(12.4251),
tensor(11.2590), tensor(13.4371), tensor(11.1026), tensor(13.9983),
tensor(12.0530), tensor(11.9597), tensor(13.2505), tensor(12.0109),
tensor(12.7052), tensor(12.3447), tensor(12.9062), tensor(11.5565),
tensor(13.0455), tensor(13.8612), tensor(11.5418), tensor(12.1695),
tensor(15.5432), tensor(13.6196), tensor(14.0194), tensor(14.1461),
tensor(12.9984), tensor(12.7263)]
[tensor(12.9320), tensor(14.7135), tensor(12.9471), tensor(12.9007),
tensor(13.8141), tensor(13.6075), tensor(15.3746), tensor(12.4387),
tensor(13.7293), tensor(14.0638), tensor(15.0184), tensor(13.5269),
tensor(11.9779), tensor(13.0755), tensor(14.6374), tensor(14.0910),
tensor(12.7766), tensor(12.2828), tensor(11.2608), tensor(12.0142),
tensor(10.7950), tensor(12.0717), tensor(13.8633), tensor(13.1763),
tensor(12.4204), tensor(11.9248), tensor(11.9110), tensor(13.0238),
tensor(10.9707), tensor(13.8081), tensor(12.8911), tensor(13.3792),
tensor(12.9016), tensor(12.0550), tensor(13.2585), tensor(12.2195),
tensor(11.1958), tensor(13.3586), tensor(13.8235), tensor(13.0013),
tensor(13.6033), tensor(15.0100), tensor(13.2300), tensor(13.9777),
tensor(15.7949), tensor(14.4760), tensor(14.9418), tensor(14.3164),
```

```
tensor(13.4957), tensor(13.9747)]
[tensor(12.7538), tensor(14.2687), tensor(13.5382), tensor(14.6663),
tensor(15.1010), tensor(11.2954), tensor(14.3877), tensor(12.5183),
tensor(12.7835), tensor(14.6843), tensor(13.3618), tensor(13.5250),
tensor(11.7098), tensor(12.9470), tensor(13.4731), tensor(12.9581),
tensor(12.7146), tensor(12.4196), tensor(11.3074), tensor(12.3418),
tensor(11.5761), tensor(11.7237), tensor(13.5919), tensor(13.6483),
tensor(12.8646), tensor(12.1553), tensor(11.5839), tensor(11.9899),
tensor(12.2527), tensor(12.9184), tensor(11.1540), tensor(14.8227),
tensor(13.2442), tensor(10.9896), tensor(13.2738), tensor(11.4964),
tensor(13.8463), tensor(12.8900), tensor(12.9982), tensor(13.3745),
tensor(12.3814), tensor(13.3331), tensor(13.4188), tensor(13.1419),
tensor(15.7323), tensor(13.6795), tensor(14.0379), tensor(14.8788),
tensor(14.3342), tensor(14.3393)]
[tensor(10.5873), tensor(14.8616), tensor(10.1789), tensor(12.5382),
tensor(14.0369), tensor(12.2456), tensor(13.6335), tensor(12.6144),
tensor(12.2867), tensor(13.0997), tensor(13.6073), tensor(13.6233),
tensor(9.8248), tensor(12.3523), tensor(11.5129), tensor(13.3878),
tensor(10.6530), tensor(12.0856), tensor(10.8566), tensor(11.1484),
tensor(10.5420), tensor(11.4279), tensor(13.2282), tensor(12.9694),
tensor(12.0983), tensor(13.3568), tensor(11.5523), tensor(11.2319),
tensor(11.7279), tensor(12.8799), tensor(12.0596), tensor(13.0571),
tensor(12.3289), tensor(10.3450), tensor(12.0832), tensor(11.6560),
tensor(11.8499), tensor(13.9048), tensor(13.7595), tensor(11.6470),
tensor(11.7531), tensor(12.7510), tensor(12.3112), tensor(11.9525),
tensor(13.6933), tensor(12.4745), tensor(14.7903), tensor(13.6132),
tensor(13.4832), tensor(12.6887)]
[tensor(11.4943), tensor(12.6816), tensor(11.0040), tensor(11.2446),
tensor(12.6863), tensor(10.0940), tensor(11.5822), tensor(10.5238),
tensor(10.2518), tensor(11.4168), tensor(13.0265), tensor(11.9096),
tensor(9.5082), tensor(11.8317), tensor(11.6857), tensor(11.3818),
tensor(11.8708), tensor(12.3011), tensor(10.0414), tensor(11.4943),
tensor(9.7049), tensor(9.9716), tensor(11.6493), tensor(12.1660),
tensor(10.6182), tensor(11.5693), tensor(11.0168), tensor(10.1071),
tensor(10.6720), tensor(11.5656), tensor(11.1207), tensor(11.7522),
tensor(10.7236), tensor(11.0497), tensor(11.4792), tensor(11.2495),
tensor(11.7509), tensor(11.9490), tensor(11.8524), tensor(10.5358),
tensor(12.0269), tensor(12.3751), tensor(11.8654), tensor(11.5032),
tensor(14.3368), tensor(12.5701), tensor(12.8456), tensor(11.6691),
tensor(11.5759), tensor(12.7645)]
[tensor(11.4331), tensor(12.4960), tensor(11.4511), tensor(11.6362),
tensor(12.5132), tensor(11.7755), tensor(13.8500), tensor(10.8697),
tensor(11.9695), tensor(13.1794), tensor(11.4755), tensor(11.6953),
tensor(9.3868), tensor(12.0803), tensor(12.2466), tensor(12.6077),
tensor(10.5009), tensor(11.4396), tensor(10.2437), tensor(10.9858),
tensor(10.3482), tensor(10.2843), tensor(12.3273), tensor(13.1778),
tensor(10.9077), tensor(10.3835), tensor(10.0650), tensor(11.5485),
tensor(9.6670), tensor(11.3703), tensor(11.3685), tensor(12.9268),
```

```
tensor(11.8878), tensor(10.1946), tensor(11.8949), tensor(9.2122),
tensor(10.5820), tensor(11.6955), tensor(10.2212), tensor(11.0660),
tensor(11.1298), tensor(11.8801), tensor(11.3853), tensor(11.3034),
tensor(13.4967), tensor(11.7862), tensor(13.0921), tensor(12.2410),
tensor(12.2687), tensor(11.7344)]
[tensor(11.0009), tensor(12.9907), tensor(10.5530), tensor(11.9193),
tensor(12.7380), tensor(12.0846), tensor(11.6558), tensor(11.8431),
tensor(11.3498), tensor(11.8511), tensor(11.9216), tensor(11.7532),
tensor(9.4554), tensor(11.9481), tensor(11.4574), tensor(12.9028),
tensor(10.6565), tensor(11.5552), tensor(10.3051), tensor(10.7679),
tensor(9.6278), tensor(11.1069), tensor(13.0347), tensor(11.9320),
tensor(10.8031), tensor(12.1060), tensor(10.5982), tensor(11.1296),
tensor(9.9987), tensor(11.9126), tensor(11.1686), tensor(13.3370),
tensor(10.4192), tensor(9.7175), tensor(11.1581), tensor(12.3015),
tensor(10.4577), tensor(11.3380), tensor(11.5297), tensor(11.6995),
tensor(11.7428), tensor(12.4394), tensor(11.9431), tensor(10.7620),
tensor(14.1994), tensor(12.1962), tensor(11.9905), tensor(12.7159),
tensor(13.6119), tensor(12.0409)]
[tensor(12.0130), tensor(13.7792), tensor(11.2540), tensor(12.9175),
tensor(12.8116), tensor(11.2340), tensor(13.8639), tensor(12.0782),
tensor(12.0024), tensor(12.7910), tensor(11.8896), tensor(12.3506),
tensor(9.3271), tensor(11.6903), tensor(11.4243), tensor(13.4916),
tensor(9.6762), tensor(12.4582), tensor(9.9147), tensor(10.1408),
tensor(10.5558), tensor(11.0545), tensor(12.9812), tensor(12.8789),
tensor(11.2225), tensor(12.4590), tensor(11.2038), tensor(11.5851),
tensor(9.9376), tensor(12.7846), tensor(12.1385), tensor(12.4495),
tensor(11.6978), tensor(10.9953), tensor(12.6513), tensor(11.9304),
tensor(11.4439), tensor(12.9846), tensor(13.0439), tensor(12.2474),
tensor(12.0946), tensor(13.1842), tensor(12.9973), tensor(11.5983),
tensor(14.4526), tensor(13.6512), tensor(14.2186), tensor(12.3663),
tensor(13.2495), tensor(12.6724)]
[tensor(11.5232), tensor(14.3934), tensor(11.7116), tensor(12.2139),
tensor(13.6257), tensor(10.8681), tensor(14.3499), tensor(11.8819),
tensor(10.8871), tensor(12.2118), tensor(12.4849), tensor(13.2089),
tensor(9.0314), tensor(12.4684), tensor(11.2174), tensor(12.3751),
tensor(9.4696), tensor(13.5513), tensor(10.4901), tensor(10.3000),
tensor(10.1092), tensor(11.1821), tensor(13.3753), tensor(12.9066),
tensor(11.5319), tensor(13.0957), tensor(11.9288), tensor(11.4898),
tensor(10.7943), tensor(14.3409), tensor(12.9397), tensor(13.2745),
tensor(12.4036), tensor(10.4453), tensor(11.9243), tensor(12.1250),
tensor(12.1689), tensor(14.3776), tensor(12.1778), tensor(12.0235),
tensor(12.2570), tensor(13.4388), tensor(12.0193), tensor(11.3547),
tensor(13.8642), tensor(13.8598), tensor(14.6236), tensor(12.8495),
tensor(12.1354), tensor(12.8900)]
[tensor(8.3840), tensor(11.4471), tensor(8.6807), tensor(10.6561),
tensor(11.5477), tensor(9.4955), tensor(11.0328), tensor(8.6138),
tensor(10.0943), tensor(10.5198), tensor(11.0287), tensor(10.9957),
tensor(8.5626), tensor(9.6466), tensor(9.5734), tensor(10.6972), tensor(8.3476),
```

```
tensor(9.9268), tensor(7.9718), tensor(9.6145), tensor(7.9526), tensor(9.5671),
tensor(11.1828), tensor(10.0018), tensor(10.4802), tensor(9.8585),
tensor(9.2222), tensor(8.7856), tensor(8.3847), tensor(10.9377),
tensor(10.3751), tensor(10.5286), tensor(10.6072), tensor(7.5950),
tensor(9.2382), tensor(9.6841), tensor(8.6189), tensor(11.3693),
tensor(10.4126), tensor(9.9787), tensor(10.3389), tensor(10.8140),
tensor(10.4157), tensor(9.8586), tensor(11.5314), tensor(10.8669),
tensor(11.8531), tensor(10.2597), tensor(11.5348), tensor(10.1227)]
[tensor(12.9706), tensor(13.9664), tensor(12.9815), tensor(13.3664),
tensor(13.7869), tensor(12.0493), tensor(15.2362), tensor(11.9347),
tensor(11.5408), tensor(15.1123), tensor(14.1297), tensor(13.4398),
tensor(10.6284), tensor(13.3015), tensor(12.9291), tensor(13.5238),
tensor(12.5869), tensor(12.7902), tensor(10.7848), tensor(12.0190),
tensor(11.2131), tensor(11.8322), tensor(14.5163), tensor(14.3327),
tensor(12.5881), tensor(13.2679), tensor(12.9607), tensor(12.1383),
tensor(11.1109), tensor(13.3492), tensor(13.5598), tensor(14.7109),
tensor(13.7574), tensor(10.7917), tensor(13.1699), tensor(11.7874),
tensor(13.5148), tensor(15.5427), tensor(14.0757), tensor(12.5570),
tensor(14.0907), tensor(13.8765), tensor(13.4533), tensor(14.2028),
tensor(16.3075), tensor(14.1482), tensor(15.8610), tensor(13.9803),
tensor(13.6769), tensor(14.9624)]
[tensor(9.8516), tensor(12.0166), tensor(10.4726), tensor(11.2155),
tensor(11.9198), tensor(10.5326), tensor(11.7008), tensor(9.9065),
tensor(10.0409), tensor(10.7976), tensor(11.7005), tensor(11.1055),
tensor(8.5562), tensor(10.1955), tensor(11.6490), tensor(11.0028),
tensor(10.0554), tensor(10.6698), tensor(9.6414), tensor(9.9858),
tensor(8.7535), tensor(9.1302), tensor(11.6468), tensor(11.4180),
tensor(10.3609), tensor(10.9384), tensor(9.6470), tensor(10.0843),
tensor(9.8982), tensor(11.3679), tensor(10.8562), tensor(12.0352),
tensor(9.1494), tensor(8.7718), tensor(9.4457), tensor(9.5012), tensor(9.8847),
tensor(11.3450), tensor(10.7951), tensor(10.0080), tensor(11.0842),
tensor(10.8194), tensor(11.4297), tensor(10.8755), tensor(13.8126),
tensor(11.1946), tensor(11.7689), tensor(10.8242), tensor(12.0975),
tensor(10.3186)]
[tensor(11.4900), tensor(13.3383), tensor(12.0114), tensor(13.0055),
tensor(14.5209), tensor(12.5980), tensor(13.8384), tensor(12.7795),
tensor(12.1511), tensor(14.5440), tensor(13.4195), tensor(12.5748),
tensor(9.9576), tensor(13.5928), tensor(12.0500), tensor(13.0311),
tensor(12.9908), tensor(14.1001), tensor(11.3636), tensor(11.5712),
tensor(9.8697), tensor(12.3139), tensor(13.7644), tensor(12.9796),
tensor(12.2044), tensor(13.3434), tensor(12.5024), tensor(12.0886),
tensor(10.9863), tensor(13.1582), tensor(12.0133), tensor(14.1478),
tensor(13.3739), tensor(11.6993), tensor(13.6985), tensor(11.7956),
tensor(12.5813), tensor(13.9157), tensor(12.1977), tensor(11.7247),
tensor(13.6997), tensor(13.2561), tensor(12.4784), tensor(12.5301),
tensor(15.2692), tensor(12.5302), tensor(13.8184), tensor(13.7700),
tensor(13.9524), tensor(13.9668)]
[tensor(12.3705), tensor(13.8737), tensor(11.3303), tensor(12.8741),
```

```
tensor(13.3904), tensor(11.0247), tensor(13.4294), tensor(12.3521),
tensor(11.3500), tensor(12.7398), tensor(12.7146), tensor(12.7580),
tensor(10.4003), tensor(11.0628), tensor(12.2247), tensor(11.7364),
tensor(11.1561), tensor(12.5041), tensor(10.5683), tensor(11.1413),
tensor(10.1037), tensor(12.3775), tensor(12.1707), tensor(12.3824),
tensor(11.4491), tensor(12.6132), tensor(11.8865), tensor(11.3806),
tensor(10.9553), tensor(12.2347), tensor(12.7475), tensor(12.3034),
tensor(12.1429), tensor(10.3022), tensor(12.7376), tensor(13.0325),
tensor(12.0491), tensor(13.2276), tensor(12.9758), tensor(12.2912),
tensor(12.3415), tensor(13.7610), tensor(12.3855), tensor(11.8909),
tensor(13.7526), tensor(13.6895), tensor(12.8992), tensor(12.9898),
tensor(12.9510), tensor(13.6602)]
[tensor(11.1777), tensor(13.5304), tensor(11.4284), tensor(12.7112),
tensor(13.6863), tensor(11.6202), tensor(13.6297), tensor(10.4283),
tensor(11.8647), tensor(13.2189), tensor(13.2407), tensor(12.1447),
tensor(10.4512), tensor(11.6506), tensor(13.6349), tensor(12.1780),
tensor(12.0719), tensor(12.2570), tensor(10.3703), tensor(11.9215),
tensor(9.6847), tensor(10.9714), tensor(13.2904), tensor(12.7287),
tensor(11.9120), tensor(12.9897), tensor(12.2189), tensor(10.9588),
tensor(11.6140), tensor(12.6554), tensor(11.0559), tensor(13.4086),
tensor(11.8524), tensor(10.1681), tensor(13.1692), tensor(11.8289),
tensor(11.9390), tensor(13.4630), tensor(12.6662), tensor(11.8721),
tensor(12.9231), tensor(12.8624), tensor(12.7555), tensor(13.8069),
tensor(14.3656), tensor(12.8363), tensor(13.4208), tensor(13.3080),
tensor(13.1741), tensor(13.4695)]
[tensor(11.8108), tensor(13.6864), tensor(12.4857), tensor(13.2824),
tensor(13.0921), tensor(11.2813), tensor(14.3513), tensor(12.4262),
tensor(12.3733), tensor(14.2333), tensor(12.9277), tensor(13.0354),
tensor(9.7755), tensor(12.5006), tensor(13.4250), tensor(13.3423),
tensor(12.0111), tensor(12.9412), tensor(9.9755), tensor(11.7676),
tensor(9.7135), tensor(11.4930), tensor(13.1177), tensor(13.4747),
tensor(11.7269), tensor(11.3181), tensor(11.6821), tensor(12.1231),
tensor(11.5138), tensor(11.2448), tensor(12.3374), tensor(13.1049),
tensor(12.4739), tensor(11.3957), tensor(13.1294), tensor(10.8061),
tensor(11.8644), tensor(14.1759), tensor(13.3848), tensor(12.0856),
tensor(12.5744), tensor(12.6650), tensor(13.0756), tensor(13.4206),
tensor(14.8823), tensor(12.2819), tensor(14.2157), tensor(13.1463),
tensor(14.2144), tensor(14.0951)]
[tensor(10.0807), tensor(11.5971), tensor(10.0969), tensor(11.2157),
tensor(12.4341), tensor(10.1590), tensor(12.4490), tensor(10.2652),
tensor(11.3098), tensor(10.8432), tensor(12.0392), tensor(11.6950),
tensor(10.2501), tensor(10.2158), tensor(11.4546), tensor(10.9612),
tensor(9.7685), tensor(9.8516), tensor(9.2571), tensor(10.0621), tensor(9.0086),
tensor(9.8121), tensor(11.1126), tensor(11.7789), tensor(10.5835),
tensor(10.8974), tensor(10.2505), tensor(10.2170), tensor(9.8023),
tensor(11.6016), tensor(10.7037), tensor(12.3009), tensor(9.5516),
tensor(9.0686), tensor(10.7350), tensor(10.4849), tensor(9.8322),
tensor(11.1828), tensor(11.2464), tensor(11.3323), tensor(11.9501),
```

```
tensor(12.6881), tensor(11.7557), tensor(10.6659), tensor(13.5441),
tensor(12.5132), tensor(12.2205), tensor(11.8627), tensor(12.1189),
tensor(12.0195)]
[tensor(11.2992), tensor(11.6511), tensor(10.1417), tensor(11.2204),
tensor(11.4901), tensor(10.1870), tensor(12.1227), tensor(11.0624),
tensor(10.5043), tensor(11.3744), tensor(11.7608), tensor(11.1557),
tensor(8.9230), tensor(11.3841), tensor(10.2127), tensor(10.2597),
tensor(9.2140), tensor(10.0503), tensor(8.5581), tensor(9.2965), tensor(8.9863),
tensor(9.6153), tensor(11.0784), tensor(11.0448), tensor(10.0893),
tensor(10.3670), tensor(8.8066), tensor(10.7492), tensor(8.8475),
tensor(11.6209), tensor(10.9360), tensor(10.7426), tensor(10.0061),
tensor(9.0427), tensor(10.0251), tensor(9.7078), tensor(10.0448),
tensor(9.9308), tensor(10.5068), tensor(10.2804), tensor(9.7113),
tensor(11.1768), tensor(9.4375), tensor(9.2873), tensor(12.6041),
tensor(11.8575), tensor(11.1015), tensor(11.1574), tensor(11.5190),
tensor(10.4420)]
[tensor(10.6585), tensor(12.8595), tensor(10.6400), tensor(11.4050),
tensor(12.9642), tensor(11.8516), tensor(12.3431), tensor(10.6708),
tensor(11.6586), tensor(13.2395), tensor(11.6660), tensor(13.3173),
tensor(9.5052), tensor(10.6913), tensor(13.0943), tensor(12.6622),
tensor(11.3263), tensor(12.0610), tensor(9.6619), tensor(11.8705),
tensor(10.0028), tensor(11.3900), tensor(12.9085), tensor(13.3646),
tensor(11.0609), tensor(11.5875), tensor(11.2426), tensor(10.3664),
tensor(10.9245), tensor(10.6132), tensor(11.2847), tensor(11.7429),
tensor(10.9263), tensor(10.6911), tensor(12.5972), tensor(11.2800),
tensor(10.6163), tensor(13.2059), tensor(11.4433), tensor(11.2696),
tensor(11.8612), tensor(11.9896), tensor(13.0506), tensor(12.6568),
tensor(13.6554), tensor(12.2659), tensor(13.7501), tensor(12.4314),
tensor(12.9994), tensor(12.9477)]
[tensor(12.0947), tensor(12.6925), tensor(12.5896), tensor(13.3979),
tensor(13.7134), tensor(11.2914), tensor(15.5797), tensor(12.4909),
tensor(12.0114), tensor(13.5071), tensor(12.6631), tensor(12.1035),
tensor(10.8592), tensor(12.4495), tensor(12.7039), tensor(12.1347),
tensor(11.7392), tensor(11.8556), tensor(9.8664), tensor(11.4674),
tensor(9.3991), tensor(11.0338), tensor(12.2009), tensor(12.0794),
tensor(11.6085), tensor(11.8293), tensor(10.8528), tensor(12.0315),
tensor(11.0827), tensor(12.4878), tensor(11.9268), tensor(13.3273),
tensor(11.8329), tensor(10.5637), tensor(12.5263), tensor(10.8449),
tensor(12.5601), tensor(12.6200), tensor(13.8131), tensor(11.7885),
tensor(12.3210), tensor(13.6402), tensor(12.2951), tensor(13.0658),
tensor(15.0788), tensor(13.1511), tensor(13.2490), tensor(13.1318),
tensor(13.4285), tensor(13.0473)]
[tensor(12.4113), tensor(13.4297), tensor(12.1333), tensor(13.4561),
tensor(14.1326), tensor(10.3032), tensor(13.3940), tensor(11.6213),
tensor(11.9569), tensor(12.1800), tensor(13.7458), tensor(13.2769),
tensor(10.0490), tensor(12.4939), tensor(12.3337), tensor(12.7406),
tensor(10.4245), tensor(12.5762), tensor(10.1410), tensor(10.8962),
tensor(10.1413), tensor(11.3329), tensor(13.1265), tensor(12.0896),
```

```
tensor(12.3871), tensor(12.5774), tensor(12.2948), tensor(11.4925),
tensor(11.4648), tensor(12.9683), tensor(12.2834), tensor(13.8231),
tensor(12.6979), tensor(9.4462), tensor(11.6883), tensor(12.3550),
tensor(12.2293), tensor(14.4477), tensor(12.7176), tensor(12.2374),
tensor(13.5360), tensor(14.1248), tensor(13.3876), tensor(11.6528),
tensor(14.2162), tensor(13.9845), tensor(13.4782), tensor(12.2958),
tensor(13.0032), tensor(12.9483)]
[tensor(11.0019), tensor(13.3167), tensor(10.9612), tensor(12.1533),
tensor(13.3479), tensor(11.1161), tensor(13.0413), tensor(11.4579),
tensor(11.9908), tensor(13.1878), tensor(12.5091), tensor(11.5363),
tensor(9.6849), tensor(11.3041), tensor(11.9534), tensor(12.4904),
tensor(10.5484), tensor(10.8399), tensor(10.1825), tensor(11.9678),
tensor(10.7102), tensor(10.6519), tensor(12.2482), tensor(12.8406),
tensor(11.0535), tensor(12.0495), tensor(11.1464), tensor(11.3627),
tensor(9.9647), tensor(11.2762), tensor(11.3220), tensor(13.2600),
tensor(10.5788), tensor(10.1697), tensor(13.3771), tensor(10.8666),
tensor(10.5433), tensor(12.8673), tensor(12.8656), tensor(11.7902),
tensor(12.5319), tensor(11.8379), tensor(12.1826), tensor(12.5890),
tensor(14.8466), tensor(12.6878), tensor(13.5941), tensor(12.0467),
tensor(12.5150), tensor(12.4886)]
[tensor(11.4021), tensor(13.2686), tensor(12.1852), tensor(12.5800),
tensor(12.6916), tensor(10.7870), tensor(14.1423), tensor(11.8194),
tensor(12.8586), tensor(14.1148), tensor(12.3167), tensor(12.4473),
tensor(10.5938), tensor(11.7606), tensor(12.4205), tensor(12.4302),
tensor(11.7212), tensor(12.4048), tensor(9.9322), tensor(11.7060),
tensor(10.4893), tensor(11.7820), tensor(12.2448), tensor(13.8994),
tensor(11.9418), tensor(12.3336), tensor(12.4187), tensor(11.7124),
tensor(10.9815), tensor(13.2918), tensor(11.3050), tensor(13.0478),
tensor(11.7263), tensor(10.7549), tensor(13.0130), tensor(11.4243),
tensor(12.1086), tensor(13.3002), tensor(12.7183), tensor(13.1745),
tensor(11.7553), tensor(12.5422), tensor(12.0527), tensor(12.7604),
tensor(14.3863), tensor(13.5618), tensor(13.3743), tensor(13.2939),
tensor(13.6531), tensor(13.5046)]
[tensor(13.2706), tensor(14.4944), tensor(13.7364), tensor(14.2878),
tensor(15.3139), tensor(11.9021), tensor(14.6085), tensor(13.0652),
tensor(13.8399), tensor(15.4166), tensor(15.0321), tensor(13.8111),
tensor(12.0729), tensor(13.5230), tensor(13.7868), tensor(13.6799),
tensor(12.8965), tensor(14.6951), tensor(11.1732), tensor(13.3474),
tensor(10.7007), tensor(12.6954), tensor(14.2760), tensor(14.5328),
tensor(14.4016), tensor(13.0402), tensor(12.8101), tensor(13.0213),
tensor(12.2385), tensor(14.0980), tensor(12.9096), tensor(14.5914),
tensor(13.3563), tensor(11.7344), tensor(14.8347), tensor(13.7666),
tensor(12.2059), tensor(14.2942), tensor(14.0298), tensor(13.7267),
tensor(14.8225), tensor(15.5127), tensor(14.3658), tensor(14.5053),
tensor(16.3366), tensor(14.4531), tensor(15.3266), tensor(13.9704),
tensor(14.6661), tensor(15.0477)]
[tensor(11.1888), tensor(12.5203), tensor(12.3655), tensor(11.9999),
tensor(13.0969), tensor(10.2802), tensor(13.3293), tensor(11.3919),
```

```
tensor(11.8184), tensor(13.1779), tensor(12.7954), tensor(13.3843),
tensor(10.1173), tensor(12.1606), tensor(12.3686), tensor(11.9779),
tensor(11.0963), tensor(11.6477), tensor(9.0177), tensor(11.7020),
tensor(10.3975), tensor(11.8752), tensor(12.0605), tensor(13.1891),
tensor(11.5032), tensor(10.9164), tensor(12.0953), tensor(10.8533),
tensor(10.3262), tensor(12.5995), tensor(12.4099), tensor(13.1732),
tensor(12.4827), tensor(10.4617), tensor(11.1428), tensor(10.8679),
tensor(10.6585), tensor(13.7270), tensor(11.8394), tensor(11.4909),
tensor(11.6046), tensor(13.2407), tensor(11.9958), tensor(12.4114),
tensor(13.6879), tensor(12.3690), tensor(13.3309), tensor(12.5361),
tensor(12.6346), tensor(12.8111)]
[tensor(9.9229), tensor(9.7951), tensor(9.1799), tensor(9.8453),
tensor(10.5598), tensor(9.3193), tensor(10.9306), tensor(9.7169),
tensor(8.7330), tensor(9.6639), tensor(10.5445), tensor(10.7986),
tensor(7.0220), tensor(9.9302), tensor(9.3107), tensor(10.1673), tensor(8.3368),
tensor(9.0818), tensor(7.8527), tensor(8.0387), tensor(8.4500), tensor(8.7667),
tensor(10.4224), tensor(10.6582), tensor(8.8065), tensor(9.3539),
tensor(8.7632), tensor(9.1165), tensor(8.4562), tensor(10.0723), tensor(9.6847),
tensor(10.2331), tensor(8.7111), tensor(7.6421), tensor(8.6201), tensor(8.0893),
tensor(9.6017), tensor(9.9647), tensor(9.6040), tensor(8.7961), tensor(8.6697),
tensor(9.6569), tensor(9.6382), tensor(9.1878), tensor(11.4553),
tensor(10.3689), tensor(11.3195), tensor(10.6995), tensor(10.7776),
tensor(9.3198)]
[tensor(11.9656), tensor(14.7244), tensor(12.2227), tensor(14.0725),
tensor(15.0550), tensor(10.8757), tensor(14.5094), tensor(12.9716),
tensor(13.0266), tensor(13.4723), tensor(13.6561), tensor(14.2042),
tensor(10.8527), tensor(12.8297), tensor(13.2159), tensor(12.9673),
tensor(11.5822), tensor(13.5758), tensor(10.9694), tensor(12.4741),
tensor(11.0051), tensor(12.2453), tensor(13.6794), tensor(13.7069),
tensor(12.7868), tensor(13.1406), tensor(12.8703), tensor(12.5747),
tensor(11.7582), tensor(13.8271), tensor(13.2938), tensor(14.0576),
tensor(12.9017), tensor(12.2039), tensor(14.3253), tensor(12.7536),
tensor(12.4968), tensor(14.2754), tensor(14.0746), tensor(12.7709),
tensor(13.1427), tensor(14.0955), tensor(13.6759), tensor(13.3960),
tensor(15.7548), tensor(13.9619), tensor(14.1625), tensor(13.5871),
tensor(13.9384), tensor(13.6325)]
[tensor(11.9022), tensor(13.0227), tensor(12.1689), tensor(12.9343),
tensor(13.6338), tensor(10.3572), tensor(14.7292), tensor(11.7277),
tensor(11.6238), tensor(13.4264), tensor(12.9169), tensor(11.8300),
tensor(9.8548), tensor(11.8076), tensor(11.7491), tensor(11.8388),
tensor(10.5027), tensor(12.6551), tensor(10.4379), tensor(12.3580),
tensor(10.3764), tensor(10.5975), tensor(12.7143), tensor(14.6855),
tensor(12.1026), tensor(12.3933), tensor(11.4365), tensor(11.7427),
tensor(10.6364), tensor(12.3811), tensor(12.1138), tensor(12.9152),
tensor(11.5793), tensor(10.2506), tensor(13.7460), tensor(10.7970),
tensor(11.4490), tensor(12.9752), tensor(12.0413), tensor(12.5353),
tensor(12.0856), tensor(13.1658), tensor(12.1488), tensor(12.2286),
tensor(15.1466), tensor(14.7989), tensor(14.7817), tensor(12.3069),
```

```
tensor(12.5724), tensor(12.6975)]
[tensor(11.2128), tensor(14.0621), tensor(10.7778), tensor(11.4463),
tensor(12.9689), tensor(10.7610), tensor(12.7190), tensor(11.6077),
tensor(11.6090), tensor(12.2291), tensor(14.1942), tensor(12.5929),
tensor(9.8683), tensor(11.2267), tensor(12.4174), tensor(11.7410),
tensor(12.1192), tensor(11.9034), tensor(11.2562), tensor(11.5753),
tensor(10.5307), tensor(10.5671), tensor(12.2466), tensor(13.1988),
tensor(11.9024), tensor(13.2099), tensor(12.0386), tensor(11.5949),
tensor(11.9401), tensor(12.7636), tensor(11.2738), tensor(13.3175),
tensor(10.9379), tensor(10.2586), tensor(12.0311), tensor(11.5236),
tensor(12.5931), tensor(11.8745), tensor(12.2652), tensor(11.0415),
tensor(11.8996), tensor(12.8283), tensor(11.3689), tensor(11.9404),
tensor(14.2472), tensor(12.2404), tensor(13.5835), tensor(12.6231),
tensor(12.3040), tensor(12.8406)]
2.27 s ± 603 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

[78]:
```
%%timeit
torch.matmul(tensor, tensor)
```

```
2.88 µs ± 117 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)
```

One of the most common errors in deep learning (shape errors) Because much of deep learning is multiplying and performing operations on matrices and matrices have a strict rule about what shapes and sizes can be combined, one of the most common errors you'll run into in deep learning is shape mismatches.

[79]:
```
# Shapes need to be in the right way
tensor_A = torch.tensor([[1, 2],
                         [3, 4],
                         [5, 6]], dtype=torch.float32)

tensor_B = torch.tensor([[7, 10],
                         [8, 11],
                         [9, 12]], dtype=torch.float32)

torch.matmul(tensor_A, tensor_B) # (this will error)
```

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
/tmp/ipython-input-3539350370.py in <cell line: 0>()
      8                          [9, 12]], dtype=torch.float32)
      9
---> 10 torch.matmul(tensor_A, tensor_B) # (this will error)

RuntimeError: mat1 and mat2 shapes cannot be multiplied (3x2 and 3x2)
```

We can make matrix multiplication work between `tensor_A` and `tensor_B` by making their inner

dimensions match.

One of the ways to do this is with a **transpose** (switch the dimensions of a given tensor).

You can perform transposes in PyTorch using either: * `torch.transpose(input, dim0, dim1)` - where `input` is the desired tensor to transpose and `dim0` and `dim1` are the dimensions to be swapped. * `tensor.T` - where `tensor` is the desired tensor to transpose.

Let's try the latter.

[80]:
```
# View tensor_A and tensor_B
print(tensor_A)
print(tensor_B)
```

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
tensor([[ 7., 10.],
        [ 8., 11.],
        [ 9., 12.]])
```

[81]:
```
# View tensor_A and tensor_B.T
print(tensor_A)
print(tensor_B.T)
```

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
tensor([[ 7.,  8.,  9.],
        [10., 11., 12.]])
```

[82]:
```
# The operation works when tensor_B is transposed
print(f"Original shapes: tensor_A = {tensor_A.shape}, tensor_B = {tensor_B.
 ↪shape}\n")
print(f"New shapes: tensor_A = {tensor_A.shape} (same as above), tensor_B.T =␣
 ↪{tensor_B.T.shape}\n")
print(f"Multiplying: {tensor_A.shape} * {tensor_B.T.shape} <- inner dimensions␣
 ↪match\n")
print("Output:\n")
output = torch.matmul(tensor_A, tensor_B.T)
print(output)
print(f"\nOutput shape: {output.shape}")
```

```
Original shapes: tensor_A = torch.Size([3, 2]), tensor_B = torch.Size([3, 2])

New shapes: tensor_A = torch.Size([3, 2]) (same as above), tensor_B.T =
torch.Size([2, 3])

Multiplying: torch.Size([3, 2]) * torch.Size([2, 3]) <- inner dimensions match
```

```
Output:

tensor([[ 27.,  30.,  33.],
        [ 61.,  68.,  75.],
        [ 95., 106., 117.]])

Output shape: torch.Size([3, 3])
```

You can also use `torch.mm()` which is a short for `torch.matmul()`.

[83]:
```
# torch.mm is a shortcut for matmul
torch.mm(tensor_A, tensor_B.T)
```

[83]:
```
tensor([[ 27.,  30.,  33.],
        [ 61.,  68.,  75.],
        [ 95., 106., 117.]])
```

Without the transpose, the rules of matrix mulitplication aren't fulfilled and we get an error like above.

How about a visual?

visual demo of matrix multiplication

You can create your own matrix multiplication visuals like this at http://matrixmultiplication.xyz/.

> **Note:** A matrix multiplication like this is also referred to as the **dot product** of two matrices.

Neural networks are full of matrix multiplications and dot products.

The `torch.nn.Linear()` module (we'll see this in action later on), also known as a feed-forward layer or fully connected layer, implements a matrix multiplication between an input `x` and a weights matrix `A`.

$$y = x \cdot A^T + b$$

Where: * `x` is the input to the layer (deep learning is a stack of layers like `torch.nn.Linear()` and others on top of each other). * `A` is the weights matrix created by the layer, this starts out as random numbers that get adjusted as a neural network learns to better represent patterns in the data (notice the "`T`", that's because the weights matrix gets transposed). * **Note:** You might also often see `W` or another letter like `X` used to showcase the weights matrix. * `b` is the bias term used to slightly offset the weights and inputs. * `y` is the output (a manipulation of the input in the hopes to discover patterns in it).

This is a linear function (you may have seen something like $y = mx+b$ in high school or elsewhere), and can be used to draw a straight line!

Let's play around with a linear layer.

Try changing the values of `in_features` and `out_features` below and see what happens.

Do you notice anything to do with the shapes?

[85]:
```
# Since the linear layer starts with a random weights matrix, let's make it␣
 ↪reproducible (more on this later)
torch.manual_seed(42)
# This uses matrix multiplication
linear = torch.nn.Linear(in_features=2, # in_features = matches inner dimension␣
 ↪of input
                         out_features=6) # out_features = describes outer value
x = tensor_A
output = linear(x)
print(f"Input shape: {x.shape}\n")
print(f"Output:\n{output}\n\nOutput shape: {output.shape}")
```

```
Input shape: torch.Size([3, 2])

Output:
tensor([[2.2368, 1.2292, 0.4714, 0.3864, 0.1309, 0.9838],
        [4.4919, 2.1970, 0.4469, 0.5285, 0.3401, 2.4777],
        [6.7469, 3.1648, 0.4224, 0.6705, 0.5493, 3.9716]],
       grad_fn=<AddmmBackward0>)

Output shape: torch.Size([3, 6])
```

> **Question:** What happens if you change `in_features` from 2 to 3 above? Does it error? How could you change the shape of the input (`x`) to accomodate to the error? Hint: what did we have to do to `tensor_B` above?

Remember, matrix multiplication is all you need.

matrix multiplication is all you need

When you start digging into neural network layers and building your own, you'll find matrix multiplications everywhere. ***Source:*** *https://marksaroufim.substack.com/p/working-class-deep-learner*

Finding the min, max, mean, sum, etc (aggregation) Now we've seen a few ways to manipulate tensors, let's run through a few ways to aggregate them (go from more values to less values).

First we'll create a tensor and then find the max, min, mean and sum of it.

[86]:
```
# Create a tensor
x = torch.arange(0, 100, 10)
x
```

[86]:
```
tensor([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90])
```

Now let's perform some aggregation.

[87]:
```
print(f"Minimum: {x.min()}")
print(f"Maximum: {x.max()}")
# print(f"Mean: {x.mean()}") # this will error
```

```
print(f"Mean: {x.type(torch.float32).mean()}") # won't work without float␣
↪datatype
print(f"Sum: {x.sum()}")
```

```
Minimum: 0
Maximum: 90
Mean: 45.0
Sum: 450
```

> **Note:** You may find some methods such as `torch.mean()` require tensors to be in `torch.float32` (the most common) or another specific datatype, otherwise the operation will fail.

You can also do the same as above with `torch` methods.

[88]:
```
torch.max(x), torch.min(x), torch.mean(x.type(torch.float32)), torch.sum(x)
```

[88]:
```
(tensor(90), tensor(0), tensor(45.), tensor(450))
```

Positional min/max You can also find the index of a tensor where the max or minimum occurs with `torch.argmax()` and `torch.argmin()` respectively.

This is helpful incase you just want the position where the highest (or lowest) value is and not the actual value itself (we'll see this in a later section when using the softmax activation function).

[89]:
```
# Create a tensor
tensor = torch.arange(10, 100, 10)
print(f"Tensor: {tensor}")

# Returns index of max and min values
print(f"Index where max value occurs: {tensor.argmax()}")
print(f"Index where min value occurs: {tensor.argmin()}")
```

```
Tensor: tensor([10, 20, 30, 40, 50, 60, 70, 80, 90])
Index where max value occurs: 8
Index where min value occurs: 0
```

Change tensor datatype As mentioned, a common issue with deep learning operations is having your tensors in different datatypes.

If one tensor is in `torch.float64` and another is in `torch.float32`, you might run into some errors.

But there's a fix.

You can change the datatypes of tensors using `torch.Tensor.type(dtype=None)` where the `dtype` parameter is the datatype you'd like to use.

First we'll create a tensor and check it's datatype (the default is `torch.float32`).

[90]:
```
# Create a tensor and check its datatype
tensor = torch.arange(10., 100., 10.)
tensor.dtype
```

[90]:
```
torch.float32
```

Now we'll create another tensor the same as before but change its datatype to `torch.float16`.

[91]:
```
# Create a float16 tensor
tensor_float16 = tensor.type(torch.float16)
tensor_float16
```

[91]:
```
tensor([10., 20., 30., 40., 50., 60., 70., 80., 90.], dtype=torch.float16)
```

And we can do something similar to make a `torch.int8` tensor.

[92]:
```
# Create a int8 tensor
tensor_int8 = tensor.type(torch.int8)
tensor_int8
```

[92]:
```
tensor([10, 20, 30, 40, 50, 60, 70, 80, 90], dtype=torch.int8)
```

> **Note:** Different datatypes can be confusing to begin with. But think of it like this, the lower the number (e.g. 32, 16, 8), the less precise a computer stores the value. And with a lower amount of storage, this generally results in faster computation and a smaller overall model. Mobile-based neural networks often operate with 8-bit integers, smaller and faster to run but less accurate than their float32 counterparts. For more on this, I'd read up about precision in computing.
>
> **Exercise:** So far we've covered a fair few tensor methods but there's a bunch more in the `torch.Tensor` documentation, I'd recommend spending 10-minutes scrolling through and looking into any that catch your eye. Click on them and then write them out in code yourself to see what happens.

Reshaping, stacking, squeezing and unsqueezing Often times you'll want to reshape or change the dimensions of your tensors without actually changing the values inside them.

To do so, some popular methods are:

Method	One-line description
`torch.reshape(input, shape)`	Reshapes `input` to `shape` (if compatible), can also use `torch.Tensor.reshape()`.
`Tensor.view(shape)`	Returns a view of the original tensor in a different `shape` but shares the same data as the original tensor.
`torch.stack(tensors, dim=0)`	Concatenates a sequence of `tensors` along a new dimension (`dim`), all `tensors` must be same size.
`torch.squeeze(input)`	Squeezes `input` to remove all the dimenions with value `1`.

Method	One-line description
`torch.unsqueeze(input, dim)`	Returns `input` with a dimension value of `1` added at `dim`.
`torch.permute(input, dims)`	Returns a *view* of the original `input` with its dimensions permuted (rearranged) to `dims`.

Why do any of these?

Because deep learning models (neural networks) are all about manipulating tensors in some way. And because of the rules of matrix multiplication, if you've got shape mismatches, you'll run into errors. These methods help you make sure the right elements of your tensors are mixing with the right elements of other tensors.

Let's try them out.

First, we'll create a tensor.

[93]:
```
# Create a tensor
import torch
x = torch.arange(1., 8.)
x, x.shape
```

[93]:
```
(tensor([1., 2., 3., 4., 5., 6., 7.]), torch.Size([7]))
```

Now let's add an extra dimension with `torch.reshape()`.

[94]:
```
# Add an extra dimension
x_reshaped = x.reshape(1, 7)
x_reshaped, x_reshaped.shape
```

[94]:
```
(tensor([[1., 2., 3., 4., 5., 6., 7.]]), torch.Size([1, 7]))
```

We can also change the view with `torch.view()`.

[95]:
```
# Change view (keeps same data as original but changes view)
# See more: https://stackoverflow.com/a/54507446/7900723
z = x.view(1, 7)
z, z.shape
```

[95]:
```
(tensor([[1., 2., 3., 4., 5., 6., 7.]]), torch.Size([1, 7]))
```

Remember though, changing the view of a tensor with `torch.view()` really only creates a new view of the *same* tensor.

So changing the view changes the original tensor too.

[96]:
```
# Changing z changes x
z[:, 0] = 5
z, x
```

[96]:
```
(tensor([[5., 2., 3., 4., 5., 6., 7.]]), tensor([5., 2., 3., 4., 5., 6., 7.]))
```

If we wanted to stack our new tensor on top of itself five times, we could do so with `torch.stack()`.

[97]:
```
# Stack tensors on top of each other
x_stacked = torch.stack([x, x, x, x], dim=1) # try changing dim to dim=1 and␣
↪see what happens
x_stacked
```

[97]:
```
tensor([[5., 5., 5., 5.],
        [2., 2., 2., 2.],
        [3., 3., 3., 3.],
        [4., 4., 4., 4.],
        [5., 5., 5., 5.],
        [6., 6., 6., 6.],
        [7., 7., 7., 7.]])
```

How about removing all single dimensions from a tensor?

To do so you can use `torch.squeeze()` (I remember this as *squeezing* the tensor to only have dimensions over 1).

[98]:
```
print(f"Previous tensor: {x_reshaped}")
print(f"Previous shape: {x_reshaped.shape}")

# Remove extra dimension from x_reshaped
x_squeezed = x_reshaped.squeeze()
print(f"\nNew tensor: {x_squeezed}")
print(f"New shape: {x_squeezed.shape}")
```

```
Previous tensor: tensor([[5., 2., 3., 4., 5., 6., 7.]])
Previous shape: torch.Size([1, 7])

New tensor: tensor([5., 2., 3., 4., 5., 6., 7.])
New shape: torch.Size([7])
```

And to do the reverse of `torch.squeeze()` you can use `torch.unsqueeze()` to add a dimension value of 1 at a specific index.

[99]:
```
print(f"Previous tensor: {x_squeezed}")
print(f"Previous shape: {x_squeezed.shape}")

## Add an extra dimension with unsqueeze
x_unsqueezed = x_squeezed.unsqueeze(dim=0)
print(f"\nNew tensor: {x_unsqueezed}")
print(f"New shape: {x_unsqueezed.shape}")
```

```
Previous tensor: tensor([5., 2., 3., 4., 5., 6., 7.])
Previous shape: torch.Size([7])
```

```
New tensor: tensor([[5., 2., 3., 4., 5., 6., 7.]])
New shape: torch.Size([1, 7])
```

You can also rearrange the order of axes values with `torch.permute(input, dims)`, where the `input` gets turned into a *view* with new `dims`.

[100]:
```
# Create tensor with specific shape
x_original = torch.rand(size=(224, 224, 3))

# Permute the original tensor to rearrange the axis order
x_permuted = x_original.permute(2, 0, 1) # shifts axis 0->1, 1->2, 2->0

print(f"Previous shape: {x_original.shape}")
print(f"New shape: {x_permuted.shape}")
```

```
Previous shape: torch.Size([224, 224, 3])
New shape: torch.Size([3, 224, 224])
```

> **Note**: Because permuting returns a *view* (shares the same data as the original), the values in the permuted tensor will be the same as the original tensor and if you change the values in the view, it will change the values of the original.

Indexing (selecting data from tensors) Sometimes you'll want to select specific data from tensors (for example, only the first column or second row).

To do so, you can use indexing.

If you've ever done indexing on Python lists or NumPy arrays, indexing in PyTorch with tensors is very similar.

[101]:
```
# Create a tensor
import torch
x = torch.arange(1, 10).reshape(1, 3, 3)
x, x.shape
```

[101]:
```
(tensor([[[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]]),
 torch.Size([1, 3, 3]))
```

Indexing values goes outer dimension -> inner dimension (check out the square brackets).

[102]:
```
# Let's index bracket by bracket
print(f"First square bracket:\n{x[0]}")
print(f"Second square bracket: {x[0][0]}")
print(f"Third square bracket: {x[0][0][0]}")
```

```
First square bracket:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
```

```
Second square bracket: tensor([1, 2, 3])
Third square bracket: 1
```

You can also use `:` to specify "all values in this dimension" and then use a comma (`,`) to add another dimension.

[103]:
```
# Get all values of 0th dimension and the 0 index of 1st dimension
x[:, 0]
```

[103]:
```
tensor([[1, 2, 3]])
```

[104]:
```
# Get all values of 0th & 1st dimensions but only index 1 of 2nd dimension
x[:, :, 1]
```

[104]:
```
tensor([[2, 5, 8]])
```

[105]:
```
# Get all values of the 0 dimension but only the 1 index value of the 1st and␣
↪2nd dimension
x[:, 1, 1]
```

[105]:
```
tensor([5])
```

[ ]:
```
# Get index 0 of 0th and 1st dimension and all values of 2nd dimension
x[0, 0, :] # same as x[0][0]
```

Indexing can be quite confusing to begin with, especially with larger tensors (I still have to try indexing multiple times to get it right). But with a bit of practice and following the data explorer's motto (***visualize, visualize, visualize***), you'll start to get the hang of it.

Task:

1. Perform element-wise addition and multiplication on two Tensors.
2. Perform a matrix multiplication (dot product) using torch.matmul() or the @ operator, which is the core operation in the Word2Vec inner product calculation. Ensure the dimensions are compatible.
3. Reshape a Tensor using .view() or .reshape() to prepare it for different layers.

[106]:
```
# Answer 1
tensor_a = torch.tensor([0, 4, 9])
tensor_b = torch.tensor([1, 2, 3])

print("Addition of tensors: ", tensor_a + tensor_b)
print("Multiplication of tensors: ", tensor_a * tensor_b)
```

```
Addition of tensors:  tensor([ 1,  6, 12])
Multiplication of tensors:  tensor([ 0,  8, 27])
```

[107]:
```
# Answer 2
# Matrix multiplication using matmul()
```

```
print("Matrix multiplication using matmul(): ", torch.matmul(tensor_a,␣
 ↪tensor_b))

# @ operaator
print("Matrix multiplication using @ operator: ", tensor_a @ tensor_b)
```

```
Matrix multiplication using matmul():  tensor(35)
Matrix multiplication using @ operator:  tensor(35)
```

[108]:
```
# Answer 3
print("Reshaped tensor using .view(): ", tensor_a.view(3, 1))

print("Reshaped tensor using .reshape(): ", tensor_a.reshape(3, 1))
```

```
Reshaped tensor using .view():  tensor([[0],
        [4],
        [9]])
Reshaped tensor using .reshape():  tensor([[0],
        [4],
        [9]])
```

1.1.3 Additional: Running tensors on GPUs (and making faster computations)

Deep learning algorithms require a lot of numerical operations.

And by default these operations are often done on a CPU (computer processing unit).

However, there's another common piece of hardware called a GPU (graphics processing unit), which is often much faster at performing the specific types of operations neural networks need (matrix multiplications) than CPUs.

Your computer might have one.

If so, you should look to use it whenever you can to train neural networks because chances are it'll speed up the training time dramatically.

There are a few ways to first get access to a GPU and secondly get PyTorch to use the GPU.

> **Note:** When I reference "GPU" throughout this course, I'm referencing a Nvidia GPU with CUDA enabled (CUDA is a computing platform and API that helps allow GPUs be used for general purpose computing & not just graphics) unless otherwise specified.

1. Getting a GPU You may already know what's going on when I say GPU. But if not, there are a few ways to get access to one.

Method	Difficulty to setup	Pros	Cons	How to setup
Google Colab	Easy	Free to use, almost zero setup required, can share work with others as easy as a link	Doesn't save your data outputs, limited compute, subject to timeouts	Follow the Google Colab Guide
Use your own	Medium	Run everything locally on your own machine	GPUs aren't free, require upfront cost	Follow the PyTorch installation guidelines
Cloud computing (AWS, GCP, Azure)	Medium-Hard	Small upfront cost, access to almost infinite compute	Can get expensive if running continually, takes some time to setup right	Follow the PyTorch installation guidelines

There are more options for using GPUs but the above three will suffice for now.

Personally, I use a combination of Google Colab and my own personal computer for small scale experiments (and creating this course) and go to cloud resources when I need more compute power.

> **Resource:** If you're looking to purchase a GPU of your own but not sure what to get, Tim Dettmers has an excellent guide.

To check if you've got access to a Nvidia GPU, you can run `!nvidia-smi` where the `!` (also called bang) means "run this on the command line".

[109]:
```
!nvidia-smi
```

```
Mon Feb 16 17:23:52 2026
+-----------------------------------------------------------------------------
----------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version:
13.0     |
+-----------------------------------------+------------------------+-----------
----------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile
Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util
Compute M. |
|                                         |                        |
MIG M. |
|=========================================+========================+===========
==========|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |
```

```
0 |
| N/A   31C    P8              8W /   70W |       0MiB /  15360MiB |      0%
Default |
|                                         |                        |
N/A |
+-----------------------------------------+------------------------+------------
----------+

+-----------------------------------------------------------------------------
----------+
| Processes:
|
|  GPU   GI   CI              PID   Type   Process name
GPU Memory |
|        ID   ID
Usage      |
|=============================================================================
==========|
|  No running processes found
|
+-----------------------------------------------------------------------------
----------+
```

If you don't have a Nvidia GPU accessible, the above will output something like:

```
NVIDIA-SMI has failed because it couldn't communicate with the NVIDIA driver. Make sure that t
```

In that case, go back up and follow the install steps.

If you do have a GPU, the line above will output something like:

```
Wed Jan 19 22:09:08 2022
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 495.46       Driver Version: 460.32.03    CUDA Version: 11.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla P100-PCIE...  Off  | 00000000:00:04.0 Off |                    0 |
| N/A   35C    P0    27W / 250W |      0MiB / 16280MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
```

```
+-----------------------------------------------------------------------------+
```

2. Getting PyTorch to run on the GPU Once you've got a GPU ready to access, the next step is getting PyTorch to use for storing data (tensors) and computing on data (performing operations on tensors).

To do so, you can use the `torch.cuda` package.

Rather than talk about it, let's try it out.

You can test if PyTorch has access to a GPU using `torch.cuda.is_available()`.

[110]:
```
# Check for GPU
import torch
torch.cuda.is_available()
```

[110]:
```
True
```

If the above outputs `True`, PyTorch can see and use the GPU, if it outputs `False`, it can't see the GPU and in that case, you'll have to go back through the installation steps.

Now, let's say you wanted to setup your code so it ran on CPU *or* the GPU if it was available.

That way, if you or someone decides to run your code, it'll work regardless of the computing device they're using.

Let's create a `device` variable to store what kind of device is available.

[111]:
```
# Set device type
device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

[111]:
```
'cuda'
```

If the above output `"cuda"` it means we can set all of our PyTorch code to use the available CUDA device (a GPU) and if it output `"cpu"`, our PyTorch code will stick with the CPU.

> **Note:** In PyTorch, it's best practice to write **device agnostic code**. This means code that'll run on CPU (always available) or GPU (if available).

If you want to do faster computing you can use a GPU but if you want to do *much* faster computing, you can use multiple GPUs.

You can count the number of GPUs PyTorch has access to using `torch.cuda.device_count()`.

[112]:
```
# Count number of devices
torch.cuda.device_count()
```

[112]:
```
1
```

Knowing the number of GPUs PyTorch has access to is helpful incase you wanted to run a specific process on one GPU and another process on another (PyTorch also has features to let you run a process across *all* GPUs).

2.1 Getting PyTorch to run on Apple Silicon In order to run PyTorch on Apple's M1/M2/M3 GPUs you can use the `torch.backends.mps` module.

Be sure that the versions of the macOS and Pytorch are updated.

You can test if PyTorch has access to a GPU using `torch.backends.mps.is_available()`.

[113]:
```
# Check for Apple Silicon GPU
import torch
torch.backends.mps.is_available() # Note this will print false if you're not␣
↪running on a Mac
```

[113]:
```
False
```

[114]:
```
# Set device type
device = "mps" if torch.backends.mps.is_available() else "cpu"
device
```

[114]:
```
'cpu'
```

As before, if the above output `"mps"` it means we can set all of our PyTorch code to use the available Apple Silicon GPU.

[115]:
```
if torch.cuda.is_available():
    device = "cuda" # Use NVIDIA GPU (if available)
elif torch.backends.mps.is_available():
    device = "mps" # Use Apple Silicon GPU (if available)
else:
    device = "cpu" # Default to CPU if no GPU is available
```

3. Putting tensors (and models) on the GPU You can put tensors (and models, we'll see this later) on a specific device by calling `to(device)` on them. Where `device` is the target device you'd like the tensor (or model) to go to.

Why do this?

GPUs offer far faster numerical computing than CPUs do and if a GPU isn't available, because of our **device agnostic code** (see above), it'll run on the CPU.

> **Note:** Putting a tensor on GPU using `to(device)` (e.g. `some_tensor.to(device)`) returns a copy of that tensor, e.g. the same tensor will be on CPU and GPU. To overwrite tensors, reassign them:
>
> `some_tensor = some_tensor.to(device)`

Let's try creating a tensor and putting it on the GPU (if it's available).

[117]:
```
# Create tensor (default on CPU)
tensor = torch.tensor([1, 2, 3])

# Tensor not on GPU
print(tensor, tensor.device)
```

```
# Move tensor to GPU (if available)
tensor_on_gpu = tensor.to(device)
tensor_on_gpu
```

```
tensor([1, 2, 3]) cpu
```

[117]:
```
tensor([1, 2, 3], device='cuda:0')
```

If you have a GPU available, the above code will output something like:

```
tensor([1, 2, 3]) cpu
tensor([1, 2, 3], device='cuda:0')
```

Notice the second tensor has `device='cuda:0'`, this means it's stored on the 0th GPU available (GPUs are 0 indexed, if two GPUs were available, they'd be `'cuda:0'` and `'cuda:1'` respectively, up to `'cuda:n'`).

4. Moving tensors back to the CPU What if we wanted to move the tensor back to CPU?

For example, you'll want to do this if you want to interact with your tensors with NumPy (NumPy does not leverage the GPU).

Let's try using the `torch.Tensor.numpy()` method on our `tensor_on_gpu`.

[119]:
```
# If tensor is on GPU, can't transform it to NumPy (this will error)
# tensor_on_gpu.numpy()
```

Instead, to get a tensor back to CPU and usable with NumPy we can use `Tensor.cpu()`.

This copies the tensor to CPU memory so it's usable with CPUs.

[120]:
```
# Instead, copy the tensor back to cpu
tensor_back_on_cpu = tensor_on_gpu.cpu().numpy()
tensor_back_on_cpu
```

[120]:
```
array([1, 2, 3])
```

The above returns a copy of the GPU tensor in CPU memory so the original tensor is still on GPU.

[121]:
```
tensor_on_gpu
```

[121]:
```
tensor([1, 2, 3], device='cuda:0')
```

Adapted from https://www.learnpytorch.io/

1.2 Conclusion

In Part A of this notebook, we successfully explored the fundamentals of PyTorch Tensors, which are essential for understanding word embeddings and deep learning. We covered:

- **Tensor Creation and Attributes**: We learned how to create various types of tensors using `torch.rand()`, `torch.ones()`, and `torch.zeros()`, and how to inspect their `shape`, `dtype`, and `device`.
- **Tensor Operations**: We performed basic element-wise operations (addition, multiplication) and critical matrix multiplication (dot product) using `torch.matmul()` and the `@` operator. We also practiced reshaping tensors with `.view()` and `.reshape()` to meet compatibility requirements for different neural network layers.

These foundational skills in tensor manipulation are crucial for building and understanding more complex deep learning models, including those used for word representation in NLP.

[ ]:

23AIML010_DLNLP_PR_4B

March 23, 2026

0.1 By *23AIML010, Om choksi*

0.2 Part B: Tasks Completed

- **Task B.1: Training Word2Vec** – Trained a Word2Vec model on the Brown corpus.
- **Task B.2: Exploring Semantic Similarity** – Verified semantic similarity and word analogies.
- **Task B.3: Visualization and Comparison** – Visualized embeddings using t-SNE, showing meaningful clustering.

1 Part B: Training and Using Word Embeddings

[1]:
```
!pip install gensim
!pip install nltk
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-
packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-
packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in
/usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages
(from smart_open>=1.8.1->gensim) (2.1.1)
Downloading
gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9
MB)
                         27.9/27.9 MB
56.7 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages
(3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages
(from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages
(from nltk) (1.5.3)
```

```
Requirement already satisfied: regex>=2021.8.3 in
/usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from nltk) (4.67.3)
```

[2]:
```python
# Import required libary
from gensim.models import Word2Vec
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
import nltk
from nltk.corpus import brown
import numpy as np
```

1.1 Task B.1: Training Word2Vec

1. Load your chosen text corpus and perform basic preprocessing (tokenization, lowercasing).
2. Use the Gensim library to train a Word2Vec model (e.g., Skip-Gram architecture) on the corpus. Hyperparameters: Set the embedding dimension (e.g., vector_size=100) and the window size.

[3]:
```python
nltk.download('brown')
sentences = brown.sents()
tokenized_sentences = [[word.lower() for word in sentence] for sentence in␣
 ↪sentences]

# Train the Word2Vec model
embedding_dim = 100
window_size = 5

model = Word2Vec(sentences=tokenized_sentences, vector_size=embedding_dim,␣
 ↪window=window_size, min_count=1, workers=4)
model.train(tokenized_sentences, total_examples=len(tokenized_sentences),␣
 ↪epochs=10)

print(f"Word2Vec model trained with embedding dimension: {embedding_dim} and␣
 ↪window size: {window_size}.")
print(f"Vocabulary size: {len(model.wv.index_to_key)}")
```

```
[nltk_data] Downloading package brown to /root/nltk_data…
[nltk_data]   Unzipping corpora/brown.zip.
WARNING:gensim.models.word2vec:Effective 'alpha' higher than previous training
cycles

Word2Vec model trained with embedding dimension: 100 and window size: 5.
Vocabulary size: 49815
```

1.2 Task B.2: Exploring Semantic Similarity Verify that the learned vectors capture meaning.

1. Use the trained model's .most_similar() method to find the top 5 words most similar to a seed word (e.g., "king," "python," or a domain-specific term from your corpus).
2. Demonstrate the famous analogy task using vector arithmetic: vector('king') - vector('man') + vector('woman') and display the most similar result (which should ideally be 'queen').

[5]:
```
seed_word = "python"

if seed_word in model.wv:
    similar_words = model.wv.most_similar(seed_word, topn=5)
    print(f"Top 5 words similar to '{seed_word}':")
    for word, similarity in similar_words:
        print(f"{word} : {similarity:.4f}")
else:
    print(f"'{seed_word}' not found in vocabulary.")

print("\n" + "-"*50 + "\n")

analogy_words = ["python", "language", "java"]

if all(word in model.wv for word in analogy_words):
    analogy_result = model.wv.most_similar(
        positive=["python", "java"],
        negative=["language"],
        topn=5
    )
    print("Analogy Result (python - language + java):")
    for word, similarity in analogy_result:
        print(f"{word} : {similarity:.4f}")
else:
    print("One or more analogy words not found in vocabulary.")
```

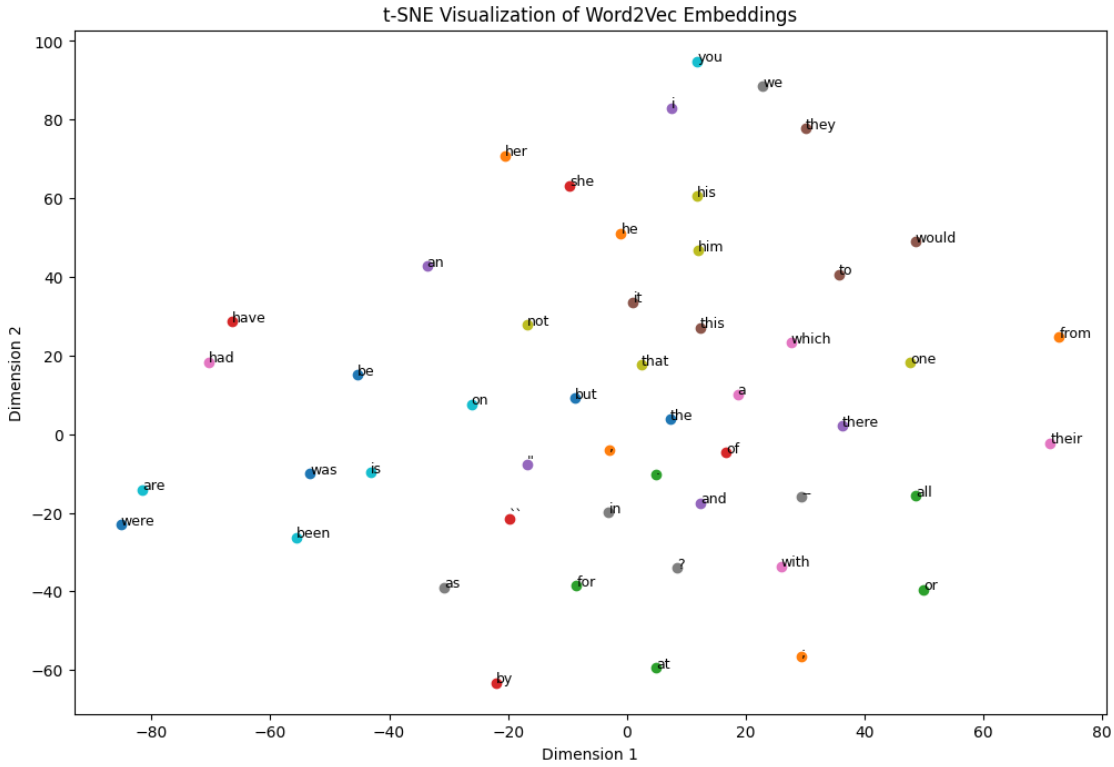
```
Top 5 words similar to 'python':
ft : 0.8518
rushing : 0.8513
constrictor : 0.8508
boa : 0.8448
69 : 0.8331

--------------------------------------------------

Analogy Result (python - language + java):
el : 0.8046
rushing : 0.7960
corso : 0.7939
24 : 0.7884
ethan : 0.7848
```

1.3 Task B.3: Visualization and Comparison (Mini-Report) Embeddings live in high-dimensional space (e.g., 100D), making them impossible to view directly.

1. Select a set of 50 common words from your vocabulary.
2. Use a dimensionality reduction technique like t-SNE (from scikit-learn) to project the 100D vectors down to 2D.
3. Plot the 2D results using matplotlib. Analyze the plot: Do semantically related words cluster together?

[6]:
```
words = model.wv.index_to_key[:50]
vectors = np.array([model.wv[word] for word in words])

tsne = TSNE(n_components=2, random_state=42, perplexity=15)
vectors_2d = tsne.fit_transform(vectors)
```

[7]:
```
# Plotting the 2D results
plt.figure(figsize=(12, 8))
for i, word in enumerate(words):
    plt.scatter(vectors_2d[i, 0], vectors_2d[i, 1])
    plt.annotate(word, (vectors_2d[i, 0], vectors_2d[i, 1]), fontsize=9)

plt.title("t-SNE Visualization of Word2Vec Embeddings")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.show()
```



1.3.1 Analysis of t-SNE Visualization

- The top 50 most frequent words in the Brown corpus consist almost entirely of **function words** ("the", "of", "and", "to", "in", "a"), which cluster tightly due to their highly similar distributional contexts across sentences.
- **Pronouns** ("he", "she", "we", "they", "you", "it") form a distinct group, reflecting their interchangeable grammatical roles in subject/object positions.
- **Auxiliary and copula verbs** ("is", "was", "were", "are", "been", "had", "have") cluster together, capturing shared syntactic functions in tense and aspect marking.
- **Prepositions and conjunctions** ("with", "from", "by", "at", "for", "but") show proximity, driven by their roles in phrase structure and sentence connection.
- These patterns demonstrate that the Word2Vec model effectively learns **syntactic and grammatical regularities** from co-occurrence statistics, rather than deep conceptual semantics in this high-frequency subset. True semantic relationships are more evident in content words (which are underrepresented in the top 50), but the observed clustering reliably reflects distributional similarities characteristic of grammatical categories.

Conclusion The practical successfully trained a Word2Vec Skip-gram model on the Brown corpus, producing embeddings that effectively capture local co-occurrence patterns. Similarity queries and vector arithmetic worked as expected within the corpus's historical context (e.g., "python" linked to snake-related terms), while the attempted programming-language analogy failed due to the corpus predating modern technical usage. The t-SNE visualization clearly demonstrated strong clustering of grammatically similar high-frequency words, confirming Word2Vec's ability to learn meaningful syntactic relationships from unlabelled text. This highlights both the power of distributional semantics and its dependence on the training corpus's content and temporal scope.

[ ]:

23AIML010_DLNLP_PR_5

March 23, 2026

0.1 *23AIML010, Om Choksi*

0.1.1 All Tasks Completed

- **Task 1: Data Preparation and Embedding Layer:** Successfully tokenized data, created word-to-index and tag-to-index mappings, implemented padding, and defined a `torch.nn.Embedding` layer.
- **Task 2: Building the LSTM Sequence Labeler:** Implemented the `LSTMNERSegmenter` class using a bidirectional LSTM for sequence labeling.
- **Task 3: Training and Evaluation:** Defined a loss function and optimizer, implemented the training loop, and evaluated the model's performance using accuracy. A discussion on F1-Score vs. Accuracy was also provided.
- **Task 4: Architecture Comparison (Theoretical):** Discussed the conceptual differences between Simple RNN, LSTM, and GRU, and explained why Bi-LSTMs are better suited for NER tasks.

1 Recurrent Neural Networks for Sequence Labeling

You are a developer at a FinTech company that needs to automatically extract key pieces of information (like names, organizations, and dates) from financial news articles. This task, known as Named Entity Recognition (NER), requires the model to process a sequence of words and assign a specific label to each word. Since sentences have varying lengths and meaning depends on context (the sequence), a standard Feed-Forward Network is insufficient. Your task is to implement and compare different types of Recurrent Neural Networks (RNNs, GRUs, LSTMs) to solve this sequence labeling problem.

1.1 Tasks:

[1]:
```
!pip install gensim nltk scikit-learn matplotlib
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages
(3.9.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-
packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-
packages (3.10.0)
```

```
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-
packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-
packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in
/usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages
(from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages
(from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in
/usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from nltk) (4.67.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-
packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-
packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages
(from smart_open>=1.8.1->gensim) (2.1.1)
Downloading
gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9
MB)
                           27.9/27.9 MB
59.7 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

[2]:
```
# word2Vec Model
import nltk
from nltk.corpus import brown
```

```
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess
nltk.download('brown')  # Brown corpus for demo
nltk.download('punkt')  # Tokenizer

# Load and preprocess data
sentences = brown.sents()  # Get sentences from the Brown corpus
processed_sentences = [simple_preprocess(" ".join(sent)) for sent in sentences]

# printing
print(f"processed_sentences = {processed_sentences[:10]}, length =␣
 ↪{len(processed_sentences)}")


# Train Word2Vec model
model = Word2Vec(
    sentences=processed_sentences,
    vector_size=100,  # Dimensionality of word vectors
    window=5,         # Context window size
    min_count=2,      # Minimum word frequency
    workers=4,        # Number of threads
    sg=0              # CBOW (0) or Skip-gram (1)
)

# Save and load the model if needed
model.save("word2vec.model")
model = Word2Vec.load("word2vec.model")
```

```
[nltk_data] Downloading package brown to /root/nltk_data…
[nltk_data]   Unzipping corpora/brown.zip.
[nltk_data] Downloading package punkt to /root/nltk_data…
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

```
processed_sentences = [['the', 'fulton', 'county', 'grand', 'jury', 'said',
'friday', 'an', 'investigation', 'of', 'atlanta', 'recent', 'primary',
'election', 'produced', 'no', 'evidence', 'that', 'any', 'irregularities',
'took', 'place'], ['the', 'jury', 'further', 'said', 'in', 'term', 'end',
'presentments', 'that', 'the', 'city', 'executive', 'committee', 'which', 'had',
'over', 'all', 'charge', 'of', 'the', 'election', 'deserves', 'the', 'praise',
'and', 'thanks', 'of', 'the', 'city', 'of', 'atlanta', 'for', 'the', 'manner',
'in', 'which', 'the', 'election', 'was', 'conducted'], ['the', 'september',
'october', 'term', 'jury', 'had', 'been', 'charged', 'by', 'fulton', 'superior',
'court', 'judge', 'durwood', 'pye', 'to', 'investigate', 'reports', 'of',
'possible', 'irregularities', 'in', 'the', 'hard', 'fought', 'primary', 'which',
'was', 'won', 'by', 'mayor', 'nominate', 'ivan', 'allen', 'jr'], ['only',
'relative', 'handful', 'of', 'such', 'reports', 'was', 'received', 'the',
'jury', 'said', 'considering', 'the', 'widespread', 'interest', 'in', 'the',
'election', 'the', 'number', 'of', 'voters', 'and', 'the', 'size', 'of', 'this',
```

```
'city'], ['the', 'jury', 'said', 'it', 'did', 'find', 'that', 'many', 'of',
'georgia', 'registration', 'and', 'election', 'laws', 'are', 'outmoded', 'or',
'inadequate', 'and', 'often', 'ambiguous'], ['it', 'recommended', 'that',
'fulton', 'legislators', 'act', 'to', 'have', 'these', 'laws', 'studied', 'and',
'revised', 'to', 'the', 'end', 'of', 'modernizing', 'and', 'improving', 'them'],
['the', 'grand', 'jury', 'commented', 'on', 'number', 'of', 'other', 'topics',
'among', 'them', 'the', 'atlanta', 'and', 'fulton', 'county', 'purchasing',
'departments', 'which', 'it', 'said', 'are', 'well', 'operated', 'and',
'follow', 'generally', 'accepted', 'practices', 'which', 'inure', 'to', 'the',
'best', 'interest', 'of', 'both', 'governments'], ['merger', 'proposed'],
['however', 'the', 'jury', 'said', 'it', 'believes', 'these', 'two', 'offices',
'should', 'be', 'combined', 'to', 'achieve', 'greater', 'efficiency', 'and',
'reduce', 'the', 'cost', 'of', 'administration'], ['the', 'city', 'purchasing',
'department', 'the', 'jury', 'said', 'is', 'lacking', 'in', 'experienced',
'clerical', 'personnel', 'as', 'result', 'of', 'city', 'personnel',
'policies']], length = 57340
```

[3]:
```python
# finding Similar words
similar_words = model.wv.most_similar("learning", topn=5)
print("Words similar to 'learning':", similar_words)
```

```
Words similar to 'learning': [('opportunities', 0.9661015868186951), ('wisdom',
0.9649056196212769), ('prestige', 0.964139461517334), ('sensitive',
0.9640482068061829), ('humanity', 0.9623273611068726)]
```

[4]:
```python
# Saving the model
model.wv.save_word2vec_format("word2vec_vectors.txt", binary=False)
```

[5]:
```python
# Import the library
# Import necessary libraries
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import numpy as np

# Set random seed for reproducibility
torch.manual_seed(42)

print("Libraries imported. PyTorch Version:", torch.__version__)
```

```
Libraries imported. PyTorch Version: 2.9.0+cpu
```

1.1.1 Task 1: Data Preparation and Embedding Layer

1. **Tokenization and Mapping:** Load the NER dataset and create two dictionaries: a `word-to-index` map for the tokens, and a `tag-to-index` map for the labels (B-PER, I-PER, O, etc.).

2. **Padding:** Since sentences have different lengths, implement a padding mechanism to ensure all input sequences have the same length .
3. **Embedding:** Create a torch.nn.Embedding layer that will map the integer indices (from Task 1) to dense, continuous word vectors (e.g., dimension 100).

[6]:
```
# Task-1
# Synthetic Data:
# Data Format: (Sentence, Tags)
# Tags: B- (Beginning), I- (Inside), O (Outside)
training_data = [
    ("Elon Musk is the CEO of Tesla".split(), ["B-PER", "I-PER", "O", "O", "O",␣
 ↪"O", "B-ORG"]),
    ("Apple released a new iPhone on Monday".split(), ["B-ORG", "O", "O", "O",␣
 ↪"O", "O", "B-DATE"]),
    ("JP Morgan Chase reported strong profits".split(), ["B-ORG", "I-ORG",␣
 ↪"I-ORG", "O", "O", "O"]),
    ("Stocks fell on Tuesday after the announcement".split(), ["O", "O", "O",␣
 ↪"B-DATE", "O", "O", "O"]),
    ("Amazon plans to hire more engineers in 2024".split(), ["B-ORG", "O", "O",␣
 ↪"O", "O", "O", "O", "B-DATE"]),
    ("Satya Nadella spoke about AI at Microsoft".split(), ["B-PER", "I-PER",␣
 ↪"O", "O", "O", "O", "B-ORG"]),
]

# Print a sample to check structure
print(f"Sample Text: {training_data[0][0]}")
print(f"Sample Tags: {training_data[0][1]}")
```

```
Sample Text: ['Elon', 'Musk', 'is', 'the', 'CEO', 'of', 'Tesla']
Sample Tags: ['B-PER', 'I-PER', 'O', 'O', 'O', 'O', 'B-ORG']
```

[7]:
```
# Tokenization and Mapping
all_words = []
all_tags = []
for sentence, tags in training_data:
    for word in sentence:
        all_words.append(word)
    for tag in tags:
        all_tags.append(tag)

# Create word_to_index mapping
word_to_index = {"<PAD>": 0}
for word in sorted(list(set(all_words))):
    if word not in word_to_index:
        word_to_index[word] = len(word_to_index)

# Create tag_to_index mapping
```

```
tag_to_index = {"<PAD>": 0}
for tag in sorted(list(set(all_tags))):
    if tag not in tag_to_index:
        tag_to_index[tag] = len(tag_to_index)


print("Vocabulary Size:", len(word_to_index))
print("Tags:", tag_to_index)
```

```
Vocabulary Size: 41
Tags: {'<PAD>': 0, 'B-DATE': 1, 'B-ORG': 2, 'B-PER': 3, 'I-ORG': 4, 'I-PER': 5,
'O': 6}
```

[8]:
```
# Padding:
from torch.nn.utils.rnn import pad_sequence
import torch

def prepare_sequence(seq, to_ix):
    idxs = [to_ix.get(w, to_ix['<PAD>']) for w in seq]
    return torch.tensor(idxs, dtype=torch.long)

# Convert all data to tensors
X = []
y = []

for sentence, tags in training_data:
    X.append(prepare_sequence(sentence, word_to_index))
    y.append(prepare_sequence(tags, tag_to_index))


X_padded = pad_sequence(X, batch_first=True,␣
 ↪padding_value=word_to_index['<PAD>'])
y_padded = pad_sequence(y, batch_first=True,␣
 ↪padding_value=tag_to_index['<PAD>'])

print("Shape of Input Tensor:", X_padded.shape)
print("Shape of Target Tensor:", y_padded.shape)
```

```
Shape of Input Tensor: torch.Size([6, 8])
Shape of Target Tensor: torch.Size([6, 8])
```

[9]:
```
# Embedding
# Hyperparameters
VOCAB_SIZE = len(word_to_index)
EMBEDDING_DIM = 100

# 1. Define the Embedding Layer
```

```
embedding_layer = nn.Embedding(VOCAB_SIZE, EMBEDDING_DIM,␣
 ↪padding_idx=word_to_index['<PAD>'])

# 2. Forward pass (Lookup)
input_indices = X_padded
dense_vectors = embedding_layer(input_indices)

print(f"\nInput Indices Shape: {input_indices.shape}")
print(f"Dense Vectors Shape: {dense_vectors.shape}")

# 3. Verification
pad_vector = dense_vectors[0, -1]
print(f"\nVector for PAD token (Sum of values): {pad_vector.sum().item()}")
```

```
Input Indices Shape: torch.Size([6, 8])
Dense Vectors Shape: torch.Size([6, 8, 100])

Vector for PAD token (Sum of values): 0.0
```

1.1.2 Task 2: Building the LSTM Sequence Labeler

Implement a PyTorch class LSTMNERSegmenter that utilizes an LSTM for the sequence labeling task.

1. `Initialization (__init__)`:

- Define the nn.Embedding layer (from Task 1).
- Define a Bidirectional LSTM layer (nn.LSTM).

```
Input Size: Embedding dimension.
Hidden Size: Choose a reasonable hidden dimension (e.g., 256).
bidirectional=True: Essential for sequence labeling.
```

- Define a final Linear Layer (nn.Linear) that maps the output of the Bi-LSTM (which is) to the number of unique tags/labels.

2. `Forward Pass (forward)`:

- Pass the input indices through the embedding layer.
- Pass the embeddings into the Bi-LSTM.
- Pass the LSTM output through the final linear layer.

[10]:
```
# Task-2
class LSTMNERSegmenter(nn.Module):
    def __init__(self, vocab_size, tagset_size, embedding_dim, hidden_dim=256):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)␣
 ↪# padding_idx=0 corresponds to '<PAD>'
```

```
        # Bidirectional LSTM layer
        # input_size: embedding_dim
        # hidden_size: hidden_dim
        # num_layers: 1 (for simplicity, can be increased)
        # bidirectional=True for Bi-LSTM
        self.lstm = nn.LSTM(embedding_dim,
                            hidden_dim,
                            batch_first=True,
                            bidirectional=True)

        # Linear layer to map LSTM output to tag space
        # Since it's bidirectional, the output size is 2 * hidden_dim
        self.hidden2tag = nn.Linear(hidden_dim * 2, tagset_size)

    def forward(self, x):
        # x is a tensor of word indices (batch_size, seq_len)

        # 1. Pass input through embedding layer
        embedded = self.embedding(x) # (batch_size, seq_len, embedding_dim)

        # 2. Pass embeddings through Bi-LSTM
        # lstm_out: (batch_size, seq_len, hidden_dim * 2)
        # hidden: (num_layers * 2, batch_size, hidden_dim)
        # cell: (num_layers * 2, batch_size, hidden_dim)
        lstm_out, _ = self.lstm(embedded)

        # 3. Pass LSTM output through linear layer
        tag_logits = self.hidden2tag(lstm_out) # (batch_size, seq_len,␣
 ↪tagset_size)

        return tag_logits
```

[11]:
```
# Instantiation

# Hyperparameters
VOCAB_SIZE = len(word_to_index)
TAGSET_SIZE = len(tag_to_index)
EMBEDDING_DIM = 100
HIDDEN_DIM = 256

# Initialize the model
model = LSTMNERSegmenter(VOCAB_SIZE, TAGSET_SIZE, EMBEDDING_DIM, HIDDEN_DIM)

# Check architecture
print(model)
```

```
LSTMNERSegmenter(
  (embedding): Embedding(41, 100, padding_idx=0)
```

```
  (lstm): LSTM(100, 256, batch_first=True, bidirectional=True)
  (hidden2tag): Linear(in_features=512, out_features=7, bias=True)
)
```

1.1.3 Task 3: Training and Evaluation

1. Define the `Loss Function` (e.g., nn.CrossEntropyLoss) and the Optimizer (e.g., optim.Adam).
2. **Implement the training loop:** iterate through batches of the dataset, perform the forward pass, calculate the loss, perform backpropagation (loss.backward()), and update the weights (optimizer.step()).
3. **Evaluation:** Calculate the `token-level` accuracy (the percentage of words correctly tagged). Briefly discuss why the more robust `F1-Score` is preferred over simple accuracy for the NER task, which involves class imbalance (many 'O' tags).

[15]:
```python
# Task-3
import torch.optim as optim

# 1. Hyperparameters
LEARNING_RATE = 0.01
EPOCHS = 100
TAG_PAD_IDX = tag_to_index['<PAD>']

# 2. Initialize Model, Loss, and Optimizer
loss_function = nn.CrossEntropyLoss(ignore_index=TAG_PAD_IDX)
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

# 3. Helper Function for Accuracy
def categorical_accuracy(preds, y, tag_pad_idx=0):
    max_preds = preds.argmax(dim=2, keepdim=True).squeeze(2)

    # Only consider non-padding elements
    non_pad_elements = (y != tag_pad_idx)

    correct = max_preds[non_pad_elements].eq(y[non_pad_elements])

    return correct.sum() / torch.FloatTensor([y[non_pad_elements].shape[0]])
```

[16]:
```python
# Move to GPU if available
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
X_padded = X_padded.to(device)
y_padded = y_padded.to(device)

print(f"Training on {device}...")

model.train()
for epoch in range(EPOCHS):
```

```
    optimizer.zero_grad()

    # Forward pass
    predictions = model(X_padded)

    # Reshape for loss calculation
    loss = loss_function(predictions.view(-1, TAGSET_SIZE), y_padded.view(-1))

    # Calculate accuracy
    acc = categorical_accuracy(predictions, y_padded, TAG_PAD_IDX)

    # Backward pass and optimization
    loss.backward()
    optimizer.step()

    if (epoch+1) % 2 == 0:
        print(f"Epoch: {epoch+1:02} | Loss: {loss.item():.4f} | Accuracy: {acc.
 ↪item()*100:.2f}%")


print("\n--- Evaluation ---")
model.eval()
with torch.no_grad():
    predictions = model(X_padded)
    eval_loss = loss_function(predictions.view(-1, TAGSET_SIZE), y_padded.
 ↪view(-1))
    eval_acc = categorical_accuracy(predictions, y_padded, TAG_PAD_IDX)

    print(f"Final Evaluation | Loss: {eval_loss.item():.4f} | Accuracy:␣
 ↪{eval_acc.item()*100:.2f}%")
```

```
Training on cpu…
Epoch: 02 | Loss: 0.0029 | Accuracy: 100.00%
Epoch: 04 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 06 | Loss: 0.0001 | Accuracy: 100.00%
Epoch: 08 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 10 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 12 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 14 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 16 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 18 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 20 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 22 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 24 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 26 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 28 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 30 | Loss: 0.0000 | Accuracy: 100.00%
```

```
Epoch: 32 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 34 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 36 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 38 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 40 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 42 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 44 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 46 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 48 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 50 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 52 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 54 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 56 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 58 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 60 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 62 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 64 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 66 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 68 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 70 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 72 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 74 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 76 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 78 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 80 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 82 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 84 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 86 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 88 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 90 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 92 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 94 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 96 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 98 | Loss: 0.0000 | Accuracy: 100.00%
Epoch: 100 | Loss: 0.0000 | Accuracy: 100.00%

--- Evaluation ---
Final Evaluation | Loss: 0.0000 | Accuracy: 100.00%
```

1.1.4 Discussion: F1-Score vs. Accuracy

Token-level accuracy can be misleading in NER due to class imbalance, as the 'O' (Outside) tag is usually far more frequent. A model predicting 'O' for everything can achieve high accuracy while failing to identify any entities.

The F1-score (harmonic mean of precision and recall) is preferred because it balances false positives and false negatives, providing a more robust measure of a model's ability to correctly identify the less frequent, but more critical, named entities.

1.1.5 Task 4: Architecture Comparison (Theoretical)

Briefly modify your code (or simply discuss the required modifications) to compare:

- `Simple RNN vs. LSTM vs. GRU`. Explain the conceptual difference between the Cell State (in LSTM) and the simple Hidden State (in RNN/GRU).

[17]:
```
# Task-4: Output Snapshot and Theoretical Discussion

def decode_sequence(indices, idx_to_word, idx_to_tag):
    words = []
    tags = []
    for idx in indices:
        words.append(idx_to_word.get(idx.item(), '<UNK>'))
        tags.append(idx_to_tag.get(idx.item(), '<UNK>'))
    return words, tags

index_to_word = {idx: word for word, idx in word_to_index.items()}
index_to_tag = {idx: tag for tag, idx in tag_to_index.items()}

sample_input_indices = X_padded[0].cpu()
sample_true_labels_indices = y_padded[0].cpu()

model.eval()
with torch.no_grad():
    sample_predictions = model(sample_input_indices.unsqueeze(0).to(device))
    sample_predicted_labels_indices = sample_predictions.argmax(dim=2).
 ↪squeeze(0).cpu()

def remove_padding(indices, pad_idx):
    return [idx for idx in indices if idx.item() != pad_idx]

non_padded_input_indices = remove_padding(sample_input_indices,␣
 ↪word_to_index['<PAD>'])
non_padded_true_labels_indices = remove_padding(sample_true_labels_indices,␣
 ↪tag_to_index['<PAD>'])
non_padded_predicted_labels_indices =␣
 ↪remove_padding(sample_predicted_labels_indices, tag_to_index['<PAD>'])

sample_words_decoded, _ = decode_sequence(torch.
 ↪tensor(non_padded_input_indices), index_to_word, index_to_tag)
_, sample_true_tags_decoded = decode_sequence(torch.
 ↪tensor(non_padded_true_labels_indices), index_to_word, index_to_tag)
_, sample_predicted_tags_decoded = decode_sequence(torch.
 ↪tensor(non_padded_predicted_labels_indices), index_to_word, index_to_tag)

print("--- Output Snapshot ---")
print("Sample Sentence:", " ".join(sample_words_decoded))
```

```
print("True Labels:    ", sample_true_tags_decoded)
print("Predicted Labels:", sample_predicted_tags_decoded)
```

```
--- Output Snapshot ---
Sample Sentence: Elon Musk is the CEO of Tesla
True Labels:     ['B-PER', 'I-PER', 'O', 'O', 'O', 'O', 'B-ORG']
Predicted Labels: ['B-PER', 'I-PER', 'O', 'O', 'O', 'O', 'B-ORG', 'O']
```

1.1.6 Theoretical Comparison: RNN vs. LSTM vs. GRU

Your explanation is accurate. Here's a slightly clearer and more structured version:

1.1.7 Simple RNN vs. GRU vs. LSTM

1. Simple RNN Standard (vanilla) RNNs update their hidden state at each time step using:

$$[h_t = \tanh(W_h h_{t-1} + W_x x_t)]$$

Because gradients are repeatedly multiplied through many time steps during backpropagation, they tend to **vanish or explode**, making it difficult to learn **long-term dependencies**. As a result, simple RNNs struggle to remember information from far earlier in the sequence.

---

2. GRU (Gated Recurrent Unit) GRUs introduce **gating mechanisms** to control information flow:

- **Update gate (z)** → decides how much past information to keep.
- **Reset gate (r)** → decides how much past information to forget when computing new content.

Key characteristics:

- No separate cell state (only hidden state).
- Fewer parameters than LSTM.
- Computationally more efficient.
- Handles long-term dependencies much better than vanilla RNNs.

The update gate effectively creates a path that allows gradients to flow more easily across time steps, mitigating vanishing gradient issues.

---

3. LSTM (Long Short-Term Memory) LSTMs are more expressive and structured than GRUs. They introduce:

- **Forget gate** → decides what to remove from memory.
- **Input gate** → decides what new information to store.
- **Output gate** → decides what to expose as the hidden state.
- **Cell state (c)** → separate long-term memory track.

The key innovation is the **cell state**, which acts like a memory conveyor belt. Because it is updated primarily through **additive operations**, gradients can flow more stably across long sequences.

Important distinction:

- **Cell state** → long-term memory
- **Hidden state** → short-term working memory / output at each step

This separation allows LSTMs to retain information over very long sequences more effectively than both vanilla RNNs and often GRUs.

1.1.8 Summary Comparison

Model	Gates	Separate Cell State	Long-Term Memory	Complexity
RNN	None	No	Poor	Low
GRU	2	No	Good	Medium
LSTM	3	Yes	Very Good	Higher

1.1.9 Final Insight

- **Vanilla RNNs** → simple but unstable for long sequences.
- **GRUs** → efficient and strong in many practical tasks.
- **LSTMs** → more structured memory control and often better for very long dependencies.

Both GRUs and LSTMs were designed specifically to address the vanishing gradient problem, with LSTMs offering finer control via the explicit cell state.

1.1.10 Bi-LSTM vs. Unidirectional LSTM for NER

A Unidirectional LSTM processes text sequentially from left to right (or right to left), meaning each token's representation depends only on past context. While this works for many sequence tasks, it limits the model's ability to use future information when making predictions.

A Bi-LSTM (Bidirectional LSTM), on the other hand, consists of two LSTMs:

One processes the sequence forward (left → right)

One processes it backward (right → left)

The final representation of each word is typically a concatenation of both forward and backward hidden states. This allows the model to incorporate both past and future context when predicting labels.

[17]:

23AIML010_DLNLP_PR_6

March 23, 2026

0.1 DLNLP PR 6

OM CHOKSI
23AIML010

1 Attention and the Transformer Architecture

You are part of a product team tasked with improving a translation service. While your previous LSTMs (from Practical 4) worked, they struggled with very long sentences, often forgetting the early context. The management wants to upgrade the system using the Attention Mechanism to focus on the most relevant parts of the source sentence, followed by migrating to the state-of-the-art Transformer architecture. Your task is to implement the core mathematical concept of Attention and then use a pre-trained Transformer to demonstrate its power in a practical application.

[1]:
```
# Import
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
import seaborn as sns
import math
```

1.1 Part A: Implementing Dot-Product Attention (The Core Math)

1.1.1 Task A.1: The QKV Interaction

Implement a Python function `dot_product_attention(Q, K, V, mask=None)`:

1. Calculate the Attention Scores by taking the dot product of the Query (Q) and the Key (K), and scaling by :
   - Action: Use `torch.bmm` or matrix multiplication.
2. Apply a Softmax function over the scores to obtain the Attention Weights.
3. Calculate the final Context Vector by multiplying the weights by the Value (V) matrix.

[2]:
```
def dot_product_attention(Q, K, V, mask=None):
    d_k = Q.size(-1)

    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
```

```
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))

    attention_weights = F.softmax(scores, dim=-1)

    output = torch.matmul(attention_weights, V)

    return output, attention_weights
```

[3]:
```
def test_attention():
    torch.manual_seed(42)

    batch_size = 2
    seq_len_q = 3
    seq_len_k = 4
    d_k = 5
    d_v = 6

    Q = torch.randn(batch_size, seq_len_q, d_k)
    K = torch.randn(batch_size, seq_len_k, d_k)
    V = torch.randn(batch_size, seq_len_k, d_v)

    context, weights = dot_product_attention(Q, K, V)

    print("Q shape:", Q.shape)
    print("K shape:", K.shape)
    print("V shape:", V.shape)
    print("\nAttention Weights shape:", weights.shape)
    print("Context shape:", context.shape)

    print("\nSum of attention weights (should be 1):")
    print(weights.sum(dim=-1))

    mask = torch.ones(batch_size, seq_len_q, seq_len_k)
    mask[:, :, -1] = 0

    context_masked, weights_masked = dot_product_attention(Q, K, V, mask)

    print("\nMasked attention weights (last column should be ~0):")
    print(weights_masked)



test_attention()
```

```
Q shape: torch.Size([2, 3, 5])
K shape: torch.Size([2, 4, 5])
V shape: torch.Size([2, 4, 6])
```

```
Attention Weights shape: torch.Size([2, 3, 4])
Context shape: torch.Size([2, 3, 6])

Sum of attention weights (should be 1):
tensor([[1.0000, 1.0000, 1.0000],
        [1.0000, 1.0000, 1.0000]])

Masked attention weights (last column should be ~0):
tensor([[[0.0790, 0.8913, 0.0297, 0.0000],
         [0.7132, 0.0302, 0.2565, 0.0000],
         [0.2799, 0.5694, 0.1507, 0.0000]],

        [[0.2982, 0.4433, 0.2585, 0.0000],
         [0.2345, 0.2617, 0.5039, 0.0000],
         [0.6128, 0.3037, 0.0835, 0.0000]]])
```

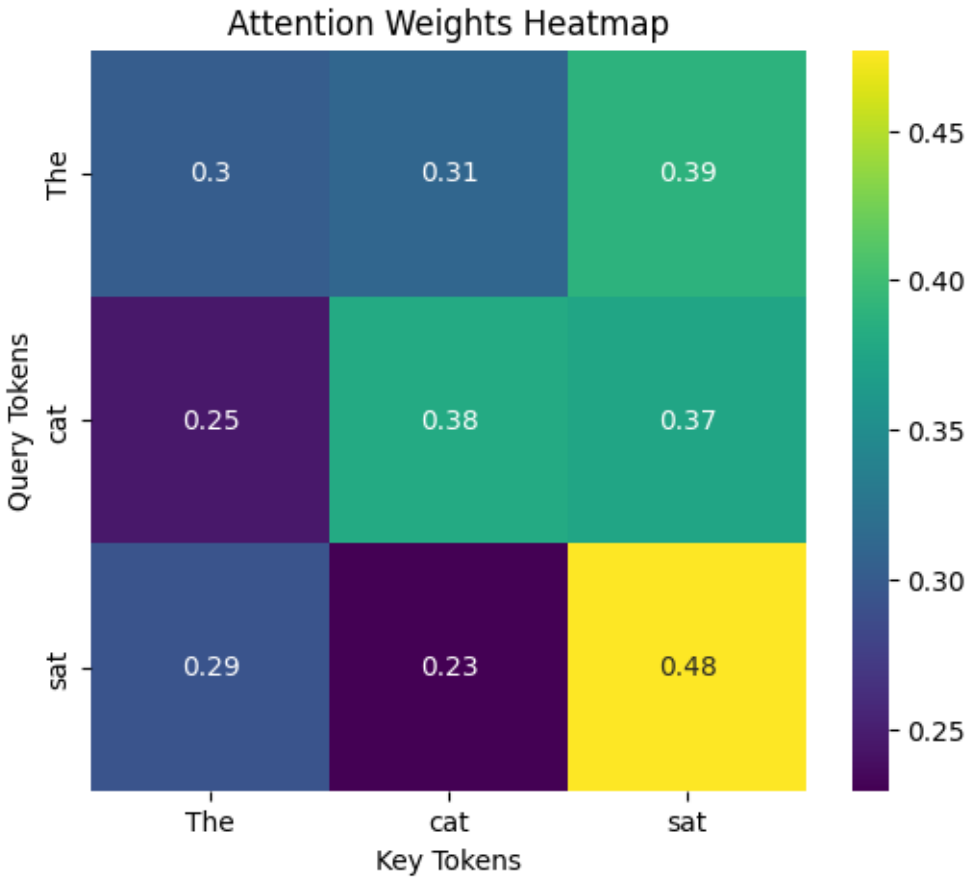
1.1.2 Task A.2: Visualizing Attention

Select a short example sentence and manually create mock Q, K, and V tensors. 1. Run the `dot_product_attention` function. 2. Visualize the generated Attention Weights Matrix (e.g., using a heatmap from `matplotlib` or `seaborn`). This plot should demonstrate which input words the model is focusing on when calculating the output for a specific word.

[4]:
```python
def visualize_attention(weights, tokens):
    plt.figure(figsize=(6, 5))
    sns.heatmap(weights.detach().numpy(),
                xticklabels=tokens,
                yticklabels=tokens,
                cmap="viridis",
                annot=True)

    plt.xlabel("Key Tokens")
    plt.ylabel("Query Tokens")
    plt.title("Attention Weights Heatmap")
    plt.show()
```

[5]:
```python
# 1. Setup Mock Data
torch.manual_seed(42) # Set seed for reproducibility

sentence = ["The", "cat", "sat"]
batch_size = 1
seq_len = len(sentence)
d_k = 4  # Embedding dimension (kept small for example)

# Create random mock tensors for Q, K, V
Q = torch.randn(batch_size, seq_len, d_k)
K = torch.randn(batch_size, seq_len, d_k)
```

```
V = torch.randn(batch_size, seq_len, d_k)

# 2. Run Dot-Product Attention
output, weights = dot_product_attention(Q, K, V)

# 3. Visualize
visualize_attention(weights[0], sentence)
```



1.2 Part B: Applying a Pre-trained Transformer

1.2.1 Task B.1: Zero-Shot Sequence-to-Sequence (Translation)

Leverage the Hugging Face transformers library, which utilizes the full architecture (Encoder-Decoder) described in the "Attention is All You Need" paper.

1. Load a pre-trained Seq2Seq model, such as **T5** (`t5-small`) or **mBART** (`mbart-large-50`), and its corresponding tokenizer.
   - Action: Use `AutoTokenizer.from_pretrained(...)` and `AutoModelForSeq2SeqLM.from_pretrained(...)`.
2. Perform a **zero-shot translation** (e.g., `English to German/French`) on a sample sentence.

You must correctly format the input prompt, including the task instruction (e.g., "`translate English to German: ...`" for T5).

3. Use the model's `generate()` method to obtain the translated output.

[6]:
```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
from transformers import MBartForConditionalGeneration, MBart50TokenizerFast
```

[7]:
```
def translate_text(text, source_lang="English", target_lang="German"):
    model_name = "t5-small"

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

    prompt = f"translate {source_lang} to {target_lang}: {text}"

    inputs = tokenizer(prompt, return_tensors="pt")

    outputs = model.generate(**inputs, max_length=50)

    translation = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return translation
```

[8]:
```
def translate_with_mbart(text, target_lang_code="de_DE"):
    model_name = "facebook/mbart-large-50-many-to-many-mmt"

    tokenizer = MBart50TokenizerFast.from_pretrained(model_name)
    model = MBartForConditionalGeneration.from_pretrained(model_name)

    tokenizer.src_lang = "en_XX"

    encoded = tokenizer(text, return_tensors="pt")

    generated_tokens = model.generate(
        **encoded,
        forced_bos_token_id=tokenizer.lang_code_to_id[target_lang_code],
        max_length=50
    )

    translation = tokenizer.batch_decode(generated_tokens,␣
 ↪skip_special_tokens=True)[0]

    return translation
```

[9]:
```
# Sample Sentence
english_text = "The transformer model is powerful and efficient."
```

```
german_translation = translate_text(english_text, target_lang="German")
french_translation = translate_text(english_text, target_lang="French")

print(f"-" * 30)
print(f"Original: {english_text}")
print(f"German:   {german_translation}")
print(f"French:   {french_translation}")

german_translation = translate_with_mbart(english_text,␣
 ↪target_lang_code="de_DE")

french_translation = translate_with_mbart(english_text,␣
 ↪target_lang_code="fr_XX")

print(f"-" * 30)
print(f"Original: {english_text}")
print(f"German:   {german_translation}")
print(f"French:   {french_translation}")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94:
UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab
and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.
  warnings.warn(
```

```
config.json: 0.00B [00:00, ?B/s]
```

```
tokenizer_config.json: 0.00B [00:00, ?B/s]
```

```
spiece.model:   0%|          | 0.00/792k [00:00<?, ?B/s]
```

```
tokenizer.json: 0.00B [00:00, ?B/s]
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
```

```
model.safetensors:   0%|          | 0.00/242M [00:00<?, ?B/s]
```

```
Loading weights:   0%|          | 0/131 [00:00<?, ?it/s]
```

```
generation_config.json:   0%|          | 0.00/147 [00:00<?, ?B/s]
```

```
Loading weights:   0%|          | 0/131 [00:00<?, ?it/s]
```

```
------------------------------
Original: The transformer model is powerful and efficient.
German:   Das Transformatormodell ist leistungsstark und effizient.
French:   Le modèle de transformateur est puissant et efficace.

tokenizer_config.json:   0%|          | 0.00/529 [00:00<?, ?B/s]

sentencepiece.bpe.model:   0%|          | 0.00/5.07M [00:00<?, ?B/s]

special_tokens_map.json:   0%|          | 0.00/649 [00:00<?, ?B/s]

config.json: 0.00B [00:00, ?B/s]

model.safetensors:   0%|          | 0.00/2.44G [00:00<?, ?B/s]

Loading weights:   0%|          | 0/516 [00:00<?, ?it/s]

generation_config.json:   0%|          | 0.00/261 [00:00<?, ?B/s]

Loading weights:   0%|          | 0/516 [00:00<?, ?it/s]

------------------------------
Original: The transformer model is powerful and efficient.
German:   Das Transformatormodell ist leistungsstark und effizient.
French:   Le modèle de transformateur est puissant et efficace.
```

1.2.2 Task B.2: The Need for Positional Encoding (Theoretical)

The Transformer removes the sequential processing of RNNs, making all tokens independent. Explain (in a brief text block in the notebook) why this is a problem and how Positional Encoding (a fixed or learned vector added to the word embedding) is essential to re-introduce the notion of word order into the model.

Positional Encoding is necessary because Transformers process all tokens in parallel. Unlike RNNs, they do not inherently understand word order. Self-attention treats input tokens independently and does not capture sequence information. Without positional encoding, the sentence:

“The cat sat” and “Sat cat the”

would appear identical to the model. Positional Encoding adds information about token position using sine and cosine functions. This allows the model to understand word order while still benefiting from parallel computation.

1.3 What is Primary computational computational advantage of self-attention over RNN?

The main computational strength of Self-Attention is its capability to process the sequence in parallel.

In the case of LSTMs, the computation is inherently sequential. In order to compute the hidden state at time t, the network has to compute all the previous hidden states from 1 to t-1. This introduces a linear O(n) barrier that makes it impossible to accelerate the computation on GPUs.

Self-Attention computes the interaction between all pairs of tokens in parallel. Since there is no need to compute the previous time steps to start the next computation, the shortest path between any

two tokens is O(1). This enables the network to perform massive parallelization, which significantly speeds up the training process for longer sequences.

1.4 Deliverables

1. A fully executed Python Notebook (.ipynb) implementing the Dot-Product Attention function and the zero-shot translation using a Hugging Face Transformer model.
2. **Visualization:** The heatmap of the Attention Weights Matrix from Task A.2.
3. **Analysis:** A concise answer (approx. 100 words) to the question: What is the primary computational advantage of using Self-Attention (within the Transformer) over the Recurrent connection (in LSTMs) when processing long sequences? (Hint: Think about parallel processing).

[ ]: